Emergence, Complexity and Computation ECC

Paul Rosin
Andrew Adamatzky
Xianfang Sun *Editors*

Cellular Automata in Image Processing and Geometry

Springer

Emergence, Complexity and Computation

Volume 10

Series editors

Ivan Zelinka, Technical University of Ostrava, Ostrava, Czech Republic
e-mail: ivan.zelinka@vsb.cz

Andrew Adamatzky, University of the West of England, Bristol, United Kingdom
e-mail: adamatzky@gmail.com

Guanrong Chen, City University of Hong Kong, Hong Kong
e-mail: eegchen@cityu.edu.hk

Editorial Board

Ajith Abraham, MirLabs, USA
Ana Lucia C. Bazzan, Universidade Federal do Rio Grande do Sul, Porto Alegre RS Brasil
Juan C. Burguillo, University of Vigo, Spain
Sergej Čelikovský, Academy of Sciences of the Czech Republic, Czech Republic
Mohammed Chadli, University of Jules Verne, France
Emilio Corchado, University of Salamanca, Spain
Donald Davendra, Technical University of Ostrava, Czech Republic
Andrew Ilachinski, Center for Naval Analyses, USA
Jouni Lampinen, University of Vaasa, Finland
Martin Middendorf, University of Leipzig, Germany
Edward Ott, University of Maryland, USA
Linqiang Pan, Huazhong University of Science and Technology, Wuhan, China
Gheorghe Păun, Romanian Academy, Bucharest, Romania
Hendrik Richter, HTWK Leipzig University of Applied Sciences, Germany
Juan A. Rodriguez-Aguilar, IIIA-CSIC, Spain
Otto Rössler, Institute of Physical and Theoretical Chemistry, Tübingen, Germany
Vaclav Snasel, Technical University of Ostrava, Czech Republic
Ivo Vondrák, Technical University of Ostrava, Czech Republic
Hector Zenil, Karolinska Institute, Sweden

For further volumes:
http://www.springer.com/series/10624

About this Series

The Emergence, Complexity and Computation (ECC) series publishes new developments, advancements and selected topics in the fields of complexity, computation and emergence. The series focuses on all aspects of reality-based computation approaches from an interdisciplinary point of view especially from applied sciences, biology, physics, or Chemistry. It presents new ideas and interdisciplinary insight on the mutual intersection of subareas of computation, complexity and emergence and its impact and limits to any computing based on physical limits (thermodynamic and quantum limits, Bremermann's limit, Seth Lloyd limits...) as well as algorithmic limits (Gödel's proof and its impact on calculation, algorithmic complexity, the Chaitin's Omega number and Kolmogorov complexity, non-traditional calculations like Turing machine process and its consequences,...) and limitations arising in artificial intelligence field. The topics are (but not limited to) membrane computing, DNA computing, immune computing, quantum computing, swarm computing, analogic computing, chaos computing and computing on the edge of chaos, computational aspects of dynamics of complex systems (systems with self-organization, multiagent systems, cellular automata, artificial life,...), emergence of complex systems and its computational aspects, and agent based computation. The main aim of this series it to discuss the above mentioned topics from an interdisciplinary point of view and present new ideas coming from mutual intersection of classical as well as modern methods of computation. Within the scope of the series are monographs, lecture notes, selected contributions from specialized conferences and workshops, special contribution from international experts.

Paul Rosin · Andrew Adamatzky
Xianfang Sun
Editors

Cellular Automata in Image Processing and Geometry

Springer

Editors
Paul Rosin
School of Computer Science & Informatics
Cardiff University
Cardiff
United Kingdom

Xianfang Sun
School of Computer Science & Informatics
Cardiff University
Cardiff
United Kingdom

Andrew Adamatzky
Unconventional Computing Centre
University of the West of England
Bristol
United Kingdom

ISSN 2194-7287 ISSN 2194-7295 (electronic)
ISBN 978-3-319-06430-7 ISBN 978-3-319-06431-4 (eBook)
DOI 10.1007/978-3-319-06431-4
Springer Cham Heidelberg New York Dordrecht London

Library of Congress Control Number: 2014936602

© Springer International Publishing Switzerland 2014
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed. Exempted from this legal reservation are brief excerpts in connection with reviews or scholarly analysis or material supplied specifically for the purpose of being entered and executed on a computer system, for exclusive use by the purchaser of the work. Duplication of this publication or parts thereof is permitted only under the provisions of the Copyright Law of the Publisher's location, in its current version, and permission for use must always be obtained from Springer. Permissions for use may be obtained through RightsLink at the Copyright Clearance Center. Violations are liable to prosecution under the respective Copyright Law.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

Preface

Image processing and mathematical morphology routines are naturally parallel. The intensity of a pixel is updated depending on the intensities of its closest neighbouring pixels. The intensities of pixels are updated simultaneously, in discrete time and by the same rule. This is exactly how a cellular automaton works if the intensity of a pixel is taken as the state of a cell. A cellular automaton is a regular array of locally connected finite state machines, or cells, which take discrete states and update their states in parallel in discrete time depending on the states of their closest neighbours. Therefore cellular automata would be a first port of call when designing computing paradigms, architectures and implementations aimed at solving problems of image processing, pattern recognition and generation.

This area of research is becoming increasingly active. Unfortunately, there are few books, and no journals or conferences that focus on cellular automata based approaches to image processing, pattern recognition and generation. A goal of this book is to provide a single source, collecting together such work. Each chapter contains a brief literature review of the related work. This will prove more convenient than searching the enormous research literature for relevant papers, which are currently widely scattered. Moreover, this concentration of material will also demonstrate the breadth and effectiveness of cellular automata based approaches to image processing, pattern recognition and generation.

We invited experts in computer science to share their findings, views and ideas on what exact problems of image processing, pattern recognition and generation can be efficiently solved in cellular automata architectures, and to provide detailed descriptions of exact cellular automaton machines they are using. The topics covered include

- **image compression and resizing:** "Cellular Automata for Efficient Image and Video Compression" by Dogaru and Dogaru, "Cellular Automata for Image Resizing" by Ioannidis, Sirakoulis and Andreadis;
- **skeletonization, erosion and dilation:** "Skeletonizing Digital Images with Cellular Automata" by Díaz-Pernil, Peña-Cantillana and Gutiérrez-Naranjo, "Image Processing Algorithms Implementation Using Quantum Cellular Automata" by Mardiris and Chatzis;

- **convex hull computation, edge detection and segmentation:** "Convex Hulls and Metric Gabriel Graphs" by Maignan and Gruau, "Edge Detection using Cellular Automata" by Rosin and Sun, "The Application of Cellular Automaton in Medical Semiautomatic segmentation" by Yang and Gao;
- **forgery detection and content based retrieval**: "Copy-move Forgery Detection Using Cellular Automata" by Tralíc, Rosin, Sun and Grgic, "Active Image Forgery Detection using Cellular Automata" by Tafti, Hassannia, "Content-based Image Retrieval With Cellular Automata" by van Zijl;
- **pattern generation**: "Cellular Automaton Shading for Building Envelopes" by Zawidzki, "Pattern Formation Using Cellular Automata and L-Systems: A Case Study in Producing Islamic Patterns" by Minoofam, Dehshibi, Bastanfard and Shanbehzadeh, "Interactive Cellular Automata Systems for Creative Projects" by Forbes.

We see the potential impact of the book in the advancement of the theory of image processing, pattern recognition and generation and the design of efficient algorithms and hardware for parallel image processing and analysis. The book aims at computer scientists, software programmers, electronic engineers, mathematicians and physicists, and at everyone who studies or develops cellular automaton algorithms and tools for image processing and analysis, or develops novel architectures and implementations of massive parallel computing devices. The book will be an attractive reading for a general audience because it has do-it-yourself appeal: all computer experiments presented in the book can be implemented with minimal knowledge of programming. Simplicity yet substantial functionality of the cellular automaton approach and transparency of the algorithms proposed makes the book an ideal supplementary reading in courses on image processing, parallel computing, automata theory and applications.

Cardiff, UK Paul Rosin
Bristol, UK Andrew Adamatzky
Cardiff, UK Xianfang Sun
March 2014

Contents

1 Cellular Automata for Efficient Image and Video Compression 1
Radu Dogaru, Ioana Dogaru
1.1 Introduction and Motivation 1
1.2 An Alternative to Compressive Sensing Based on Cellular Automata Scan 5
1.2.1 Principle of Chaotic Scan 5
1.2.2 Properties of the Chaotic Counters 7
1.2.3 Designing Good Chaotic Counters as Hybrid Cellular Automata 10
1.2.4 Message Recovery and Examples 12
1.3 Image Compression Based on Dictionaries Generated by Cellular Automata (CA-VQ) 15
1.3.1 The General Framework of Dictionary Based Compression 15
1.3.2 Learning CA-Based Dictionaries and Performance Evaluation of the CA-VQ System 16
1.4 Hardware Description and Synthesis of CA Using Algebraic Normal Form 20
References 21

2 Cellular Automata for Image Resizing 25
Konstantinos Ioannidis, Georgios Ch. Sirakoulis, Ioannis Andreadis
2.1 Introduction 25
2.2 Cellular Automata Fundamentals and Canny Edge Detector 29
2.2.1 CA Fundamentals 29
2.2.2 Canny Edge Detector 30
2.3 Proposed Method 31
2.3.1 Edge Detection 31
2.3.2 CA Resizing 32
2.3.3 Remapping Process 33
2.4 Experimental Results 34
2.5 Hardware Implementation 40

2.6 Discussion and Conclusions 42
References 43

3 Skeletonizing Digital Images with Cellular Automata 47
Daniel Díaz-Pernil, Francisco Peña-Cantillana, Miguel A. Gutiérrez-Naranjo
3.1 Introduction 47
3.2 Skeletonizing Algorithms 49
3.3 Guo and Hall Algorithm 51
3.4 Cellular Automata 53
3.5 Parallel Implementation 55
3.5.1 Examples 56
3.6 Conclusions 60
References 60

4 Image Processing Algorithms Implementation Using Quantum Cellular Automata 65
Vassilios Mardiris, Vassilios Chatzis
4.1 Introduction 65
4.2 Mathematical Morphology 67
4.3 Quantum-dot Cellular Automata 69
4.4 Methodology 73
4.4.1 Analysis 73
4.4.2 Circuit Design 73
4.4.3 Simulation 75
4.5 QCA Implementation of Morphological Operations 75
4.5.1 QCA Implementation of Morphological Erosion 76
4.5.2 QCA Implementation of Morphological Dilation 77
4.6 Conclusions 80
References 80

5 Edge Detection Using Cellular Automata 85
Paul L. Rosin, Xianfang Sun
5.1 Introduction 85
5.2 Boundary Detection 86
5.3 Edge Detection in Intensity Images 89
5.4 Post-Processing of Edges 92
5.4.1 A Simple Edge Linking Scheme 93
5.5 Experiments 94
5.6 Conclusions 100
References 101

6 Copy-Move Forgery Detection Using Cellular Automata 105
Dijana Tralic, Paul L. Rosin, Xianfang Sun, Sonja Grgic
6.1 Introduction 106
6.2 Copy-Move Forgery (CMF) 107

6.3 Copy-Move Forgery Detection (CMFD) 107
6.3.1 Block-Based Method for CMFD 109
6.3.2 Possible Feature Vectors 111
6.4 CA for CMF Detection 112
6.4.1 Representation of Image in Binary 112
6.4.2 Plain CMF 116
6.4.3 Application on Post-processed Images 119
6.5 Future Work 122
6.6 Conclusion 122
References 123

7 Active Image Forgery Detection Using Cellular Automata 127
Ahmad Pahlavan Tafti, Hamid Hassannia
7.1 Introduction 127
7.2 Scenario 1: Using Cellular Automata and LU Decomposition . . . 129
7.2.1 Proposed Model 130
7.3 Scenario 2: Using Cellular Automata and Singular Value Decomposition 134
7.3.1 Proposed Model 134
7.4 Dataset and Experimental Results 136
7.4.1 Performance and Visual Quality 137
7.4.2 Time Consumption 138
7.4.3 True and False Alert 138
7.4.4 PSNR 139
7.4.5 Secret Key Sensitivity 139
7.4.6 Diffusion 141
7.4.7 Comparison with Non-CA Works 142
7.5 Limitations 142
7.6 Conclusion and Future Work 144
References 145

8 Content-Based Image Retrieval with Cellular Automata 147
Lynette van Zijl
8.1 Introduction 147
8.2 Content-Based Image Retrieval: A Background 147
8.3 Cellular Automata in Image Processing and Feature Analysis . . . 149
8.3.1 Noise Reduction 149
8.3.2 Edge Detection 152
8.3.3 Segmentation 153
8.3.4 Colour Matching and Histograms 153
8.3.5 Shape Matching 155
8.4 Practical Case Study: Recognition of LEGO Bricks 156
8.5 Conclusion 160
References 160

9 The Application of Cellular Automaton in Medical Semiautomatic Segmentation 163
Yonghui Gao, Jie Yang
9.1 Introduction 164
9.2 Cellular Automaton Segmentation Rule 166
9.3 Cellular Automaton Interactive Segmentation 168
9.3.1 Labels in Image Plane 168
9.3.2 Regional Cellular Automaton Segmentation 169
9.3.3 Volume Cellular Automaton Segmentation 173
9.4 Block Cellular Automaton Segmentation in Medical Applications 177
References 180

10 Convex Hulls and Metric Gabriel Graphs 183
Luidnel Maignan, Frédéric Gruau
10.1 Notational and Naming Convention 183
10.2 Introduction 184
10.3 The Angular Point of View 186
10.3.1 Majority Rule and θ-Convexity 187
10.3.2 Complete θ-Convex Hull 187
10.4 The Metrical Point of View 188
10.4.1 Majority Rule and Metric Convexity 190
10.4.2 Complete Metric Convex Hulls 192
10.5 Detectable Middles and Metric Gabriel Graphs 196
10.5.1 Pairwise Construction in Euclidean Space 196
10.5.2 (Metric) Gabriel Graphs in Cellular Spaces 198
10.6 The Complete Cellular Automaton 201
References 203

11 Cellular Automaton Shading for Building Envelopes 205
Machi Zawidzki
11.1 Building Envelope and Daylighting 205
11.2 Why Cellular Automata to Drive Shading of Building Envelopes? 207
11.2.1 The Nomenclature 209
11.3 One-Dimensional Cellular Automata Applied on Surfaces 211
11.3.1 Elementary Cellular Automata 212
11.3.2 The Original CA for Shading 212
11.3.3 Half-Distance Automata 212
11.3.4 Higher Order Cellular Automata 213
11.3.5 Other Regular Tessellations: Hexagonal and Triangular 215
11.3.6 PFSS on Triangular Grid: $PFSS_T$ 219
11.4 Two-Dimensional Cellular Automata on Surfaces 220
11.4.1 Triangular Cellular Automata 222

11.4.2 Cellular Automata on Triangulated Free-Form Surfaces . . . 223
11.5 Application of Evolutionary Algorithms for Optimization of CA Shading . . . 223
11.6 Prototypes . . . 226
References . . . 228

12 Pattern Formation Using Cellular Automata and L-Systems: A Case Study in Producing Islamic Patterns . . . 233
Seyyed Amir Hadi Minoofam, Mohammad Mahdi Dehshibi, Azam Bastanfard, Jamshid Shanbehzadeh
12.1 Introduction . . . 234
12.2 Preliminary Definitions and Terminologies . . . 237
12.3 Proposed Method . . . 238
12.3.1 Ma'qeli Character Generation Using Margolus Neighborhood . . . 239
12.3.2 Word and Sentence Generation Using Ma'qeli Patterns . . . 240
12.3.3 Holy Word Formation Using L-Systems . . . 241
12.4 Experimental Results . . . 244
12.4.1 Ma'qeli Script Generation Using 1D Synchronous Cellular Automata . . . 245
12.4.2 Ma'qeli Script Generation Using 2D Synchronous Cellular Automata . . . 245
12.4.3 Holy Words Formation Using L-Systems . . . 249
12.5 Conclusion . . . 250
References . . . 250

13 Interactive Cellular Automata Systems for Creative Projects . . . 253
Angus Graeme Forbes
13.1 Creative Projects Based on Cellular Automata Systems . . . 253
13.2 The *Fluid Automata* Project . . . 256
13.3 Simulation and Visualization in the *Fluid Automata* Project . . . 257
13.3.1 Fluid Simulation . . . 257
13.3.2 Flow Visualization . . . 261
13.4 Single-User Interaction Techniques . . . 264
13.5 Multi-user Interaction Techniques . . . 265
13.6 The *Annular Genealogy* Project . . . 266
13.7 Conclusion . . . 268
References . . . 270

References . . . 273

Index . . . 301

List of Contributors

Andrew Adamatzky
University of the West of England, Bristol, United Kingdom
e-mail: andrew.adamatzky@uwe.ac.uk

Ioannis Andreadis
Department of Electrical and Computer Engineering,
Democritus University of Thrace, Xanthi, Greece
e-mail: iandread@ee.duth.gr

Azam Bastanfard
Department of Computer Engineering,
Islamic Azad University, Karaj, Iran
e-mail: bastanfard@kiau.ac.ir

Vassilios Chatzis
Technological Educational Institute of Eastern Macedonia and Thrace,
Kavala, Greece
e-mail: chatzis@teikav.edu.gr

Mohammad Mahdi Dehshibi
Department of Computer Engineering, Islamic Azad University,
Science and Research Branch, Iran
e-mail: mohammad.dehshibi@piau.ac.ir

Daniel Díaz-Pernil
Department of Applied Mathematics, University of Seville, Spain
e-mail: sbdani@us.es

Ioana Dogaru
Dept. of Applied Electronics and Information Engineering,
University "Politehnica" of Bucharest, Bucharest, Romania
e-mail: ioana.dogaru@upb.ro

Radu Dogaru
Dept. of Applied Electronics and Information Engineering,
University "Politehnica" of Bucharest, Bucharest, Romania
e-mail: radu_d@ieee.org

Yonghui Gao
School of Medical Instrument and Food Engineering,
University of Shanghai for Science and Technology
e-mail: gaoyonghui1978@163.com

Angus Graeme Forbes
University of Arizona, USA
e-mail: angus.forbes@gmail.com

Frederic Gruau
University Paris Sud, Orsay, France
e-mail: frederic.gruau@lri.fr

Sonja Grgic
Faculty of Electrical Engineering and Computing, University of Zagreb, Zagreb
e-mail: Sonja.Grgic@fer.hr

Miguel A. Gutiérrez-Naranjo
Department of Computer Science and Artificial Intelligence,
University of Seville, Spain
e-mail: magutier@us.es

Hamid Hassannia
Department of Advanced Computing, Fan Pardaz Higher Education Institute, Iran
e-mail: hamid.h.h@ieee.org

Konstantinos Ioannidis
Department of Electrical and Computer Engineering,
Democritus University of Thrace, Xanthi, Greece
e-mail: kioannid@ee.duth.gr

Luidnel Maignan
University Paris, Est-Créteil, France
e-mail: Luidnel.Maignan@u-pec.fr

Vassilios Mardiris
Technological Educational Institute of Eastern Macedonia and Thrace,
Kavala, Greece
e-mail: mardiris@teikav.edu.gr

Seyyed Amir Hadi Minoofam
Department of Computer Engineering, Islamic Azad University,
Nazar Abad Center, Karaj, Iran
e-mail: minoofam@qiau.ac.ir

Francisco Peña-Cantillana
Department of Computer Science and Artificial Intelligence,
University of Seville, Spain
e-mail: frapencan@gmail.com

Paul L. Rosin
School of Computer Science and Informatics, Cardiff University, United Kingdom
e-mail: Paul.Rosin@cs.cardiff.ac.uk

Jamshid Shanbehzadeh
Department of Computer Engineering, Kharazmi University, Tehran, Iran
e-mail: jamshid@saba.tmu.ac.ir

Georgios Ch. Sirakoulis
Department of Electrical and Computer Engineering,
Democritus University of Thrace, Xanthi, Greece
e-mail: gsirak@ee.duth.gr

Xianfang Sun
School of Computer Science and Informatics, Cardiff University, United Kingdom
e-mail: Xianfang.Sun@cs.cardiff.ac.uk

Ahmad Pahlavan Tafti
University of Wisconsin Milwaukee, USA
e-mail: pahlava2@uwm.edu,

Dijana Tralic
Faculty of Electrical Engineering and Computing, University of Zagreb, Zagreb
e-mail: Dijana.Tralic@fer.hr

Lynette van Zijl
Stellenbosch University, Stellenbosch, South Africa
e-mail: lvzijl@sun.ac.za

Jie Yang
Shanghai Jiaotong University, China
e-mail: jieyang@sjtu.edu.cn

Machi Zawidzki
Massachusetts Institute of Technology, USA
e-mail: zawidzki@mit.edu

Chapter 1
Cellular Automata for Efficient Image and Video Compression

Radu Dogaru and Ioana Dogaru

Abstract. This chapter focuses on applications of cellular automata for image and video compression leading to high efficiency implementations (i.e. relatively simple operators and algorithms, and extra functionality such as encryption, achieved with no additional resources). First, a CA-based alternative to compressive sensing is presented, assuming that an image sensor is available where pixels can be addressed randomly. The key idea is to replace the traditional raster scan counter addressing the sensing units, with a "chaotic counter" behaving like a pseudo-random number generator but having in addition the binary synchronization property. Consequently, only a fraction of all pixels (the most relevant) are rapidly scanned. A certain class of hybrid CA (HCA) implements the chaotic counter. A recovery algorithm, implemented in the receiving unit, reconstructs the missing (less relevant) pixels. Another CA-based method presented herein is a vector-quantization method where color or gray images are decomposed in binary bit-planes and compression is achieved by replacing binary blocks within bit-planes with similar binary vectors from an indexed dictionary, previously generated by a properly designed CA. Finally, aspects of efficiently implementing the CA modules present in both schemes, are briefly discussed.

1.1 Introduction and Motivation

In various applications (remote sensing, secure data transmission, compressive sensing [1] some message (represented by the generic sequence of samples) must be passed from a transmitter system (next abbreviated "Tx") to a remote receiver

Radu Dogaru · Ioana Dogaru
University "Politehnica" of Bucharest,
Dept. of Applied Electronics and Information Engineering,
Natural Computing Laboratory, Room B232, Bvd. Iuliu Maniu 1-3,
Sector 6, Bucharest, Romania
e-mail: radu_d@ieee.org, ioana.dogaru@upb.ro

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_1, © Springer International Publishing Switzerland 2014

(next abbreviated "Rx") or storage medium with an efficient use of the bandwidth or storage memory. In addition, in order to maintain data security certain encryption algorithms may be also used. The problem as stated above is usually approached by various methods form the mature areas of image and video compression and cryptography. Yet, some applications require low power consumption and consequently a simple mechanism is needed for encoding and decoding processes.

Cellular automata (CA) hold the promise [25] [6] [5] of a very convenient way to achieve both compression and encryption with the benefit of using simple circuit models leading to low power consumption, as often desired when the source of signal is a stand-alone sensor unit powered by battery or/and solar energy. In cryptology, cellular automata are widely used [8], patents on cellular-automata random number generators being among the first associated with CA applications [31].

In this chapter recent research results in this area are summarized, within a general compression framework depicted in Figure 1.1. The particularity of our approach is to exploit complex dynamics emerging in Boolean cellular automata for various tasks in compression and encryption, usually approached with computationally intensive algorithms using various arithmetic operations such as cosine and other kind of transforms, multiplications etc. Locating useful emergent behaviors in CA and their potential applications are topics described in more detail in [12] and in a series of recent papers [13] [17].

The use of cellular automata in various stages of the compression and decompression phases allows the avoidance of arithmetic operators such as multiplication, summation etc. with a dramatic impact on the architectural complexity of the algorithms implementation. Within this paper only the case of lossy compression is considered, which is effective for image and video content associated to the message. Also the focus is on algorithms that would lower the complexity of the Tx unit (often a stand alone smart sensor with critical power consumption requirements).

As seen in Figure 1.1, in order to compress the string x_t two stages (depicted here as A and B) will be considered. As detailed next, they can be applied independently (i.e. A only, or B only) or consecutively. In stage A, the redundancy present in the original message is exploited to reduce the number N of original samples (i.e. image pixels) into a smaller K number of representative samples. In the corresponding A-stage of the receiver (Rx), the missing samples are recovered (with a certain loss) using various interpolation schemes. Stage A is reminiscent of the compressive sampling approach but the similarity is only at the functional level; while compressive sampling approaches produce the $K-$sized vector s_k as the result of multiplying the message with an $N \times K$ matrix with random (non-binary) elements, our approach dubbed "chaotic scan" [19] simply picks some samples from the original message without performing any arithmetic operation at all. It is the role of a cellular automaton to select (by addressing) the K samples from the original message, as it will be detailed in Section 1.2. Consequently, a highly intensive computational algorithm (usually embedded into a sensor with low power consumption requirements) is replaced now with a much simpler implementation of a CA with $n = \log_2 N$ cells addressing the selection of K samples. In terms of FPGA implementation each CA cell is allocated to one single LUT (basic computational unit in FPGA), a far more

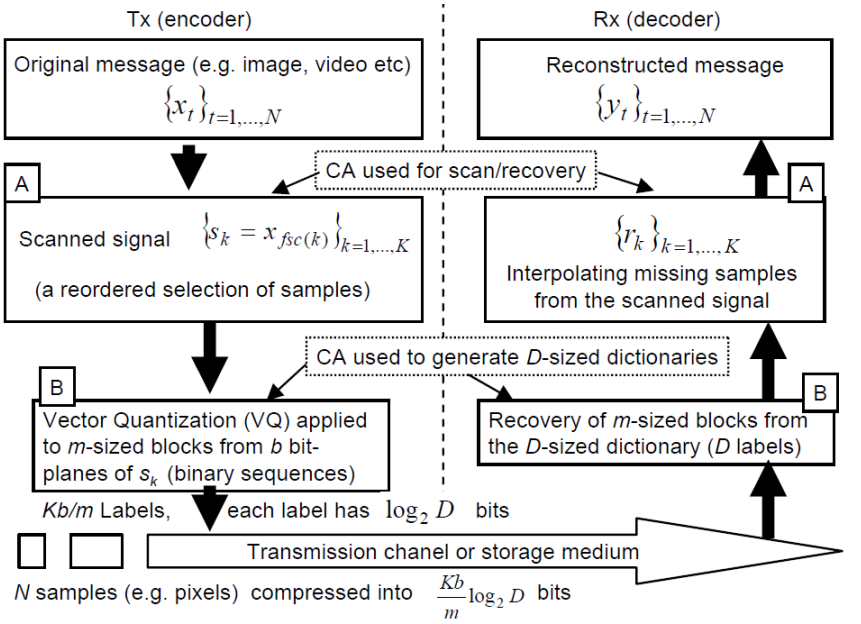


Fig. 1.1 A general framework for message compression and recovery

effective solution than implementing the matrix multiplication required by the compressive sampling approach. It was shown [19] that using a ratio $K/N = 5\%$ ensures a decent recovery of the original signal, while preserving the most important features of the image (or video sequence). Consequently a compression of up to 20 times can be achieved in the A-stage of the encoder with very little computational effort. More computational effort is required in the A-stage of the Rx unit [19] [16] but in most applications of interest the Rx is usually implemented on a desktop computer with no critical requirementes, while the Tx is often an autonomous low-power sensor where the issue of low complexity is critical.

An identical CA structure as the one used to scan the original message is required in the Rx unit. In order to ensure the correct recovery of information, the CA in the Rx unit must evolve (as a dynamic system) in synchrony with the CA used in the Tx unit. This property is supported, as recently was demonstrated [14] by a proper choice of particular CA rules. Such CA rules ensure both **binary synchronization** and **longest cycle properties**. Binary synchronization means that the entire state of the CA (n bits addressing the t sample of the original message) can be recovered from one single bit per clock received from the Tx CA. Among various chaos synchronization schemes [24] [27] presented in the literature this one is the most robust and requires the less synchronization information. The longest cycle property means that eventually all (or almost all, with an insignificant loss of samples) N samples of the original message are addressed (much like a counting automaton, except the pseudo-random ordering or scan) ensuring that, if desired, all information from the

original message is preserved (yet scrambled) without compression i.e. $K \cong N$. In fact, the user-selected K parameter represents the number of cycles advanced by the properly designed CA used in chaotic scan and can be traded off for the quality of the reconstructed message.

Compared to the CA-based A-stage, in terms of functionality, the closest approach found in the literature seems to be the "holographic scan" [3] although the implementation of this algorithm appears to be more complex than implementing a cellular automaton. Other traditional approaches implementing stage-A are the use of various image transforms (requiring intensive arithmetic computations) such as DCT, Karhuenen-Loeve or PCA, kernel-PCA [26] [28] or neural auto-encoders [23].

Stage-B is basically a Vector Quantization (VQ) stage. Blocks of the original message (or compressed, resulting from the A-stage) are m-sized vectors that would be compared with codebook vectors in a D-sized dictionary (previously prepared to optimize the compression efficiency for a class of messages). Consequently a label (among all D possible) is selected indicating the closest codebook vector. Traditional approaches to VQ employ computationally intensive arithmetic operations performed in fixed-point representations of the variables (including message samples). In contrast, our CA-based approach, to be detailed in Section 1.3, has two simplifying features: i) **the message is divided in binary bit-planes** (i.e. binary message sequences associated to one rank bit in the original sequence) as seen in Figure 1.2 such that m-sized vectors are now m-bit binary words; ii) **the codebook is a list of m-sized binary vectors**, generated by a 2-dimensional CA with a certain rule. The CA rule may be obtained using a training approach (as detailed in this chapter) or it can be the result of a selection process [21] [11]. In effect, computationally intensive arithmetic operators are replaced with simple logic operators and the codebook is simply generated based on CA rule information only (there is no need to send the entire codebook from Tx to Rx). In terms of performance (evaluated here as the PSNR - Power to Signal Noise Ratio - between original and recovered message) the CA-VQ approach is comparable to traditional image compression algorithms (e.g. JPEG) and in fact is more effective for low bit-rates (under 0.25 bits / sample). An FPGA implementation is reported in [32].

Mixed approaches involve exploiting both A and B stages. For instance, one may scan only $K < N$ samples from the original message and submit to the VQ system an incomplete m-sized vector (some bits are not specified since they belong to samples that were not scanned). Still a codebook search can be performed for the best match and the result is an improved compression rate (with an N/K factor) at a slightly degraded PSNR performance.

In order to implement the proposed compression methods various technologies may be used. Of a particular interest is the possibility to embed compression algorithms into smart sensors with low power requirements. Consequently, it is important to choose convenient synthesis solutions such that the whole algorithm is described in a hardware description language (e.g. VHDL or Verilog).

s_1^0	s_2^0	$\cdots$	s_k^0	$\cdots$	s_{K-1}^0	s_K^0	Bitplane 0
s_1^1	s_2^1	$\cdots$	s_k^1	$\cdots$	s_{K-1}^1	s_K^1	Bitplane 1
s_1^2	s_2^2	$\cdots$	s_k^2	$\cdots$	s_{K-1}^2	s_K^2	Bitplane 2
s_1^3	s_2^3	$\cdots$	s_k^3	$\cdots$	s_{K-1}^3	s_K^3	Bitplane 3
s_1^4	s_2^4	$\cdots$	s_k^4	$\cdots$	s_{K-1}^4	s_K^4	Bitplane 4
s_1^5	s_2^5	$\cdots$	s_k^5	$\cdots$	s_{K-1}^5	s_K^5	Bitplane 5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_1^b	s_2^b	$\cdots$	s_k^b	$\cdots$	s_{K-1}^b	s_K^b	Bitplane b

Fig. 1.2 Organization of signals in b binary sequences called *bit-planes*. Bitplane j is associated to the binary sequence $s_k^j, k = 1 \ldots K$. Each sample $s_k$is represented on b bits.$s_k = [s_k^0, s_k^1, \cdots, s_k^{b-1}]$ with $s_k^j \varepsilon \{0,1\}$(binary).

In Section 1.4 an efficient method to implement CA systems in FPGA technologies is briefly discussed in relation with the representation of the CA local rule using Algebraic Normal Form (ANF).

1.2 An Alternative to Compressive Sensing Based on Cellular Automata Scan

1.2.1 Principle of Chaotic Scan

This approach to message compression is a compact alternative to the compressive sensing methods [1] and was first proposed in [19]. The simplified model is given in Fig. 1.3, with regards to an image sensor. The idea may be further expanded to any other kind of multi-dimensional sensor in order to reduce the number of samples effectively transmitted from the sensor. The method is effective assuming that adjacent elements in the image array are highly correlated.

The nonlinear dynamic system (discrete-time, discrete-state) present in both Tx (encoding) and Rx (decoding) units can be any kind of automaton as long as it ensures certain properties to be detailed next. Since its role is to address (count) K pixels (or in general, K samples) from the N samples of the original message it will be called next a "chaotic counter". The term "chaotic" is a simplification from "pseudo-random", the above mentioned system implementing in fact a pseudo-random number generator. As discussed next in more detail, a convenient chaotic counter belongs to a class of 1-dimensional cellular automata having all desired properties for such a system. The contrast with a raster scanning counter is exemplified in Fig. 1.4. Here only 5% of the pixels in the original image) were addressed sequentially during the

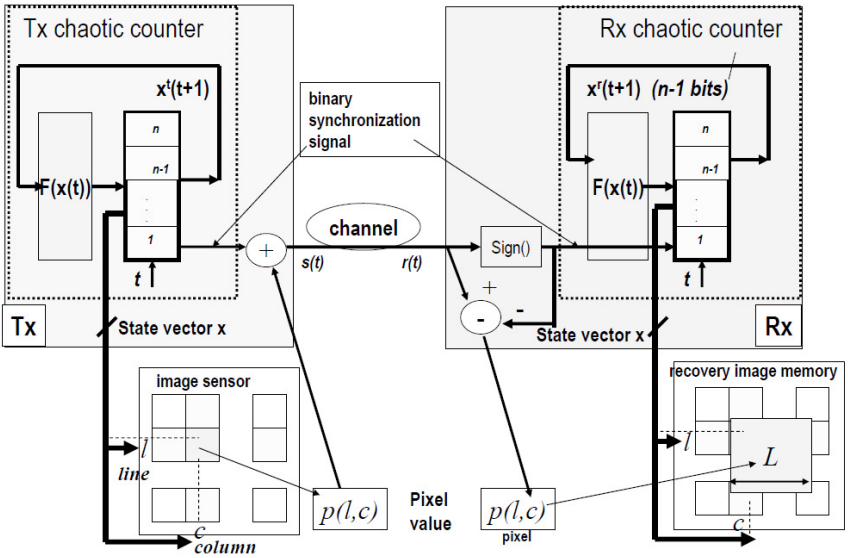


Fig. 1.3 An alternative model for compressed sensing based on chaotic scan: Pixels in a sensor array are chaotically selected by the chaotic counter. At receiving point a similar chaotic counter, synchronized with the one in Tx, is recomposing the image form the serially received pixel value while estimating its L-sized neighborhood. A small fraction of samples from the Tx image suffices to recover a good quality replica of the original image with a certain acceptable information loss.

counting process. While in the case of the raster scan a single image strip with no meaningful content is obtained, the chaotic counter locates distant uncorrelated pixels (at apparently random locations) making the image content recognizable. Based on the assumption that pixels in the neighborhood are often correlated, a simple reconstruction scheme is employed to recover the missing pixels. It consists in filling a $L \times L$ window with the same intensity value (the one of the received pixel). As seen in Fig. 1.4, when an optimal L is selected, this reconstruction scheme allows the recovery of most the important semantic image content.

Consequently, the scanning process of K samples is given by the following low complexity and easy to implement algorithm, where $X(i, j)$ represents the original square image message with N samples:

RESET the counting automaton
FOR *k=1,..K*
i=**rand_i**;
j=**rand_j**;
s(k) =X(i,j);
END

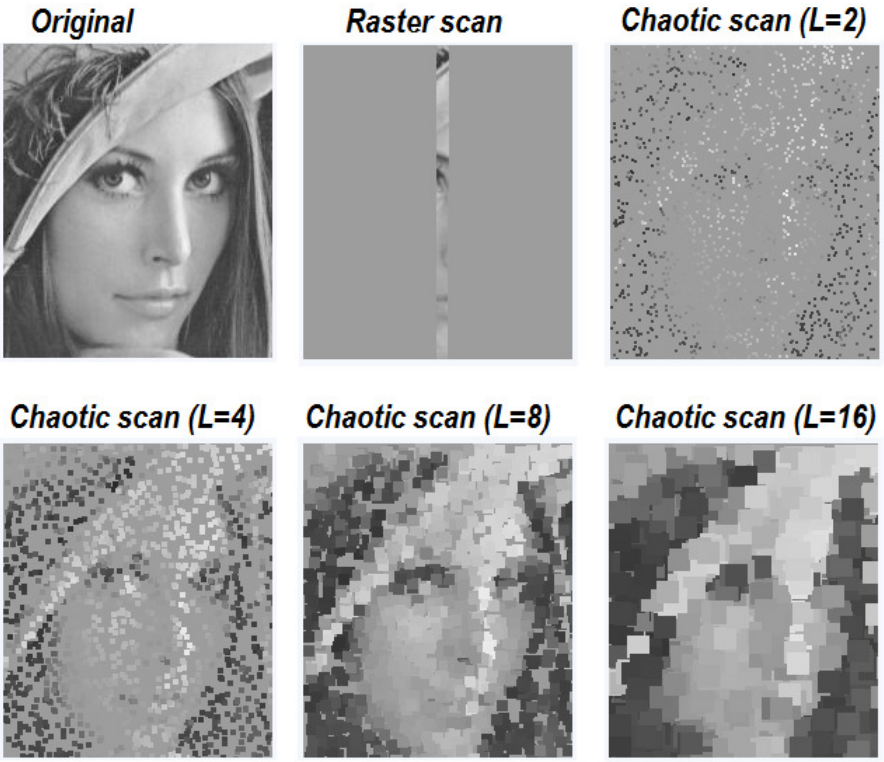


Fig. 1.4 Image recovery with chaotic scan after receiving $K = 0.05N$ pixels (5% of all N pixels of the original image); Too small values of the recovery block size (L) makes the received image difficult to read while using too large L values impede on image details. Depending on the image content there is an optimal L value (here, $L = 8$) such that the semantic content is best revealed.

Without loss of generality in the next we will use **rand_i rand_j** to denote the reading of the next state of the counting automaton (split into $n/2$ bits for line "i" and $n/2$ bits for column "j").

1.2.2 Properties of the Chaotic Counters

In order to understand the properties required for the chaotic counter, we shall consider it as a particular case of discrete-time, discrete-space dynamic system (automaton). Any such dynamic system, also called a *nonlinear map* is defined by a feedback function F imposing a certain profile of the state space (partitioned into certain attractors and their basins of attraction).

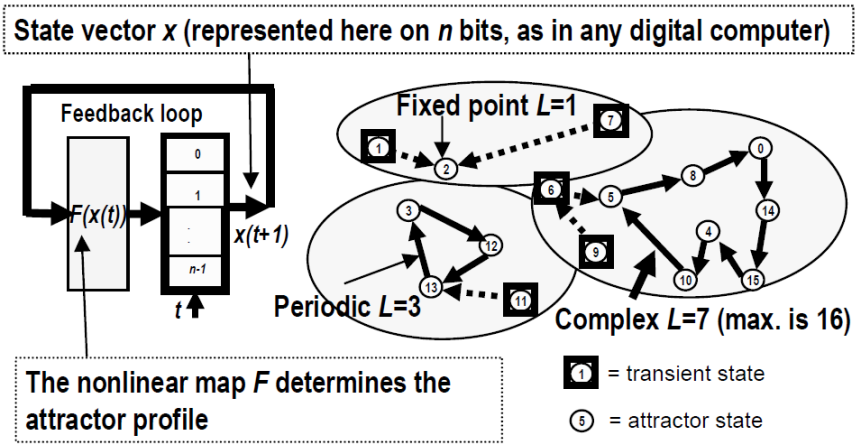


Fig. 1.5 General structure of a nonlinear map or automaton in a digital implementation (discrete-time, finite precision of n bits). In the case of cellular automaton, each bit is associated with a cell. The nonlinear mapping F induces a structure of the state space which is partitioned in attractors, each attractor may also collect "transient states" associated with its basin of attraction. The complexity of attractors is proportional with their length (maximal length is $L = 2^n$) and average distance between consecutive states.

Good chaotic counters must rely on:

i) The existence of a **dominant long attractor cycle** with a length L as close as possible to the maximal length i.e. $N = 2^n$; This property ensures that almost all pixels in the original image are addressed. The existence of transients and of many shorter cycles, imply some loss of pixels (samples) from the original message. Recently, considering CA as a nonlinear dynamical systems [7] defined *conservative* CA (definition applies to any automaton model as depicted in Fig. 1.5) as those having the property that *no state is a transient (or ephemeral) state*. Both LFSR and NLFSR (Linear or Non Linear Feedback Shift Register) used for long as pseudo-random sequence generators have this property as well. The cellular automata in this paper are conservative and consequently they have no transients;

ii) **A "chaotic" character of the dominant longest cycle**; A very long cycle is not necessarily a random one. A good counter-example is the counting automaton used in the traditional raster scan of images. It has a maximal cycle length $L = N$ but the transition from one state to the consecutive one is rather smooth, often only one bit is changing. As discussed above we are interested in pseudo-random counting automata ensuring consecutive distant "jumps" between the coordinates of pixels. To characterize such behaviours, in [18] we introduced a randomness measure that may be conveniently computed. We are in particular interested on the randomness of the *dominant cycle*. The measure of randomness was defined observing that in a "chaotic" automata the average Hamming distance between consecutive binary vector states (as given by the n cell outputs) becomes $n/2$ instead of 1 for counters.

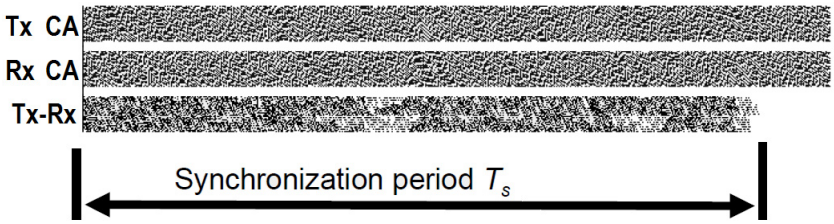


Fig. 1.6 Exemplification of binary synchronization using two identical CA with $n = 31$ cells and rule ID=13474665135 (5 cells neighborhood). Although the initial states of both transmitter (Tx-CA) and receiver (Rx-CA) are different by driving the Rx-CA with 1 single bit from Tx-CA ensures that after a synchronization period T_s the entire state of Rx-CA is identical to the state of Tx-CA.

Therefore, for any arbitrary cycle C_j of length L_j a *scattering coefficient* S_j is defined by averaging the Hamming distances between all consecutive binary vector states in that cycle: $S_j = \frac{1}{nL_j} \sum_{k=1}^{L_j} \sum_{i=1}^{n} |x_i(k) - x_i(k-1)|$ where k is the time index of consecutive states in the cycle j. A *degree of chaos* $\lambda_j = 1 - |2S_j - 1|$ is then defined such that it becomes maximum if $S_j = 0.5$ and zero for the extreme, non-chaotic cases of both fixed points and period 2 cycles (with $S_j = 0$ and $S_j = 1$ respectively). The *degree of chaos* may be regarded as qualitatively similar to the Lyapunov exponent used in continuous-state systems to characterize chaotic behaviours. In our case its largest value is $\lambda_j = 1$ indicates the highest degree of randomness in a finite-length cycle of an automata network;

iii) **Binary synchronization property:** Unlike traditional chaos synchronization [24] in the case of binary synchronization sending only 1 bit from the Tx automaton allows the recovery of the entire state (n bits) in a similar Rx automaton (as seen in Fig. 1.6).

In this case the information needed to resynchronize the Rx is minimal and consists of only 1 bit per clock cycle. Consequently it may be easily embedded and recovered in various forms of modulation/demodulation. This property is not common to CA and it was first investigated for all elementary cellular automata (ECA) in [18]. Within all 256 ECA we found that the binary synchronization property holds only for the conservative rules ID=45 (and its 3 equivalents ID=75,89, and 101). Further work [17] indicates that a precondition to achieve binary synchronization in CA is the existence of an asymmetric cell (i.e. a cell that is sensitive to mirroring the inputs from left to right with respect to the central cell). Also, in order to optimize the dominant cycle length it was found [14] that a hybrid CA model with some of the cells having inverted outputs has a better behavior and allows to design a cellular automaton satisfying all 3 properties mentioned above.

Next, CA or automata systems fulfilling all the above three properties will be mentioned as “good chaotic counters”. As seen in the next subsection, expanding the neighborhood to 5 cells allows the identification of more CA rules holding the

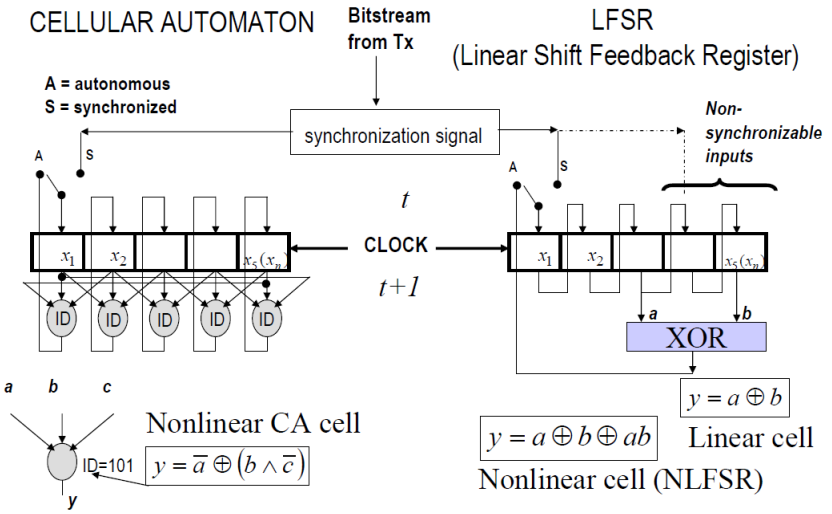


Fig. 1.7 Structure of HCA (left) and LFSR or NLFSR (right) chaotic counters. Unlike HCA, binary synchronization can be used with LFSR/NLFSR only for the cells in the shift-register area, i.e. without cryptographic protection. In the case of HCA cryptographic protection is ensured since synchronization is only possible if the the Rx-CA structure (ID, mask vector) is identical for the Tx-CA. Consequently, the key is given by the specific CA structure.

"good chaotic counter" property. The problem of locating good chaotic counters in the very large space of all possible 2^{32} rules for the 5-cell neighborhood CA is a computationally demanding problem, and so far we solved it only for the case of up to 4 inputs cell within a 5-cell neighborhood.

1.2.3 Designing Good Chaotic Counters as Hybrid Cellular Automata

Ordinary chaotic maps (i.e. logistic, tent, etc.) cannot be used as good chaotic counters because their finite computing precision implementations often produce cycles with only a very small fraction of state vectors (each addressing a pixel in the image sensor array) belonging to the counting cycle. Consequently, only a small fraction of the sensing elements will be addressed, compromising the information acquisition process [15]. As discussed above, a hybrid cellular automaton model proved to be very effective in ensuring the properties of a good chaotic counter. To explain the HCA structure let us consider the case $m = 3$ (3 cells neighbourhood). It expands naturally to a 5-cell or larger neighbourhood. Figure 1.7 presents the HCA automaton structure compared to the widely known LFSR. Note that both of them may be operated in either autonomous mode (as it is the case in the transmitter system Tx) or with one input forced by the synchronization signal (as it is the case in the receiving system Rx).

The discrete-time dynamics of the hybrid cellular automata (HCA) is given by the next equation, which applies synchronously to all n cells (a cell is identified by an index $i \in \{1,2,..n\}$):

$$x_i^T(t+1) = m_i \oplus Cell\left(x_{i-1}^T(t), x_i^T(t), x_{i+1}^T(t)\right), ID\big) \tag{1.1}$$

where the upper index "T" stands for the transmitting CA counter, $\oplus$ is the logical XOR operator and $Cell(u1,u2,u3,ID)$ is a Boolean function with 3 binary inputs ($u1,u2$, and $u3$), also called the CA (local) rule. A periodic boundary condition is assumed i.e. the leftmost cell ($i=1$) is connected to the rightmost one ($i=n$). The binary mask vector $\mathbf{m} = [m_1, m_2, .., m_n]$ can be optimized [14] (so far our programs perform optimization in reasonable time for $n \leq 29$) to obtain a maximal cycle length ($r = L/2^n \to 1$). The above equation (1.1) easily extends to larger neighborhoods such as m =*5* by adding 2 additional inputs to the cell, located on the rightmost and leftmost positions (*i-2*, and *i+2*). For any neighborhood the relationship between inputs and the output local CA rule can be characterized in two different ways while conversion functions are available via [30] :

a) Truth-Table (TT) representation: This is the most widely used representation. The rule is characterized by a binary vector $Y = [y_{N-1}, y_{N-2}, ..., y_0]$. Its representation in decimal basis is called a rule identifier (ID). The output y_k is a binary number assigned to the cell's output when its inputs ordered as a binary vector $[u_n, u_{n-1}, .., u_1]$ are the binary representation of k;

b) Algebraic Normal Form (ANF): This form is described by a binary vector $C = [c_0, c_1, ...c_N]$ (using the method in [30] a unique conversion from Y to C and vice-versa exists) such that its coefficients are multipliers of an algebraic representation on the GF2 exemplified next for the case of m=3 neighborhood:

$$y = c_0 \oplus c_1 u_1 \oplus c_2 u_2 \oplus c_3 u_2 u_1 k_3 u_3 \oplus c_4 u_3 \oplus c_5 u_3 u_1 \oplus c_6 u_3 u_2 \oplus c_7 u_3 u_2 u_1 \tag{1.2}$$

Note that in general (for any size m of the neighbourhood) c_k is the multiplier of a product (logical AND) of all input variables in a binary vector $[u_n, u_{n-1}, \ldots, u_1]$ corresponding to 1 in the associated binary vector representing k. For example, in the case of $m=3$ for $k = 5 = 101_2$ only the inputs corresponding to 1 in the input string $u_3u_2u_1$are selected to be multiplied resulting in the term $c_5u_3u_1$.

The most important results of our research in designing HCA that are good chaotic counters are summarized in Figure 1.8:

i) First, we have **a number of HCA with 3 inputs** where the ANF representation of the rules is given by the formula: $y = m_i \oplus u_a \oplus u_b \oplus u_c u_b$ where a,b,c are cell position indexes such that $a-b = b-c = h$ where h is an integer. In all these cases a computationally intensive program is run to find the best mask (the one maximizing the dominant cycle length L). For instance the HCA with rule ID=1347465135 has the best mask found so far 19801 (decimal representation of mask vector $\mathbf{m} = [m_1, m_2, .., m_n]$) leading to L =*N-1=131071* in the case of n =17 cells (i.e. addressing an image of size 512×1024). The problem with all HCA in the above category is that they are conservative (no transient and possible to optimize

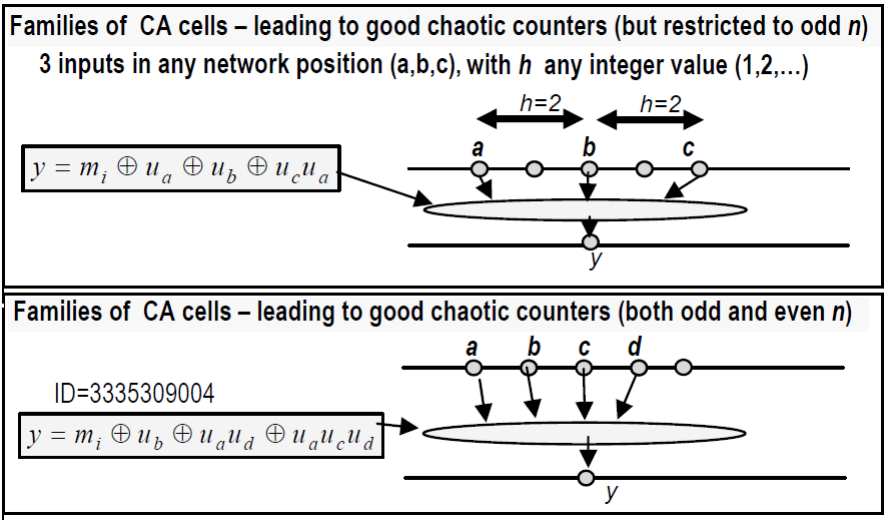


Fig. 1.8 Cell rules in a 5-cell neighbourhood in order to obtain good HCA chaotic counters

for the longest cycle) only for an *odd* number of cells n; This will make them useful in image and video only for sensing arrays with an aspect ratio of 2:1.

ii) Recently we were able to locate a good counting automaton within the class of 4 inputs positioned in a 5-cell neighbourhood. This automaton (ID=3335309004) is **conservative for any number of cells, including even ones**, as required by the commonly used square image sensors. A near-optimal mask for the case $n=18$ (i.e. addressing a 512×512 pixels image) is 17855. For this mask $L=261990=N-154$. The loss of 154 pixels is insignificant for such a compression scheme where the number of scanned samples $K << N$

1.2.4 Message Recovery and Examples

It is assumed that in order to reconstruct the image $\mathbf{Y}$ a counting automaton similar to the one used in the measurement process is available. Also, it is operated in synchrony with the one available in the Tx unit. They would either start from the same initial state or they may be synchronized using the binary synchronization property. The recovery counting automata follows the same dynamics as in the measurement process during the scanning of the$K$ received samples.

The reconstruction algorithm is:
RESET the counting automaton
INITIALIZE all pixels in Y with 0.5;
FOR *k=1,..K*
i=**rand_i**; *j*=**rand_j**;

$$\mathbf{Y} = \mathbf{B}_{i,j} s(k) + \mathbf{Y} \circ (\mathbf{1} - \mathbf{B}_{i,j}) \tag{1.3}$$

END

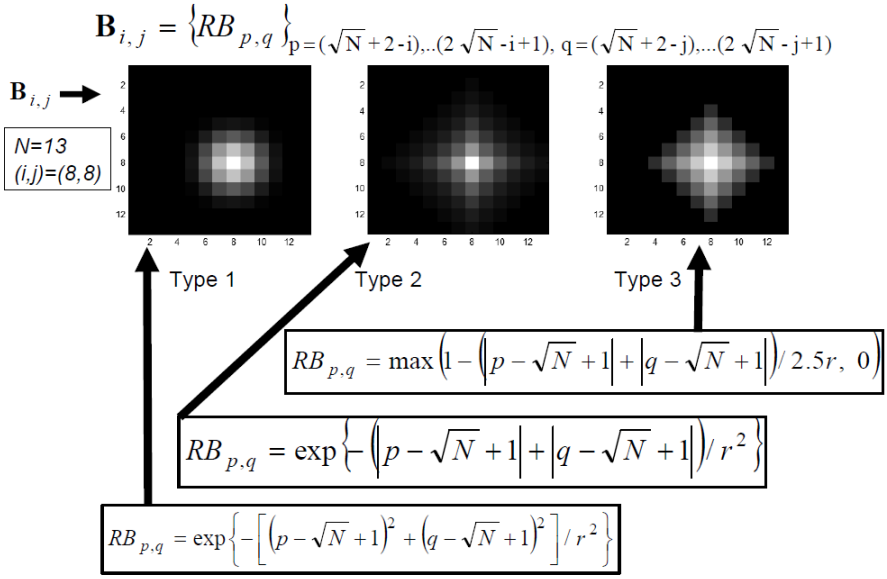


Fig. 1.9 Various sliding bases and their associated radial basis functions

In the above, $\mathbf{1}$ is a square matrix with all unity elements, both $\mathbf{Y}$ and $\mathbf{B}_{i,j}$ are matrices with the same size as the original image $\mathbf{X}$ and $\circ$ denotes element wise matrix multiplication (i.e. if $\mathbf{C} = \mathbf{A} \circ \mathbf{B}$ then $c_{i,j} = a_{ij}b_{ij}$). Consequently the recovery process is an iterative process given by recursive equation (1.3), with a total of at most $2N$ multiplications (using a pice-wise linear basis function, the number of multiplications can be further reduced). The *sliding basis* $\mathbf{B_{i,j}}$ is computed using a radial basis kernel inspired from [20]. The detailed formulae and an example for 3 types of basis functions are given in Figure 1.9.

The sliding basis implements the equivalent of a fuzzy membership function with a maximal value 1 on the position (i, j) of the current measurement $(d = 0)$. Neighboring pixels at distance d will correspond to the decaying amplitude of the kernel function according to a *radius parameter* r to be determined. Such a radial basis function is necessary to reconstruct the neighboring pixels that were not sampled during the measurement process and it is expected to be in relationship with the correlation model of the signal $\mathbf{X}$.

In terms of recovery performance we estimate the PSNR (measured in dB) of the reconstructed image $\mathbf{Y}$ (in this case the noise is the difference between original and the reconstructed image) as a function of K. Such a curve may be also considered as a rate-distortion curve since the compression rate is proportional to K for a given N.

In the following, the "Lena" image with $N = 256 \times 256$ samples (pixels) is considered. It was found that the choice of the basis function (among the 3 types mentioned in Fig. 1.9) has little influence on the PSNR value. Consequently, in order to

reduce the computational effort, the type 3 basis (a piecewise-linear approximation of the Gaussian) is the most suitable. The type-3 basis is 0 almost everywhere except $(2r+1)^2$ pixels. Consequently the number of multiplications required in equation (1.3) reduces from $2N$ *to only* $2(2r+1)^2$.

The pre-computing of RB and its storage allows to avoid calculations of $B_{i,j}$ for each new measurement position as the counter automaton advances. In the case of a fixed radius (e.g. choosing $r = 2$, for a similar reconstruction error as in [4]) the number of multiplications in the recovery algorithm is of the order of 10^6 for $K = 20000$ (easily computed in less than 0.1 second on actual computers). For reference, in [2] where an improved, highly effective reconstruction algorithm is considered, more than 100 seconds are reported for a similar number of measurements. This comparison indicates a significant speed-up of our image scan method with several orders of magnitude, not only in the measurement but also in the reconstruction phase, when compared to compressive sensing.

The Influence of Radius *r*: As expected, the radius r has a significant role on the quality of the reconstructed image. It corresponds to L in the naïve reconstruction scheme discussed in Fig. 1.4. As seen in Figure 1.10, for a given r the PSNR improves almost linearly up to a value K, then it saturates. In order to avoid saturation r must be smaller. But on the other hand, a small r is in contrast with a small K because the sliding basis cannot provide good reconstruction of all pixels located in the neighborhood of the measurement. The reconstructed image will look noisy (as it happens for $r = 1.5$ in Fig. 1.10) but will improve when $r = 3$. Consequently, it follows that r must adapt to the value of K. Based on experiments we propose the following formula: $r(K) = log_2(N) - log_2(K)$. Such a value may be used as a fixed one in the recovery scheme but an adaptive radius is also possible, for instance by updating the radius value any time$K$ becomes a power of 2. Experiments with both schemes revealed no major quality differences in the reconstructed image, except that the adaptive radius scheme would lead to more computational effort (recalculating the sliding basis each time the radius value changes).

Note that in the above experiments $K = 5000$ is about 7.6% of all pixels in the original image corresponding thus to a compression rate of about 13 times or a rate of 0.61 bpp (bits per pixel) assuming a coding of each pixel with 8 bits. In order to provide a comparison with compressive sensing approaches, the same medical image as in [4] is considered. While our method leads to a slight degradation in the PSNR when compared to the above mentioned compressive sensing method (PSNR = 24,1 dB for ours, instead of 26.5 dB in [4] for K= 10000 measurements) the visual quality of reconstructed images is rather similar (Figure 1.11). Such a slight degradation is acceptable given the important reduction in the overall complexity, particularly the complexity of the sensing device.

Further improvements (i.e. obtaining a better PSNR for a given K) are expected, at the expense of computational complexity, by introducing novel reconstruction sliding bases, tailored to the nature of the images to be compressed. Though, given the very low complexity of the compression stage such a method is suitable for embedding in various smart sensing devices with low power consumption

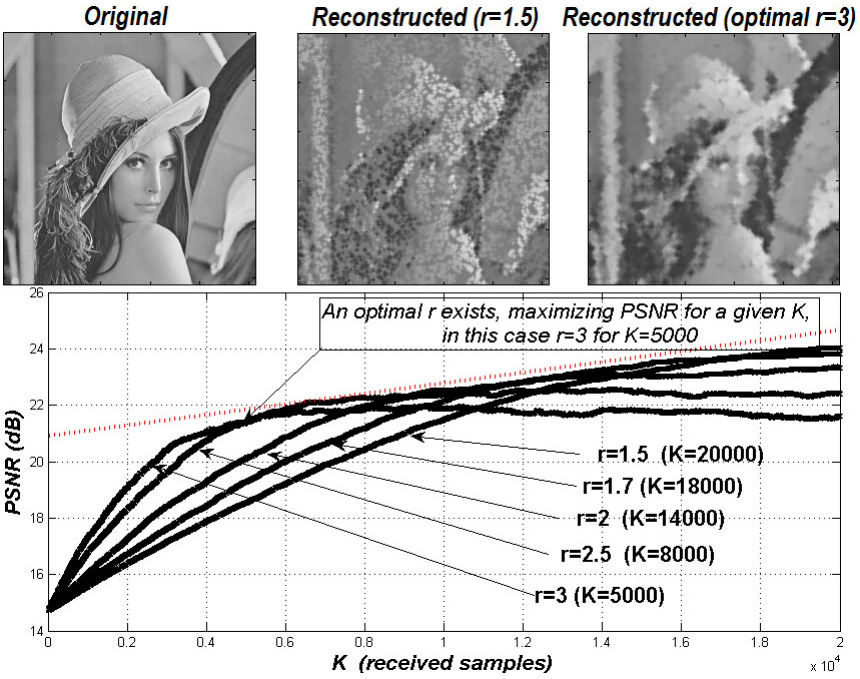


Fig. 1.10 Influence of radius r and K on the quality of the reconstructed image; Here the basis function is of type 3 (piecewise-linear) having the lowest computational complexity

requirements. The method can be applied for video sequences as well, and recent work [29] suggests that it can help reduce the complexity of the motion detection algorithms and may have other benefits given by the simplicity of adjusting the scanning rate depending on the presence or absence of motion (low rate or small K per frame for still or almost still images and higher rates when motion occurs). Furthermore, given the longest cycle property one still has the choice to reconstruct the original image without loss. In all cases (for any degree of compression) the chaotic scan introduces a form of encryption (a key associated with the mask and the ID of the cellular automaton must be known at the Rx unit, in order to properly recover the original message) for free.

1.3 Image Compression Based on Dictionaries Generated by Cellular Automata (CA-VQ)

1.3.1 The General Framework of Dictionary Based Compression

The main idea in this case is to use a properly designed cellular automaton to generate a codebook formed of a number D of *m-sized* binary vectors. The compression

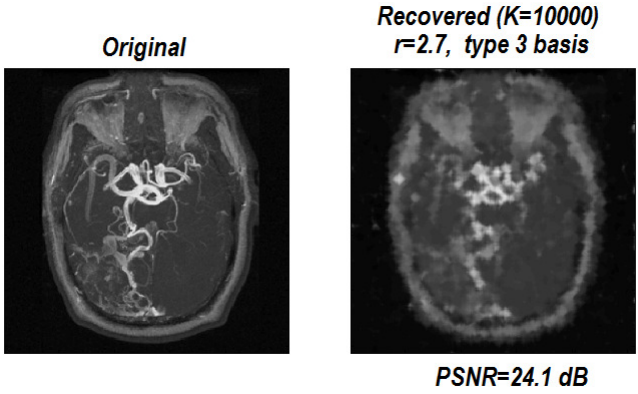


Fig. 1.11 The result of chaotic scan for $K = 10000$ and optimal radius $r = 2.7$

of the original message is done on a certain number b of bit-planes from the original message (e.g. an image).

Within this section we consider the message signal to be identical to the original one i.e. $K = N$. For each block, a search through the codebook reveals the best match (associated to a label encoded with $\log_2 D$ bits) and consequently in the decoding stage (assuming the existence of the same codebook in the Rx unit) the initial block is replaced with one of the D code-vectors in the codebook. Consequently, each bit-plane may be treated independently i.e. algorithm parameters such as the CA rule (ID) generating the codebook, its size D, the block size m, may be optimized such that the bit error (B_{err}) for that specific bit-plane is minimized. In the following $b = 0$ denotes the most significant plane, there is also an option in choosing the exact number b of bit-planes knowing that the most important in term of overall performance (measured as the PSNR of the recovered image with respect to the original one) is bit-plane 0. The compression performance in this case is always expressed in *bits per pixel* (*bpp*). For instance, while using for each $b = 6$ bit-planes $m = 64$ (8×8 blocks), $D = 64$, the rate is computed with the following general formula:

$$bpp = (b \log_2 D)/m \tag{1.4}$$

For the above particular values (quite usual with this compression scheme) a 0.56 bpp rate results. An exemplification of the encoding and decoding stages for the CA-based dictionary VQ system is given in Figures 1.12 and 1.13.

1.3.2 Learning CA-Based Dictionaries and Performance Evaluation of the CA-VQ System

As indicated above, the most important aspect in optimizing the CA-based compression scheme is the choice of the CA structure and its rule, since it generates the dictionary. In previous works [21] [11] a guided search through the space of

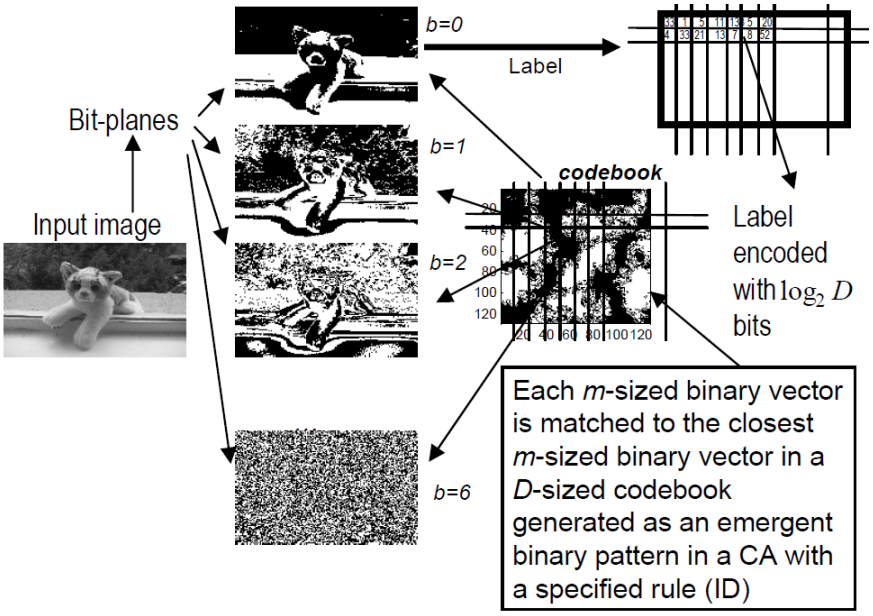


Fig. 1.12 Encoding process using CA-generated dictionaries and binary vector quantization

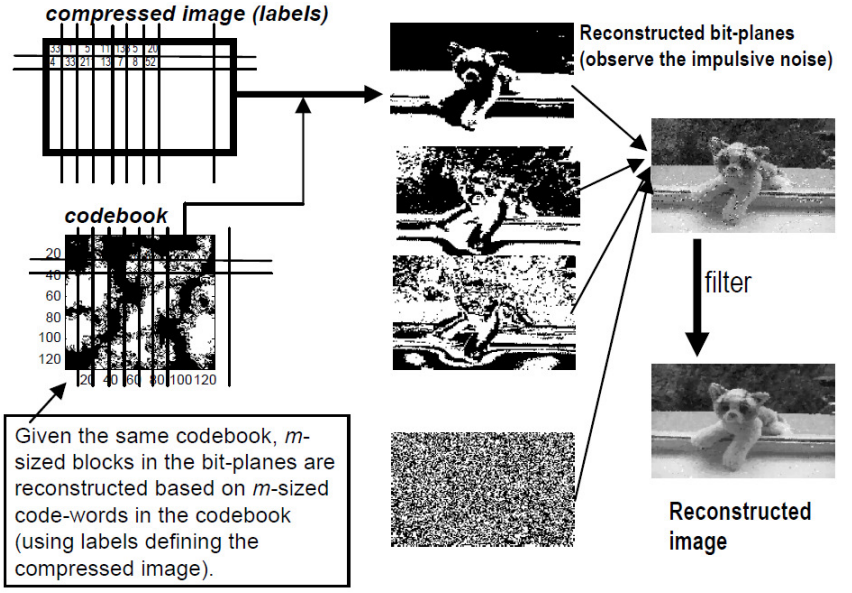


Fig. 1.13 Decoding process using the CA-generated dictionaries

all 1024 outer-totalistic two-dimensional CA with 5-cell (von Neumann) neighborhood was done and several useful ID rules were proposed, maximizing the PSNR for the a given compression rate. In the same work, various block sizes and number of bit-planes were considered, typical values being $D = 32$ or 64 and $m = 64$ to 256 (for very high compression rates). Comparing our compression method to JPEG led to the conclusion that CA-VQ gives better performance than JPEG for high compression rates ($bpp < 0.3$) such compression rates being also associated with faster computation.

Herein we present a different approach for choosing the rule of a CA codebook suitable for compressing a class of images. Unlike in the previous approaches, the local rule is no more restricted to outer-totalistic and although the same von-Neumann neighborhood is used, the space of possible rules is now extremely large (2^{32} rules). Moreover, the CA rule can be individually tuned per each bit-plane.

The CA-based dictionary has the following structure: Given m and D (usually both are powers of 2) the array size M of the 2-dimensional $M \times M$ square CA with periodic boundary condition is determined as $M = \sqrt{Dm}$. For instance, if $D = 64$ and $m = 64$, the size of the dictionary CA is $M = 64$. The codebook is composed of all square D blocks of m size each cropped from the emergent pattern obtained in this CA after a certain number T of iterations when the initial state is an arbitrary (but known at Rx) random binary state with equal number of bits in 0 and 1. The initial state, the rule ID, and the number of iterations T may be regarded as a key of the compression scheme providing a rudimentary form of encryption in addition to the basic compression functionality.

The CA rule is obtained from a simplified learning process using a certain bit-plane image (binary image) of arbitrary size. The original bit-plane image is perturbed using a uniform distribution i.e. a percentage α of its bits are flipped (1 becomes 0 and vice-versa) resulting in an input image for the 5 inputs CA cell (a sliding 5-cell neighborhood is scanning all N pixels of the image as inputs, while the central cell from the associated image is taken as the desired output. For each of the 32 possible 5-bit entries (each encoded as an integer i in the associated truth table) two numbers are stored: n_0^i indicating the number of times the desired output was 0 and n_1^i the number of times the desired output was 1. For N_i occurrences of the input code i it follows that $n_0^i + n_1^i = N_i$. Finally each output for the line i of the truth table is assigned 1 if $n_1^i > N_i/2$ and 0 else. For the rare cases with $N_i = 0$, 1 or 0 is picked randomly with a probability 0.5 as the corresponding output in the truth table. The resulting truth table is then associated with the cell ID, as shown in Fig. 1.14, where $\alpha = 0.12$ was optimized such that the recovery scheme will minimize the bit error on the most significant bit-plane. Given a choice for m and D, the CA codebook is generated by running the CA with the previously determined ID for a certain number of iterations.

The above strategy was motivated by the goal to have a cell ID such that the resulting CA will converge to a stationary pattern preserving most of the relevant details in the original bit-plane. A noisy input was found necessary in order to ensure the diversity of input codes (5-bit words) necessary to construct the associated truth table.

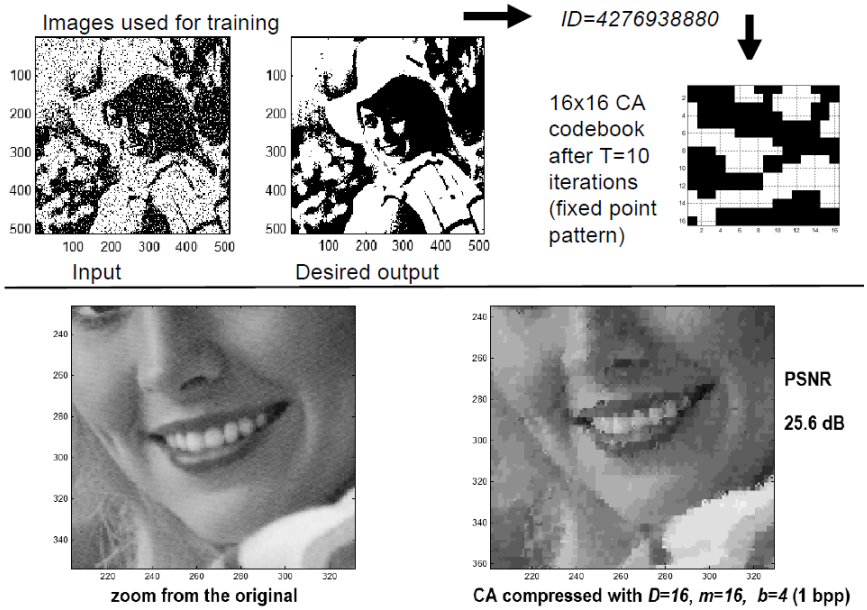


Fig. 1.14 Generating a CA codebook via a learning mechanism and its use in a CA-VQ compression scheme

As seen in Fig. 1.14, for a particular choice of the codebook size $D = 16$ and block size $m = 16$ (4×4 window) the running of the 16×16 CA with ID=4276938880 for $T = 10$ iterations would reveal the codebook. A zoom of both the original image (512×512 pixels) and the recovered one using this codebook for the most significant *4* bit-planes shows the typical losses of this compression scheme. The PSNR is about 25 dB and the quality of the image is acceptable given the rate of 1 bit per pixel. Further optimization of performance is still possible, for instance the use of different, optimized codebooks, for each bit-plane, or a different learning scheme.

In terms of computational complexity, the CA-VQ approach requires more computational effort in the encoding stage and less effort in the decoding stage. It is an opposite situation from the case of chaotic scan compression. For a message with N bits and a particular bit rate (codebook size D and block size m) assuming that the codebook is calculated and stored, the compression process requires comparisons (between blocks and code-words in the codebook) being more effective for large block sizes and small dictionaries (also ensuring highest compression but lowest reconstruction quality). The above compares not so favorably to simply reading $K < N$ samples in the chaotic scan method. But on the other hand, the complexity of the CA-VQ compression process still remains linear in the number of pixels, being much lower than for traditional compression methods. The decompression stage in CA-VQ is m times faster (i.e. N/m operations of reading the codebook) and involves

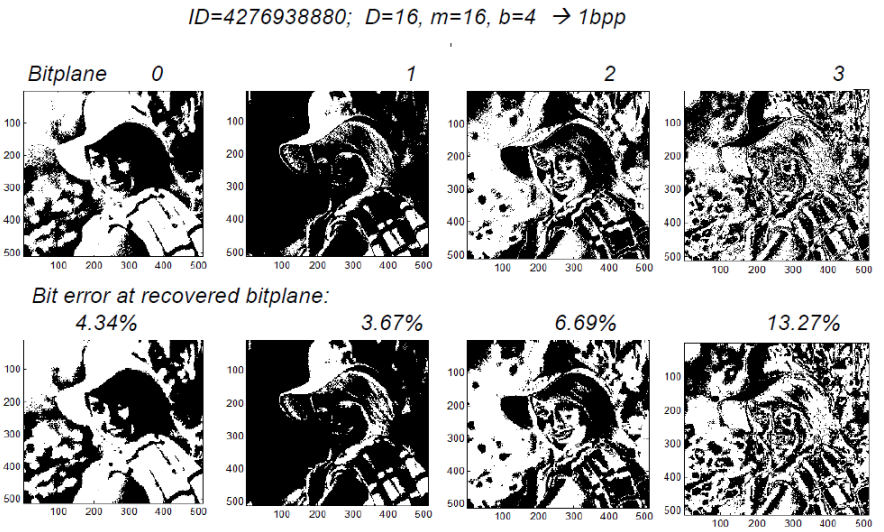


Fig. 1.15 Distribution of bit errors on each of the 4 bit-planes used in the CA-VQ compression scheme for the particular example shown in Fig. 1.14

no comparison. This situation compares favorably to the relatively large number of operations (multiplications) required by the basis-based recovery scheme of the chaotic scan method.

1.4 Hardware Description and Synthesis of CA Using Algebraic Normal Form

In the above, two different image (or video) compression approaches were proposed, both based on Cellular Automata (CA). Since CA is an important part of both algorithms, it is important to find convenient and efficient ways to implement them in various technologies. While usual PCs or microcontrollers may be used to implement the cellular automata, maximal speed and efficiency is achieved when CA models are implemented in a fully parallel fashion. Particularly, when the aim is of implementing the above algorithms as part of a low-power smart sensing device it is of interest to provide CA code for hardware description languages.

In [9] we provide a convenient methodology to generate VHDL representations using the Algebraic Normal Form conversion discussed previously. The result would be a fully integrated sensor system with encryption, compression and other capabilities, as discussed in [19]. Such a sensor would perform compressed sensing using a different, computationally more efficient approach.

A detailed description of our FPGA implementations is also given in [10]. Specific to all of them is the development of a software module, called next CA-description module and written in C++ to automatically generate the VHDL code

used for the usual synthesis steps. The same code may be used for FPGA designs as well as for dedicated VLSI chips. The CA-description module inputs a description of the CA (neighborhood size, rule, mask, number of cells, etc.) in a user-friendly manner and generates VHDL modules to be used in various hardware implementations of the above mentioned algorithms. The key issue in generating the most important part of the VHDL code is the very good correspondence between the ANF description and the possibility to express it in a few VHDL line codes. A particular example is give next: A one-dimensional CA is defined as having 7 cells, a certain mask vector (1100010) and ID=101. The resulting VHDL line describing the entire HCA is:

REG<= "1111111" xor c xor a xor (b and a) xor mask;

The above corresponds to the following particular form of the ANF representation:

$$y = 1 \oplus u_3 \oplus u_1 \oplus u_2 u_1 \tag{1.5}$$

The above variable REG represents the entire CA array and the variables *a,b,c* are constructed to represent shifted versions of REG according to specific neighborhood to be implemented.

So far the CA-description module can implement either 3-cell neighborhoods or 5-cell neighborhoods for the HCA model (the traditional, homogeneous CA model is a particular case of HCA with all 0 elements in the mask vector). Various FPGA target devices were considered and in all cases the resulted implementation was found to be very efficient. For instance, in the case of Xilinx FPGA's 1 LUT was assigned for HCA designs with up to 3 inputs and mask vector while 2 LUTs suffice to implement an entire cell (including its local memory) in the maximal case considered so far of 5 cell neighborhoods. Similarly, for FPGA devices from Altera (Cyclone II EP2C35F672C6 device on the DE2 board provided by the University Program) one basic computational unit (LE – logic element) is assigned for 3-inputs HCA cells and 2 times more for the case of 5-inputs. Note the very efficient allocation of one cell per FPGA logic register. The above results confirm that cellular automata with very large number of cells ($n = 33216$ in the case of the chip on the DE2 board) can be easily realized in low cost series FPGA. The same VHDL description may be used to generate part of specialized sensor chips (e.g. in addition to low power image sensors e.g. [22]) using an ASIC design flow.

References

1. Baraniuk, R., Cevher, V., Duarte, M., Hedge, C.: Model-based compressive sensing. IEEE Trans. Inform. Theory 56, 1982–2001 (2010)
2. Baron, D., Sarvotham, S., Baraniuk, R.G.: Bayesian compressive sensing via belief propagation. IEEE Trans. Signal Process. 58, 269–280 (2010)
3. Bruckstein, A.M., Holt, R.J., Netravali, A.N.: Holographic representation of images. IEEE Trans. Image Processing 7, 1583–1597 (1998)
4. Candes, E., Romberg, J.: Practical signal recovery from random projections. In: Proc. SPIE Conf., Wavelet Applications in Signal and Image Processing XI (2005)

5. Capellari, L., Milani, S., Cruz-Reyes, C., Calvagno, G.: Resolution scalable image coding with reversible cellular automata. IEEE Trans. on Image Processing 20(5), 1461–1468 (2011)
6. Chen, R.J., Lai, J.L.: VLSI implementation of the universal 2-D CAT/ICAT systems. In: Proceedings of the 11th IEEE International Conference on Electronics, Circuits and Systems (ICECS), pp. 187–190 (2004)
7. Chua, L.O.: A nonlinear dynamics perspective of Wolfram's New Kind of Science (vol. I-IV). World Scientific Series on Nonlinear Science, Series A, vol. 57,68,76. World Scientific Publishing Company (2011)
8. Das, D.: A survey on cellular automata and its applications. In: Krishna, P.V., Babu, M.R., Ariwa, E. (eds.) ObCom 2011, Part I. CCIS, vol. 269, pp. 753–762. Springer, Heidelberg (2012)
9. Dogaru, I., Dogaru, R.: Algebraic normal form for rapid prototyping of elementary hybrid cellular automata in FPGA. In: Proceedings of the ISEEE Conference, Galati, Romania (2010)
10. Dogaru, I., Dogaru, R., Damian, C.: FPGA implementation of chaotic cellular automaton with binary synchronization property. In: Proceedings of 8th International Conference on Communications (COMM), Bucharest, Romania (2010)
11. Dogaru, R.: CA-VQ: A simple compression scheme using codebooks generated by cellular automata. In: Proceedings NSIP 2007 (International Workshop on Nonlinear Signal and Image Processing), Bucharest, Romania (2007)
12. Dogaru, R.: Systematic design for emergence in cellular nonlinear networks with applications in natural computing and signal processing. SCI, vol. 95. Springer, Heidelberg (2008)
13. Dogaru, R.: A fast method for classification of emergent dynamics in cellular automata based on uncertainty profiles. Journal of Control Engineering and Applied Informatics 11, 18–25 (2009)
14. Dogaru, R.: Hybrid cellular automata as pseudo-random number generators with binary synchronization property. In: Proceedings of the International Symposium on Signals Circuits and Systems (ISSCS 2009), Iasi, Romania (2009)
15. Dogaru, R.: HCA101: A chaotic map based on cellular automata with binary synchronization properties. In: Proceedings of the 8th Int. Conference on Communications (COMM 2010), Bucharest, Romania (2010)
16. Dogaru, R.: A low complexity image sensing method using pseudo-random scan and recursive reconstruction with radial basis functions. In: Proceedings of the ISEEE Conference, Galati, Romania (2013)
17. Dogaru, R., Dogaru, I.: Uncertainty profiles for predicting complex nonlinear dynamics in cellular automata: the case of five cells neighborhood. In: Proceedings of the International Symposium on Nonlinear Theory and its Applications (NOLTA 2010), Krakow, Poland, September 5-8 (2010)
18. Dogaru, R., Dogaru, I., Kim, H.: Binary chaos synchronization in elementary cellular automata. Int. J. Bifurcation Chaos 19, 2871–2884 (2009)
19. Dogaru, R., Dogaru, I., Kim, H.: Chaotic scan: A low complexity video transmission system for efficiently sending relevant image features. IEEE Trans. on Circuits and Systems for Video Technology 20, 317–321 (2010)
20. Dogaru, R., Murgan, A.T., Ortmann, S., Glesner, M.: A modified RBF neural network for efficient current-mode VLSI implementation. In: Proceedings Micro-Neuro 1996, February 12-14, pp. 265–270. IEEE Press, Laussane (1996)

21. Dogaru, R., Tetzlaff, R., Glesner, M.: Semi-totalistic CNN genes for compact image compression. In: The IEEE Proceedings of 10th International Workshop on Cellular Neural Networks and Their Applications, Istanbul, Turkey (2006)
22. Fernandez-Berni, J., Carmona-Galan, R., Carranza-González, L.: FLIP-Q: A QCIF resolution focal-plane array for low-power image processing. IEEE Journal of Solid-State Circuits 46, 669–680 (2011)
23. Hinton, G.E., Salakhutdinov, R.: Reducing the dimensionality of data with neural networks. Science 313, 504–507 (2007)
24. Kolumban, G., Kennedy, M.P., Chua, L.O.: The role of synchronization in digital communication using chaos-part ii: Chaotic modulation and chaotic synchronization. IEEE Trans. Circuits and Syst. I 45, 1129–1140 (1998)
25. Lafe, O.: Data compression and encryption using cellular automata transforms. Engng. Applic. Artif. Intell. 10, 581–591 (1997)
26. Lee, J., Verleysen, M.: Nonlinear dimensionality reduction, 3rd edn. Springer (2007)
27. López-Mancilla, D., Cruz-Hernández, C.: Output synchronization of chaotic systems: Model-matching approach with application to secure communication. Nonlinear Dynamics and Systems Theory 5, 141–156 (2005)
28. van der Maaten, L., Postma, E., van den Herik, J.: Dimensionality reduction: a comparative review. Tilburg University Technical Report TiCC-TR 2009-005 (2009)
29. Mitrea, C., Ionescu, B., Dogaru, R.: A pseudo-random scan perspective to the motion detection paradigm. In: Proceedings of the ISEEE Conference, Galati, Romania (2013)
30. Ronjom, S., Abdelraheem, M., Danielsen, L.E.: TT and ANF representations of boolean functions. Online database of Boolean functions (2013), `http://www.selmer.uib.no/odbf/help/ttanf.pdf` (last accessed September 28, 2013)
31. Wolfram, S.: Random sequence generators. US Patent 4691291A (1987)
32. Zipf, P., Hinkelmann, H., Shao, H., Dogaru, R., Glesner, M.: An area-efficient FPGA realisation of a codebook-based image compression method. In: Proceedings of FPT 2008, pp. 349–352 (2008)

Chapter 2
Cellular Automata for Image Resizing

Konstantinos Ioannidis, Georgios Ch. Sirakoulis, and Ioannis Andreadis

Abstract. During the last years, several methods have been applied to tackle the image resizing problem. Most of these methods are derived from image interpolation techniques for image enlargement. Among them, the edge-directed interpolation methods succeed to preserve the edges of the low resolution image and produce crisper results compared to the space invariant models. In this chapter, we present an edge-directed method which exploits the simplicity and the inherent parallelism of the Cellular Automata (CA) computational tool to generate high resolution images from low resolution acquired images. This task is accomplished with the help of the Canny Edge Detector so as to discriminate the edge regions from the homogenous ones. Moreover, appropriate CA states and transition rules were designed to evolve the CA, which, eventually, attempt to enhance the quality of the edge areas. The orientation of the edge cells are considered in order to preserve effectively the edges of the initial image. The presented experimental results in terms of PSNR values and processing time demonstrate the effectiveness of the proposed method when compared to well-known methods as well as its suitability, especially for systems with low requirements specifications when further image processing is required.

2.1 Introduction

Today, digital images and video sequences are usually stored and transmitted in compressed form using different standards resulting in huge amounts of image data found in every place of modern cyberlife. To tackle this situation, image resizing methods able to generate high resolution images from their low resolution versions are applied. Nevertheless, image enlargement is a non-trivial process that involves a trade-off between efficiency, smoothness and sharpness. As a result, efficient

Konstantinos Ioannidis · Georgios Ch. Sirakoulis · Ioannis Andreadis
Laboratory of Electronics, Department of Electrical and Computer Engineering,
Democritus University of Thrace, Xanthi, Greece
e-mail: {kioannid,gsirak,iandread}@ee.duth.gr

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_2, © Springer International Publishing Switzerland 2014

manipulation of these image data types in systems with low technical specifications is a significant issue in their overall performance. Image interpolation techniques focus on expanding images originally acquired with low resolution cameras or sensors ($M \times N$ pixels) into up-sampled images ($M' \times N'$ pixels) and are found to be the most commonly adopted methods for image enlargement. Image interpolation works using known data to estimate values at unknown points, i.e. it works in two directions, trying to achieve the best approximation of a pixel's color or intensity based on the values of surrounding pixels.

A usual classification of image interpolation techniques includes adaptive and non-adaptive algorithms. Non-adaptive algorithms usually apply to all image pixels with the same manner while adaptive techniques take into account what they are interpolating (sharp edges vs. smooth texture) and change accordingly the treatment of the corresponding pixels. The latter algorithms are primarily designed to maximize artifact-free detail in enlarged photos, so some cannot be used to distort or rotate an image. Several commonly non-adaptive used interpolation methods have been suggested for image resizing, such as the nearest neighbor interpolation [13], bilinear interpolation [13], bicubic interpolation [18] and spline interpolation [11]. Linear approaches are the most frequently applied for the resizing process due to their low computational burden. However, those methods produce image artifacts like blurring on edges since no information related to abrupt changes of light intensity is considered, usually failing to preserve the quality of the edges and consequently produce resized images with blurred edges or annoying zigzag artifacts. On the contrary, nonlinear methods produce better results. Nevertheless, they have a larger computational burden and involve blurring, as well. Various generic approaches have been proposed to improve the subjective quality of the interpolated images and overcome such deficiencies. In addition, the method in [14] is based on variation models with smoothing and orientation constraints. The nonlinear Partial Differential Equation (PDE) problem is simplified into a series of problems with explicit solutions. Furthermore, the area based interpolation scheme in [1] computes each interpolated pixel by proportional area coverage of a window filter which is applied to the input image. A quadratic image interpolation method [23] has been proposed with adequate visual results but its computational cost remains high. Finally, a method to estimate the model parameters piecewisely is proposed in [36] using an autoregressive image model. The method utilizes the covariance matrix of the high resolution image itself, with missing pixels properly initialized. Nevertheless, conventional linear interpolation schemes based on space-invariant models also fail to preserve the quality of edges and, consequently, result to images with blurred edges and artifacts.

An alternative type of approaches has been introduced, namely edge-directed interpolation methods, in order to preserve the edges of the low resolution image and produce more crisp results. Edge-directed interpolation methods apply a variety of operators according to the edge directions [6]. A fuzzy interpolation approach is proposed in [7] for two dimensional signal resampling however additional processing for edge identification is required. In addition, a neural network approach has been proposed in [10] to approximate the computational rules of interpolation algorithms

for learning statistical inter-pixel correlation of interpolated images. The method in [21] comprises a hybrid artificial intelligence system. A fuzzy decision system is proposed to classify all the pixels of the input image into human perception non-sensitive class and sensitive class. The bilinear interpolation is applied to the non-sensitive regions while a neural network is used to interpolate the sensitive regions along the edges. Furthermore, the method proposed in [20] initially estimates local covariance coefficients from a low resolution image. These covariance estimates are used to adapt the interpolation at a higher resolution based on the geometric duality between the low resolution and the high-resolution covariances. Despite the visually accurate resulted images, the above edge directed approaches display high levels of computational cost and thus, their application in real-time systems is restricted. In order to achieve frame rates close to real time limits while enhancing the quality of the edges, an edge-oriented method is proposed in [8]. The main idea is to seperate the image homogenous areas and edges areas the latter processed using different interpolation methods. The method achieves real-time image enlargement nevertheless, the classification of the areas depends on a predefined threshold as well as two stages of process are required. Finally, Shi *et al.* in [33] initially expand the low resolution image using a bilinear interpolation method and a Canny edge detector is applied to identify the edges of the up-scaled image. The final light intensity values are calculated by applying some refinement functions. Despite the satisfactory visual results and the high frame rates, the inaccuracies introduced by the initial bilinear enlargement lead to blurred edges and thus, the Canny edge detector [4] is unable to detect significant edges.

On the other hand, non conventional computational techniques like CA have been applied to the field of image processing with great success. More specifically, CA have been successfully applied to image processing due to their discrete, fully parallel with local interconnections nature. Taking into consideration that most of the common image processing methods are characterized by high complexity and their high requirements for memory and computational resources, the usage of CA in image processing has intrigued the scientists a long time ago. More specifically, Preston and Duff [28] demonstrated how major image processing tasks, such as image segmentation, skeletonization, and filtering may be approached using the CA methodology. Furthermore, specific applications of these image processing tasks in both science and biomedicine are also presented. Lafe has also proposed CA methods, by which information building blocks, called basis functions (or bases), can be generated from the evolving states, and called it Cellular Automata Transforms (CAT) applying them to image and video compression in [19]. In recent years, it has also been shown by several researchers [2, 12, 26, 30–32] that CA can be used to perform some standard image processing tasks to a high level of performance, as well as in up-to-date computer vision fields, such as stereo vision [9, 22, 24]. For example, Rosin in [30] proposed the training of CA to perform several image processing tasks, namely noise filtering (also applied to grayscale images using threshold decomposition), thinning, and convex hulls, while the same author proposed in [31] the application of CA in intensity images instead of binary ones, able to perform many different image processing tasks, and that the quality of these

results is in many cases comparable or better than established specialised algorithms. On the other hand, CA have been extensively used as a VLSI architecture [34]. In contrast to the serial computers, the implementation of the model is motivated by parallelism, an inherent feature of CA that contributes to further acceleration of the model operation. The CA approach is consistent with the modern notion of unified space-time. In computer science, space and time correspond to memory and processing unit, respectively. In CA, memory (CA cell state) and processing unit (CA local rule) are inseparably related to a CA cell [34]. In terms of circuit design and layout, ease of mask generation, silicon-area utilization, and maximization of clock speed, CA are perhaps one of the most suitable computational structures for hardware realization. To combine the inherent advantages of both CA models and their implementation on hardware, when applied to image processing, some researchers such as Andreadis *et al.* [2] proposed a Application Specific Integrated Circuit (ASIC), which performs the conversion, in real time, of the R, G and B colour co-ordinates to the CIE standard L*, a* and b* colour coordinates to be used in colorimetry instrumentation for colour measurement and control and in colour machine vision in autonomous applications such as robotics and military systems, where the need for short processing times is crucial. In [3] a hardware CA module for detecting circular objects is introduced, where targeted applications include inspection tasks (accept/reject operations) of circular objects, such as tablets in the pharmaceutical industry, and detection of uncoated areas, foreign objects and level of bake in the confectionery and food industry. Moreover, Karafyllidis *et al.* [16] calculate the mean velocity of a moving object, using the CA properties, along the centra axis perpendicular to the lens of the vision system where the motion of the object is restricted to translation (angular velocity is zero) and to one moving object in the scene. It should be also mentioned that cellular logic image processors, like Clip have been developed in the past and cellular neural networks, an extension of CA that includes weight matrices, chips with both continuous time and discrete time versions have been applied to a variety of image processing tasks [30]. In this aspect novel nanoelectronic structures like Quantum Cellular Automata (QCA) have been also applied for image processing mathematical morphology operations [5, 25] chapter 4. Finally, Porter *et al.* [27] proposed a CA reconfigurable framework which includes a highly pipelined architecture for multi-scale cellular image processing as well as support for several different pattern recognition applications, while Katis and Sirakoulis [17] designed specialized FPGAs that achieve automated image processing such as noise filtering, edge thinning and convex hull detection with the help of corresponding CA algorithms.

Although it is clear that there many applications of CA to different image processing tasks, and that while some such as image denoising are quite well known, the application of CA to image resizing has not been explored in detail. In this chapter, we present an edge-directed method which exploits the simplicity and the inherit parallelism of the CA. The edges of a low resolution image are initially determined by applying the Canny edge detector leading to a bit-wise edge map. This map is then considered as a CA grid along with a CA state which corresponds to the undefined pixel. The CA evolves its state by applying the appropriate CA rules

which were constructed based on the orientation of the edges. Finally, a simple remapping of each cell state to light intensity values is applied which is based on the weighted sum of the neighboring light intensity values of every pixel. The method manages to preserve the initial edges adequately while achieving high frame rates for both color and grayscale images. In order to evaluate the performance of the proposed method, a quantitative comparison with other related methods was applied proving its effectiveness. More specifically, the presented experimental results in terms of PSNR values and processing time prove the effectiveness of the proposed method when compared with well-known methods as well as its suitability, especially for systems with low technical specifications when further image processing is required.

The rest of the chapter is organized as follows. Section 2.2 provides the basic fundamentals of the CA as used in this chapter and the Canny edge detector principles. In Sects. 2.3 and 2.4, an analysis of the proposed CA method and the experimental results are presented, respectively. More specifically, experiments show that in both visual comparisons and quantitative analysis, the results extracted by the proposed CA based algorithm are better than those extracted from zero-ground, bilinear, bicubic interpolation, near edge orientation. Finally, the hardware implementation of the presented CA method for image resizing and conclusions are drawn in Sects. 2.5 and 2.6, respectively.

2.2 Cellular Automata Fundamentals and Canny Edge Detector

2.2.1 CA Fundamentals

CA are decentralized, discrete space-time systems where interactions are local and can be used to model physical systems [35]. A finite automaton could be defined by the quadruple:

$$\{d, q, N, F\} \tag{2.1}$$

where d denotes the CA dimensions, q the total number of the used cell states, N the applied neighborhood and f the transition rules.

At each time step, every cell update its state based on its current state and the state of its adjacent cells according to the defined transition set rule f. It must be highlighted that all cells of the CA grid update their state simultaneously leading to a completely parallel system. The neighborhood of each cell is defined by variable N. For a two-dimensional CA, two neighborhoods of range are often considered: Von Neumann and Moore neighborhood. The Von Neumann neighborhood is a diamond shaped neighborhood and can be used to define a set of cells surrounding a given cell (x_0, y_0). Equation 2.2 defines the Von Neumann neighborhood of range r.

$$N_(x_0, y_0)^N = \{(x, y) : |x - x_0| + |y - y_0| \leq (r)\} \tag{2.2}$$

For a given cell (x_0, y_0) and range r, the Moore neighborhood can be defined by the following equation:

$$N_(x_0, y_0)^M = \{(x,y) : |x - x_0| \leq (r), |y - y_0| \leq (r)\} \tag{2.3}$$

In most practical applications, when simulating a CA rule, it is impossible to deal with an infinite lattice. The system must be finite and have boundaries. Clearly, a site belonging to the lattice boundary does not have the same neighborhood as other internal sites. In order to define the behavior of these sites, the neighborhood is extending for the sites at the boundary. Extending the neighborhood leads to various types of boundary conditions such as periodic (or cyclic), fixed, adiabatic or reflection.

The transition rule f determines the way in which each cell of the automaton is updated. The state of each cell is affected by the cell values in its neighborhood and its value on the previous time step, according to the transition rule or a set of rules. The state of every cell in the CA is updated simultaneously by applying the transition rule f, thus, providing an inherent parallel system.

2.2.2 Canny Edge Detector

The Canny edge detector [4] is an edge detection operator that uses a multi-stage algorithm to detect a wide range of edges in images. The Canny edge detector is actually an optimal technique of edge detection and creation and its application relies on the following criteria: correct detection, accurate localization and minimal response. To satisfy these requirements, a technique which finds the function which optimizes a given functional was used, namely the calculus of variations. The method produces binary edge maps by applying sequentially the following processing stages:

- Stage 1: Filtering the image. The image is initially filtered by a 2-D Gaussian filter of zero mean value and a predefined standard deviation σ in order to eliminate possible noise. The result is a slightly blurred version of the original image.
- Stage 2: Defining the intensity gradient of the filtered image. At this stage, elementary edge detection operators, like Sobel, are used in order to define the first derivative both in the horizontal and the vertical direction. Thus, the gradient and direction of each pixel are defined by the following equations:

$$E_g = \sqrt{I_{Gx}^2 + I_{Gy}^2},\ E_d = \arctan\left(\frac{I_{Gy}}{I_{Gx}}\right) \tag{2.4}$$

- Stage 3: Non maximum suppression. Given estimates of the image gradients, a search is applied to determine if the gradient magnitude assumes a local maximum in the gradient direction. A pixel is defined as an edge pixel if its direction is larger than the average direction of its area.
- Stage 4: Hysteresis thresholding. The last stage intends to further reduce the number of edge pixels that resulted during the above stages. For this purpose, two thresholds are used. The process starts by applying a high threshold and using the

directional information; thus, edges can be traced in the image. While an edge is traced, the lower threshold is applied in order to trace faint sections of edges. The most frequent value for the high threshold is considered to be related to the highest value of the gradient magnitude of the image while the low threshold is usually equal to $0.4 \times (high\ threshold)$.

Once the process is completed, a binary image is extracted where each pixel is marked as either an edge pixel or a non-edge pixel. Essentially, a new image with the same dimensions is produced representing the edges of the initial image.

2.3 Proposed Method

The proposed method aims at calculating the unknown light intensity values of pixels which are produced in the resizing process. The basic concept of the method is to initially classify the pixels of the initial image into two categories: homogenous areas and edge areas. The method exploits the capability of the Canny edge detector to accurately determine the edges of the image. Since the bitwise edge map is resulted, the logical array is enlarged and is considered as a CA lattice. The unknown cells update their state according to the proposed transition rules. Finally, a simple transformation is applied to calculate the unknown light intensity values based on the state of each CA cell. For color images, the above procedure is applied to each of the RGB vector components separately. The overall process is illustrated in Fig. 2.1.

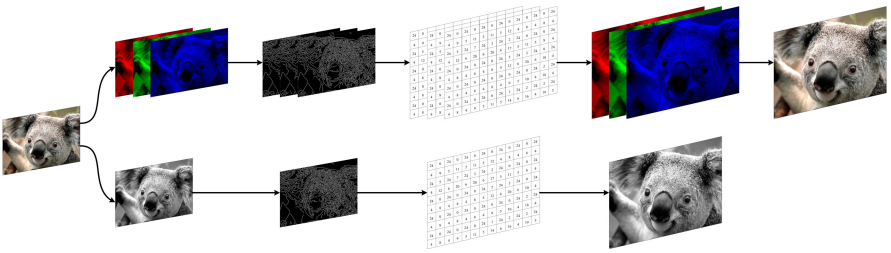


Fig. 2.1 Proposed CA based image resizing method

2.3.1 Edge Detection

The first stage of the method includes the application of an edge detection technique in order to discriminate the homogeneous areas from the edge areas. Thus, any method reported in the literature could be applied. However, most of the methods which produce more accurate edges have high computational complexity. Therefore,

their application in real-time systems is not prohibitive. In order to produce accurate edge maps with high frame rates, the Canny edge detector was incorporated. As mentioned in Subsect. 2.2.2 the proposed edge detector algorithm is considered as one of the optimal edge detectors found in literature. Taking into account the fact that the edges occurring in images should not be missed while there should be no responses for non-edges the Canny detector succeeds to achieve low error rates. Moreover, it attempts to keep the distance between the selected edge pixels and the actual edges as closer as possible to minimum. Finally, it seeks for only one response per edge thus eliminating any multiple responses to an edge. In brief, the most significant criteria of this selection are as found below:

- Correct detection: Edges are detected with high probability when these exist in the original images.
- Accurate localization: Marked edges are accurately close to the edges in the original images.
- Minimal response: A defined edge is detected only once, and where possible, noise should not create false edges.

2.3.2 CA Resizing

Motivated by the binary nature of the calculated edge maps, a CA is proposed to define the state of the additional cells that resulted after the resizing process and eventually the pixels' values. Without the loss of generality, it is assumed that the high resolution edge map $Y_{i,j}$ of size $2M \times 2N$ directly comes from of size $M \times N$. Thus, it yields $Y_{2i,2j} = X_{i,j}$. Figure 2.2 provides a schematic illustration of the resulted enlarged edge map.

The enlarged edge map is considered to be a 2-D lattice of cells where every binary pixel is represented by a cell. Thus, the proposed CA grid has the same dimensions with the enlarged image. Moreover, in order to update the state of each cell, Moore neighborhood is adopted; therefore, Equation 2.3 is used with a radius r equal to 1. In addition, the set of states must be defined. Since the enlarged edge map includes non-edge cells (stated as "0"), edge cells (stated as "1") and undefined cells, we assume that the undefined cells are marked with the state "2". In conclusion, the cells of the CA before its evolution can be marked with three states: "0" (non-edge cell), "1" (edge cell) and "2" (undefined cell). Taking advantage of the CA flexibility, the transition rules as well as the states of the cells after the evolution are created in order to preserve the edges. The basic motive is to create states that eventually will produce more crisp transitions of light intensities from non-edge pixels to edge pixels during the remapping process. By using such states, the orientation of each edge is considered.

For example, based on the lattice of Fig. 2.2, let us assume that the cell $Y_{2i,2j}$ is marked as a non-edge cell while the cell $Y_{2i,2j+2}$ is an edge cell. The intermediate

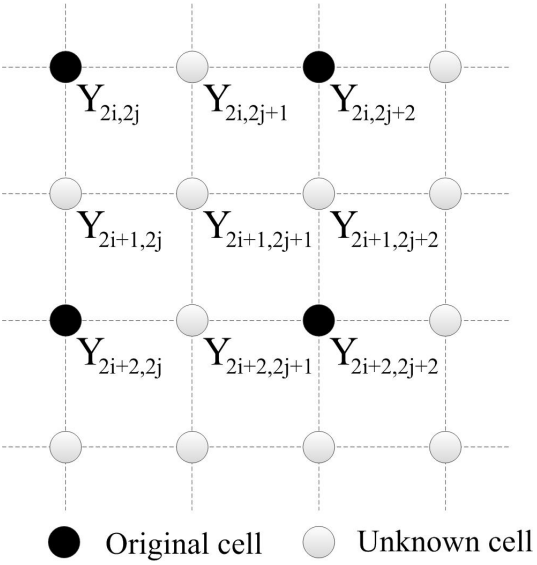


Fig. 2.2 Edge map after the enlargement

cell $Y_{2i,2j+1}$ must be evolved to a cell that will produce a light intensity which is more closer to the value of the edge pixel and less close of the non-edge pixel. In addition, if no cell in its immediate neighbor is marked as a non-edge cell, its next state must correspond to a value which leads to a homogenous area.

It must be mentioned that all non-edge and edge cells of the enlarged map are surrounded by eight (8) unknown cells based on the aforementioned selected Moore neighborhood with $r = 1$ and after the evolution; they are marked with states which leads to equal values with the corresponding pixels of the low resolution image. In addition, every cell stated as "2" appears either two known cells (horizontal or vertical cells) or four known cells (central cell) in its Moore neighborhood which will eventually designate the next state. The total number of the used states and the constructed transition rules are twenty four (24) therefore, each CA cell is marked with one discrete number in the range of $[0, 23]$ after the evolution of the CA. Finally, null boundary conditions are used in order to evolve the state of the boundary cells. Figure 2.3 presents simple cases of the above rationality.

2.3.3 Remapping Process

At this stage of process, every pixel value of the resized image is defined based on the state of the corresponding cell. Let us consider that $f_{i,j}$ and $F_{2i,2j}$ correspond to

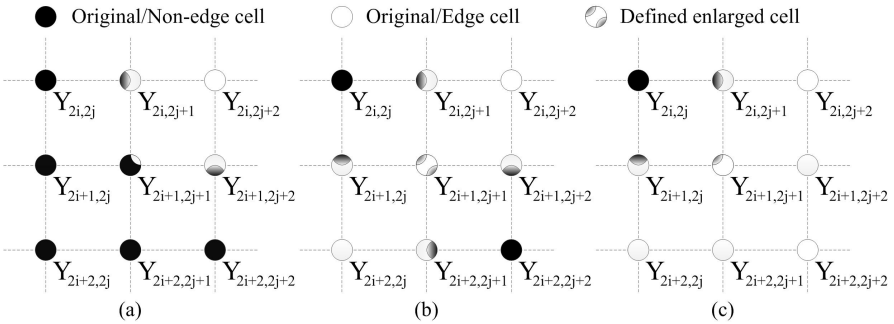


Fig. 2.3 Some examples of the applied CA transition rules

the low resolution image and the high resolution image, respectively. At the previous stage, cells indexed by $(2i, 2j)$ were marked with states that simply apply the following: $F_{2i,2j} = f_{i,j}$. On the contrary, in order to keep the computational cost low, pixels with the indices $(2i, 2j+1)$, $(2i+1, 2j)$ and $(2i+1, 2j+1)$ are expressed as a weighted summation of their adjacent pixel values of the low resolution image. Thus, for the remapping process of these pixels, the following expressions are introduced:

$$F_{2i,2j+1} = a_{11} \times f_{i,j} + a_{12} \times f_{i,j} \tag{2.5}$$

$$F_{2i+1,2j} = b_{11} \times f_{i,j} + b_{12} \times f_{i,j} \tag{2.6}$$

$$F_{2i+1,2j+1} = c_{11} \times f_{i,j} + c_{12} \times f_{i,j+1} + c_{21} \times f_{i+1,j} + c_{22} \times f_{i+1,j+1} \tag{2.7}$$

Each of the above weights is defined based on the state of the corresponding cell of the CA grid. Also, the sum of each of the factors a, b and c must be equal to one. For example, assuming that cell $(2i, 2j)$ is defined as a non-edge cell and cell $(2i, 2j+2)$ as an edge cell (Fig. 2.3(a)), weight a_{12} must be greater than a_{11} to produce a more crisp transition between the non-edge and the edge pixel. In addition, if both pixels are denoted as non-edge or edge pixels, the weights are equal in order to produce an expanded homogenous or edged area. Based on the case that Fig. 2.3(c) presents, the cell $Y_{2i,2j+1}$ is surrounded by the non-edge cell $Y_{2i,2j}$ and the edge cell $Y_{2i,2j+2}$. Thus, the following values are assigned to the $a_{i,j}$ weights: $a_{11} = 0.25$ and a_{12}=0.75. On the contrary, the cell $Y_{2i+1,2j+2}$ is surrounded by edge cells, and thus, for the corresponding pixel, the following weights are used: $b_{11} = b_{21} = 0.5$.

2.4 Experimental Results

To assess the performance of the tested resizing methods including the proposed method, several tested were performed. Zero-order, bilinear, bicubic [13], the New

Edge-Directed Interpolation (NEDI) [20], the edge-oriented [8] and the proposed method were tested on both color and grayscale images of various resolutions. The key idea of bilinear interpolation is to perform linear interpolation first in one direction, and then again in the other direction. The whole interpolation, although each step is linear in the sampled values and in the position, is considered not linear but rather quadratic in the sample location. In general, bilinear interpolation can be used where perfect image transformation with pixel matching is impossible, so that one can calculate and assign appropriate values to pixels with less computational burden. The nearest neighborhood interpolation selects the value of the nearest point and does not consider the values of neighboring points at all, yielding a piecewise-constant interpolant. In bicubic interpolation method [13], interpolated surface is smoother than corresponding surfaces obtained by bilinear interpolation or nearest neighbor interpolation and usually applies when speed is not an issue. In contrast to bilinear interpolation, which only takes 4 pixels (2×2) into account, bicubic interpolation considers 16 pixels (4×4) resulted to smoother resampled images with fewer interpolation artifacts with the handicap of less computational speed compared with bilinear interpolation execution. In NEDI interpolation method, the basic idea is first to estimate local covariance coefficients from a low resolution image and then use these covariance estimates to adapt the interpolation at a higher resolution based on the geometric duality between the low resolution covariance and the high-resolution covariance [20]. The edge-directed property of covariance-based adaptation attributes to its capability of tuning the interpolation coefficients to match an arbitrarily oriented step edge. A hybrid approach of switching between bilinear interpolation and covariance-based adaptive interpolation is proposed to reduce the overall computational complexity. Finally, the basic idea of the edge-oriented algorithm is to partition digital images into homogeneous and edge areas based on the analysis of the local structure on the images. In addition, in order to have better performance on interpolating images, specified algorithms are assigned to interpolate each classified areas, respectively. In this way, the edge-oriented interpolation method succeeds to lower the resulting computational complexity when compared with the aforementioned image interpolation techniques.

To compare all the above resizing methods with the proposed one CA based method several images were selected based on their appropriateness as found in relevant literature. All original images were initially down sample and then up sampled by the same algorithm to meet the initial dimensions. The results shown in Figs. 2.4, 2.5, 2.6 and 2.7 are a comparison of all the applied algorithms for gray as well as color images of the Koala, Cameraman, Lena, Box, Building, Teddy, Statue, Butterfly, Port and Garden images, respectively. In order to quantify the effectiveness of every method, the Peak Signal-to-Noise Ratio (*PSNR*) metric was calculated by the following formula:

$$PSNR_{(db)} = 20 \times log_{10}(255/\sqrt{MSE}) \tag{2.8}$$

Table 2.1 Resulted PSNRS (dB) values. NN: Nearest; BL: Bilinear; BC:Bicubic; ND: NEDI; EO: Edge - oriented; CA-R method

Image	Method					
	NN	**BL**	**BC**	**ND**	**EO**	**CA-R**
Koala (RGB: 256×192)	23.70	25.37	25.14	32.45	29.30	30.32
Cameraman (GR:128×128)	22.37	23.96	23.70	30.77	25.80	26.54
Lena (RGB:150×150)	26.93	28.88	28.77	34.57	30.90	31.57
Box (GR:320×240)	28.68	30.19	29.99	32.1	29.13	30.02
Building (RGB:640×480)	29.92	31.71	31.73	33.91	30.62	31.58
Teddy (RGB:450×376)	10.9374	11.0288	10.96	16.356	13.3263	15.137
Statue (RGB:384×288)	24.9064	26.4758	26.2662	29.167	27.3648	28.473
Butterfly (RGB:133×100)	18.5155	20.1773	20.1355	25.1367	23.164	24.526
Port (RGB:800×600)	24.4053	25.9650	25.8321	29.3648	27.816	28.7912
Garden (RGB:352×240)	16.7522	18.0961	19.7002	23.9463	20.1834	21.648

where MSE denotes the Mean-Squared-Error and is calculated by:

$$MSE = \frac{1}{M \times N} \sum_{i=0}^{M-1} \sum_{i=0}^{N-1} [I(i,j) - F(i,j)]^2 \tag{2.9}$$

where $I(i,j)$ corresponds to the original image, $F(i,j)$ is the approximated version of the image and M, N are the dimensions of the image, respectively. For color images, the definition of $PSNR$ is the exact same except the MSE is the sum over all squared value differences divided by image size and by three.

All resulted PSNR values for every tested image are provided in Tab. 2.1; while in Tab. 2.2, the resulted execution time for each method is demonstrated. As it is depicted in Tab. 2.2, the proposed method produces sufficiently high PSNR values. Despite their low processing time, the Nearest neighbor method produces zigzag artifacts over the edge areas while the Bilinear method results in blurred edges. The Bicubic method also produces blurred edges requiring more processing time. The highest PSNR values are produced by the NEDI algorithm nevertheless; the required execution time prohibits its use for real-time applications. The edge-oriented method produces adequate PSNR values with low processing time however; it is a threshold dependent method since it is required to determine the edge areas. On the contrary, the proposed method produces sufficiently high PSNR values preserving the edges of the initial image. Moreover, exploiting the inherent parallelism of the CA, the method displays low execution time making appropriate for real time systems.

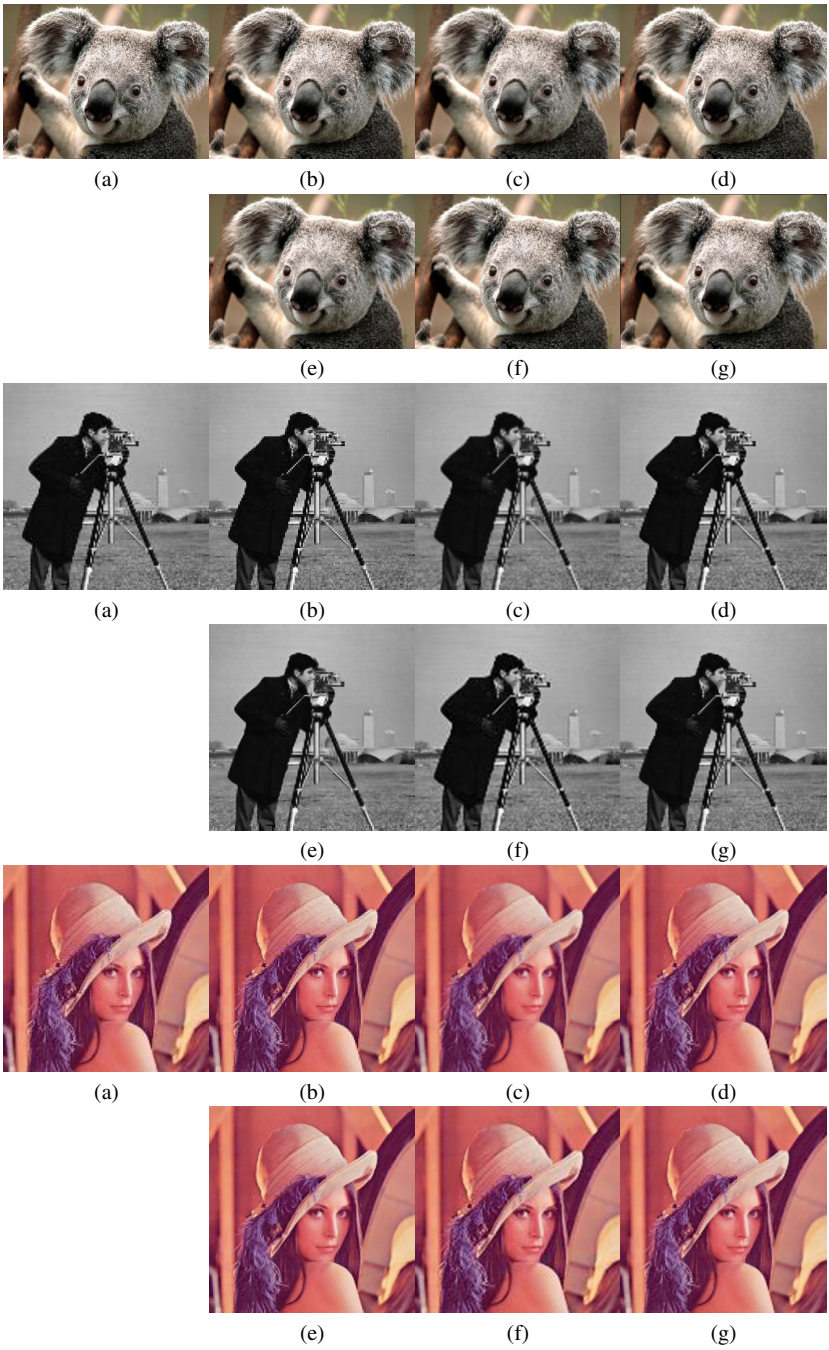


Fig. 2.4 Resized images: (I) Koala, (II) Cameraman, and (III) Lena: (a) Original image, (b) Nearest neighbor, (c) Bilinear, (d) Bicubic, (e) NEDI, (f) Edge-oriented and (g) CA based method

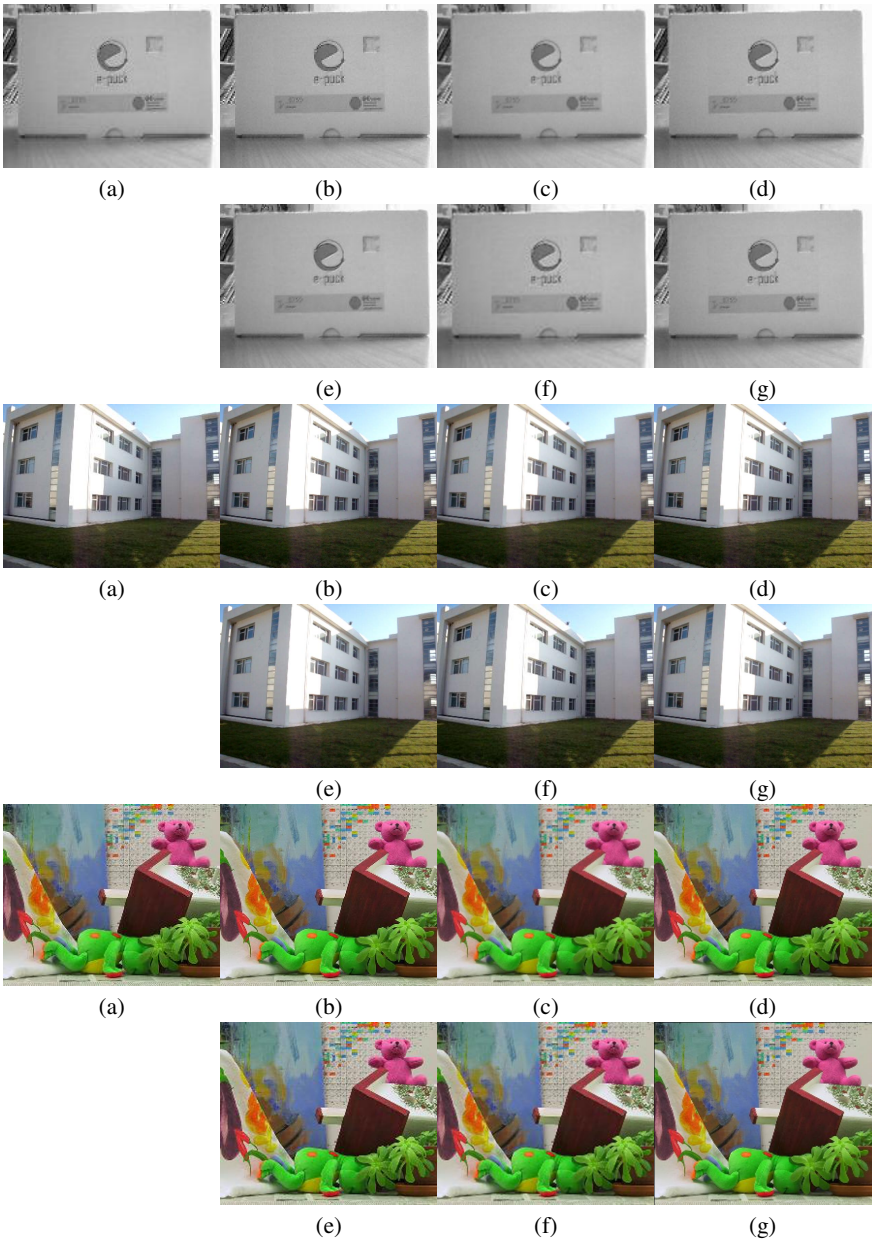


Fig. 2.5 Resized images: (I) Box, (II) Building, and (III) Teddy: (a) Original image, (b) Nearest neighbor, (c) Bilinear, (d) Bicubic, (e) NEDI, (f) Edge-oriented and (g) CA based method

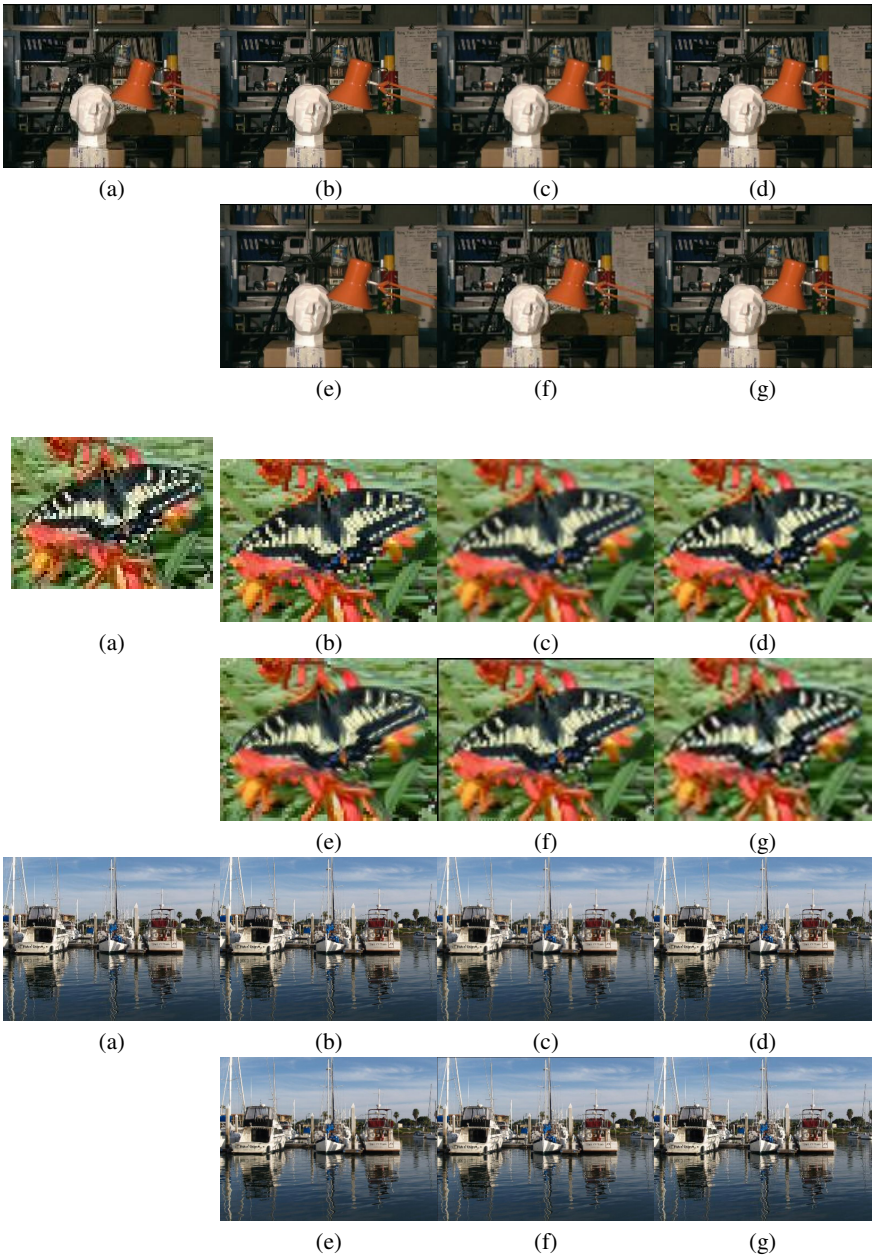


Fig. 2.6 Resized images: (I) Statue, (II) Butterfly, and (III) Port : (a) Original image, (b) Nearest neighbor, (c) Bilinear, (d) Bicubic, (e) NEDI, (f) Edge-oriented and (g) CA based proposed method

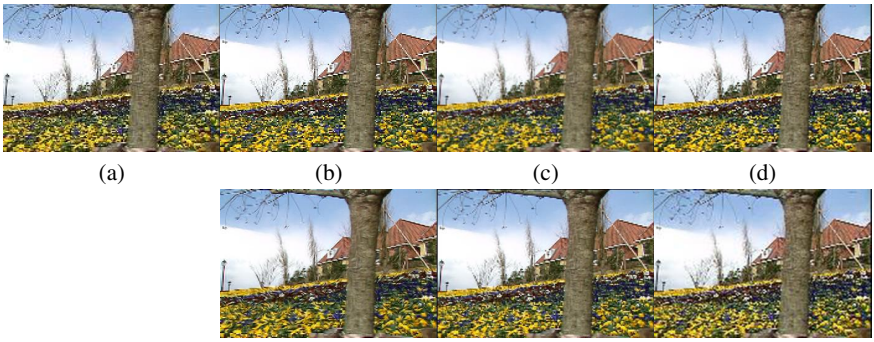


Fig. 2.7 Resized images: (I) Garden: (a) Original image, (b) Nearest neighbor, (c) Bilinear, (d) Bicubic, (e) NEDI, (f) Edge-oriented and (g) CA based method

Table 2.2 Execution time (msec). NN: Nearest; BL: Bilinear; BC: Bicubic; ND: Nedi, EO: Edge - oriented; CA-R method

Image	Method					
	NN	BL	BC	ND	EO	CA-R
Koala (RGB: 256×192)	3.6	19	21	44860	91.2	84.4
Cameraman (GR:128×128)	3.1	8.7	9.7	4500	38.2	36.9
Lena (RGB:150×150)	3.9	13.2	15.9	18600	41.2	39.3
Box (GR:320×240)	3.6	9.4	10.6	5000	63.2	58.6
Building (RGB:640×480)	5	26.2	29.5	65290	102.4	99.4
Teddy (RGB:450×376)	5.052	9.829	16.002	10526	61.235	59.331
Statue (RGB:384×288)	4.878	12.574	10.083	6537	43.216	43.808
Butterfly (RGB:133×100)	4.395	6.493	9.228	2930	12.648	13.633
Port (RGB:800×600)	6.006	22.002	34.731	128494	152.367	169.085
Garden (RGB:352×240)	4.157	11.445	12.894	21507	34.504	36.135

2.5 Hardware Implementation

The proposed model was implemented on an FPGA platform. Although the model performed well as software on conventional, commercially available computers, there are many reasons why a specialized, custom made integrated circuit could be preferred in the case of CA parallel computation tool. Conventional general-purpose computers display two main disadvantages: large sizes and increased operational energy amounts. On the other hand smaller, embedded general-purpose processors may pose limitations on the computational complexity of the simulated model. The

way to overcome the limitations on embedded systems is through customization. A custom-made FPGA can combine (the advantages of a conventional computer with those of an embedded processor, meaning the computational efficiency and energy consumption, respectively).The main drawback of this option lies on the design and the technical specifications of the specialized microprocessor. However, due to their simplicity and repeatability, designing a custom FPGA system for a CA model requires the formation of a single, simple cell and the connections to its neighbors.

In more detail, CA are probably the most suitable structures that best match to a complete parallelized hardware implementation. In contrast to serial computers, the motivation of model's hardware implementation is detected on the parallel processing capacity of CA. It is an intrinsic feature that contributes to further accelerate the model's operation. The FPGA based CA hardware implementation presents advantages of low-cost, portability and unified and repeated structure. General purpose computers provide sufficient processing power in the analysis of complicated phenomena. Nonetheless, such an option may be prohibited or even impossible due to high power consumption. Portable general purpose computers can be unable to handle more complicated computational processes. A possible acceleration of a model's execution can be achieved in such embedded systems provided that available features of FPGA structures are utilized. These structures enable parallel data processing using standardized digital modules. Besides, the circuitry of a CA demands the design of a single cell and the rest of the configuration is the same across. The total mask for a large CA pattern (cells, internal connections and cells interconnections) could be generated by a simple repetitive procedure, thus preventing from silicon's area overhead and long interconnection lines. Furthermore, due to the local processing character, the length of the critical interconnections is minimized and it is independent of the total number of cells.

As a result, the presented CA based image resizing method has been implemented with the use of VHDL code in a single FPGA device. The discrimination between the homogenous pixels and the edge pixels is achieved by the application of the Canny edge detector which was also embedded in the tested FPGA platform as the initial processing stage. The resulted map is typically increased in order for the cells with the unknown pixel's light intensity value to be produced. Each cell of the resulting CA grid receives as inputs all of its neighboring current states, and it is further supplied with a clock, a start/stop and a reset signal. The cell outputs after the application of the aforementioned transition rules result to all of its next step states. All light intensity values are determined during the remapping process based on the state of each corresponding cell. The process is repeated separately for each one of the RGB channels. Moreover, the data output of the proposed hardware implementation is made in a semi-parallel manner [9, 15, 29]. Essentially, instead of providing the output data for each cell simultaneously on its own output, the cells are grouped by column and the output of the n cells of each column appear serially on the output bus that corresponds to the column. The adoption of such an initialization method considerably shortens the number of input pins as well as the length of interconnections, thus accelerating the operation of the resulting FPGA processor.

The use of a task oriented embedded system such as FPGA can significantly decrease the total execution time of the method. Due to their features, a CA-based implementation using an FPGA can exploit their potentiality for an entirely parallel system executing simultaneously multiple threads and thus, the unknown cell states can be defined. On the other hand, the implementation with a general purpose processor poses some speed related restriction. With the assumption that the used processor embeds more than one core, it simulates the parallel execution of multiple threads which eventually corresponds to a CA cell evolution. Thus, the total number of the FPGA executed threads are significantly superior related to the corresponding threads of a general purpose processor. In addition, both implementations of the Canny edge detector display major differences related to their execution time. The FPGA implementation handles multiple execution threads while a single process is available with the corresponding general purpose processor. Moreover, images with an increased number of edges require additional execution circles in order to produce the desired edge maps. In concluding, these variations between the two tested systems pose many restrictions to accurately evaluate the total speed improvement since no common execution base can be identified.

2.6 Discussion and Conclusions

In this paper, a new image resizing method based on the CA and the Canny edge detector was introduced. The Canny edge detector is initially applied in order to discriminate the edge areas from the homogenous areas. The idea was to enhance the performance of the proposed CA based image resizing method with a robust edge detector resulting in less computational burden. Towards this direction, as a future work the application of CA oriented edge detector coupled with the CA algorithm already proposed could possibly lead to advantageous results in terms of parallelism and computational speed.

The resulted binary edge map is then upscaled and it is processed as a CA grid. Appropriate CA states and transition rules were constructed to evolve the CA which eventually attempt to enhance the quality of the edged areas. The orientation of the edge cells is considered in order to preserve effectively the edges of the initial image. Finally, a simple linear transformation is applied to re-evaluate the light intensity value of each pixel for the final resized image. In terms of quantitative comparison based on the PSNR values, the method demonstrates sufficient performance while the required processing time is kept at low levels due to the parallel nature of the CA. It is clear the proposed CA-based algorithm successfully achieves the goals of real-time interpolation and good subjective quality. Consequently, the method could be considered as appropriate for systems with low technical specifications, i.e. low resolution cameras, when further image processing is required.

Moreover, in order to enhance the performance of the proposed CA image resizing technique the usage of Genetic Algorithms (GA) so as to provide CA rules in correspondence with the under study image should be also examined. More specifically, CA have been successfully linked with GA in the past resulting in quite

robust models able to overcome complex limitations of the examined systems and processes. Here, the idea is to advance the selection of the CA rules and neighborhoods depending on the specific texture and edges of the examined images. In such a way we will be able to tackle with the complexity imposed by the different characteristics of different images and to propose a robust adaptive CA resizing technique. It should be mentioned that when CA neighborhood is increased the corresponding results are advantageous in terms of PSNR comparison while the computational burden also increases. To overcome the computational speed decrement the GA could possible introduce different rules while the demand of low computational burden could serve through appropriate selection of the fitness function resulting in more adaptive CA rules and neighborhoods as a compromise between quality and computational speed.

References

1. Amanatiadis, A., Andreadis, I., Gasteratos, A.: A log-polar interpolation applied to image scaling. In: IEEE International Workshop on Imaging Systems and Techniques, pp. 1–5. IEEE, Cracovia (2007)
2. Andreadis, I., Illiades, P., Karafyllidis, Y., Tsalides, P., Thanailakis, A.: Design and VLSI implementation of a new ASIC for colour measurement. IEE Proceedings - Circuits, Devices and Systems 142(3), 153–157 (1995)
3. Andreadis, I., Karafyllidis, I., Tzionas, P., Thanailakis, A., Tsalides, P.: A new hardware system for automated visual inspection based on a cellular automaton architecture. J. Intellig. Robot. Sys. 16, 89–102 (1996)
4. Canny, J.: A computational approach to edge-detection. IEEE Trans. Pattern Anal. Mach. Intell. 8, 679–700 (1986)
5. Cardenas-Barrera, J.L., Plataniotis, K.N., Venetsanopoulos, A.N.: QCA implementation of a multichannel filter for image processing. Mathematical Problems in Engineering 8, 87–99 (2002)
6. Cha, Y., Kim, S.: The error-amended sharp edge (EASE) scheme for imaging zooming. IEEE Trans. Image Process. 16, 1496–1505 (2007)
7. Chen, J.L., Chang, J.Y., Shieh, K.L.: 2D discrete signal interpolation and its image resampling application using fuzzy rule-based inference. Fuzzy Sets Syst. 114, 225–238 (2000)
8. Chen, M.J., Huang, C.H., Lee, W.L.: A fast edge-oriented algorithm for image interpolation. Image and Vision Computing 23, 791–798 (2005)
9. Georgoulas, C., Kotoulas, L., Sirakoulis, G.C., Andreadis, I., Gasteratos, A.: Real-time disparity map computation module. Microprocess. Microsyst. 32(3), 159–170 (2008)
10. Huang, Y., Fan, H.: Learning from interpolated images using neural networks for digital forensics. In: IEEE Conference on Computer Vision and Pattern Recognition, pp. 177–182. IEEE, San Francisco (2010)
11. Hwang, J.W., Lee, H.S.: Adaptive image interpolation based on local gradient features. IEEE Signal Process. Lett. 29, 359–362 (2004)
12. Ioannidis, K., Andreadis, I., Sirakoulis, G.C.: An edge preserving image resizing method based on cellular automata. In: Sirakoulis, G.C., Bandini, S. (eds.) ACRI 2012. LNCS, vol. 7495, pp. 375–384. Springer, Heidelberg (2012)
13. Jain, A.K.: Fundamentals of Digital Image Processing. Prentice-Hall, Upper Saddle River (1978)

14. Jiang, H., Moloney, C.: A new direction adaptive scheme for image interpolation. In: International Conference on Image Processing, Rochester, New York, USA, pp. 369–372 (2002)
15. Kalogeropoulos, G., Sirakoulis, G.C., Karafyllidis, I.: Cellular automata on FPGA for real-time urban traffic signals control. Journal of Supercomputing 65(2), 1–18 (2013)
16. Karafyllidis, I., Andreadis, I., Tzionas, P., Tsalides, P., Thanailakis, A.: A cellular automaton for the determination of the mean velocity of moving objects and its VLSI implementation. Pattern Recognition 29(4), 689–699 (1996)
17. Katis, I., Sirakoulis, G.: Cellular automata on FPGAs for image processing. In: 16th Panhellenic Conference on Informatics, pp. 308–313. IEEE Computer Society, Piraeus (2012)
18. Keys, R.G.: Cubic convolution interpolation for digital image processing. IEEE Trans. Acoust., Speech, Signal Process. 29, 1153–1160 (1981)
19. Lafe, O.: Cellular Automata Transforms: Theory and Applications in Multimedia Compression. Encryption and Modeling. Kluwer Academic Publishers (2000)
20. Li, X., Orchard, M.T.: New edge-directed interpolation. IEEE Trans. Image Process. 10, 1521–1527 (2001)
21. Lin, C.T., Fan, K.W., Pu, H.C., Lu, S.M., Liang, S.F.: An HVS-directed neural network based image resolution enhancement scheme for image resizing. IEEE Trans. Fuzzy Syst. 15, 605–615 (2007)
22. Michailidis, G., Andreadis, I.: A real-time stereo correspondence algorithm based on 2-D cellular automata. In: Int. Workshop on Advanced Imaging Technology, Kuala Lumbur, Malaysia, pp. 1–6 (2010)
23. Muresan, D., Parks, T.: Adaptively quadratic (aqua) image interpolation. IEEE Trans. Image Process. 13, 690–698 (2004)
24. Nalpantidis, L., Amanatiadis, A., Sirakoulis, G.C., Gasteratos, A.: Efficient hierarchical matching algorithm for processing uncalibrated stereo vision images and its hardware architecture. IET Image Processing 5(5), 481–492 (2011)
25. Panagiotopoulos, F.K., Mardiris, V.A., Chatzis, V.: Quantum–dot cellular automata design for median filtering and mathematical morphology operations on binary images. In: Sirakoulis, G.C., Bandini, S. (eds.) ACRI 2012. LNCS, vol. 7495, pp. 554–564. Springer, Heidelberg (2012)
26. Popovici, A., Popovici, D.: Cellular automata in image processing. In: Proceedings of the 15th International Symposium on the Mathematical Theory of Networks and Systems, p. 6 (2002)
27. Porter, R., Frigo, J., Conti, A., Harvey, N., Kenyon, G., Gokhale, M.: A reconfigurable computing framework for multi-scale cellular image processing. Microprocess. Microsyst. 31(8), 546–563 (2007)
28. Preston, K., Duff, J.: Modern Cellular Automata: Theory and Applications. Plenum Press (1984)
29. Progias, P., Sirakoulis, G.C.: An FPGA processor for modelling wildfire spread. Mathematical and Computer Modeling 57(5-6), 1436–1452 (2013)
30. Rosin, P.L.: Training cellular automata for image processing. IEEE Trans. Image Process. 15(7), 2076–2087 (2006)
31. Rosin, P.L.: Image processing using 3-state cellular automata. Computer Vision and Image Understanding 114(7), 790–802 (2010)
32. Rosin, P.L., Sun, X.: Cellular automata as a tool for image processing. In: Chen, C.H. (ed.) Emerging Topics in Computer Vision and its Applications, pp. 233–251 (2011)
33. Shi, H., Ward, R.: Canny edge based image expansion. In: IEEE International Symposium on Circuits and Systems, pp. 785–788. IEEE, Scottsdale (2002)

34. Sirakoulis, G.C., Karafyllidis, I., Thanailakis, A.: A CAD system for the construction and VLSI implementation of cellular automata algorithms using VHDL. Microprocess. Microsyst. 27(8), 381–396 (2003)
35. Wolfram, S.: Theory and applications of Cellular Automata. World Scientific, Singapore (1986)
36. Xiong, R., Ding, W., Ma, S., Gao, W.: Improved autoregressive image model estimation for directional image interpolation. In: 28th Picture Coding Symposium, Nagoya, Japan, pp. 442–445 (2010)

Chapter 3
Skeletonizing Digital Images with Cellular Automata

Daniel Díaz-Pernil, Francisco Peña-Cantillana, and Miguel A. Gutiérrez-Naranjo

Abstract. The skeletonization of an image consists of converting the initial image into a more compact representation. In general, the skeleton preserves the basic structure and, in some sense, keeps the *meaning*. The most important features concerning a shape are its *topology* (represented by connected components, holes, etc.) and its *geometry* (elongated parts, ramifications, etc.), thus they must be preserved. Skeletonization is usually considered as a pre-processing step in pattern recognition algorithms, but its study is also interesting by itself for the analysis of line-based images such as texts, line drawings, human fingerprints classification or cartography.

Since the introduction of the concept by Blum in 1962 under the name of medial axis transform, many algorithms have been published in this topic and there are many different approaches to the problem, among them the ones based on distance transform of the shape and skeleton pruning based on branch analysis. In this chapter, we focus on how the skeletonization of an image can be studied in the Cellular Automata framework and, as a case study, we consider in detail the Guo and Hall skeletonizing algorithm.

3.1 Introduction

Skeletonization is one of the approaches for representing a shape with a small amount of information by converting an image into a more compact representation and keeping the meaningful features. The conversion should remove redundant

Daniel Díaz-Pernil
Research Group on Computational Topology and Applied Mathematics,
Department of Applied Mathematics, University of Seville, Spain
e-mail: sbdani@us.es

Francisco Peña-Cantillana · Miguel A. Gutiérrez-Naranjo
Research Group on Natural Computing,
Department of Computer Science and Artificial Intelligence, University of Seville, Spain
e-mail: frapencan@gmail.com, magutier@us.es

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_3, © Springer International Publishing Switzerland 2014

information, but it should also keep the basic structure. The concept of skeleton was introduced by Blum in [10, 11], under the name of medial axis transformation, but other previous approaches can be found in the literature (see, e.g., [16, 28]). The skeleton of an image is useful to characterize objects by compact representation while preserving the connectivity and topological properties of any image. The most important features concerning a shape are its *topology* (represented by connected components, holes, etc.) and its *geometry* (elongated parts, ramifications, etc.), thus they must be preserved.

Currently, there are many different definitions of the skeleton of a black and white image and, according to Saeed *et al.* [39], more than one thousand algorithms have been published on image skeletonization. Nevertheless, roughly speaking, we can say that the image B is a skeleton of the black and white image A, if the former has fewer black pixels than the latter, preserves its topological properties and, in some sense, keeps its *meaning*. Figure 3.1 illustrates this idea. The skeletonized image keeps the *meaning* of the original one and it uses fewer black pixels. It keeps the basic geometry of the original image and also its topology. Let us remark that the white regions inside the hand-made words are also white regions in the skeletonized one and the connectedness is preserved.

As pointed out by Rosenfeld [36], the concept of skeletonizing[1] is not easy to be defined mathematically; however it seems reasonable to require that any skeletonizing algorithm should preserve the connectedness for both objects and their complement; leave unchanged 1-pixel wide and isolated points; and change objects whose length and width are both greater than 1.

From the computational side, a digital image can be roughly defined as a function from a two dimensional surface which maps each point from the surface onto a set of attributes as brightness or color. Technically, a digital image can be considered as a bi-dimensional array of $n \times m$ pixels. Each pixel can be characterized by a triplet (i, j, a) (usually written as a_{ij}) where (i, j) represents its position in the array and a encodes brightness, color or any other feature associated to the position (i, j). As in many other image processing algorithms, the basic procedure for skeletonizing an image involve a discrete transformation of the features associated with the position (i, j) according to the current values of such features (i.e., the current value of a) together with the values of the features of the neighbour positions. From this point of view, a cellular automaton can be considered as a natural computer device for such transformations [23, 40]. As usual, pixels are identified with cells of the cellular automaton and the encoding a represent the current *state* of the pixel. In many cases, the transformation of all the pixels can be done in parallel, since the *state* of a pixel at the step i only depends on the *states* of a set of pixels at the step $i - 1$.

Such parallelism in skeletonizing algorithms has been broadly studied (see, e.g., [21, 32, 43, 45]). The development of new hardware architectures has also contributed to new parallel implementations of these algorithms [19, 24, 26]. Recently, a parallel implementation of a cellular automata skeletonizing algorithm developed by

[1] Rosenfeld used the word *thinning* instead of *skeletonizing*. In this paper, both terms are considered synonymous. Nevertheless, in the literature many different definitions of *skeletonizing* and *thinning* can be found, not all of them equivalent.

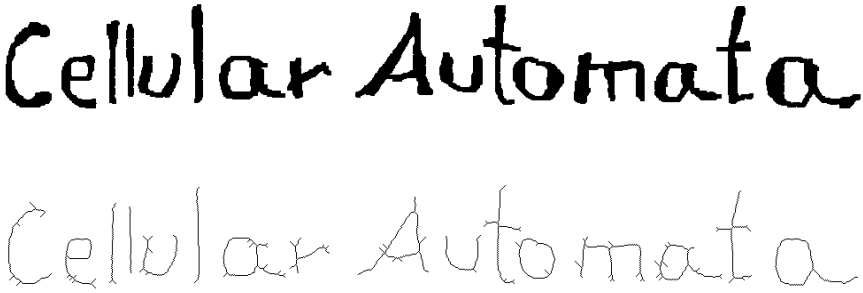


Fig. 3.1 Hand-written words and their skeletonization. This skeletonization has been performed with the parallel implementation on GPU of the cellular automata adaptation of the Guo and Hall algorithm presented in [35].

using a device architecture called Compute Unified Device Architecture (CUDA™) has been presented [35]. CUDA™ is a general purpose parallel computing architecture that allows the parallel NVIDIA[2] Graphics Processors Units (GPUs) to solve many complex computational problems in a more efficient way than on a Central Processing Unit (CPU). In Section 3.5, we explore how this new computer architecture can be used in order to improve the efficiency of the algorithms from a practical point of view.

Skeletonization has been found useful for data compression and pattern recognition in a wide range of applications in the industrial and scientific fields. It is usually considered as a pre-processing step in pattern recognition algorithms, but its study is also interesting by itself for the analysis of line-based images such as coronary arteries [17], human fingerprints classification [27], cartography [29], data compression and data storage [20], automated inspection of printed circuit boards [44] or optical character recognition (OCR) [42] among others.

The chapter is organized as follows: Firstly, a brief overview on different skeletonizing algorithms is presented (Section 3.2). In Section 3.3, as an example, the Guo and Hall skeletonizing algorithm is studied in detail. Next, some hints on a general skeletonizing algorithm on cellular automata are shown (Section 3.4) and, as a case study, the Guo and Hall algorithm is adapted to cellular automata. In Section 3.5, some hints for a parallel implementation in CUDA are presented. Finally, some examples and some conclusions are provided.

3.2 Skeletonizing Algorithms

The first definition of the skeleton of a region was provided by Blum as the medial axis transformation (MAT) [10, 11]. According to the original definition, in order to find the MAT region with border B from a region R, the closest neighbor in B

[2] `http://www.nvidia.com`

of each pixel in R should be found. If the pixel has more than one such neighbor, it is said to belong to the medial axis (skeleton) of R. The definition of *closest* depends, of course, on the definition of distance. Obviously, the implementation of such definition is typically prohibitive computationally. Many algorithms have been proposed for improving computational efficiency while at the same time attempting to produce a medial axis representation. Such skeletonizing algorithms iteratively delete edge points of a region subject to the constraints that deletion of these points (1) does not remove end points, (2) does not break connectedness and (3) does not cause excessive erosion of the region.

Due to the enormous amount of algorithms, it is not easy to classify them. Different criteria can be used in a a preliminary approach. A first classification can be done according to the method used to obtain the skeleton. Following this criterion, the algorithms are split into iterative and non-iterative. A second classification method focuses on the properties which a black pixel must satisfy in order to be marked as erasable. In this way, the properties can be *local* or *global*.

According to the first criterion, the skeletonizing algorithms can be classified as either iterative or non-iterative. In iterative methods, skeletonizing algorithms produce a skeleton by examining and deleting contour pixels through an iterative process in either sequential or parallel way. Parallel algorithms may also be further classified according to their performance, i.e., in 4-, 2-, or 1-subcycle manners. The latter (1-subcycle parallel algorithms) have always received more considerable attention in the research area of parallel skeletonizing as they have reduced the computation time in the number of iterations, and that is why they are sometimes called one-pass or fully parallel algorithms [14, 15, 22]. In sequential skeletonizing algorithms, contour points are examined for deletion in a predetermined order. In parallel skeletonizing algorithms, pixels are examined for deletion on the basis of results obtained only from the previous iteration. That is why parallel skeletonizing algorithms are suitable for implementation in parallel processors. Non-iterative (non-pixel based) skeletonizing algorithms produce a certain median or centre line of the pattern to be thinned directly in one pass, without examining all the individual pixels.

According to the second criterion, (see [19]), two different methods of pattern analysis can be applied to determine the skeleton of an image or scene: Global Pattern Analysis, where pixels are labeled depending on their distance from the contours and Local Pattern Analysis, based on the repetition of the simultaneous deletion of border points verifying certain conditions. This classification is not strict, and some *hybrid* methods can be found in the literature, as Liu's method [30, 31], based on the notion of *cell complex*. This method combines an iterative process where *outer* cells are removed with two difference measures which provide some ideas of the size of the maximum disk inscribed in the object.

Since Dinneenn [16] found in the 1950s that an averaging operation over a square window with a high threshold resulted in a skeletonizing of the input image and later Blum presented his definition of medial axis transformation [10, 11], many different authors have contributed to the skeletonizing theory. After some attempts of defining the skeleton of an image and different skeletonizing algorithms, (e.g. [25, 38]),

Rosenfeld [36] was the first to evaluate the necessary and sufficient conditions for preserving topology while deleting border points in parallel process. Only a few years later, Pavlidis [34] proposed a combination of parallel and sequential operations.

In 1980s, many other algorithms were proposed (see, e.g., [3, 18, 45]). Among them, it is Baruch's remarkable algorithm [7]. It is a non iterative, non pixel-based algorithm. The skeleton is produced in one pass, by line following. Another important contribution is the Guo and Hall algorithm, which will be showed in detail in Section 3.3. In this algorithm, the contour pixels are examined for deletion in an iterative process and it has been the basis for further refinements (see, e.g., [46]). There are many different approaches to the problem of skeletonizing and a detailed survey of the different methods is out of the scope of this chapter[3]. Among the different research areas around the skeletonization problem, we can cite the studies based on distance transform of the shape and skeleton pruning based on branch analysis (see, for example, [4–6, 9, 41]). The research of skeletonizing algorithms also includes different computational models, as the algorithm presented by Altuwaijri and Bayoumi [2] where self-organizing neural networks are used or the use of skeletonizing in color images [33].

As pointed out above, there exists an enormous amount of skeletonizing algorithms. Generally, each of the presented algorithms has its own advantages and disadvantages, and each has its applications where it performs better than others. Therefore, it is often difficult to directly compare the results.

3.3 Guo and Hall Algorithm

As an example of skeletonizing algorithm, we will see the implementation of a classical algorithm, the Guo and Hall algorithm [21, 22]. It is a so-called 1-subcycle parallel algorithm or fully parallel algorithm. It is an iterative edge-point erosion algorithm where a 3×3 window is considered around each pixel of the image with a set of rules applied to the contents of the window. In a sequential simulation of the algorithm, a *unique* window is moved along the image, whereas in the parallel one, *all the windows* are considered simultaneously. The skeleton is obtained by an iterative procedure of skeletonizing: the border points are removed as long as they are not considered significant. The remaining set of points is called the *skeleton*.

In this algorithm, the contour pixels are examined for deletion in an iterative process. The decision is based on a 3×3 neighbourhood. The image is divided into two disjoint areas (sub-sections), similarly to a chess board. One of the sections is composed by the pixels a_{ij} such that $i + j$ is even. Alternatively, the second subsection corresponds to the pixels a_{ij} such that $i + j$ is odd. The algorithm consists on two sub-iterations where the removal of redundant pixels from both sub-sections are alternated, i.e., in each step only the pixels of one of the subsections are evaluated for its deletion. This is repeated until there are no redundant pixels left.

[3] A good introduction can be found in [39].

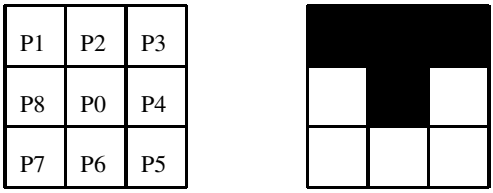


Fig. 3.2 (a) Enumeration of the pixels in a 3×3 neighborhood. (b) An example of 3×3 neighborhood where the central pixel $P0$ has $B(P0) = 3$ and $C(P0) = 1$.

Given a pixel $P0$, a clockwise enumeration $P1, \ldots, P8$ of its eight neighbor pixels is considered, as shown in Fig. 3.2 (a). As usual, for each $i \in \{1, \ldots, 8\}$, Pi is considered as a Boolean variable, with the (truth) value 1 if Pi is *black* and 0 if Pi is *white*.

In order to decide if a pixel $P0$ is deleted in the corresponding iteration subcycle, two parameters are evaluated:

$$\begin{aligned} B(P0) &= \textstyle\sum_{i=1}^{i=8} Pi \\ C(P0) &= (\neg P2 \wedge (P3 \vee P4)) + (\neg P4 \wedge (P5 \vee P6)) \\ &\quad + (\neg P6 \wedge (P7 \vee P8)) + (\neg P8 \wedge (P1 \vee P2)) \end{aligned}$$

$B(P0)$ counts how many pixels in the neighborhood of $P0$ are black. $C(P0)$ evaluates the *connectivity* of the pixel $P0$. Notice that for isolated black pixels, the connectivity is 0. If the pixel is surrounded by eight black pixels, the connectivity is also 0. According to the Guo and Hall algorithm, in each iteration, an evaluated black pixel $P0$ is deleted (changed to white) if and only if all of the following conditions are satisfied.

Guo and Hall Conditions

1. $B(P0) > 1$;
2. $C(P0) = 1$; This condition is necessary for preserving local connectivity when P is deleted.
3. $(P1 \wedge P3 \wedge P5 \wedge P7) \vee (P2 \wedge P4 \wedge P6 \wedge P8) = FALSE$; Intuitively, this condition is satisfied if $P0$ is not the central pixel of a *cross*.

For example, let us consider as $P0$ the central pixel in the image of Fig. 3.2 (b). In this case, $B(P0) = 3 > 1$, $C(P0) = 1$, and the third condition is also satisfied. Hence, $P0$ will be deleted in the corresponding subcycle iteration.

3.4 Cellular Automata

Cellular automata (CA) are dynamical systems discrete in time and space. These features make CA suitable for dealing with some problems in the analysis of digital images, where pixels are identified with cells and the changes in a cell depends on the current state plus the current state of its neighbours [13, 23, 37, 40].

The dynamics of CA has been extensively studied across a variety of disciplines from different perspectives (see, e.g., [1, 8, 12]). Technically, a CA consists of two components. The first one is a *cellular space*: a lattice of N identical finite-state machines (cells) each with an identical pattern of local connections to other cells, with boundary conditions if the lattice is finite. The second component is a set of transition rules that gives the update state of each cell. The formal description of the algorithm in the framework of CA requires to provide the cellular space and the set of rules.

Given a $n \times m$ image, we will consider as *cellular space* a rectangular grid $(n+2) \times (m+2)$ with 8-adjacency. We will consider a cell of the cellular space for each pixel on the image plus two extra columns and rows surrounding such cells. The intuition behind these extra pixel lines is to consider that they are *white pixels*, which do no affect to the skeletonizing process. The states in these extra cells never change and we will consider that the rules are not applied on them, but they contribute to the neighborhood of the border pixels of the original image.

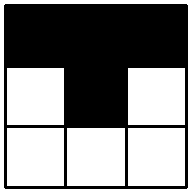
With respect to the set of states, they must encode the information of each pixel along all the steps of the algorithm. The basic information for the skeletonizing process of a black and white image is, of course, the color of the pixel, but, depending on the algorithm, different features can be associated to the pixel. If these features change dynamically along the skeletonizing process, they should be encoded in the set of states.

In order to fix ideas, we will consider the Guo and Hall algorithm. On the one hand, the set of states should inform about the *color* B or W (black or white) of the pixel at each discrete step, but, according to the algorithm, the set of pixels are split into two subsections and only one of them is evaluated at each step. In this way, the state of a cell at time t should inform if the cell corresponds to an evaluable pixel or to a non-evaluable pixel. Putting together both pieces of information, the color and the evaluable situation (E for evaluable and N for non evaluable), we obtain four possible states for each cell of the cellular space: BE, WE, BN and WN.

Bearing in mind this interpretation for the states, the initial state of each cell is determined by the color of the corresponding pixel in the input image and the corresponding sub-section. In the initial configuration, we will take the pixels of the first sub-section, corresponding to the pixels a_{ij} such that $i+j$ is even, are *evaluable* and the pixels corresponding to the second sub-section, i.e., pixels a_{ij} such that $i+j$ is odd, are non evaluable[4] (see Fig. 3.3).

In the description of the set of rules, it is necessary to describe the conditions necessary for changing the state of a cell. Such conditions are taken directly from

[4] If the second subsection is taken as evaluable in the initial configuration, the pixels of the final skeleton are different.



(a)

WN	WE	WN	WE	WN
WE	BN	BE	BN	WE
WN	WE	BN	WE	WN
WE	WN	WE	WN	WE
WN	WE	WN	WE	WN

(b)

Fig. 3.3 A toy example of black and white image and the initial configuration of the associated CA for skeletonizing it according to the Guo and Hall algorithm

the Guo and Hall algorithm. A cell will change its state if the corresponding pixel in the image satisfies the Guo and Hall conditions. Since each cell can be in one of four different states, a full description of the rules needs to consider all the possibilities of the 3×3 neighborhood, i.e., $4^9 = 2^{18}$ rules. Obviously, they are too much from a practical point of view. Nevertheless, we can provide an intensive description.

- If the central pixel is in the state WN, the next state is WE, regardless the states of the surrounding pixels.
- If the central pixel is in the state WE, the next state is WN, regardless the states of the surrounding pixels.

The intuition is clear. If the cell corresponds to a white pixel, it remains white to the end of the process. The unique change is that the pixel alternates the condition of evaluable and non evaluable in each step.

- If the central pixel is in the state BN, the next state is BE, regardless the remaining cells of the neighborhood.

If a cell correspond to a black pixel and the pixel belongs to the non evaluable section, it remains black, but in the next step it will be evaluable.

With these descriptions, it only remains to describe the rules in case of the central pixel is BE. In this case, it should be evaluated according to the Guo and Hall conditions in order to decide if the next state will be BN or WN. Nevertheless, the number of possibilities is enormous. By fixing the state of the central pixel, four possibilities for the remaining eight cells, i.e., $4^8 = 2^{16}$ possibilities, should be considered.

In order to reduce the number of different CA rules, we observe than the decision of change the current state BE (to the state BN or WN) only depends on the color of the surrounding pixels and not on the evaluable or non evaluable condition. This consideration reduces drastically the number of possibilities, since only $2^8 = 256$ cases should be considered.

In order to encode each of these cases, we can use the enumeration of the pixels used in the Section 3.3 to represent the neighborhood of the pixel $P0$ in (i, j). Given

$$DEL = \left\{ \begin{array}{cccccccccccc} 6 & 12 & 14 & 20 & 22 & 24 & 28 & 30 & 48 & 52 & 54 & 56 \\ 60 & 62 & 80 & 84 & 86 & 88 & 92 & 94 & 96 & 112 & 116 & 118 \\ 120 & 124 & 126 & 192 & 208 & 212 & 214 & 216 & 220 & 222 & 224 & 240 \\ 244 & 246 & 248 & 252 & 258 & 260 & 262 & 268 & 270 & 276 & 278 & 284 \\ 286 & 308 & 310 & 316 & 318 & 320 & 322 & 324 & 326 & 332 & 334 & 336 \\ 338 & 344 & 346 & 352 & 354 & 356 & 358 & 364 & 366 & 368 & 370 & 376 \\ 378 & 384 & 386 & 388 & 390 & 396 & 398 & 404 & 406 & 412 & 414 & 436 \\ 438 & 444 & 448 & 450 & 452 & 454 & 460 & 462 & 464 & 466 & 472 & 474 \\ 480 & 482 & 484 & 486 & 492 & 496 & 498 & 504 & & & & \end{array} \right\}$$

Fig. 3.4 Set of the codings for erasing a black pixel in an evaluable section

a cell (i, j) at a time t, the states of its eight surrounding pixels will be represented as a list $[H1, \ldots, H8]$, where, for $r \in \{1, \ldots, 8\}$, $Hr = 1$ if the state of the cell is BE or BN (i.e., if the cell corresponds to a black pixel) and $Hr = 0$ if the state of the cell is WE or WN (a white pixel). This representation of the neighborhood can be done in a more compact way, by encoding the neighborhood as a number[5] in $\{2i : i \in \{0, \ldots, 255\}\}$:

$$cod(i, j) = \sum_{r=1}^{8} Hr \times 2^r$$

For example, in Fig. 3.2 (b), the states of the cells surrounding the central one can be encoded as $[1, 1, 1, 0, 0, 0, 0, 0,]$, or, shortly, $2^1 + 2^2 + 2^3 = 14$.

Since the decision of removing a black pixel (changing to white) depends on its surrounding pixels and there is a bijective correspondence among the sets of all the possibilities and the possible encodings $\{2i : i \in \{0, \ldots, 255\}\}$. It is easy to check that the cell in (i, j) must change its state to WN (i.e., the corresponding black pixel must be deleted) if the encoding of the surrounding cells belong to the set DEL showed in Fig. 3.4. A quick check on the set DEL will decide if the state BE changes to WN or BN.

3.5 Parallel Implementation

The parallel iterative algorithms are the result of processing a pixel at the i-th iteration just depending on the value of the neighbor pixels in the $(i-1)$-th iteration. Thus parallel iterative algorithms can process all pixels in the image simultaneously.

As pointed above, the parallelism in skeletonizing algorithms has been deeply studied, but the real implementation is associated to the development of new parallel hardware [19, 24, 26]. Recently, in [35], a parallel implementation of a cellular automata skeletonizing algorithm developed by using CUDA™ has been presented.

[5] Similar ideas are also used in [39].

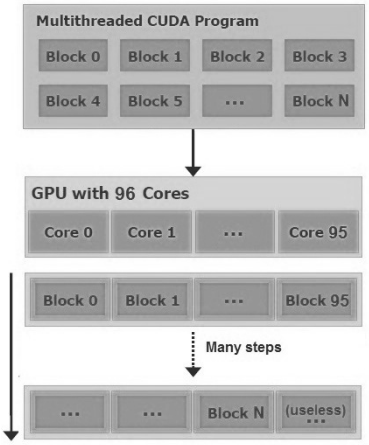


Fig. 3.5 Scheme of threads for the CUDA implementation

It shows some important features that make this new hardware architecture suitable for further implementations of CA algorithms.

CUDA™ is a general purpose parallel computing architecture that allows the parallel NVIDIA Graphics Processors Units (GPUs) to solve many complex computational problems in a more efficient way than on a Central Processing Unit (CPU). GPUs constitute nowadays a solid alternative for high performance computing, and the advent of CUDA allows programmers a friendly model to accelerate a broad range of applications. The way GPUs exploit parallelism differs from multi-core CPUs, which raises new challenges to take advantage of its computing power. GPU is especially well-suited to address problems that can be expressed as data-parallel computations.

3.5.1 Examples

In [35], a parallel implementation of the Guo and Hall algorithm for CA based on the principles described above was presented. It has been implemented by using Microsoft Visual Studio 2008 Professional Edition (C++) with the plug-in Parallel Nsight (CUDA™) under Microsoft Windows 7 Professional with 32 bits. CUDA™ C, an extension of C for implementations of executable kernels in parallel with graphical cards NVIDIA has been used to implement the CA. It has been necessary to use the *nvcc compiler* of CUDA™ Toolkit and some libraries from openCV for image input and output.

The experiments have been performed on a computer with a CPU AMD Athlon II x4 645, which allows to work with four cores of 64 bits to 3.1 GHz. The computer has four blocks of 512KB of L2 cache memory and 4 GB DDR3 to 1600 MHz of main memory. The used graphical card (GPU) is an NVIDIA Geforce GT240

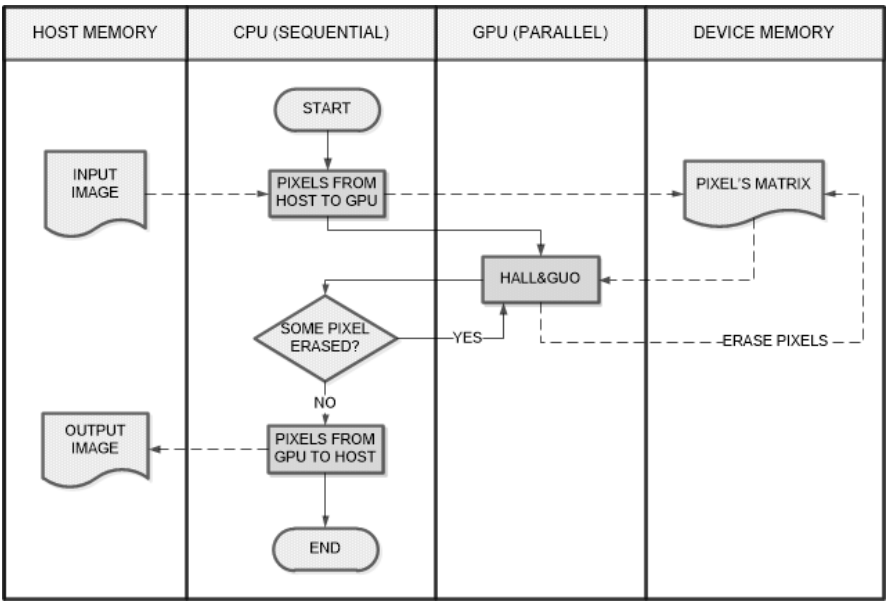


Fig. 3.6 Flowchart of the implementation on CUDA™ of the CA algorithm

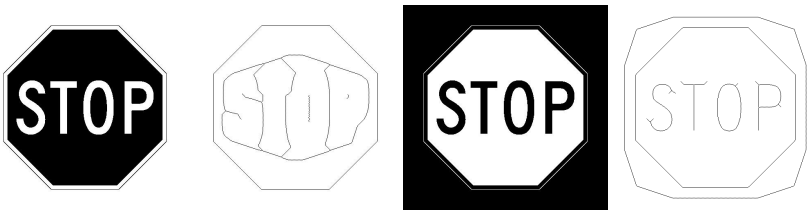


Fig. 3.7 The binarized STOP signal, its skeletonization, its inverted binarization and its skeletonization

composed by 12 *Stream Processors* with a total of 96 cores to 1340 MHz. It has 1 GB DDR3 main memory in a 128 bits bus to 700 MHz. So, the transfer rate obtained is by 54.4 Gbps. The used Constant Memory is 64 KB and the Shared Memory is 16 KB. Figure 3.6 shows a flowchart of the implementation on CUDA of the algorithm.

Next, we show the results of some experiments of skeletonizing with our parallel CA implementation of the Guo and Hall algorithm. As a first example[6], Fig. 3.7, shows the skeletonization of the STOP traffic signal. This example illustrates an old problem in black and white images. In such images, the *meaning* is usually associated to black pixels, whereas white pixels constitute the background. In this

[6] These examples are borrowed from [35].

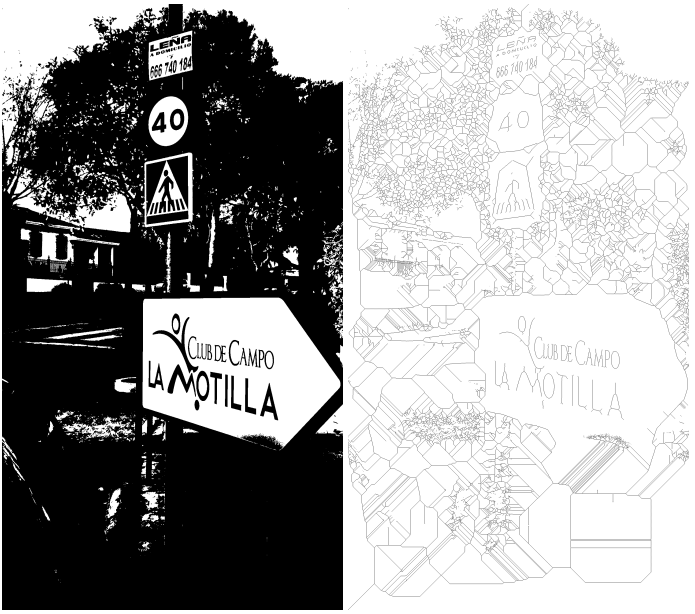


Fig. 3.8 The binarization of a real photograph and its skeleton

case, the skeletonization of the image with inverted colors is more *meaningful*. In Fig. 3.8, we can see two black and white images of 789×1317 pixels. On the left, the result of the binarization of a real photograph and its skeletonization on the right.

A last example is presented in Fig. 3.9. On the left, a fingerprint is shown and on the right, its skeletonization. The basic problem with fingerprints is to determine whether two fingerprints come from the same finger. There exist multiple algorithms that do fingerprint matching in many different ways. Some methods involve matching minutiae points between the two images, while others look for similarities in the

Fig. 3.9 A fingerprint and its skeleton

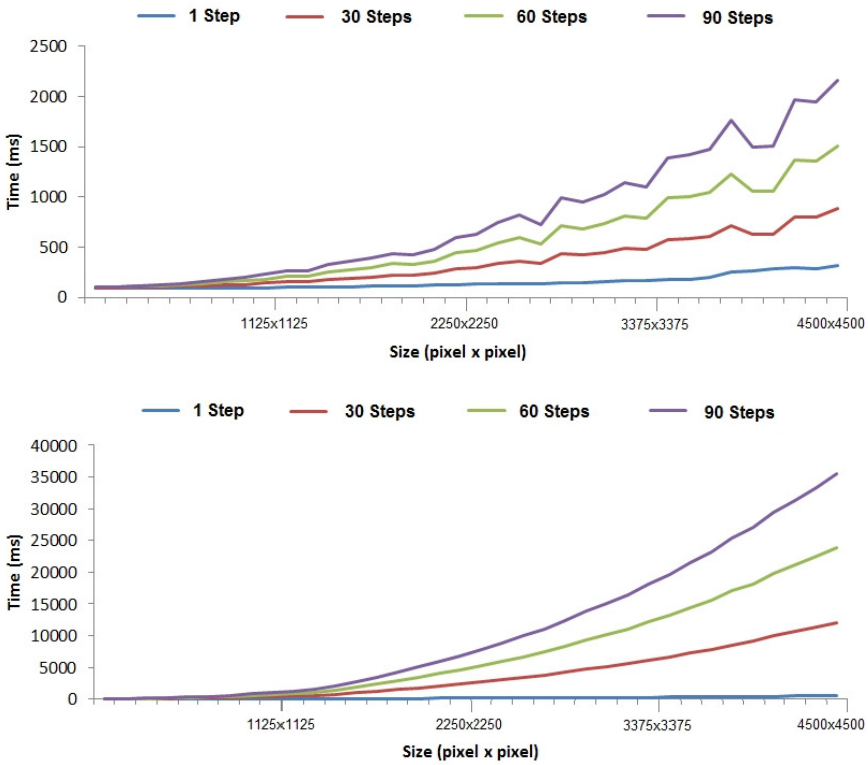


Fig. 3.10 Experimental time obtained for the Guo and Hall algorithm applied on 36 totally black images of $n \times n$ pixels, from $n = 125$ to $n = 4500$ with a regular increment of 125 pixels. Top image shows the time of our parallel implementation in CA. Bottom image shows the time for a sequential implementation.

bigger structure of the fingerprint. In many cases, the thickness of each line print is not important and the skeletonized image provides the same information as the original one.

We finish this section by showing the results of some experiments performed with our CA implementation. We have taken 36 totally black images of $n \times n$ pixels, from $n = 125$ to $n = 4500$ with a regular increment of 125 pixels of side. Figure 3.10 (top) shows the time in milliseconds of the application of our implementation of the Guo and Hall algorithm in CA for 1, 30, 60 and 90 steps in the skeletonizing process. Figure 3.10 (bottom) shows the same study for a sequential implementation of the algorithm. Finally, Figure 3.11 shows a comparison of our implementation vs. the sequential one by taking 90 steps in the Guo and Hall algorithm.

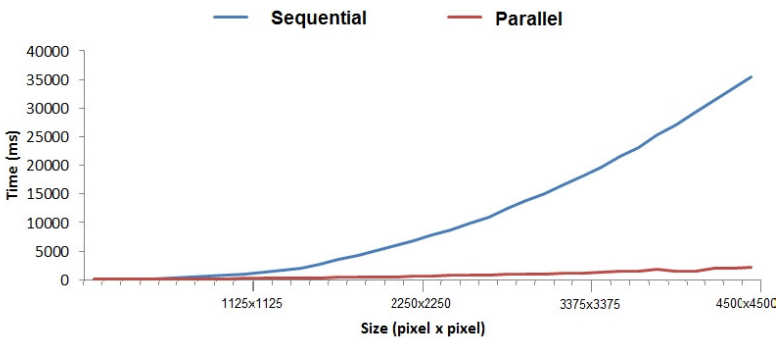


Fig. 3.11 A comparison of our implementation vs. the sequential one by taking 90 steps in the Guo and Hall algorithm

3.6 Conclusions

Computer vision is a hard task and a challenge in the next years. Classical sequential algorithms need to be revisited and adapted to the novel technologies, but the new developments also need the support of deep theoretical foundations. On the one hand, it is necessary to develop new algorithms in the framework of CA or other computational paradigms which can put a new light on classical problems. A deep study of the properties of such algorithms can improve their practical efficiency and accelerate their use in digital image industry.

On the other hand, the inherent features of CA for dealing with Image Analysis have found the limits of sequential computers. The theoretical parallel framework of CA could not be efficiently implemented in one-processor computers. In this way, the theoretical study should be also oriented to the most recent advances in hardware architecture. In this chapter, we have considered the implementation on GPUs. It is a good alternative for future implementations for CA algorithms, but many other possibilities should be explored. A strong link between the research on theoretical models and the development of new hardware architectures is the key for realistic answers to the challenges of the future in digital image areas.

Acknowledgements. MAGN acknowledges the support of the project TIN2012-37434 of the Ministerio de Economía y Competitividad of Spain.

References

1. Acerbi, L., Dennunzio, A., Formenti, E.: Conservation of some dynamical properties for operations on cellular automata. Theoretical Computer Science 410(38-40), 3685–3693 (2009)

2. Altuwaijri, M., Bayoumi, M.: A new thinning algorithm for Arabic characters using self-organizing neural network. In: 1995 IEEE International Symposium on Circuits and Systems, ISCAS 1995, vol. 3, pp. 1824–1827 (1995)
3. Ammann, C., Sartori Angus, A.: Fast thinning algorithm for binary images. Image Vision and Computing 3(2), 71–79 (1985)
4. Arcelli, C., di Baja, G.S.: Euclidean skeleton via centre-of-maximal-disc extraction. Image and Vision Computing 11(3), 163–173 (1993)
5. Attali, D., Boissonnat, J.D., Edelsbrunner, H.: Stability and Computation of Medial Axes - a State-of-the-Art Report. In: Mathematical Foundations of Scientific Visualization, Computer Graphics, and Massive Data Exploration, ch. 6, pp. 109–125. Springer, Heidelberg (2009)
6. di Baja, G.S., Thiel, E.: Skeletonization algorithm running on path-based distance maps. Image and Vision Computing 14(1), 47–57 (1996)
7. Baruch, O.: Line thinning by line following. Pattern Recognition Letters 8(4), 271–276 (1988)
8. Betel, H., Flocchini, P.: On the relationship between fuzzy and boolean cellular automata. Theoretical Computer Science 412(8-10), 703–713 (2011)
9. Biasotti, S., Attali, D., Boissonnat, J.D., Edelsbrunner, H., Elber, G., Mortara, M., Baja, G.S., Spagnuolo, M., Tanase, M., Veltkamp, R.: Skeletal structures. In: Floriani, L., Spagnuolo, M. (eds.) Shape Analysis and Structuring, Mathematics and Visualization, pp. 145–183. Springer, Heidelberg (2008)
10. Blum, H.: An associative machine for dealing with the visual field and some of its biological implications. In: Bernard, E.E., Kare, M.R. (eds.) Biological Prototypes and Synthetic Systems, vol. 1, pp. 244–260. Plenum Press, New York (1962), Proceedings of the 2nd Annual Bionics Symposium, held at Cornell University (1961)
11. Blum, H.: An associative machine for dealing with the visual field and some of its biological implications. In: Computer and Mathematical Sciences Laboratory, Electronics Research Directorate, Air Force Cambridge Research Laboratories, Office of Aerospace Research. United States Air Force (1962)
12. Cattaneo, G., Dennunzio, A., Margara, L.: Solution of some conjectures about topological properties of linear cellular automata. Theoretical Computer Science 325(2), 249–271 (2004), Theoretical Aspects of Cellular Automata
13. Chauhan, S.: Survey paper on training of cellular automata for image. International Journal of Engineering and Computer Science 2(4), 980–985 (2013)
14. Chen, Y.S.: The use of hidden deletable pixel detection to obtain bias-reduced skeletons in parallel thinning. In: Proceedings of the 13th International Conference on Pattern Recognition, vol. 2, pp. 91–95. IEEE Computer Society, Washington, DC (1996)
15. Chen, Y.S., Hsu, W.H.: Systematic approach for designing 2-subcycle and pseudo 1-subcycle parallel thinning algorithms. Pattern Recognition 22(3), 267–282 (1989)
16. Dinneen, G.P.: Programming pattern recognition. In: Proceedings of the Western Joint Computer Conference, AFIPS 1955 (Western), pp. 94–100. ACM, New York (1955)
17. Dufresne, T.E., Sarwal, A., Dhawan, A.P.: A gray-level thinning method for delineation and representation of arteries. Computerized Medical Imaging and Graphics 18(5), 343–355 (1994)
18. Favre, A., Keller, H.: Parallel syntactic thinning by recoding of binary pictures. Computer Vision, Graphics, and Image Processing 23(1), 99–112 (1983)
19. Gil Montoya, M., Garcia, I.: Implementation of parallel thinning algorithms on multicomputers: analysis of the work load balance. In: Proceedings of the Sixth Euromicro Workshop on Parallel and Distributed Processing, PDP 1998, pp. 257–263 (1998)
20. González, R.C., Woods, R.E.: Digital image processing. Pearson/Prentice Hall (2008)

21. Guo, Z., Hall, R.W.: Parallel thinning with two-subiteration algorithms. Communications of the ACM 32, 359–373 (1989)
22. Guo, Z., Hall, R.W.: Fast fully parallel thinning algorithms. CVGIP: Image Understanding 55, 317–328 (1992)
23. Hernandez, G., Herrmann, H.: Cellular-automata for elementary image-enhancement. Graphical Models and Image Processing 58(1), 82–89 (1996)
24. Heydorn, S., Weidner, P.: Optimization and performance analysis of thinning algorithms on parallel computers. Parallel Computing 17(1), 17–27 (1991)
25. Hilditch, C.: An application of graph theory in pattern recognition. Machine Intelligence 3, 325–347 (1968)
26. Holt, C., Stewart, A.: A parallel thinning algorithm with fine grain subtasking. Parallel Computing 10(3), 329–334 (1989)
27. Hongbin, P., Junali, C., Yashe, Z.: Fingerprint thinning algorithm based on mathematical morphology. In: 8th International Conference on Electronic Measurement and Instruments, ICEMI 2007, pp. 2–618–2–621 (2007)
28. Kirsch, R.A., Cahn, L., Ray, C., Urban, G.H.: Experiments in processing pictorial information with a digital computer. In: Papers and Discussions Presented at the December 9-13, Eastern Joint Computer Conference: Computers with Deadlines to Meet, IRE-ACM-AIEE 1957 (Eastern), pp. 221–229. ACM, New York (1958)
29. Lee, K.H., Cho, S.B., Choy, Y.C.: Automated vectorization of cartographic maps by a knowledge-based system. Engineering Applications of Artificial Intelligence 13(2), 165–178 (2000)
30. Liu, L.: 3D thinning on cell complexes for computing curve and surface skeletons. Washington University (2009)
31. Liu, L., Chambers, E.W., Letscher, D., Ju, T.: A simple and robust thinning algorithm on cell complexes. Computer Graphics Forum 29(7), 2253–2260 (2010)
32. Lü, H.E., Wang, P.S.P.: A comment on a fast parallel algorithm for thinning digital patterns. Communications of the ACM 29(3), 239–242 (1986)
33. Nedzved, A., Ilyich, Y., Ablameyko, S., Kamata, S.: Color thinning with applications to biomedical images. In: Skarbek, W. (ed.) CAIP 2001. LNCS, vol. 2124, pp. 256–263. Springer, Heidelberg (2001)
34. Pavlidis, T.: Algorithms for Graphics and Image Processing. Digital system design series. Computer Science Press (1982)
35. Peña-Cantillana, F., Berciano, A., Díaz-Pernil, D., Gutiérrez-Naranjo, M.A.: Parallel skeletonizing of digital images by using cellular automata. In: Ferri, M., Frosini, P., Landi, C., Cerri, A., Di Fabio, B. (eds.) CTIC 2012. LNCS, vol. 7309, pp. 39–48. Springer, Heidelberg (2012)
36. Rosenfeld, A.: A characterization of parallel thinning algorithms. Information and Control 29(3), 286–291 (1975)
37. Rosin, P.L.: Training cellular automata for image processing. IEEE Transactions on Image Processing 15(7), 2076–2087 (2006)
38. Rutovitz, D.: Pattern recognition. Journal of the Royal Statistical Society 129, 504–530 (1966)
39. Saeed, K., Tabedzki, M., Rybnik, M., Adamski, M.: K3M: A universal algorithm for image skeletonization and a review of thinning techniques. Applied Mathematics and Computer Science 20(2), 317–335 (2010)
40. de Saint Pierre, T., Milgram, M.: New and efficient cellular algorithms for image processing. CVGIP: Image Understanding 55(3), 261–274 (1992)
41. Siddiqi, K., Pizer, S.: Medial representations: mathematics, algorithms and applications. Computational imaging and vision. Springer (2008)

42. Smith, S.J., Bourgoin, M.O., Sims, K., Voorhees, H.L.: Handwritten character classification using nearest neighbor in large databases. IEEE Transactions on Pattern Analysis and Machine Intelligence 16(9), 915–919 (1994)
43. Suzuki, S., Abe, K.: Binary picture thinning by an iterative parallel two-subcycle operation. Pattern Recognition 20(3), 297–307 (1987)
44. Ye, Q.Z., Danielsson, P.E.: Inspection of printed circuit boards by connectivity preserving shrinking. IEEE Transactions on Pattern Analysis and Machine Intelligence 10(5), 737–742 (1988)
45. Zhang, T.Y., Suen, C.Y.: A fast parallel algorithm for thinning digital patterns. Communications of the ACM 27(3), 236–239 (1984)
46. Zhang, Y.Y., Wang, P.S.P.: A parallel thinning algorithm with two-subiteration that generates one-pixel-wide skeletons. In: Proceedings of the 13th International Conference on Pattern Recognition, vol. 4, pp. 457–461 (1996)

Chapter 4
Image Processing Algorithms Implementation Using Quantum Cellular Automata

Vassilios Mardiris and Vassilios Chatzis

Abstract. This chapter presents the implementation of Quantum-dot Cellular Automata (QCA) technology for image processing mathematical morphology operations. The basic concepts for both mathematical morphology and QCA technology are briefly described. QCA is a very promising emerging technology in the field of nanoelectronics that seems to suit well with image processing needs. The design of QCA circuits fundamental components are presented and the methodology that should be followed for a robust design of more complex circuits is recorded. Two QCA circuits for the implementation of morphological erosion and dilation are designed, simulated and tested. Results show that the QCA architecture provide better performance by exploiting parallel processing, ease of mask generation, better silicon-area utilization, maximization of clock speed and very low power consumption.

4.1 Introduction

Cellular Automata (CA) are models of physical systems, with local interactions in a discrete space and time [41]. In the literature, many published works have proven the efficiency of CA in providing solutions to scientific problems and simulating physical systems. Their ability to capture essential features of a system is based on the idea that the collective effect of simple locally interactive components can describe the global behaviour of a system [9, 69]. Moreover, CA can easily handle inhomogeneities and anisotropies as well as initial conditions and complex boundaries [60]. The CA approach is also compliant with the modern theory of unified space-time. In computer science is usually accepted that space corresponds to memory and time to processing unit. But, in CA, space and time are inseparably related to a CA cell, since memory corresponds to a CA cell state and processing unit to

Vassilios Mardiris · Vassilios Chatzis
Technological Educational Institute of Eastern Macedonia and Thrace, 65404 Kavala, Greece
e-mail: {mardiris,chatzis}@teikav.edu.gr

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_4, © Springer International Publishing Switzerland 2014

a CA local rule [55]. CA can also be considered as a Very-Large-Scale Integration (VLSI) architecture. Many special computing machines have been developed based on the CA architecture. Computational structures for VLSI realization, which have been developed based on CA architecture, are more suitable in terms of circuit design and layout, ease of mask generation, silicon-area utilization, and maximization of clock speed [54, 60]. Applications include image or video processing in several cases e.g. in [4, 25].

The VLSI technology is now a mature integrated circuit technology, that has made possible the development of smaller, faster, and cheaper special-purpose digital processing devices. This technology also contributed in the construction of digital systems, capable of performing the complex digital signal processing tasks that were usually too difficult or too expensive to be performed by analog circuitry. Today, many of the functions usually performed by analog means in the past, are realized by less expensive and more reliable digital hardware. VLSI design and implementation played a crucial role in turning theory into reality. Therefore, both signal processing and VLSI technologies were developed in parallel during the last decades [47, 48, 57, 63].

Hardware implementations of image and video processing algorithms by using VLSI technology, have been extensively used in the past and have been proven as one of the most viable solution for improving the performance of image and video processing systems. In such systems, non-linear image processing filters and mathematical morphology operators are usually used as parts of more sophisticated algorithms. Some examples of successful implementations are: a separable 2-D FIR filter for finite input matrix which has been implemented using VLSI [37]. A universal filter architecture that provides a multi-functional image processing solution for applications that require the output of nonlinear filters, such as edge preservation smoothing, noise filtering, and image segmentation [15]. A VLSI filter architecture for video noise reduction based on a nonlinear rational operator filter [49] and an implementation using Application Specific Integrated Circuits (ASICs) suitable for performing non-linear image processing operations [11].

Nowadays, electronic device sizes continue to shrink deeper into the nanometer regime but physical limitations of conventional electronics including power consumption, interconnect and lithography have become increasingly difficult to overcome. It is soon expected that nanoelectronic circuits will gradually replace the CMOS circuits. There are several emerging nanoelectronic technologies, which have been proposed by researchers during the last years. Carbon nanotubes, silicon nanowires, Quantum-dot Cellular Automata, single-electron transistors (SETs) and circuits, resonant tunnelling diodes, and single-molecule devices are some of the most promising emerging technologies as reported by ITRS [19]. The QCA computational scheme uses highly pipelined architectures and extremely high speeds. It does not use electric current flow to codify the information, but the positions of electrical charges in the interior of the QCA cells. An efficient design of circuits based on QCA technology would lead to the reduction of computational complexity and power consumption. Therefore, QCA is considered as one of the most promising emerging nanoelectronic technologies [28, 29, 44, 62].

Although QCA and CA as architectures share some common characteristics they are mostly different as it is described later in this chapter. A basic CA cell implementation using QCA has already been proposed at the literature [33]. The implemented QCA circuit is presenting much better characteristics compared to its conventional VLSI implementation in terms of circuit area, clock frequency and power consumption. According to these results it can be easily assumed that all CA systems which have been designed and implemented using conventional VLSI technology now can be implemented using QCA [32, 55]. These new QCA circuits will operate near 1 THz clock frequencies, they will cover 100 times (or more) less area than their previous implementations and they will present extremely low power consumptions [29].

In the following section, basic concepts of mathematical morphology will be presented, and a simple description of the quantum-dot cellular automata will follow in Sect. 4.3. The methodology that should be followed for a correct and efficient design of a QCA circuit will be given in Sect. 4.4. The QCA design of the morphological erosion and dilation operations will be presented in Sect. 4.5 as implementation examples. Finally, conclusions will be drawn in Sect. 4.6.

4.2 Mathematical Morphology

In recent years, the evolution of digital recording technology has made digital image and video applications an important aspect of daily life. But, in many cases and due to a variety of reasons, the quality of digital images and videos reduces to unsatisfactory levels. Common reasons for this corruption are the unconstrained camera motion, the uncontrolled indoor or outdoor environments, the existence of clutter and the exposure to noise during signal transmission. There are many algorithms proposed in the literature trying to improve the quality of corrupted images. Such algorithms are usually implemented in hardware in order to improve the performance of image and video processing systems and satisfy the real time processing requirements desired in many cases. In such systems, mathematical morphology operators and filters are extensively used as parts of more complex algorithms.

Mathematical morphology is a field of image processing that offers a unified approach and powerful tools applied to image and video processing applications [13, 51, 52], including thinning, thickening, skeletonising, shape extraction, segmentation, object detection and tracking, noise reduction, feature point selection, face recognition and verification and many more [14, 31, 36, 56]. There are also some more complex morphological algorithms such as interpolation of 3D binary objects [6], robust 2D and 3D object representation [5]. In other cases, morphological filters are applied as parts of more complicated algorithms in order to achieve better performance [21, 46].

The basic operations of mathematical morphology are: erosion, dilation, opening and closing. They are explicitly defined, and due to their simplicity they are used in many image analysis and computer vision problems. All basic operations take two sets of data as input: the input image and the structuring element. The simplest version of mathematical morphology is applied on binary images and uses binary

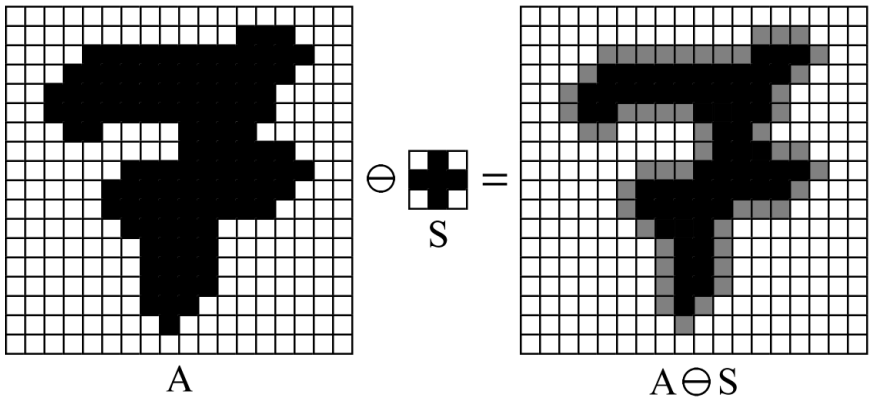


Fig. 4.1 The erosion of an object by a structuring element

structuring elements. It will be assumed that in binary images, white pixels represent background regions, while black pixels denote foreground, although in some implementations this convention is reversed.

The implementation details for all operations can be generalized, by considering that the output image is produced by applying a set operator (intersection, union, inclusion, complement) between the input image and the structuring element.

To compute a mathematical operation on a binary image using a structuring element, each of the pixels in the input image (usually called input pixel) should be considered in turn. For each pixel, we superimpose the structuring element over the input image, so that the origin of the structuring element (usually the central pixel) coincides with the input pixel position. The set operator applied on the image's and the structuring element's pixels values gradually produce the output image.

The most basic morphological operation is that of erosion. As it is shown in Fig. 4.1, suppose A is a binary object in the input image and S is a structuring element (only the black pixels). If S is placed with its origin pixel at every pixel of the input image (i, j) we denote it by $S(i, j)$. Then the erosion of A by S is defined to be the set of all pixel locations for which S placed at that pixel is contained within A. This is denoted $A \ominus S$ and is written as:

$$A \ominus S = \left\{ (i, j) : S_{(i,j)} \subset A \right\} \tag{4.1}$$

Fig. 4.1 presents the input image with the binary object A, the structuring element S which is a 3×3 cross and the eroded image $A \ominus S$ where the grey pixels are the eroded pixels that are subtracted from the object (in fact they are white).

Dilation can be defined as a complementary to the erosion operation, but an alternative definition that helps to understand how it is applied is the following. Let as assume that S' is the reflection of S. Obviously, $S' = S$ for symmetrical structuring elements. Then $A \oplus S$ contains all the pixels lying in any $S'(i, j)$ for which $(i, j) \in A$.

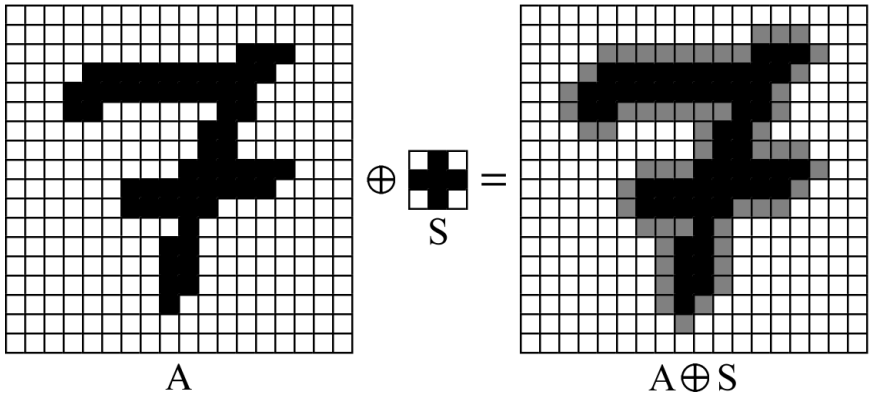


Fig. 4.2 The dilation of an object by a structuring element

So, the dilation is the union of the pixels, of a copy of S' at every pixel of the binary object A in the input image. This definition is written as:

$$A \oplus S = \bigcup_{(i,j)\in A} S'(i,j) \tag{4.2}$$

Fig. 4.2 presents another object A in an input image, the same structuring element S which is symmetrical and the dilated image $A \oplus S$ where the grey pixels are the dilated pixels that are considered as part of the output image (in fact they are black).

The design and simulation of morphological erosion and dilation with nanoelectronic circuits will be presented in Sect. 4.5.

4.3 Quantum-dot Cellular Automata

Single electron technology is defined as the capability to control the transport and the position of a single electron or a small number of electrons in nanometer structures. Significant research is being done on the field of modelling and simulation of SET devices and circuits [10, 68, 70], which is accompanied by the development of an efficient single electron circuit theory [16].

QCA is considered as a single electron technology. The conventional digital technologies use ranges of voltage or current to represent binary values "0" and "1". In contrast, QCA uses the position of electrons in quantum dots to represent the binary values. The information is stored as configurations of electron pairs in quantum dot arrays. The unit of information is kept in a QCA cell which is presented in Fig. 4.3. The QCA cell is the basic building block of QCA devices and it consists of four quantum dots in a square array coupled by tunnel barriers. The physical mechanisms for interactions between dots are the Coulomb interactions and quantum-mechanical tunnelling. Electrons are able to tunnel between the dots, but cannot leave the cell. If two mobile electrons are placed in the cell, in the ground state and in absence of

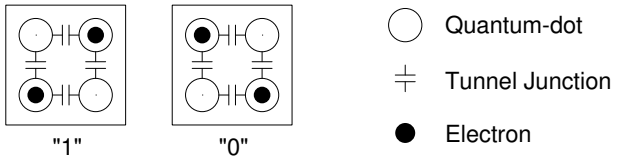


Fig. 4.3 The two possible charge configurations of a QCA cell

external electrostatic influence, Coulomb repulsion will force the electrons to dots on opposite corners [28, 29, 44, 62]. The two possible charge configurations are presented in Fig. 4.3 and correspond to binary "1" and "0".

In the following paragraphs, the implementations of the basic components of QCA circuits will be presented. The binary wire, the inverter and the three-input majority gate are fundamental components. The logic AND and OR gate are implemented using the majority gate.

The successive placement of QCA cells one after the other in one direction, acts like a wire and is usually called binary wire [28]. Fig. 4.4 presents the binary wire component, where the input A that is fed in the left, is transmitted towards the right direction to the output. In this example the input A equals to logic "0" as the electrons are placed in the upper left and down right dots. In the second cell the electrons, due to Coulomb repulsion, will have the same state as in the first cell. The same happens in all the cells that follow in the same direction, so the last cell (the output) will be in the same state "0" as the input. In the case that the input A changes to logic "1", the second cell will also change to "1", due to Coulomb repulsion. The same will happen successively to all cells and the output cell will also become "1" . So, in any case, the input is transmitted to the output and this function simulates the classical wire.

The three-input QCA majority gate is one of the most important processing elements in QCA circuits and its implementation was proposed in [62]. The majority gate is constructed with a cross structure. The logic function of the three-input majority gate is $M(A,B,C) = AB + AC + BC$ and the output has to be the same as the majority of the three inputs A,B,C. The implementation is composed of five cells as shown in Fig. 4.5. Here $A = 1$, $B = 0$ and $C = 0$. The electrostatic forces between the electrons of the three input cells constrain the electrons of the centric cell to get the state of logic "0". Generally, the state of the centric cell is set by electrostatic forces from the logic state of the majority of the inputs. This state is then transmitted to the output cell at the right.

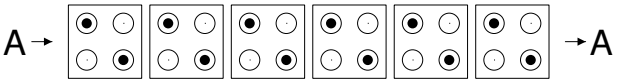


Fig. 4.4 The QCA binary wire

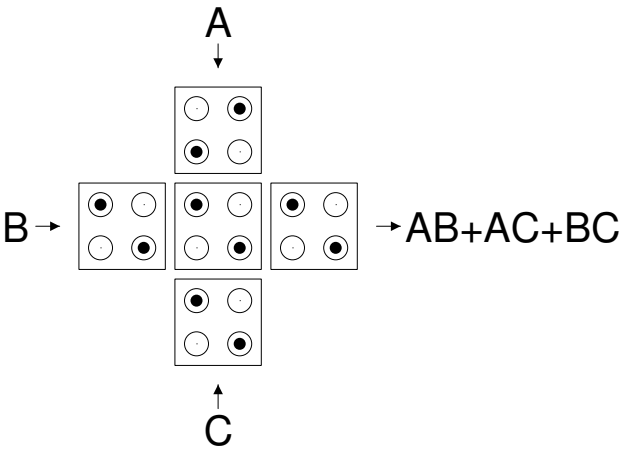


Fig. 4.5 The implementation of the majority gate using QCA cells

The inverter is constructed with a fork structure [62] and its design is shown in Fig. 4.6. The input A (in this example $A = 0$) is fed at the left cell and is transmitted by electrostatic forces between the electrons to the upper and the lower cells of the design. The second last cell is properly placed in order to change the state of A. In this example the second last cell, due to the electrostatic forces of its two previous cells, will get the state of "1". In general, it will be $\overline{A}$ which is the inversion of the input A.

The AND and OR logic gates can be implemented as special cases of the majority gate. When in a majority gate one of the inputs is steadily set to 0, then this gate operates as the AND gate. As it is shown in Fig. 4.7 the design has the remaining two inputs A and B and the outcome will always be their logic AND, symbolized as AB.

When in a majority gate one of the inputs is steadily set to 1, then this gate operates as the OR gate. As it is shown in Fig. 4.8 the design has the remaining two inputs A and B and the outcome will always be their logic OR, symbolized as $A + B$.

More logic gates and circuits have been proposed in the literature such as XOR gates [62], bit-serial adder, full adder [7, 12, 24, 39], multiplier [7], multiplexer [34,

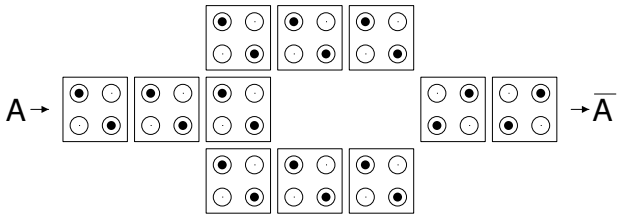


Fig. 4.6 The implementation of the inverter using QCA cells

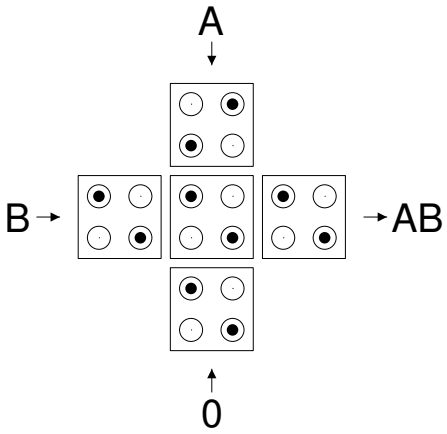


Fig. 4.7 The logic AND gate designed using QCA cells

35, 65], flip-flop [17, 38, 53], arithmetic logic units (ALUs) [43, 59] and serial or parallel memories [35, 58, 64, 65].

By using QCA technology, the integration can reach densities of 10^{12} devices/cm^2 and the circuit switching frequency can be close to terahertz [19]. Although, the main advantages of QCA designs are the improvement that offer in size, speed and power consumption, there are also some other characteristics that give advantage to QCA technology, such as the ability to cross wires in a plane [62] and a new computation and information representation method which is referred to as processing-in-wire [28]. Therefore, QCA is a new opportunity for the design of highly parallel algorithms and architectures.

But, there are also some problems in the realization of QCA technology. The stability of a QCA circuit is based on the assumption that the system falls to the ground

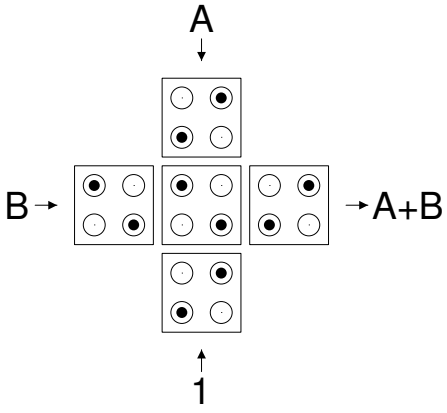


Fig. 4.8 The logic OR gate designed using QCA cells

state every time is excited by the inputs. However, this is not always guaranteed so in this case the system settles in a metastable state, affecting the functionality of the design. This problem can be solved by adiabatic switching [44].

Experiments have shown that QCA cells created by using metallic dots operate at cryogenic temperatures. By contrast, other realizations of QCA cells by using either single molecules or nano-magnets can be more promising for room temperature operation [18, 20, 26, 27, 30].

4.4 Methodology

The methodology that should be followed for the implementation of a new circuit, consists of three basic steps: Analysis, Circuit Design and Simulation. These steps are described in the following paragraphs.

4.4.1 Analysis

During the step of Analysis, any algorithm should be split into smaller procedures, taking into account the advantages and the requirements of the QCA technology. It is well known that an algorithm can be implemented in several ways. Usually, the better implementation is the one that minimizes the computational time and/or the area covered by the QCA circuit.

The research should focus on analyzing an algorithm to provide easier and better QCA design in terms of space, stability and time. This can be achieved when the advantages of QCA are taking into account during the analysis step. For example an advantage is the fact that some operations such as the majority gate are easily implemented by using QCA. So, the majority gate is a module that could be extensively used for the analysis of several algorithms.

4.4.2 Circuit Design

The circuit design step is the major part of the research work. Designing with QCA is a very interesting but challenging work for the following reasons. Theoretically, all Boolean logic functions can be implemented on a QCA combinational circuit, by properly placing QCA cells so that their coulombic interactions produce the requested output [61]. But, in QCA circuit design it is very important to produce a circuit characterized by advanced stability [2]. Many times small QCA designs succeed in simulation when they are not connected with other designs, but fail when they are used as parts of a larger design. In other cases the designers tweak simulation model parameters in order to manage their circuits to succeed in simulation. As a result many QCA designs are not robust and consequently not operational.

There are some design concerns that should be kept into account to increase the robustness of the QCA circuits [8, 22]. When generating QCA circuit designs, a significant effort should be made to keep the length of a wire within a given

clocking zone to a minimum. The reason to do this is that as wire length grows, the probability that a QCA cell will switch successfully decreases, in proportion to the distance of this particular cell from a clamped (frozen) input at the beginning of the wire. Consequently, for shorter wires, it is most likely that all cells making up the wire will switch successfully. Additionally, short wire lengths result in circuit operation in higher clock rates [23]. That is because, before the cells in a given zone can change phase, every cell within this zone must make appropriate polarization changes that need time. Therefore, the longer the wire is, the longer time is needed for a signal to propagate along its length. In general, the maximum wire length that can successfully propagate a signal from one end to the other is twenty eight (28) 90-degree cells, or twenty seven (27) 45-degree cells, acquired by simulation on an experimental setup [22].

At nonzero Kelvin temperatures, higher operating temperatures lead to higher thermal fluctuations which increase the probability of a kink occuring (in which a QCA cell aligns differently from its expected polarization) [1]. It has been proven [44, 66] that the maximum QCA line length for kink-free operation is given by:

$$N \leq e^{\frac{E_k}{k_b T}} \tag{4.3}$$

where E_k represents the energy required for a QCA cell to encounter a kink, k_b is the Boltzmann constant and T is the operating temperature. For the proposed designs the maximum wire length would be kept under a desired value so that it can operate to higher possible temperatures.

It is also important to keep the area of the clocking zones to a minimum. This has to be done to increase uniformity and consequently manufacturability. Furthermore, by keeping the area to minimum, wire lengths are also kept to a minimum and consequently the circuit can operate at higher temperatures with no kink occurrence. On the other hand, if we shrink a phase block to a single cell this block will cause functional failures. So to make up with this vulnerability we must set a minimum of 2 cells for the length of a phase block.

Another problem arising with complex QCA designs is that usually large amounts of white space-wasted area is left between cells [40, 42]. In the proposed designs the use of clocking zones with many cells should be avoided and consequently the QCA cells should be uniformly distributed into the clocking zones. The clocking zones should also be designed in such a way so that the uncovered areas would be as small as possible. Finally, to confront this problem, the total area covered by QCA cells should be minimized by keeping the distance between binary wires as close as possible according to QCA design rules [22, 23]. As a result, the proposed architecture would not leave large amount of area unused.

In order to obtain a successful and stable design the following issues should be considered:

- keep the length of the wires to a minimum
- keep the area of the clocking zones to a minimum
- set the minimum of 2 cells for the length of a phase block
- the uncovered areas would be as small as possible

Another desirable characteristic for the QCA designs is modularity. The design procedure should provide the possibility for the produced circuit to be modular. Although this procedure is based on the division of the circuit functionality in simpler pieces, the reverse procedure of putting them all together is not really straightforward. So, any particular QCA block, must be designed very carefully in order to produce a functional final circuit. This modular approach in the design methodology of QCA circuits, includes a two step procedure. First a number of basic building block circuits are designed and simulated, and then these blocks are properly assembled to produce the requested designs.

More specifically, when assembling two neighbouring blocks, the relative position of the blocks and the flow direction of the signals reaching the block edges must be carefully handled. In order for these signals to propagate correctly from one block to another, the QCA cells which are involved with the inter-block signal propagation must be controlled by successive clocks, so that the signal will flow in the correct direction. Additionally, when assembling two elementary building blocks, the output signals of every block must be synchronized with the output signals of any other block in the circuit to avoid wrong results for the case inputs from different blocks are not received simultaneously. Consequently, all blocks must produce the same amount of delay in clock phases, in order to be synchronized with each other.

4.4.3 Simulation

All circuits designed according to the previous constraints should be simulated using the simulator of the well known QCADesigner tool. QCADesigner is a QCA layout and simulation tool developed at the University of Calgary [67] and has been used extensively for the design and simulation of QCA circuits. First, the functionality of each elementary building block is verified and finally the whole circuit composed of these blocks is simulated. The circuits are redesigned in case of functionality problems and the new circuits should be re-simulated. The last procedure is repeated until the whole circuit presents the requested behaviour. A large number of simulation test vectors are produced for every circuit in order a full functionality test to be carried out. Many simulation sessions using different simulation parameters should be done in order to arise any possible circuit problems related with the simulation procedure. After the functional verification of each circuit an estimation of QCA circuit area and the maximum operational clock frequency will be calculated.

4.5 QCA Implementation of Morphological Operations

As it was presented in previous sections, the implementations of logic gates using QCA are quite simple and regular. QCA seems to be an ideal hardware for digital signal processing applications, since it is well suited for the implementation of many nonlinear signal and image processing algorithms [3, 45, 50]. It would be very interesting to discover how existing algorithms could be realized using this

new technology. The logic of using the majority gate, and the uniformity of wires and processing elements, modify the way of thinking in designing circuits. For example, the majority gate could simplify many nonlinear filters since it is, by itself, a binary three point median.

In the rest of this section, the image processing morphological operations of erosion and dilation, described in Sect. 4.2, will be implemented by using QCA technology, applying the methodology described in Sect. 4.4 and utilizing the simple QCA circuit components presented in Sect. 4.3.

For the hardware implementation let us assume that black pixels correspond to logic "1" and that white pixels correspond to logic "0". It is also assumed that the inputs are binary images with white background and black objects and that the structuring element is a 3×3 pixels cross with origin its centric pixel, as they are presented in Fig. 4.1 or 4.2. Following the rules presented in methodology in the designs that follow, the wire lengths are less than 10 cells, the length of phase blocks are 2 cells or more and the areas of clocking zones and uncovered areas have been kept as small as possible.

4.5.1 QCA Implementation of Morphological Erosion

In the following, the QCA implementation of morphological erosion operation will be described. By using the cross 3×3 pixels structuring element, the procedure that applies the erosion can be simplified to the following steps. First, superimpose the structuring element over every 3×3 region of the image. Then, if all the black pixels of the structuring element coincide with black pixels on the input image, paint black the pixel of the output image that corresponds to the structuring element origin. In other cases the pixels of the output image are left white.

This operation can be designed in hardware by implementing a 5-input AND gate as shown in Fig. 4.9, applying as inputs (i1 to i5) the values of the five input image pixels that coincides to the five structuring element black pixels. In this figure the "-1" denotes the polarization of the fixed cell that corresponds to logic "0".

As it was presented in the previous section, a 2-input AND gate in QCA is realized by clamping one input of the 3-input majority gate to logic "0". The realization of the 5-input AND gate is made by the proper connection of four 2-input AND gates. In the design the maximum wire length in a clocking zone is kept to 5 cells and the minimum distance between to wires is 2 cells.

The output of the 5-input AND gate will be logic "1" only if all the inputs are set to logic "1" and the state of the output cell will define the value of the pixel of the output image with coordinates that corresponds to the coordinates of the structuring element origin pixel. So, if all the pixels of the input image (which coincides with the structuring element) are black, then the pixel of the origin on the output image will be black. In other cases, the output will be logic "0", so the output pixel will be white.

This QCA design was simulated and tested with the use of the QCADesigner tool. According to this tool, the design consists of 35 cells covering an area of

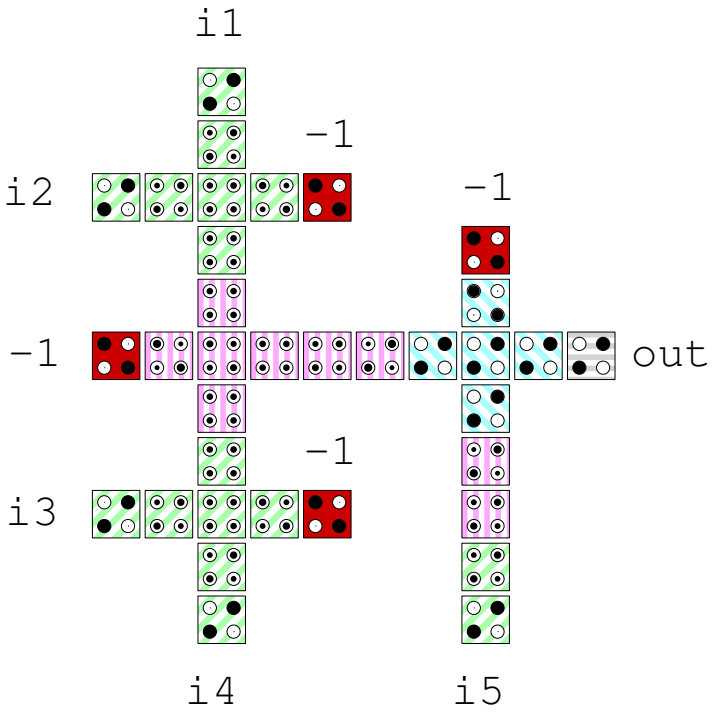


Fig. 4.9 The QCA circuit for morphological erosion

198×218 nm^2 that is approximately 0.043 μm^2. It is covering an orthogonal region of 10×11 grids and the ratio of the area covered by QCA cells to the overall orthogonal region area of the layout is 0.318. Fig. 4.10 presents the simulation results of the QCA morphological erosion operation, which confirms the correctness of the design. As it is shown all possible value combinations of 5 inputs are applied to the circuit. The output is set to logic "1" only when all inputs stay at logic "1" otherwise is set to logic "0" indicating the AND gate behaviour.

4.5.2 QCA Implementation of Morphological Dilation

The QCA implementation of morphological dilation operation that will be described in the following uses a cross 3×3 pixels structuring element. The procedure that applies this dilation can be simplified as follows. First, superimpose the structuring element over every 3×3 region of the image. Then, if the black pixels of the structuring element coincides with one or more black pixels on the input image, paint black the pixel of the output image that corresponds to the structuring element origin. In other cases the pixels of the output image are left white.

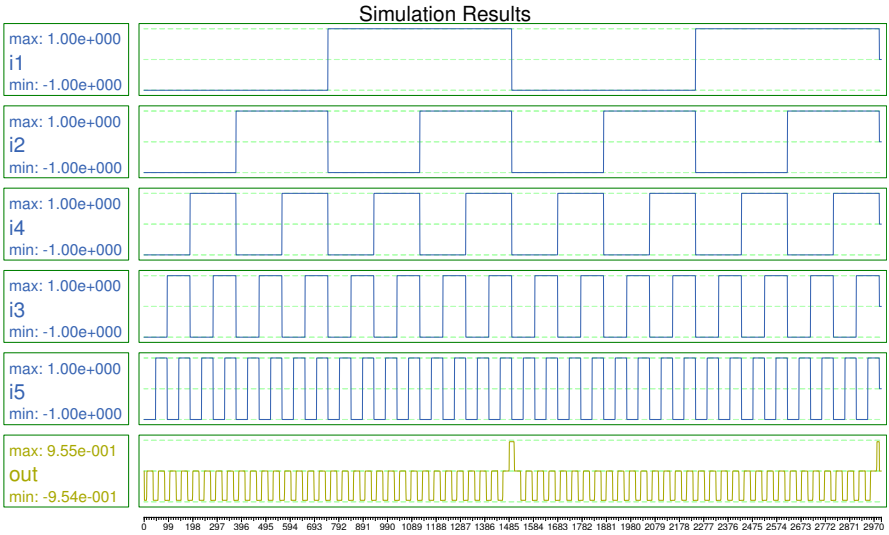


Fig. 4.10 Simulation of the QCA circuit for morphological erosion

This operation can be designed on hardware by implementing a 5-input OR gate as shown in Fig. 4.11, applying as inputs (i1 to i5) the values of the five input image pixels that coincides to the five structuring element black pixels. In this figure the "1" denotes the polarization of the fixed cell that corresponds to logic "1".

The realization of the 5-input OR gate is made by the proper connection of four 2-input OR gates, which are realized by clamping one input of the 3-input majority gate to logic "1". In the design the maximum wire length in a clocking zone is kept to 5 cells and the minimum distance between to wires is 2 cells.

The output of the 5-input OR gate will be logic "1" if one or more of the inputs are set to logic "1". So, if one or more of the pixels of the input image (which coincides with the structuring element) are black, then the pixel of the origin on the output image will be black. In other cases, the output will be logic "0", so the output pixel will be white.

According to QCADesigner tool the design consists of 35 cells covering an area of 198×218 nm^2 that is approximately 0.043 μm^2. It is covering an orthogonal region of 10×11 grids and the ratio of the area covered by QCA cells to the overall orthogonal region area of the layout is 0.318.

Fig. 4.12 presents the simulation results of the QCA morphological dilation operation, which confirms the correctness of the design. As it is shown, all possible value combinations of 5 inputs are applied to the circuit. The output is set to logic "1" when one or more of the inputs stay at logic "1" otherwise is set to logic "0" indicating the OR gate behaviour.

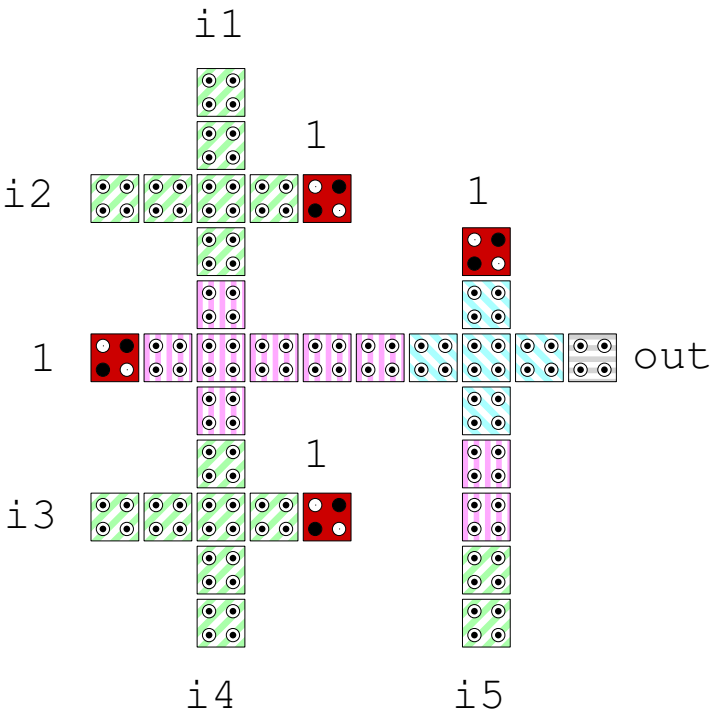


Fig. 4.11 The QCA circuit for morphological dilation

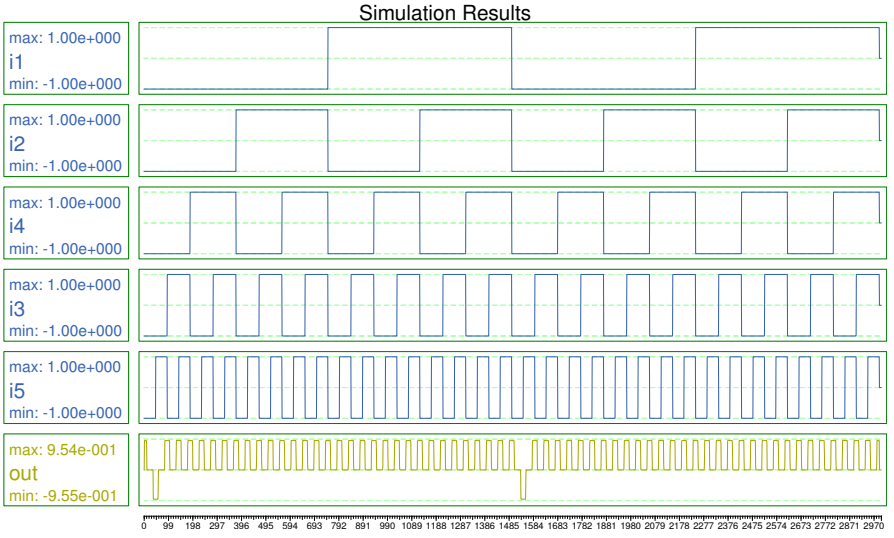


Fig. 4.12 Simulation of the QCA circuit for morphological dilation

4.6 Conclusions

This chapter presents the design and simulation of the dilation and erosion morphological operations on binary images with QCA technology. The new QCA implementations provide parallel processing, high circuit performance, very low dimension and very low power consumption compared to similar implementations with conventional VLSI technology.

These advantages make the proposed implementations useful for image processing systems applied on mobile or autonomous devices where real time processing and low power consumption is needed. The image processing algorithms considered in this chapter are limited to basic algorithms which are applied only on binary images, but further research could cope with more complicated mathematical morphology algorithms.

Moreover, image and video applications require intensive computations since the processing usually has to be applied to each image pixel and in some cases recursively to the whole image. For example, nonlinear filtering operations could be designed by using advanced QCA circuits, including all order statistics filters such as medians, L-filters, ranked-order statistic filters and stack filters. Performing the required large number of comparisons and sorting operations, which are necessary in such applications, in real time with a conventional sequential digital system would require extremely high clock speeds, leading to high power consumption. A more efficient design based on QCA technology would eventually lead to the reduction of computational complexity and power consumption.

References

1. Bhanja, S., Sarkar, S.: Thermal switching error versus delay tradeoffs in clocked QCA circuits. IEEE Transactions on Very Large Scale Integration (VLSI) Systems 16(5), 528–541 (2008)
2. Bubna, M., Roy, S., Shenoy, N., Mazumdar, S.: A layout-aware physical design method for constructing feasible QCA circuits. In: Proceedings of the ACM Great Lakes Symposium on VLSI, GLSVLSI, pp. 243–248 (2008)
3. Cardenas-Barrera, J.L., Plataniotis, K.N., Venetsanopoulos, A.N.: QCA implementation of a multichannel filter for image processing. Mathematical Problems in Engineering 8(1), 87–99 (2002)
4. Chatzichristofis, S.A., Mitzias, D.A., Sirakoulis, G.C., Boutalis, Y.S.: A novel cellular automata based technique for visual multimedia content encryption. Optics Communications 283(21), 4250–4260 (2010)
5. Chatzis, V., Pitas, I.: A generalized fuzzy mathematical morphology and its application in robust 2-D and 3-D object representation. IEEE Transactions on Image Processing 9(10), 1798–1810 (2000)
6. Chatzis, V., Pitas, I.: Interpolation of 3-D binary images based on morphological skeletonization. IEEE Transactions on Medical Imaging 19(7), 699–710 (2000)
7. Cho, H., Swartzlander Jr., E.E.: Adder and multiplier design in quantum-dot cellular automata. IEEE Transactions on Computers 58(6), 721–727 (2009)

8. Choi, M., Patitz, Z., Jin, B., Tao, F., Park, N., Choi, M.: Designing layout-timing independent quantum-dot cellular automata (QCA) circuits by global asynchrony. Journal of Systems Architecture 53(9), 551–567 (2007)
9. Feynman, R.E.: Simulating physics with computers. International Journal of Theoretical Physics 21(6), 467–488 (1982)
10. Fonseca, L.R.C., Korotkov, A.N., Likharev, K.K., Odintsov, A.A.: A numerical study of the dynamics and statistics of single electron systems. Journal of Applied Physics 78(5), 3238–3251 (1995)
11. Gasteratos, A., Andreadis, I.: Non-linear image processing in hardware. Pattern Recognition 33(6), 1013–1021 (2000)
12. Gladshtein, M.: Quantum-dot cellular automata serial decimal adder. IEEE Transactions on Nanotechnology 10(6), 1377–1382 (2011)
13. Haralick, R.M., Sternberg, S.R., Zhuang, X.: Image analysis using mathematical morphology. IEEE Transactions on Pattern Analysis and Machine Intelligence PAMI-9(4), 532–550 (1987)
14. Haris, K., Efstratiadis, S.N., Maglaveras, N., Katsaggelos, A.K.: Hybrid image segmentation using watersheds and fast region merging. IEEE Transactions on Image Processing 7(12), 1684–1699 (1998)
15. Hernandez, O.J., Keohane, T., Steponanko, J.: A combined VLSI architecture for nonlinear image processing filters. In: Conference Proceedings - IEEE SOUTHEASTCON, vol. 2006, pp. 261–266 (2006)
16. Hoekstra, J.: On circuit theories for single-electron tunneling devices. IEEE Transactions on Circuits and Systems I: Regular Papers 54(11 SPEC. ISS.), 2353–2359 (2007)
17. Huang, J., Momenzadeh, M., Lombardi, F.: Analysis of missing and additional cell defects in sequential quantum-dot cellular automata. Integration, the VLSI Journal 40(4), 503–515 (2007)
18. Imre, A., Csaba, G., Ji, L., Orlov, A., Bernstein, G.H., Porod, W.: Majority logic gate for magnetic quantum-dot cellular automata. Science 311(5758), 205–208 (2006)
19. International Technology Roadmap for Semiconductors: Emerging Research Devices (2007), www.itrs.net
20. Jiao, J., Long, G.J., Grandjean, F., Beatty, A.M., Fehlner, T.P.: Building blocks for the molecular expression of quantum cellular automata. isolation and characterization of a covalently bonded square array of two ferrocenium and two ferrocene complexes. Journal of the American Chemical Society 125(25), 7522–7523 (2003)
21. Kim, C.: Segmenting a low-depth-of-field image using morphological filters and region merging. IEEE Transactions on Image Processing 14(10), 1503–1511 (2005)
22. Kim, K., Wu, K., Karri, R.: Towards designing robust QCA architectures in the presence of sneak noise paths. In: Proceedings -Design, Automation and Test in Europe, DATE 2005, vol. II, pp. 1214–1219 (2005)
23. Kim, K., Wu, K., Karri, R.: Quantum-dot cellular automata design guideline. IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences E89-A(6), 1607–1614 (2006)
24. Kim, K., Wu, K., Karri, R.: The robust QCA adder designs using composable qca building blocks. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 26(1), 176–183 (2007)
25. Konstantinidis, K., Sirakoulis, G.C., Andreadis, I.: Design and implementation of a fuzzy-modified ant colony hardware structure for image retrieval. IEEE Transactions on Systems, Man and Cybernetics Part C: Applications and Reviews 39(5), 520–533 (2009)
26. Lent, C.S., Isaksen, B.: Clocked molecular quantum-dot cellular automata. IEEE Transactions on Electron Devices 50(9), 1890–1896 (2003)

27. Lent, C.S., Isaksen, B., Lieberman, M.: Molecular quantum-dot cellular automata. Journal of the American Chemical Society 125(4), 1056–1063 (2003)
28. Lent, C.S., Tougaw, P.D.: Lines of interacting quantum-dot cells: A binary wire. Journal of Applied Physics 74(10), 6227–6233 (1993)
29. Lent, C.S., Tougaw, P.D., Porod, W., Bernstein, G.H.: Quantum cellular automata. Nanotechnology 4(1), 49–57 (1993)
30. Manimaran, M., Snider, G.L., Lent, C.S., Sarveswaran, V., Lieberman, M., Li, Z., Fehlner, T.P.: Scanning tunneling microscopy and spectroscopy investigations of QCA molecules. Ultramicroscopy 97(1-4), 55–63 (2003)
31. Maragos, P., Schafer, R.W.: Morphological filters - Part I: Their set - theoretic analysis and relations to linear shift - invariant filters. IEEE Transactions on Acoustics, Speech, and Signal Processing ASSP-35(8), 1153–1169 (1987)
32. Mardiris, V.A.: Design and simulation of quantum cellular automata nanoelectronic circuits. PhD thesis, Democritus University of Thrace (2011)
33. Mardiris, V.A., Karafyllidis, I.G.: Universal cellular automaton cell using quantum cellular automata. Electronics Letters 45(12), 607–609 (2009)
34. Mardiris, V.A., Karafyllidis, I.G.: Design and simulation of modular 2n to 1 quantum-dot cellular automata (QCA) multiplexers. International Journal of Circuit Theory and Applications 38(8), 771–785 (2010)
35. Mardiris, V.A., Karafyllidis, I.G.: Design and simulation of modular quantum-dot cellular automata multiplexers for memory accessing. Journal of Circuits, Systems and Computers 19(2), 349–365 (2010)
36. Meyer, F., Beucher, S.: Morphological segmentation. Journal of Visual Communication and Image Representation 1(1), 21–46 (1990)
37. Mohanty, B.K., Meher, P.K.: New scan method and pipeline architecture for VLSI implementation of separable 2-D FIR filters without transposition. In: IEEE Region 10 Annual International Conference, Proceedings/TENCON (2008)
38. Momenzadeh, M., Huang, J., Lombardi, F.: Defect characterization and tolerance of QCA sequential devices and circuits. In: Proceedings - IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems, pp. 199–207 (2005)
39. Navi, K., Farazkish, R., Sayedsalehi, S., Rahimi Azghadi, M.: A new quantum-dot cellular automata full-adder. Microelectronics Journal 41(12), 820–826 (2010)
40. Neto, O.P.V., Pacheco, M.A.C., Hall Barbosa, C.R.: Neural network simulation and evolutionary synthesis of QCA circuits. IEEE Transactions on Computers 56(2), 191–201 (2007)
41. Neumann, J.V.: Theory of Self-Reproducing Automata. University of Illinois Press, Champaign (1966)
42. Niemier, M.T., Kogge, P.M.: Problems in designing with QCAs: Layout = timing. International Journal of Circuit Theory and Applications 29(1), 49–62 (2001)
43. Niemier, M.T., Kontz, M.J., Kogge, P.M.: Design of and design tools for a novel quantum dot based microprocessor. In: Proceedings - Design Automation Conference, pp. 228–232 (2000)
44. Orlov, A.O., Amlani, I., Bernstein, G.H., Lent, C.S., Snider, G.L.: Realization of a functional cell for quantum-dot cellular automata. Science 277(5328), 928–930 (1997)
45. Panagiotopoulos, F.K., Mardiris, V.A., Chatzis, V.: Quantum–dot cellular automata design for median filtering and mathematical morphology operations on binary images. In: Sirakoulis, G.C., Bandini, S. (eds.) ACRI 2012. LNCS, vol. 7495, pp. 554–564. Springer, Heidelberg (2012)
46. Pesaresi, M., Benediktsson, J.A.: A new approach for the morphological segmentation of high-resolution satellite imagery. IEEE Transactions on Geoscience and Remote Sensing 39(2), 309–320 (2001)

47. Pirsch, P., Stolberg, H.J.: VLSI implementations of image and video multimedia processing systems. IEEE Transactions on Circuits and Systems for Video Technology 8(7), 878–891 (1998)
48. Rabaey, J.M., Gass, W., Brodersen, R., Nishitani, T.: VLSI design and implementation fuels the signal-processing revolution: The design and implementation of signal-processing systems technical committee. IEEE Signal Processing Magazine 15(1), 22–37 (1998)
49. Saponara, S., Fanucci, L., Terreni, P.: Design of a low-power VLSI macrocell for nonlinear adaptive video noise reduction. Eurasip Journal on Applied Signal Processing 2004(12), 1921–1930 (2004)
50. Sen, B., Anand, A.S., Adak, T., Sikdar, B.K.: Thresholding using quantum-dot cellular automata. In: 2011 International Conference on Innovations in Information Technology, IIT 2011, pp. 356–360 (2011)
51. Serra, J.: Image analysis and mathematical morphology: Theoretical advances. Image Analysis and Mathematical Morphology. Academic Press (1988)
52. Serra, J.: Image Analysis and Mathematical Morphology. Acad. Press (1993)
53. Shamsabadi, A.S., Ghahfarokhi, B.S., Zamanifar, K., Movahedinia, N.: Applying inherent capabilities of quantum-dot cellular automata to design: D flip-flop case study. Journal of Systems Architecture 55(3), 180–187 (2009)
54. Sirakoulis, G.C., Karafyllidis, I., Thanailakis, A.: A CAD system for the construction and VLSI implementation of cellular automata algorithms using VHDL. Microprocessors and Microsystems 27(8), 381–396 (2003)
55. Sirakoulis, G.C., Karafyllidis, I., Thanailakis, A., Mardiris, V.: A methodology for VLSI implementation of cellular automata algorithms using VHDL. Advances in Engineering Software 32(3), 189–202 (2000)
56. Soille, P.: Morphological Image Analysis: Principles and Applications. Springer (2010)
57. Strauss, W.: Digital signal processing. IEEE Signal Processing Magazine 17(2), 52–56 (2000)
58. Taskin, B., Hong, B.: Improving line-based QCA memory cell design through dual phase clocking. IEEE Transactions on Very Large Scale Integration (VLSI) Systems 16(12), 1648–1656 (2008)
59. Teja, V.C., Polisetti, S., Kasavajjala, S.: QCA based multiplexing of 16 arithmetic and logical subsystems-a paradigm for nano computing. In: 3rd IEEE International Conference on Nano/Micro Engineered and Molecular Systems, NEMS, pp. 758–763 (2008)
60. Toffoli, T.: Cellular automata as an alternative to (rather than an approximation of) differential equations in modeling physics. Physica D: Nonlinear Phenomena 10(1-2), 117–127 (1984)
61. Tougaw, P.D.: A device architecture for computing with quantum dots. Proceedings of the IEEE 85(4), 541–557 (1997)
62. Tougaw, P.D., Lent, C.S.: Logical devices implemented using quantum cellular automata. Journal of Applied Physics 75(3), 1818–1825 (1994)
63. Tseng, P.C., Chang, Y.C., Huang, Y.W., Fang, H.C., Huang, C.T., Chen, L.G.: Advances in hardware architectures for image and video coding - a survey. Proceedings of the IEEE 93(1), 184–197 (2005)
64. Vankamamidi, V., Ottavi, M., Lombardi, F.: A line-based parallel memory for QCA implementation. IEEE Transactions on Nanotechnology 4(6), 690–698 (2005)
65. Vankamamidi, V., Ottavi, M., Lombardi, F.: A serial memory by quantum-dot cellular automata (QCA). IEEE Transactions on Computers 57(5), 606–618 (2008)
66. Vankamamidi, V., Ottavi, M., Lombardi, F.: Two-dimensional schemes for clocking/timing of QCA circuits. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 27(1), 34–44 (2008)

67. Walus, K., Dysart, T.J., Jullien, G.A., Budiman, R.A.: QCADesigner: A rapid design and simulation tool for quantum-dot cellular automata. IEEE Transactions on Nanotechnology 3(1 SPEC. ISS.), 26–31 (2004)
68. Wang, J.M., Sukhwani, B., Padmanabhan, U., Ma, D., Sinha, K.: Simulation and design of nanocircuits with resonant tunneling devices. IEEE Transactions on Circuits and Systems I: Regular Papers 54(6), 1293–1304 (2007)
69. Wolfram, S.: Theory and Applications of Cellular Automata: Including Selected Papers, 1983-1986. Advanced Series on Complex Systems. World Scientific Publishing Company, Incorporated (1986)
70. Zardalidis, G., Karafyllidis, I.G.: SECS: A new single-electron-circuit simulator. IEEE Transactions on Circuits and Systems I: Regular Papers 55(9), 2774–2784 (2008)

Chapter 5
Edge Detection Using Cellular Automata

Paul L. Rosin and Xianfang Sun

Abstract. Edge detection has been a long standing topic in image processing, generating hundreds of papers and algorithms over the last 50 years. Likewise, the topic has had a fascination for researchers in cellular automata, who have also developed a variety of solutions, particularly over the last ten years. CA based edge detection has potential benefits over traditional approaches since it is computationally efficient, and can be tuned for specific applications by appropriate selection or learning of rules. This chapter will provide an overview of CA based edge detection techniques, and assess their relative merits and weaknesses. Several CA based edge detection methods are implemented and tested to enable an initial comparison between competing approaches.

5.1 Introduction

Edge detection is a fundamental tool for computer vision systems. The original use of boundaries detected in a scene was for object detection, as they provide a set of features suitable for model matching. However, edge maps are nowadays applied to a host of different ways, such as preventing removal of significant structures in anisotropic blurring [23], indicating salient regions [33], selecting redundant seams in image resizing [1], image registration [25], depth from focus [8], extended depth of field, etc.

From Roberts' early work [31] in 1963 through to Canny's influential paper [4] in 1986, most methods have used first-order derivatives to estimate edge magnitude and orientation. Subsequently, the edgels (i.e. edge pixels) are often thresholded, linked, and thinned. Edge detection is normally carried out using local operators, and since edges may not be locally distinct in the image this means that edge maps are prone to fragmentation. Thus, research into new methods of edge

Paul L. Rosin · Xianfang Sun
School of Computer Science & Informatics, Cardiff University, Cardiff, CF24 3AA, UK
e-mail: {Paul.Rosin,Xianfang.Sun}@cs.cf.ac.uk

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_5, © Springer International Publishing Switzerland 2014

detection continues to be an active area. For instance, more recent approaches try to combine changes in brightness, colour, and texture [18], and use machine learning to determine how these multiple cues are combined.

During the last decade or so there has been considerable interest in using cellular automata to perform edge detection. They have several potential benefits, such as:

- **Efficiency** – Cellular automata are both inherently parallel and computationally simple, which means that they can implemented very efficiently in hardware.
- **Global properties** – Since multiple iterations of cellular automata rules can lead to emergent global behaviour it is feasible that a cellular automaton approach could provide benefits regarding the tendency of existing methods to produce fragmented output where there is locally inconsistent data.
- **Application specificity** – Rules can be selected (e.g. learnt from training data) to operate better than general purpose edge detectors for demanding applications, e.g. noisy modalities such as ultrasound and SAR.

There are many applications of CA based edge detection. For instance, in medical image processing some examples are: detection of tumours [7], identification of the pectoral muscle in mammograms [43] and diagnosis of lung cancer [30]. Other applications include: analysis of antibiotic images [19], detection of fabric defects [42], detection of grain boundaries in rocks [11], horizon tracking [9] and content based image retrieval (see chapter 8).

However, it is difficult for the general reader to gain an understanding of the state of the art in cellular automata based edge detection as the relevant papers are dispersed over many conferences and journals, which are often not specific to either cellular automata or image processing/computer vision. This chapter aims to collect together descriptions of these methods, and to provide a critical assessment of their merits.

Another obstacle in the path of the reader is the lack of performance evaluation. In the general area of computer vision there has been considerable work on comparison of edge detectors, using various methodologies, e.g. human assessment, comparison to (human) ground truth [12, 13, 40]. However, the majority of papers on developing cellular automata methods for edge detection simply show a few examples of their results alongside the Sobel or Canny outputs. Ideally this should be replaced by quantitative evaluation scores. However, this process is generally complicated by the need to process the raw outputs of the edge detectors before comparison, and so would depend on parameters for thresholding, thinning, etc.

5.2 Boundary Detection

The classical or the most popular cellular automata (CA) are binary, so it is natural to use CA for binary image processing. Edge detection of binary images can be considered as finding the boundaries of objects or regions in an image, and thus it is also called boundary detection.

When CA are used for image edge detection, the image to be detected is usually considered as a cellular automaton, each pixel of which is taken as a cell that

connects to its neighbouring pixels (cells), and the pixel values (0 or 1 in binary images) are the state values. Basically, the process of edge detection takes several iterations of state transition from the initial states (the input image pixel values) to the final states (the output image with 1 representing edge pixels and 0 the others). It is critical to find good state transition rules for the CA.

The state transition rule proposed by Wongthanavasu and Sadananda [45] is probably the simplest one, where a cell changes its state from 1 to 0 only if all its von Neumann neighbours have state value 1, otherwise it retains its original state (either 1 or 0). Though simple, this rule can produce reasonable results. It is intuitively understandable why this simple rule works. Suppose that we have a contiguous region of 1 as the foreground, then one iteration of CA evolution using this rule will result in all the inner pixels (which have all 1 in their neighbourhoods) changing to 0 (non-edge), while the boundary pixels (those have at least one 0 in their neighbourhoods) to remain as 1 (i.e. edge). More iterations will not change the pattern. The weakness of this rule is also obvious: if noisy pixels (value 0) exist inside a foreground region, the neighbouring pixels of a noisy pixel will remain unchanged, and wrongly classified as edges. Thus, this approach is not able to work effectively in the presence of noise.

To find optimal state transition rules, several researchers resorted to genetic algorithms. As early as in 1994, Sahota *et al.* [35] used a genetic algorithm to train CA to find optimal rules for edge detection. They used three simple original/edge image pairs as the training samples, and the obtained rules were then used in edge detection. They did not describe the rules they obtained, and only showed the detection results of two manually designed toy images.

Selvapeter and Hordijk [37] adopted the same method as in [35] to train CA, where they used similar original/edge image pairs as the training samples. The contribution of their work was to deal with real images with noise. They simply used a CA-based image noise filter [38] as the first step to denoise the image, and then used the trained CA rules for edge detection. They showed that the CA-based method can produce comparable results to the other methods such as Canny, Prewitt, Sobel, and Laplacian of Gaussian operators when the image is noiseless, and it can also produce reasonable results when the image is noisy, while the other methods failed to produce meaningful edge detection results. However, their comparisons in noisy cases are unfair because they had first denoised the image before using the CA to detect the edges, while they used the other methods to directly detect the edges in the noisy image.

Batouche *et al.* [3] also applied genetic algorithms to train CA for binary image edge detection. Instead of training rules for all the $2^9 = 512$ pixel patterns in a Moore neighbourhood, they assigned those rotational symmetric patterns (that rotate 0°, 90°, 180°, 270°, or flip horizontally or vertically) the same state transition rule. A weak matching criterion was introduced, so that some patterns with a difference less than a similarity threshold were further merged into a single rule. They assembled 15 state transition rules into a packet, which is represented by a chromosome, and trained to produce optimal rules. Experiments showed that edges are successfully detected, but are a little bit thick.

Slatnia *et al.* [41] assigned rotational symmetric patterns the same rule, but they were not training symmetric state transition rules as was done in [3]. Inspired by Rosin's work [32], they instead used genetic algorithms to train only a single powerful rule, that is, the central cell changes its state only when its Moore neighbourhood matches a specific pixel pattern. Interestingly, they got an optimal rule which was the same as that proposed by Wongthanavasu and Sadananda [45]. Again, they compared their results with Canny's, and claimed that they had better results, although this is not obvious from the figures they provided.

Yang *et al.* [46] proposed a CA approach for a specialised form of edge detection. Instead of using binary state transition rules to evolve the CA, they complicated the algorithm by introducing more state values. Their method consists of two steps. The first step sets the state value of each pixel. It uses the Prewitt operator, rotating in eight angles (from 0° with interval 45°, numbered as direction 1 to 8), to compute eight direction values for each pixel, and takes the direction number (called the location coordinate in their paper) with maximal direction value as the state value (called the characteristic vector). The state value is changed to 16 if the maximal direction value is less than 3. The second step evolves the CA based on the states (represented by values in $1 \sim 8$, or 16) in the neighbourhood of each cell. The differences of the state values between the central cell and its neighbours are calculated, and the number of neighbours with differences 0, 1, or 7 is counted. If the number is not equal to 2, the state of the central cell is changed to 16. After several iterations, the pixels with state values other than 16 are classified as edge pixels. One iteration of the algorithm will result in reasonable edge detection results. More iterations will delete irregular edges, and only the edges of the objects that are strictly rectangular survive.

In fact, we can simply describe Yang *et al.*'s algorithm by binary state transition rules also in two steps. The first step performs only one iteration with the rule that if three contiguous neighbours of a central pixel have the value 1 and the other three contiguous neighbours symmetric to them about the central pixel have the value 0, then the central pixel keeps its state (either as edge or non-edge), otherwise it is non-edge. The second step performs several iterations based on the rule that an edge pixel remains as edge if exactly two pixels in its von Neumann neighbourhood are edge pixels, and all the other pixels in its Moore neighbourhood are non-edge.

It should be mentioned that most of the proposed CA-based binary image edge detection methods were compared against Canny's method. This is inappropriate because Canny's method was not designed specifically for binary image edge detection, but rather for grey-scale image edge detection. In fact, Canny's method is not poorer than the above-mentioned methods even when dealing with binary images.

Another issue is that, from an image processing perspective, detecting boundaries in binary images is a relatively trivial task, and is not generally considered to be a research problem. In comparison, detecting edges in intensity images (which often also involves estimating edge magnitude and orientation as well) still regularly generates many papers in high ranking journals and conferences in the field of image processing/computer vision.

5.3 Edge Detection in Intensity Images

For edge detection of intensity images, some approaches first convert the intensity images into binary ones, and then evolve two-state cellular automata using specific state transition rules to determine edge pixels, while the others directly update pixel states based on the relationship of the central pixel with its neighbourhood, mostly a 1-ring von Neumann or Moore neighbourhood.

Popovici and Popovici [26] proposed an edge detection approach based on the state differences between the central pixel and the pixels in its von Neumann neighbourhood. If all the absolute state differences are less than a threshold ε, then the state of the central pixel becomes 0, otherwise it remains unaltered. The rule can be formulated as:

$$v_c^+ = \begin{cases} 0 & \text{, if } |v_i - v_c| \leq \varepsilon,\ \forall i \in N_c \\ v_c & \text{, otherwise.} \end{cases}$$

where v_c and v_c^+ are the current and the updated states of the central cell c, N_c is the von Neumann neighbourhood of the cell c, and v_i is the current state value of the cell i in N_c.

Gorsevski *et al.* [11] used Popovici and Popovici's approach to detect the grain boundaries in deformed rocks, but they did not cite [26].

Wongthanavasu and Sadananda [45] proposed a conditional rule to update the cellular state as:

$$v_c^+ = \begin{cases} v_c & \text{, if } v_c \leq v_{\max} - v_{\min} \\ v_{\max} - v_{\min} & \text{, otherwise.} \end{cases}$$

where $v_{\max}$ and $v_{\min}$ are the maximum and minimum states, respectively, in the von Neumann neighbourhood of the central cell c. The above rule can be described theoretically by a state transition table. However, it is hard to construct and use such a table in practice, because there are 256^5 entries in the table for a 256 greyscale image with the von Neumann neighbourhood. Wongthanavasu and Sadananda provided a partial transition table for illustration. Our experiments show that only a single iteration of this state transition will produce reasonable results. Multiple iterations actually degrade the results.

Wongthanavasu and Lursinsap [44] also extended the above-described conditional rule into 3D image edge detection with the only difference that the neighbourhood is now 3D von Neumann. Their experiments showed that the CA approach exhibited better performance than the Sobel and the Laplacian detection algorithms on average.

Kumar and Sahoo's method [15] also directly utilizes the intensities of the central pixel and its neighbourhood to detect edges, but the algorithm description is unclear, and it is hard to figure out how the algorithm is actually implemented.

Diwakar *et al.* [7] presented an approach that first convert an intensity image to a binary image through thresholding, and then use rules similar to Conway's Game

of Life to detect the edges. Unfortunately, their description is unclear.[1] Essentially they are using the thresholding to do the majority of the work, and then the CA just finds the boundaries in the binary image. Otsu's method [21] is used for thresholding. The resulting pixel value 0 represents background, and 1 represents a potential edge pixel. The cellular automaton rule used is totalistic, and (to the best of our understanding given the inconsistencies of their description) is:

$$v_c^+ = \begin{cases} 1 & \text{, if } |M_c| = 5 \\ v_c & \text{, if } |M_c| = 3,4,6,7 \\ 0 & \text{, otherwise.} \end{cases}$$

where $|M_c|$ is the number of edge pixels (value 1) in the Moore neighbourhood of the central cell c (including c itself). Since the thresholding is global, it will miss many edges and also find spurious edges!

Another method that first converts an intensity image to a binary image and then uses CA to detect edges, was proposed by Qadir and Khan [29]. Although 2^{512} possible rule sets exist for a CA with a Moore neighbourhood, there are only 512 linear rule sets among them. Qadir and Khan tested all the 512 linear rule sets, and found that some of them showed no edge detection, some showed strong edge detection, while the others showed weak detection. They compared their results with Sobel's and Canny's, and claimed that their results are better, but this is not obviously clear from their example images provided.

The state transition rules of the above-mentioned approaches are all specified manually, and are not necessarily optimal. Some researchers tried to find optimal rules for edge detection using genetic algorithms or evolutionary algorithms.

Kazar and Slatnia [14] used genetic algorithms to construct CA rules for edge detection of intensity images. One novelty in their approach is that they do not use pixel intensities as state values, but rather label different intensities in the Moore neighbourhood of a central pixel, and take the labels as the state values. In this way, they significantly reduced the possible number of neighbourhood patterns from $256^8/5$ to $8^8/5$ for 256 greyscale images, and thus constructing state transition tables becomes computationally affordable.

Sato and Kanoh [36] introduced rule-changing cellular automata for edge detection, and used a form of genetic programming, namely gene expression programming (GEP) – an evolutionary algorithm to optimise the CA rules. The idea of rule-changing CA is to execute an array of transition rules R_i for M_i iterations in sequence. Each rule R_i is represented by a binary expression tree with the leaf nodes being the pixel states in the Moore neighbourhood of a central pixel or the constants 0, 127, and -128, and the non-leaf nodes functions max(), min(), saturated addition, or saturated subtraction. GEP were used to optimise both R_i and M_i. Experiments showed that better results are obtained using two rules (the rule-changing CA) than

[1] Diwakar *et al.*'s neighbourhood appears to contain the central pixel, which is not consistent with the standard descriptions of Conway's Game of Life. Moreover, they describe their system as implementing Wolfram's rule 124, which is however normally used to describe a one dimensional (rather than two dimensional) cellular automaton.

using only one rule (the ordinary CA). The paper provided their optimal rules and iterations R_1, M_1 and R_2, M_2. Unfortunately, it seems that errors or typos existed in R_1 because it is not a binary expression tree with two arguments for each function as stated in their algorithm specification.

Priego *et al.* [27] describe an approach for edge detection from hyperspectral (i.e. multi-band) images. Rather than input the hyperspectral values directly into the CA, they first extract a set of features from each pixel's neighbourhood (first the eight spectral angles are computed, and these are then described by their mean, directional gradients and standard deviation). A genetic algorithm is then employed to learn $M = 20$ rules, each consisting of a tuple mapping an instance of the six feature elements to a real valued output value in the range $[0, 1]$. Once the rules have been learnt, the CA is run by applying at each pixel the closest matching rule (in terms of its Euclidean distance to the six feature elements). The output at each pixel is thresholded at 0.5 to produce a binary edge map.

Beside traditional CA approaches, cellular automata combined with other techniques were also proposed for edge detection of intensity images.

Mirzaei *et al.* [20] used fuzzy cellular automata for edge detection. They designed eight masks, each of which separates the Moore neighbourhood into two groups. The average of the absolute state differences between the central pixel and the pixels within each group is calculated, and then fuzzified into a fuzzy description of 'High' and 'Low'. Thirty-two fuzzy rules are used for fuzzy state transition, and the resulting fuzzy description is defuzzified to produce an updated numerical state. The authors claim that their method has better efficiency than the Robert and Sobel edge detectors, and moreover that they have largely overcome detection errors (missed edges and false edges). However, their published results do not support this claim.

Chen and Yan [6] proposed an approach for edge detection which first uses a CA-based diffusion model for image smoothing, and then detects the image edges by finding the zero-crossings of the second order derivative of the image defined as the difference between the smoothed and the original images divided by the diffusion time. Experiments showed better results were obtained by the proposed method than that by Laplacian of Gaussian, Laplacian, Canny, and Sobel operators. However, the final results are heavily dependent on the suitable choice of the number of iterations in the smoothing stage.

Other methods, including cellular neural networks [2, 17], fuzzy cellular neural network [39], cellular learning automata [10], and cellular automata transformation methods [24], all produced reasonable results, which shows that CA combined with other techniques are promising tools for image edge detection.

Finally, we describe a method developed by one of this chapter's authors. In an attempt to retain the simplicity of binary CA with the ability to process intensity (i.e. non-binary) images, Rosin [34] applied threshold decomposition, a technique often used in image processing. This involves decomposing a gray level image into the set of binary images obtained by thresholding it at all possible gray levels. A single set of two-state CA rules is learnt which is applied independently to each binary image.

The resulting binary output images are then combined as a simple summation to produce the edge magnitudes.

To find suitable rules a deterministic approach was used, namely sequential floating forward search (SFFS) [28], which starts from the empty set and iteratively adds the best performing rule. The objective function was such that the edge magnitudes constructed from the CA with threshold decomposition should provide a good match – in terms of root mean square (RMS) – to the gray level target ground truth edge map.

The final rule set that was learnt from the training data simply consisted of a single rule. It specified that any white pixel in a 3×3 homogeneous (i.e. all white) neighbourhood is set to black (inverted). For each of the binary images that the input is decomposed into, this causes all white pixels to be replaced by black except for pixels adjacent to black pixels in the input image. Thus, a black image is formed containing a one pixel wide white strip along the original black/white transitions, and these images are summed at the reconstruction stage of threshold decomposition.

5.4 Post-processing of Edges

In the introduction we stated how, after an initial edge detection, the results are often thresholded, linked, and thinned. Clearly, cellular automata have been designed for thinning – see chapter 3. They are also suitable for performing linking, although this is less common.

Lee and Bruce [16] describe a method for edge detection that is initialised with computing a gradient angle and edge magnitude for each pixel using an algorithm mimicking the Prewitt edge detector. They then use CA to perform adaptive thresholding based on the gradient angle and edge magnitude information. The edges are typically overestimated after adaptive thresholding, and each edge pixel is tested, and removed if it has the lowest edge magnitude in its neighbourhood and removal of this pixel will not bisect the edge. Further post-processing is performed using CA to thin wide edges, remove short edges, and delete the edges which enclose small regions.

Chang *et al.*'s [5] approach to edge detection involves using an "orientation information measure" that essentially estimates edge strength, and is used to provide an under-thresholded sparse binary edge map (the mark matrix). CA rules are then applied to link the edge pixels. A set of 3×3 edge (line) patterns are defined based on the assumption that the width of edges is one pixel, and the CA updates a pixel state to edge if it matches an edge pattern when taking as edge the pixel with maximum orientation information measure in its neighbourhood or semi-neighbourhood.

Like Chang *et al.*, the method of Peer *et al.* [22] starts with a mark matrix, which is however undefined. Therefore, it is not possible to replicate their algorithm. The next stage of their method is to apply Conway's Game of Life to the mark matrix to generate a binary edge map.

Cloud models are the extension of fuzzy models, which combine fuzziness and randomness into the transformation between the qualitative linguistic description and the quantitative numerical values. Zhang *et al.* [47] combined a cloud model and cellular automata in edge detection. They used the direction information, the neighbour edge intensity, and the width of the neighbour edge isolation as the input to a cloud reasoning system, which produces a binary edge map as the mark matrix. The same CA rules as Chang *et al.*'s [5] are then used to link the edge pixels. Their experiments showed that the proposed method can detect edges appropriately, but the resulting edges are wide.

5.4.1 A Simple Edge Linking Scheme

To link disconnected thin edges using cellular automata, we propose a simple four-step method whose input is a thresholded edge map. Some of the rules are based on the 3×3 Moore neighbourhood, while other rules use just a 1×3 neighbourhood. In addition to the two states of the initial image, two more states are introduced, to represent the ends of open curves and 'T'-junctions. These extra states enable us to use a 1D rather than 2D neighbourhood at times. The rules use X, the crossing number at a pixel which is the number of transitions from white to black and vice versa when the pixel's eight neighbours are traversed in a circular fashion.

Essentially, each iteration of the CA extends open end points of edge curves by one pixel. Any further extension is terminated if the end point touches an existing edge (i.e. the point becomes a junction). After completing the extension of edges, those that did not reach an existing edge, i.e. end points at the end of open curves, are contracted back to their initial length.

Using the 3×3 neighbourhood, edges can be extended in the eight principal directions (0°, 45°, ...). If a larger neighbourhood were used, then not only could more orientations be used, but curved extensions could also be accommodated.

1. **detect end points (one iteration):**
   if p = `edge` and $X = 2$ then p = `end`
2. **extend open ends (n iterations):**
   a. **detect junctions (one iteration):**
      if p = `end` and $X = 6$ then p = `junction`
   b. **extend open ends (one iteration):**
      if p = `end` and number of non-background = 1
      and any of the 1D patterns in figure 5.1 match then update
3. **contract open ends (n iterations):**
   if p = `end` or `junction` and number of non-background = 1
   then p = `non-edge`
4. **relabel pixels:**
   if $p \neq$ `non-edge` then p = `edge`

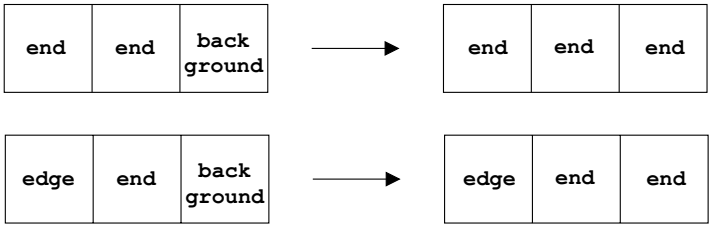
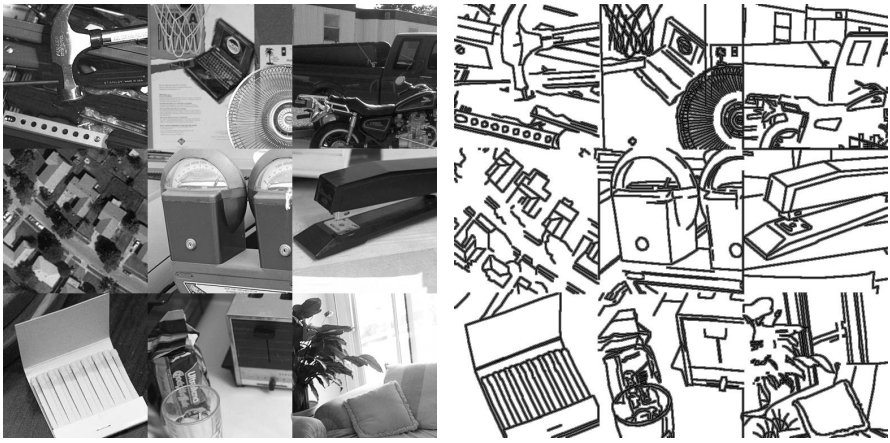


Fig. 5.1 1D CA rules for extending open ends for edge linking. The rules use a 1×3 pixel neighbourhood containing pixel labels: `edge`, `end`, `background`. All 45° rotated versions are also required.

5.5 Experiments

In this section we show results of applying several cellular automata edge detection methods. We start with Rosin's [34] method; the original training data was a 750×750 image mosaic containing sub-images from the University of South Florida data set which contains images along with manually generated ground truth edges; see figure 5.2. Note that in this chapter all edge maps are inverted for display purposes. Since there is likely to be some positional error in the ground truth edges (which are one pixel wide) the target edge map was dilated twice, with the new edges set each time to an increasingly lower intensity.

The CA rules were tested on four independent images, not included in the training data (figure 5.3), and produced the results shown in figure 5.4. The cellular automaton converged after a single iteration of the single rule. It can be seen that the results are fairly similar to the Sobel edge maps.

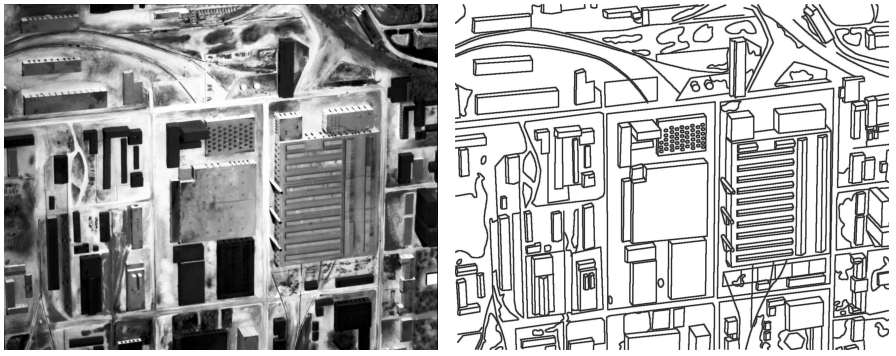


a b

Fig. 5.2 USF training data: (a) input image, (b) target ground truth image (inverted for display purposes)

Fig. 5.3 Sample test images containing indoor, outdoor, man-made and natural scenes. Also are shown the Sobel edge maps for comparison.

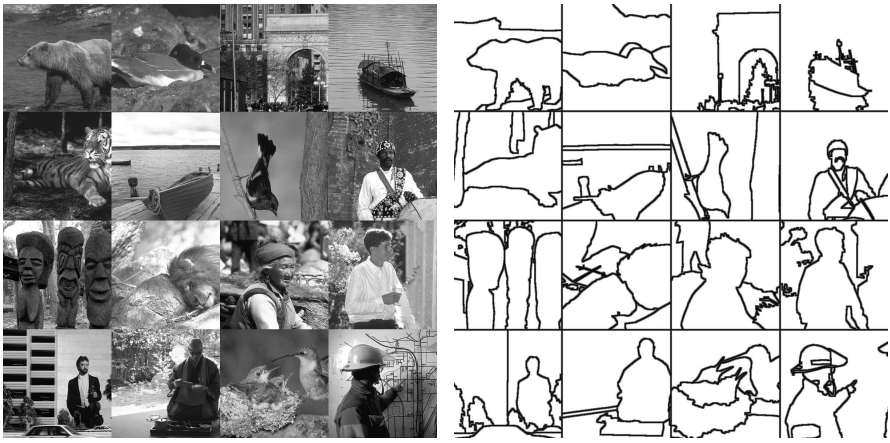
Fig. 5.4 Edges detected using Rosin's [34] method trained on data in figure 5.2



a b

Fig. 5.5 RADIUS training data: (a) input image, (b) target ground truth image

For comparison, several other training sets were used to learn rule sets for edge detection. Figure 5.5 shows the 1314×1044 image `J25` of a 40×40 inch model board especially created for the RADIUS project, along with its associated manually generated ground truth edges. After training a single rule was learnt that was identical to that learnt for the USF data set.



a b

Fig. 5.6 BSDS training data; (a) input image, (b) target ground truth image

Fig. 5.7 Edges detected using Rosin's [34] method trained on data in figure 5.6

Figure 5.6 shows a 1200×1200 image mosaic created from a subset of sub-images from the BSDS300 Berkeley Segmentation Dataset and Benchmark. Whereas the previous two sets of ground truth were explicitly made up of edges, the BSDS300 contains object boundaries. Since objects are defined at a higher semantic level than low level edges, it can be seen that there is often a poor correspondence between the two, and thus it poses a greater challenge to the cellular automaton. A larger set of rules is learnt from this data compared to the previous training data. The new rules consist of the single rule learnt previously plus another eight. The effect (see figure 5.7) is to emphasise the corners more than previously, and to reduce the response at some edges. Since there was a less direct match between the

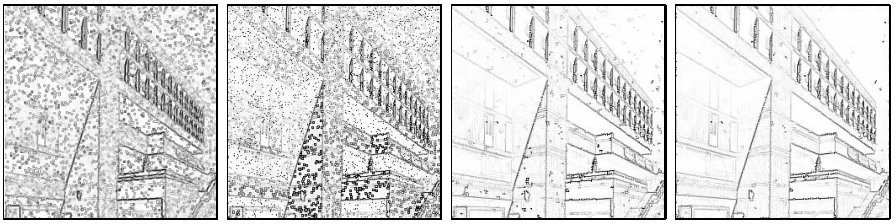


Fig. 5.8 Edge detection applied to an image which has had salt and pepper noise added. a) Sobel, b) CA rules learnt from training set in figure 5.2, c) CA rules learnt from a version of the training set in figure 5.2 with added salt and pepper noise, d) CA rules learnt from noisy data applied to both the image and an inverted version.

Fig. 5.9 Edges detected using Wongthanavasu and Lursinsap's [45] method (upper row: single iteration; lower row: iterated until convergence)

training source data and the ground truth, the CA requires more iterations to achieve convergence (typically about four iterations).

A final example of Rosin's method [34] is given where the USF training data had salt and pepper noise with probability of 0.1. This enables a set of seven rules to be learnt that are robust to similar noise. Results are shown in figure 5.8 for edge detection on a noisy version of the MIT image. Application of the Sobel produces a very noisy edge map (figure 5.8a), while applying a CA with the original rule learnt from the clean USF data also fares poorly (figure 5.8b). Using the set of rules learnt from the noisy training data produces a much better result (figure 5.8c), requiring on average about fifteen iterations of the rules. However, since the rule learnt for edge detection is restricted to inverting white pixels then the 'salt' (white noise) is effectively removed, but the 'pepper' (black noise) remains. The solution taken in Rosin [34] is to also apply the rules to the inverted image so as to remove the

Fig. 5.10 Edges detected using Popovici and Popovici's [26] method (upper row threshold = 4; middle row threshold = 16; lower row threshold = 64)

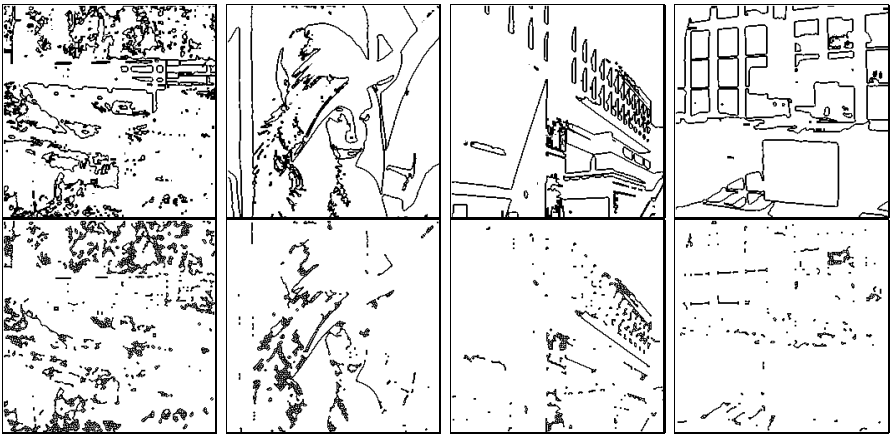


Fig. 5.11 Edges detected using Diwakar *et al.*'s [7] method (upper row: single iteration; lower row: iterated until convergence)

Fig. 5.12 Binary edge maps (left) post-processed to link fragmented edges by running five iterations of the rules described in section 5.4.1 (right); inserted edgels are coloured red. The bottom row shows two close ups from the second example.

inverted pepper. The logical AND operation is applied to the two edge maps which effectively eliminates both salt and pepper (figure 5.8d).[2]

[2] There can also be a single pixel translation between the edge responses of the two outputs, and so better results were achieved by translating one of the images by $\{-1,0,1\}$ in X and Y before the logical AND operation. The outputs were then combined using a logical OR operation.

We now show results for a variety of other CA based edge detectors. In figure 5.9 is shown Wongthanavasu and Lursinsap's [45] method. For a single iteration the results look reasonable. Additional iterations degrade the results.

Figure 5.10 shows results from Popovici and Popovici's [26] method (which converged after three iterations).[3] They require an application specific threshold parameter to be specified, and it can be seen to alter the density of the detected edges. The best results in figure 5.10 might be considered to be with the threshold set to 16, although even then the results are generally inferior, containing thick edge regions whilst also retaining many scattered and disconnected single pixel edges.

Figure 5.11 shows results from Diwakar *et al.*'s [7]. As expected, since the method is based on global thresholding, it has missed many edges and found many spurious ones. If more than a single iteration of the CA is run, the results degrade even further.

The results of applying a post-processing step of edge linking are shown in figure 5.12. The input image is edge detected using Rosin's method [34], and then thresholded to create a binary edge map. In addition, isolated edge pixels were removed. The linking method described in section 5.4.1 was applied for five iterations. It can be seen that many fragmented edges have been successfully linked (as shown by the red edgels).

5.6 Conclusions

As stated in the introduction, it is difficult for the general reader to gain an understanding of the state of the art in cellular automata based edge detection since papers are dispersed over many conferences and journals. Our brief survey shows that there exists a relatively large number of relevant papers, although a number of them were not clearly written, with details missing or occasionally inconsistent. Moreover, a number of papers are misleading, in that, according to common usage within image processing, they actually perform (binary image) boundary detection rather than (intensity image) edge detection.

The papers and the results of the experiments included in this chapter demonstrate that cellular automata are indeed capable of performing edge detection, i.e. process a grey level input image to produce an edge magnitude image (either binary or e.g. 256 values) as output. The results were of variable quality, but in order to be able to confidently evaluate and compare edge detectors a more systematic and quantitative analysis should be carried out. This has not been done to date.

The experiments revealed that for all the CA only a very few iterations were necessary to achieve their optimal results (such details were often missing from the descriptions of the methods in their original publications). Specifically, the methods of Rosin [34] (when trained on the USF dataset), Wongthanavasu and Lursinsap [45] and Diwakar *et al.* [7] should generally run for only a single iteration.

[3] Popovici and Popovici's [26] paper described a von Neumann neighbourhood, but we found better results (those shown in figure 5.10) to be obtained using a Moore neighbourhood.

Although for Rosin's method the CA converged after that iteration, this was not the case for the other methods, whose results steadily degraded when further iterations were applied. Popovici and Popovici's [26] method converged after three iterations, and Rosin's [34] method also converged after a similar number of iterations when trained on the BSDS300 dataset. This suggests that none of the above approaches are using the full power of CA to capture more global image structure by propagating information across the image via a larger number of iterations. The version of Rosin's method [34] trained on the noisy version of the USF training data required more (typically fifteen) iterations, and this is consistent with the denoising rules in [32]. The latter were found to be competitive with alternative denoising methods, and also required several tens of iterations of the rules (depending on the level of noise).

Nevertheless, cellular automata based edge detection holds promise since it is computationally efficient, and can moreover be tuned to specific domains (i.e. applications and/or image types) by appropriate selection/learning of rules. Not only that, but pre-processing and post-processing stages such as noise filtering, thinning and edge linking can also be easily included in the cellular automata framework.

References

1. Avidan, S., Shamir, A.: Seam carving for content-aware image resizing. ACM Trans. Graph. 26(3), 10 (2007)
2. Baştürk, A., Günay, E.: Efficient edge detection in digital images using a cellular neural network optimized by differential evolution algorithm. Expert Syst. Appl. 36(2), 2645–2650 (2009)
3. Batouche, M., Meshoul, S., Abbassene, A.: On solving edge detection by emergence. In: Ali, M., Dapoigny, R. (eds.) IEA/AIE 2006. LNCS (LNAI), vol. 4031, pp. 800–808. Springer, Heidelberg (2006)
4. Canny, J.: A computational approach to edge detection. IEEE Trans. Pattern Analysis and Machine Intelligence 8, 679–698 (1986)
5. Chang, C., Zhang, Y., Gdong, Y.: Cellular automata for edge detection of images. Int. Conf. on Machine Learning and Cybernetics 6, 3830–3834 (2004)
6. Chen, Y., Yan, Z.: A cellular automatic method for the edge detection of images. In: Huang, D.-S., Wunsch II, D.C., Levine, D.S., Jo, K.-H. (eds.) ICIC 2008. LNCS (LNAI), vol. 5227, pp. 935–942. Springer, Heidelberg (2008)
7. Diwakar, M., Patel, P., Gupta, K.: Cellular automata based edge-detection for brain tumor. In: Advances in Computing, Communications and Informatics, pp. 53–59 (2013)
8. Ens, J., Lawrence, P.: An investigation of methods for determining depth from focus. IEEE Trans. Pattern Analysis and Machine Intelligence 15(2), 97–108 (1993)
9. Georgilas, I., Gale, E., Adamatzky, A., Melhuish, C.: UAV horizon tracking using memristors and cellular automata visual processing (2013)
10. Gharehchopogh, F., Ebrahimi, S.: A novel approach for edge detection in images based on cellular learning automata. Int. J. Computer Vision and Image Processing 2(4), 51–61 (2012)
11. Gorsevski, P., Onasch, C., Farver, J., Ye, X.: Detecting grain boundaries in deformed rocks using a cellular automata approach. Computers & Geosciences 42, 136–142 (2012)

12. Heath, M., Sarkar, S., Sanocki, T., Bowyer, K.: Robust visual method for assessing the relative performance of edge detection algorithms. IEEE Trans. Pattern Analysis and Machine Intelligence 19(12), 1338–1359 (1997)
13. Heath, M.D., Sarkar, S., Sanocki, T.A., Bowyer, K.W.: Comparison of edge detectors: A methodology and initial study. Computer Vision and Image Understanding 69(1), 38–54 (1998)
14. Kazar, O., Slatnia, S.: Evolutionary cellular automata for image segmentation and noise filtering using genetic algorithms. Journal of Applied Computer Science and Mathematics 5(10), 33–40 (2011)
15. Kumar, T., Sahoo, G.: A novel method of edge detection using cellular automata. International Journal of Computer Applications 9(4), 38–44 (2010)
16. Lee, M., Bruce, L.: Applying cellular automata to hyperspectral edge detection. In: Int. Geoscience and Remote Sensing Symposium, pp. 2202–2205 (2010)
17. Li, H., Liao, X., Li, C., Huang, H., Li, C.: Edge detection of noisy images based on cellular neural networks. Communications in Nonlinear Science and Numerical Simulation 16(9), 3746–3759 (2011)
18. Martin, D., Fowlkes, C., Malik, J.: Learning to detect natural image boundaries using local brightness, color, and texture cues. IEEE Trans. Pattern Analysis and Machine Intelligence 26(5), 530–549 (2004)
19. Men, H., Zhang, J., Wang, C.: Measurement of inhibition zone based on cellular automata edge detection method. In: Int. Workshop on Education Technology and Computer Science, vol. 2, pp. 357–360 (2009)
20. Mirzaei, K., Motameni, H., Enayatifar, R.: New method for edge detection and denoising via fuzzy cellular automata. Int. J. Phy. Sci. 6(13), 3175–3180 (2011)
21. Otsu, N.: A threshold selection method from gray-level histograms. IEEE Trans. SMC 9, 62–66 (1979)
22. Peer, M., Qadir, F., Khan, K.: Investigations of cellular automata game of life rules for noise filtering and edge detection. Int. J. Information Engineering and Electronic Business 4(2), 22–28 (2012)
23. Perona, P., Malik, J.: Scale-space and edge detection using anisotropic diffusion. IEEE Trans. Pattern Analysis and Machine Intelligence 12(7), 629–639 (1990)
24. Piao, Y., Kim, S., Cho, S.J.: Two-dimensional cellular automata transforms for a novel edge detection. In: IComputability in Europe 2008, Logic and Theory of Algorithms (2008)
25. Pluim, J.P.W., Maintz, J.B.A., Viergever, M.A.: Image registration by maximization of combined mutual information and gradient information. IEEE Trans. Med. Imaging 19(8), 809–814 (2000)
26. Popovici, A., Popovici, D.: Cellular automata in image processing. In: Int. Symp. on the Mathematical Theory of Networks and Systems (2002)
27. Priego, B., Bellas, F., Souto, D., López-Peña, F., Duro, R.: Evolving cellular automata for detecting edges in hyperspectral images. In: Int. Conf. on Fuzzy Systems, pp. 1–6 (2012)
28. Pudil, P., Novovicova, J., Kittler, J.: Floating search methods in feature-selection. Pattern Recognition Letters 15(11), 1119–1125 (1994)
29. Qadir, F., Khan, K.: Investigations of cellular automata linear rules for edge detection. Int. J. Computer Network and Information Security 3, 47–53 (2013)
30. Qadir, F., Peer, M., Khan, K.: Efficient edge detection methods for diagnosis of lung cancer based on two-dimensional cellular automata. Advances in Applied Science Research 3(4), 2050–2058 (2012)

31. Roberts, L.: Machine Perception of Three-Dimensional Solids. In: Outstanding Dissertations in the Computer Sciences. Garland Publishing, New York (1963)
32. Rosin, P.: Training cellular automata for image processing. IEEE Trans. on Image Processing 15(7), 2076–2087 (2006)
33. Rosin, P.: A simple method for detecting salient regions. Pattern Recognition 42(11), 2363–2371 (2009)
34. Rosin, P.: Image processing using 3-state cellular automata. Computer Vision and Image Understanding 114(7), 790–802 (2010)
35. Sahota, P., Daemi, M., Elliman, D.: Training genetically evolving cellular automata for image processing. In: Int. Symp. Speech, Image Processing and Neural Networks, pp. 753–756 (1994)
36. Sato, S., Kanoh, H.: Evolutionary design of edge detector using rule-changing cellular automata. In: Nature & Biologically Inspired Computing, pp. 60–65 (2010)
37. Selvapeter, J., Hordijk, W.: Genetically evolved cellular automata for image edge detection. In: Proceedings of the International Conference on Signal, Image Processing and Pattern Recognition, SIPP 2013 (2013)
38. Selvapeter, P.J., Hordijk, W.: Cellular automata for image noise filtering. In: Nature & Biologically Inspired Computing, pp. 193–197 (2009)
39. Senthilkumar, S., Piah, A.R.M.: An improved fuzzy cellular neural network (IFCNN) for an edge detection based on parallel RK(5,6) approach. International Journal of Computational Systems Engineering 1(1), 70–78 (2012)
40. Shin, M.C., Goldgof, D.B., Bowyer, K.W.: Comparison of edge detector performance through use in an object recognition task. Computer Vision and Image Understanding 84(1), 160–178 (2001)
41. Slatnia, S., Batouche, M., Melkemi, K.E.: Evolutionary cellular automata based-approach for edge detection. In: Masulli, F., Mitra, S., Pasi, G. (eds.) WILF 2007. LNCS (LNAI), vol. 4578, pp. 404–411. Springer, Heidelberg (2007)
42. Suyi, L., Qian, W., Heng, Z.: Edge detection of fabric defect based on fuzzy cellular automata. In: Int. Workshop on Intelligent Systems and Applications, pp. 1–3 (2009)
43. Wongthanavasu, S.: Cellular automata for medical image processing. In: Salcido, A. (ed.) Cellular Automata – Innovative Modelling for Science and Engineering, pp. 395–410 (2011)
44. Wongthanavasu, S., Lursinsap, C.: A 3-D CA-based edge operator for 3-D images. In: Int. Conf. Image Processing, pp. 235–238 (2004)
45. Wongthanavasu, S., Sadananda, R.: A CA-based edge operator and its performance evaluation. J. Visual Communication and Image Representation 14(2), 83–96 (2003)
46. Yang, C., Ye, H., Wang, G.: Cellular automata modeling in edge recognition. In: 7th Int. Symp. on Artificial Life and Robotics, pp. 128–132 (2002)
47. Zhang, K., Zhang, W., Yuan, J.: Edge detection of images based on cloud model cellular automata. In: Chinese Control Conference, pp. 249–253 (2008)

Chapter 6
Copy-Move Forgery Detection Using Cellular Automata

Dijana Tralic, Paul L. Rosin, Xianfang Sun, and Sonja Grgic

Abstract. Thanks to the availability of many sophisticated image processing tools, digital image forgery is prevalent nowadays. One of the common methods is copy-move forgery (CMF), where part of an image is copied to another location in the same image. Detection of copy-move forgery has been widely researched recently, and many different solutions have been proposed. This chapter introduces a different approach, in which cellular automata (CA) are applied to the task of copy-move forgery detection (CMFD). The basic idea is to learn, for each overlapping block in the image, a set of CA rules that represents the intensity changes within that block. These rules are then used as features for the detection of copied blocks.

A problem arises when applying CA to image processing. If pixel intensities are used as cell states, then the large range of image intensities leads to a combinatorial explosion in the number of possible rules, making it difficult to both learn and represent rules efficiently. We describe a solution in which a reduced description of a neighbourhood is accomplished by a proper binary representation of the image based on local binary patterns (LBPs).

In the case of plain copy-move forgery, a simple 1D CA are sufficient for detection purposes, but any transformation of the copied area (for example, rotation and scaling) introduces large changes into the binary representation of the image, resulting in the need for more complicated forms of the CA's neighbourhood. However, the main issue of CMFD using CA rules is its sensitivity to processing after the copy-move operation applied to hide traces of the forgery, for example, addition of noise. Nevertheless, in some cases the CA can effectively cope with such forgeries if image pre-processing (for example, simple image filtering) is applied before forgery detection.

Dijana Tralic · Sonja Grgic
Faculty of Electrical Engineering and Computing, University of Zagreb,
Unska 3, 10000 Zagreb
e-mail: {Dijana.Tralic,Sonja.Grgic}@fer.hr

Paul L. Rosin · Xianfang Sun
School of Computer Science and Informatics, Cardiff University, Cardiff CF24 3AA
e-mail: {Paul.Rosin,Xianfang.Sun}@cs.cardiff.ac.uk

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_6, © Springer International Publishing Switzerland 2014

6.1 Introduction

Nowadays analogue images are completely replaced by digital images thanks to the simplicity of their acquisition, processing and storage. Many sophisticated digital image processing tools allow modification (tampering/manipulation) of digital image content to be done without any visible traces, leading to many forged images used in everyday life. Different approaches for revealing the manipulation of the content of digital image have been developed recently, with the aim to ensure the credibility of digital images. However, there is still no methodology to verify the integrity of digital images in an automatic manner. Therefore, digital image forensics [7] is an emerging research field that includes methods for determining the authenticity and the origin of digital images.

Image authentication methods can generally be classified into two main categories [6]: active and passive methods. Active methods involve embedding of some information into an image in the archiving process, such as digital watermarks [13] and signatures. Tampering of images usually destroys or modifies that embedded information, so any kind of content manipulation can be easily detected by analyzing information extracted from image. One active approach for digital image forensics is described in Chapter 7, where cellular automata are used as a tool for generating digital signatures.

Passive methods on the other hand involve checking the integrity of an image by analysis of image statistics and properties (for example, sensor noise [8], illumination conditions [10], etc.). Detection methods in this category are usually aimed at solving some special kind of tampering attempts so there is no unique technique for detection of all types of forgeries.

One common type of forgery attack is a copy-move forgery (CMF) [20] in which part of an image is copied and moved to a new location. CMF is often used because of the simplicity of performing that type of forgery. Detection of CMF can be done using different techniques, but the most basic method is based on dividing the image into overlapping blocks and calculating some set of features from every block. Those features are then used to determine similarity between blocks in image. Different feature sets were proposed for this purpose, such as average or intensity [14] or frequency coefficients [9], etc.

In this chapter, a new approach for copy-move forgery detection based on cellular automata (CA) is presented [23]. The basic idea is to use cellular automata to learn a set of rules for each sub-image block in an image. Those rules can serve as features for determining the similarity of blocks. A problem that arises when using CA with grayscale images is the large number of possible rules, which leads to an even larger number of possible combinations of those rules. A reduced representation can be accomplished by binary representation of images in a proper manner using techniques based on local binary patterns (LBP's) [16]. The problem of plain copy-move detection can be solved by applying simple 1D CA, but detection of different types of CMFs (for example, rotation or scaling of copied area) requires defining more complicated types of neighbours for CA. Also, post-processing

methods such as JPEG compression or the addition of noise can reduce the accuracy of the detection algorithm.

6.2 Copy-Move Forgery (CMF)

One of the most common used digital image forgery methods is a copy-move forgery (CMF) [20], where part of an image is copied and moved to another location in the same image. The purpose of this kind of forgery is usually to hide or to add some content or object in the image. Since the forged region came from the same image, it is impossible to use some statistical properties (for example, camera noise or illumination conditions) for forgery detection because they are very well matched within the image. Taking the forged region from the same image also simplifies the forgery process because it is easier to fit the forged region into the image due to the similarity of properties of the copied region and the rest of image.

Plain copy-move forgery is a type of forgery where the copied area is translated to a new location in the same image without changing any properties of the copied area. Therefore, in that kind of forgery, there are two completely identical areas in the image which makes plain copy-move forgery detection quite easy to implement.

More complicated types of forgery can be done by transformation of a copied region before translation to a new location. Some possible transformations of copied regions are:

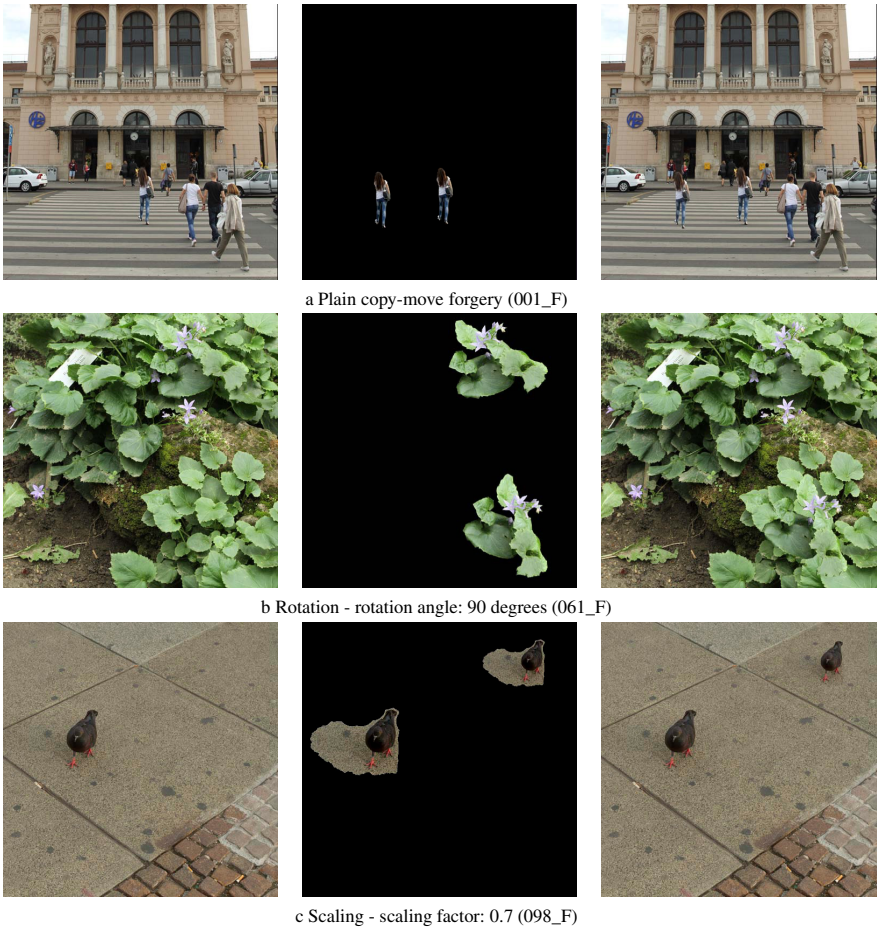
1. scaling – enlarging or shrinking of a copied area by an equal scale factor in all directions,
2. rotation – circular moving of a copied area around a centre of rotation by a arbitrary angle,
3. distortion – enlarging or shrinking of a copied area by a scale factor that is not the same in all directions,
4. combination – application of more than one transformation of a copied area.

The result of applying any of these transformations is a change in the properties of the copied area. Therefore, searching for forgeries is not as simple as in the case of plain CMF. Figure 6.1 shows some examples of CMFs from the CoMoFoD database [24].

Hiding of forgery traces can be done by applying some post-processing methods. It is possible to apply a post-processing method on the whole image after forgery, but sometimes post-processing is applied only on copied region borders to assure better fitting with the new background. The most common post-processing methods used in digital image forgery are JPEG compression, addition of noise and image blurring (Fig. 6.2).

6.3 Copy-Move Forgery Detection (CMFD)

Detection of copy-move forgery has been widely researched [9]. Developed methods for copy-move forgery detection can be categorized as keypoint-based and



a Plain copy-move forgery (001_F)

b Rotation - rotation angle: 90 degrees (061_F)

c Scaling - scaling factor: 0.7 (098_F)

Fig. 6.1 Examples of CMF (left: original image, middle: regions with copy-move version, right: forged image). The image identifiers in brackets are provided for comparison with results in Tab. 6.1.

block-based methods. Keypoint-based methods include scanning of the whole image with the aim of finding points of interest (for example, point with high entropy). Those points are then analyzed to select only point with the same properties and detect similar areas in the image. Some popular examples of keypoint-based methods are SIFT (Scale-invariant feature transform) [1] and SURF (Speeded Up Robust Features) [21].

Block-based methods involve dividing an image into small overlapping blocks as a first step of the algorithm. A set of features is then calculated for every defined block, and those features are used for detection of similar blocks in the image. Different sets of features, such as DCT (Discrete Cosine Transform) [9] / DWT (Discrete Wavelet Transform) [2] coefficients, PCA (Principal Component Analysis)

a JPEG compression - quality factor: 30 (005_F)

b Image blurring - 3×3 averaging filter (018_F)

c Noise adding - zero mean, variance: 0.001 (032_F)

Fig. 6.2 Examples of postprocessing methods applied on whole image (left: original image, right: post-processed image)

[17] or Zernike moments [19], have been proposed for use in block-based methods, but the use of cellular automata for this purpose is a completely new approach.

6.3.1 Block-Based Method for CMFD

Generally all block-based copy move forgery detection approaches follow similar steps (Fig. 6.3 illustrates each step):

1. First the image is pre-processed because most algorithms require only the luminance component information, and so it is necessary to convert images to grayscale space. Sometimes a Gaussian pyramid decomposition is also applied (for example, in [25]).
2. After pre-processing, an image is divided into overlapping blocks by sliding a predefined window by one pixel through the whole image. The size of the window is usually small (for example, 8×8, 16×16, 24×24 pixels) to assure detection of areas of all sizes. Dividing an $N \times M$ image into overlapping blocks of size $b \times b$ leads to a very large number of different blocks according to equation (6.1) (for illustration: dividing a 512×512 image using a 8×8 window produces 255,025 different blocks).

$$N_b = (N - b + 1) \times (M - b + 1) \tag{6.1}$$

3. For every defined block a feature vector f is calculated by same method. The feature vector is used as a reduced description of a block because it contains information about shape, texture, orientation or some other properties of a block. The size of the feature vector depends on the selected method for its calculation.
4. Applying brute-force search to find similar blocks by mutual comparison of all pairs of blocks requires a lot of computational time and resources. Therefore, all feature vectors are stored in one matrix that is sorted by some algorithm (for example, lexicography sorting) to accomplish grouping of similar blocks. Beside sorting, some other methods for finding similar blocks can be applied, for example, kd-tree.
5. Neighbor feature vectors in the sorted matrix are than compared by analyzing the similarity between them, using the Euclidean distances between feature vector elements according to equation (6.2). All pairs of blocks with distance v higher than some predefined threshold T_s are removed from the set of possible results. Selection of threshold T_s depends on the type of forgery, for example, it can be set to zero for plain CMF, or it has to be adjusted to some higher values if any transformations/post-processing methods are applied. After this step only similar pairs of blocks are kept as possible results.

$$v = \sqrt{\sum_{i=1}^{size(f)} (f_1(i) - f_2(i))^2} \tag{6.2}$$

6. The set of possible results is analyzed again and Euclidean distance d is calculated between coordinates of blocks of every pair according to equation (6.3). All pairs with distance d smaller than predefined threshold T_d are removed from the set of possible results. Threshold T_d is usually defined according to selected block size (for example, $k \times b$, where k is some small positive constant) to remove all close blocks (it can be assumed that a block is moved more than T_d pixels). After this step only similar pairs of blocks that are not close to each other are kept as possible matches.

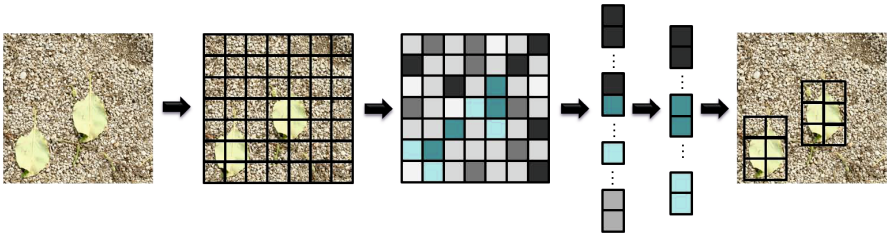


Fig. 6.3 Block-based copy-move forgery detection: 1. preprocessing; 2. overlapping blocks; 3. feature vectors; 4. sorting; 5. detection of similar blocks, 6. detection of CMF

$$d = \sqrt{(x_{f1} - x_{f2})^2 + (y_{f1} - y_{f2})^2} \tag{6.3}$$

7. The detection image is generated by marking all remaining pairs of blocks. Some simple post-processing can be applied to remove small, falsely detected areas in the image (for example, morphological opening).

6.3.2 Possible Feature Vectors

Defining an appropriate feature set is a common problem in block-based methods, because features have to yield similar results for duplicated blocks despite the transformation of the copied area or applied post-processing methods. Different sets of feature vectors for block-based CMFD have been proposed [5].

One of the first approaches used quantized frequency coefficients of the Discrete Cosine Transform (DCT) [9] as features. Thanks to the properties of DCT, it gives good results in cases of added noise, compression, and retouching. A similar approach is presented by Bashar et al. [2], where the coefficients of a Discrete Wavelet Transform (DWT) using Haar-Wavelets was introduced. Bayram et al. [3] recommended using the Fourier-Mellin Transform (FMT) for generating feature vectors.

Popescu and Farid [17] computed Principal Component Analysis (PCA) to reduce the feature set size. This representation is robust to compression and adding of noise, but any transformation of the copied area (for example, scaling, rotation) would affect the eigenvalues. Later this approach is expanded by dividing every block into 4 sub-blocks using the Discrete Wavelet Transform (DWT) [12]. A similar approach to [17] was proposed in [11], where Singular Value Decomposition (SVD) was used.

Luo et al. [15] introduced features based on the intensity of pixels in blocks. The first three values of the feature vectors included the average of the red, blue and green color components. The rest of the feature vector was defined by dividing of the block into 2 equal parts in 4 directions and calculating the ratio of each part's intensity with respect to the intensity of the whole block. Bravo-Solorio et al. [4] used the same three components as in the previous method with the addition of the entropy of a block. A similar approach is presented in [14] where every block was

divided into 4 sub-blocks and the feature vector is defined as a ratio of intensities of those sub-blocks. In the circle approach, proposed in [25], the image is first reduced in dimension by Gaussian pyramid decomposition and every block is divided into four concentric circles. The feature vector is calculated as a mean of the image pixel value in each circular region of every block.

Wang et al. [26] introduced the first four Hu moments as features. The image is first reduced in dimension by Gaussian pyramid decomposition, and the Hu moments are computed from the overlapping blocks of the low-frequency image. The use of Zernike moments of degree 5 as features was proposed by Ryu et al. [19].

6.4 CA for CMF Detection

Cellular automata can be described as a discrete system that contains a regular grid of cells. Each cell can be in only one finite-state determined by the previous states of a surrounding neighbourhood of cells (reference on CA). The use of cellular automata for image processing is interesting because of the property that very simple rules can result in very complex behaviour. Detection of CMF by CA is based on the fact that similar areas in an image should produce similar rules.

The basic approach can be described as a variation of block-based methods for CMFD with a new set of feature vectors. In this approach, the feature vector for each block is obtained using a CA to generate a set of rules that describe the texture of that block [23]. This process can be described as a selection of a subset of rules from all possible rules.

Applying a CA on a greyscale image results in the combinatorial explosion in the number of possible rules, because a whole range of image intensities (256 levels) is used as cell states. For example, using a neighbourhood of 8 pixels for learning rules, leads to 256^8 possible rules. Moreover, the large number of possible rules leads to an even larger number of possible subsets of those rules, which makes it difficult to learn and represent rules efficiently. A reduced description of a neighbourhood can be accomplished by a proper binary representation of image. In that case only two values (0 and 1) are used as cells states leading to a compact description. For comparison, a neighbourhood of 8 pixels on binary image generates only 2^8 possible rules. It is clear that a binary representation of image is more suitable for this purpose, but the problem that arises is to find a proper binary representation. In the following section, we describe several possible solutions.

Another important part of CMFD with CA is the proper selection of neighbourhood. Some tasks can be solved by using simple 1D neighbourhood, but in cases of some transformations, complicated versions of neighbourhood have to be applied.

6.4.1 Representation of Image in Binary

Applying a CA to a grayscale image requires taking the whole range of intensities as a cell states which leads to a large number of possible rules, and a large number of possible subsets of rules. One way to avoid that is to represent the grayscale image

as a binary image, but this must be done in a proper manner that ensures that enough information is kept for further analysis.

6.4.1.1 Image Thresholding

A grayscale image can be represented in binary by simple thresholding of pixel intensity values g_i using a predefined threshold T_b according to equation (6.4).

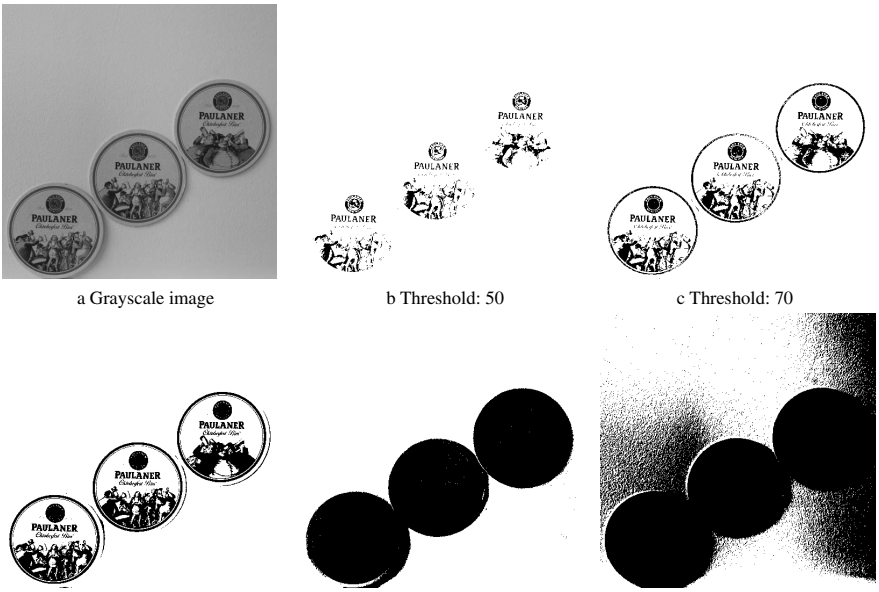
$$g_i = \begin{cases} 1, & g_i \geq t_b \\ 0, & g_i < t_b \end{cases} \tag{6.4}$$

Threshold T_b has to be selected according to image content with the aim of preserving sufficient information about image textures so enough descriptive information is kept for further analysis. Figure 6.4 shows a few examples of grayscale image thresholding using different thresholds. The use of smaller values (for example, 50) results in not enough details in the image, as can be seen in Fig. 6.4b. Larger values for thresholding (Fig. 6.4d) leads to loss of some details in the image. For this example, a good threshold value would be around 70, as can be seen in Fig. 6.4c. Even if using mean or median values for image thresholding appears to be an intuitive solution, global use of those values can lead to unreliable results (see Fig. 6.4e and 6.4f) where most information of image content and texture is lost.

The use of binary images obtained by simple thresholding of grayscale images as an input into CA is not the best solution for CMFD. A problem arises since applying the same (global) threshold to all pixels values results in many homogeneous regions in the binary image, especially in areas with smooth textures. A possible solution for this problem is to generate more binary images with different thresholds, so-called threshold decomposition [18], and use all of them as an input into the algorithm. The final result is then accomplished by combining (summing) the results from all binary images. Even if this approach can lead to good results, better results can be accomplished by different binary representation.

6.4.1.2 Image Binary Planes

Another way of representing a greyscale image in a binary way is to convert all pixel values into binary codes and use each plane of bits as one binary image. The result of this process is 8 different binary images, as can be seen in Fig. 6.5). Naturally, the use of the least significant bits will produce an image similar to noise (Fig. 6.5b), but that effect drops with moving to more significant bits (Fig. 6.5f– 6.5i). Binary images obtained by using more significant bit planes are more similar to images accomplished by thresholding of the greyscale image, especially the image for the most significant bit (Fig. 6.5i). According to that, the use of those images would lead to the same problem as in the case when thresholding is used. To avoid that, images obtained by using lower bits should be used. Even if they look like noise (especially Fig. 6.5b– 6.5d), they contain a detailed description of texture. Also, the lack of uniform white or black areas on those images leads to fewer falsely detected areas.



a Grayscale image b Threshold: 50 c Threshold: 70

d Threshold: 100 e Threshold: mean value f Threshold: median value

Fig. 6.4 Thresholding of a grayscale image

The main problem with this approach is noise sensitivity because adding even a small amount of noise significantly affects the lower bits. In order to reduce noise sensitivity, it is possible to use a binary image obtained by some higher bits. For example, in Fig. 6.5 proper results can be accomplished by using the image produced by the 4th significant bit of pixels values (Fig. 6.5e). That binary image is less sensitive to noise, but still contains enough information about image texture. However, selection of the proper binary level strongly depends on image content. For images with less homogeneous areas, higher levels could also be used as a proper representation.

6.4.1.3 Local Binary Pattern

Local Binary Pattern (LBP) is a simple and efficient texture operator obtained by thresholding the neighbourhood of central pixels and representing the result as a binary number [16]. It is simple to calculate but also has the important property of robustness to illumination change.

The LBP of neighbourhood P and radius R is obtained by using the value of central pixel g_c as the threshold for defining values of neighbourhood pixels g_p according to equation (6.5).

$$LPB(P,R) = \sum_{p=0}^{P-1} s(g_p - g_c) \times 2^p, \qquad s(x) = \begin{cases} 1, & x \geq 0, \\ 0, & \text{otherwise.} \end{cases} \tag{6.5}$$

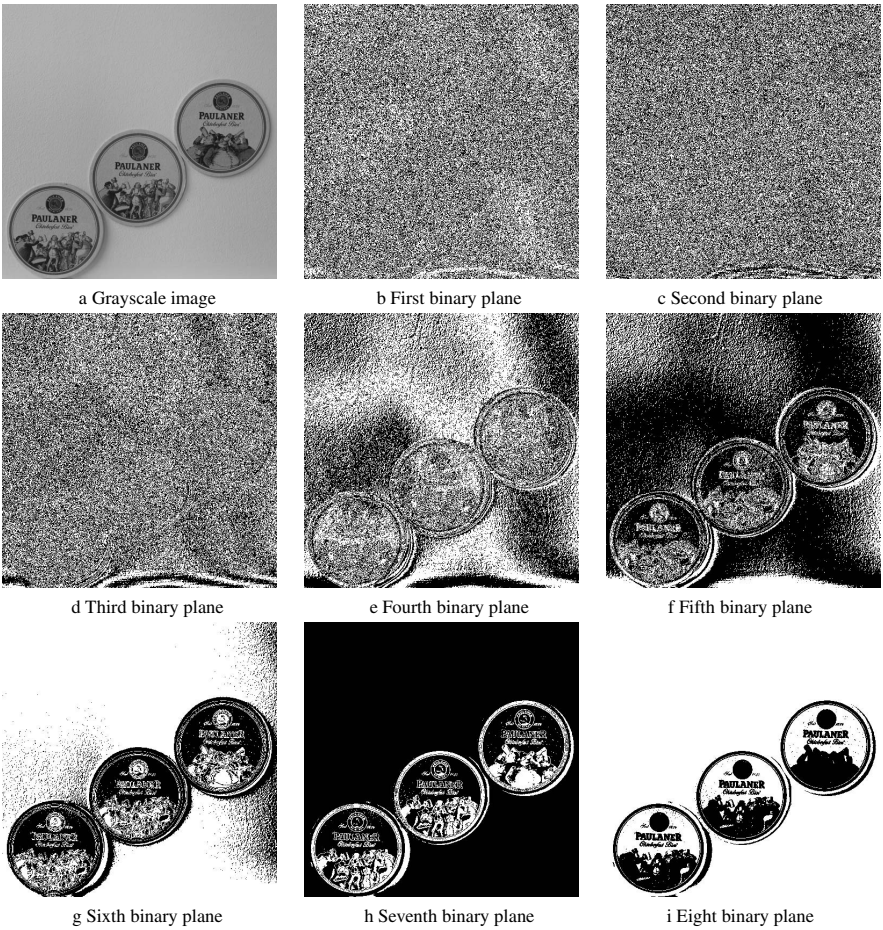
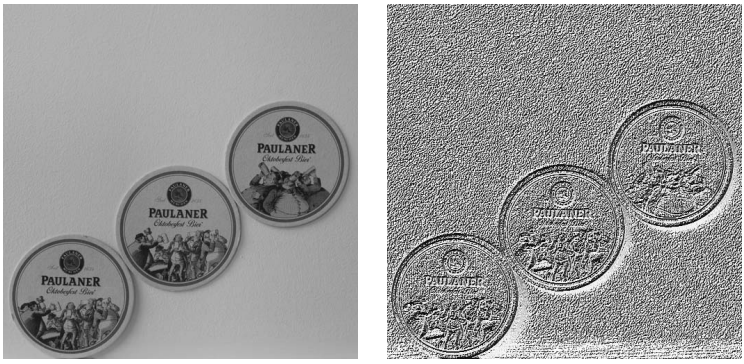


Fig. 6.5 Image binary planes

All values of neighbourhood pixels are represented by one bit according to the value of central pixel, making a binary code that is assigned to the central pixel. An example of a LBP image is shown in Fig. 6.6. It can be seen that LBP combines good properties of thresholding and binary planes, because it preserves global shapes and texture, but it also treats homogeneous region locally leaving enough information for analysis.

Although the LBP image properly presents texture, it still has the same number of intensity levels as a greyscale image (256) so it is not appropriate as image representation for cellular automata. Consequently, representation is defined using LBP but only for calculating binary values of neighbourhood pixels. Every neighbourhood is treated separately, and binary values of pixels in that neighbourhood are defined according to the difference between current pixel and the mean value of all pixels in the neighbourhood and current pixel. Those values are then used as a binary



a Grayscale image b Local binary pattern

Fig. 6.6 Local binary pattern example

representation of texture in every neighbourhood [23]. It is important to notice that representation for every neighbourhood in all blocks is based on the same technique, but it still produce different results thanks to its dependence on mean values.

6.4.2 Plain CMF

Detection of a plain copy-move forgery is the easiest task for any detection algorithm because the goal is simply to find two equal areas in the image. This refers to the fact that properties are not changed during the translation of the copied area to a new location because no transformation (for example, scaling or rotation) is used. Also, in this case we assume that no post-processing is used (handling with post-processing is described in Subsect. 6.4.3). Detection task for plain CMF can be easily solved with the simple 1D CA applied on every block of image [23]:

1. For every pixel p_c in the block define neighbourhood N for CA as a group of k pixels from the row of the image above pixel p_c. Pixels are chosen so that one pixel straight above pixel p_c and an equal number of neighbouring pixels from both sides of p_c is selected according to equation (6.6).

$$N(p_c) = N(p_{x,y}) = \{p_{x+i,y-1}, \quad i = (-\lfloor k/2 \rfloor, ..., \lfloor k/2 \rfloor)\} \tag{6.6}$$

2. For every neighbourhood calculate the mean value using the pixel p_c and its neighbour pixels' intensities. Use a mean value to threshold all pixels' values p_i to binary values b_i according to equation (6.7).

$$b_i = \begin{cases} 1, & p_i \geq mean(N(p_c) \cup p_c) \\ 0, & \text{otherwise} \end{cases} \tag{6.7}$$

3. Use the fast rule identification method proposed by Sun et al. [22] to generate a rule in the block that describes the relation between each pixel p_c and its

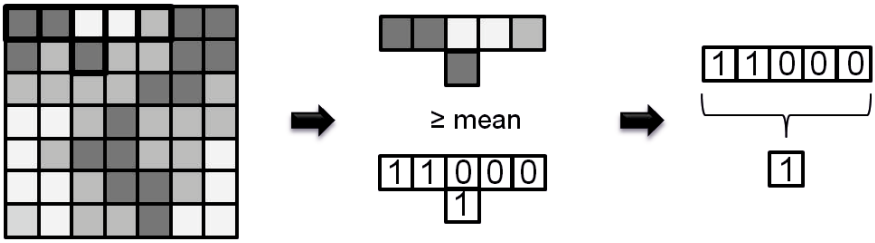


Fig. 6.7 Rule generation using 1D CA

neighbourhood. Note that Sun et al.'s method contains a step of neighbourhood selection, but we can simply ignore it because we have a fixed neighbourhood for each pixel.

The result of this process is a subset of rules for every block in the image that describes the block's texture and is used as a feature vector to distinguish copied areas in the image. Figure 6.7 shows generation of one rule for one neighbourhood in 7×7 block of the image. Neighbourhood of 5 pixels from one row of the image is used for learning pixel p_c. Intensity values of all pixels from the neighbourhood and pixel p_c are used for calculation of mean value and thresholding of pixels intensities to binary values. After thresholding binary values of neighbourhood pixels are used for learning binary value of pixel p_c.

The use of neighbourhood of size k gives total of N_s possible rules, which leads to even higher number of possible subsets of those rules N_{ss}, according to equation (6.8). Thus, number of neighbourhood pixels should be selected considering the computational time and the accuracy of texture description.

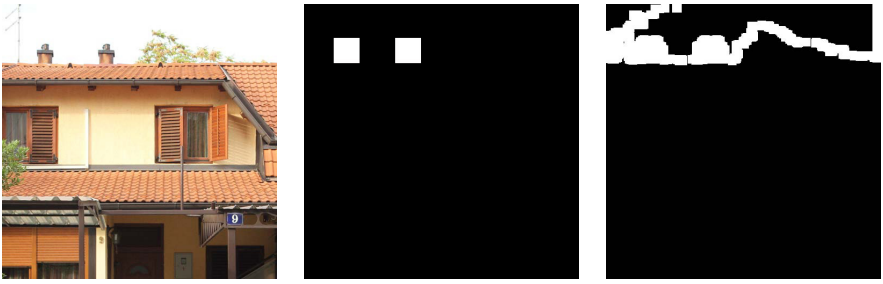
$$N_{ss} = 2^{N_s} = 2^{2^k} \tag{6.8}$$

Figure 6.8 shows a few examples of plain CMF detection on images from the CoMoFoD database [24]. A block size b of 32×32 pixels is used, and neighbourhood size k is set to 7. Values of thresholds are defined as follows: $T_s = 0$ and $T_d = 1.5 \times b = 48$ pixels. The only pre-processing is just for conversion of RGB images into greyscale images, and no post-processing is used (for example, removal of falsely detected areas). Detection is very accurate under different conditions, such as presence of homogeneous regions (Fig. 6.8a) or complex textures (Fig. 6.8b).

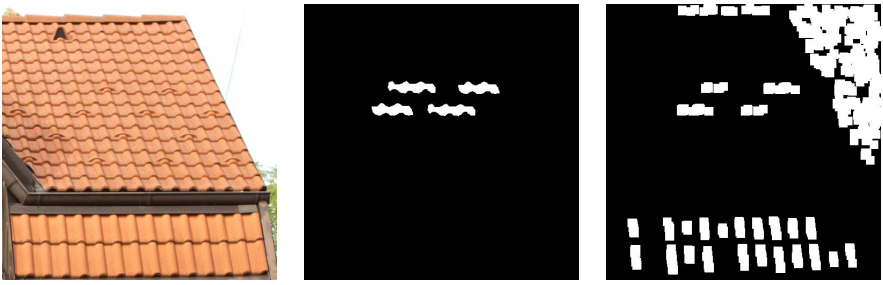
Detection is less accurate in conditions where many areas have very similar properties, for example, areas with very small differences of pixels values. Figures 6.8c and 6.8d show two examples of images where part of the area includes sky and homogeneous regions with very similar values. Beside detection of copied regions, much false detected area is also present, so detection in those cases is not satisfactory. The problem is caused by very small differences between pixels values so that in the thresholding process all blocks have very similar binary representations resulting in similar sets of rules.

a Successful CMFD on image with homogeneous regions (005_F)

b Successful CMFD on image with complex textures (011_F)



c Problem with detection of sky (003_F)



d Problem with detection of homogeneous regions and sky (007_F)

Fig. 6.8 Plain CMFD (left: forged image, middle: expected result, right: actual result)

Table 6.1 shows detection results for 40 plain CMF images from the CoMoFoD database [24]. Block size and threshold T_d are chosen according to the image content in order to maximise the F-measure, threshold T_s is set to zero, and no post-processing is applied. Results show that 82.5% of the images have a F-measure score higher than 0.6, indicating that detection is very accurate. The main problem for detection represents homogeneous regions because in that case many blocks are falsely detected.

Furthermore, results achieved with this method are comparable with results of other CMFD methods. Lower accuracy and weaker performances are noticeable only in the case of large homogeneous regions with many pixels having similar values. In that case, better results are achieved with methods that do not require representation of the image in binary, such as DCT [9] or Zernike moments [19]. Better performance in this case can be accomplished by changing the binary representation to produce a more discriminative description of such similar areas. However, the advantage of the CA method is the very small number of false detected regions in all other tested cases, for example images with more complex textures.

6.4.3 Application on Post-processed Images

Post-processing of forged images introduces differences in pixel values resulting in differences in the binary representation and generated set of rules. One of the common problems is detection of forgery in the image after the addition of noise.

Figure 6.9 shows degradation of performance with adding of Gaussian noise on plain CMF image (simple 1D CA is used for detection). The result is accomplished using image with added noise of zero mean and variance equal to 0.00001 (image intensities were normalized to range [0,1] prior to addition of noise). Block size is set to 32×32 pixels and neighbourhood size of 7 pixels is used. Values of thresholds are $T_s = 6$ and $T_d = 1.5\times b = 48$ pixels. Conversion of the RGB image into a greyscale image is used before noise adding, and no post-processing is applied (for example, removing of false detected areas). Even when the amount of noise is very small, only part of the copied area is successfully detected (Fig. 6.9c). A larger part of the copied area can be detected by adjusting the similarity threshold for detection of similar blocks, but that would also introduce much false detected area.

Coping with noise can be done by using simple pre-processing of image in the form of filtering [23]. Figure 6.10 shows example of CMF detection on image with Gaussian zero mean noise with variance equal to 0.0001. In the case when no pre-processing is applied, detection is not possible at all (Fig. 6.10c). After application of an averaging filter of size 3×3 on noise image, detection becomes very accurate (Fig. 6.10d).

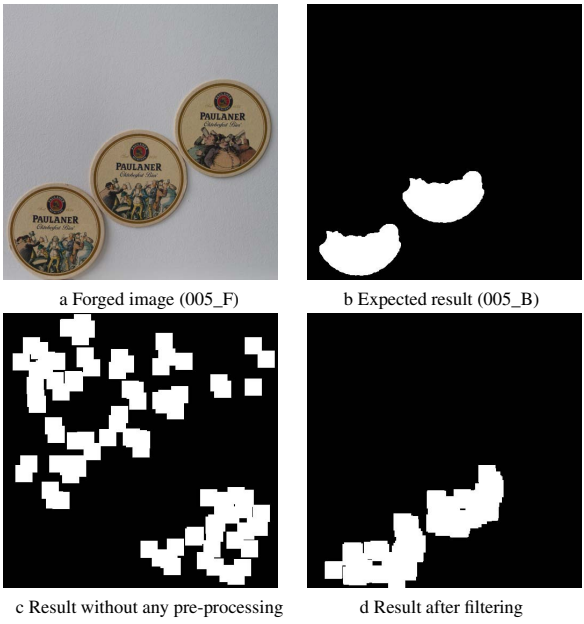
Another very common post-processing method is image blurring. Figure 6.11 shows one example of detection of blurred image accomplished by applying of 3×3 filter. It is possible to notice that detection is quite accurate because blurring does not affect pixels values in a way that changes properties of copied regions.

Table 6.1 Plain CMF detection for images from the CoMoFoD database

Image	Block size	Threshold T_d	Precision	Recall	F-measure
001_F	16	40	0.6759	0.5778	0.6230
002_F	16	35	0.7031	1.0000	0.8257
003_F	16	40	0.2062	1.0000	0.3419
004_F	16	25	0.5643	1.0000	0.7214
005_F	32	40	0.9529	0.9522	0.9526
006_F	32	40	0.9133	1.0000	0.9546
007_F	16	25	0.0766	0.7401	0.1388
008_F	any	any	0.0000	0.0000	0.0000
009_F	32	40	1.0000	1.0000	1.0000
010_F	16	25	0.0384	0.9305	0.0738
011_F	16	40	0.9133	1.0000	0.9546
012_F	16	40	0.1760	0.9930	0.2989
013_F	16	40	0.9756	0.9807	0.9782
014_F	16	40	0.8907	0.9804	0.9334
015_F	16	40	0.9822	0.9932	0.9877
016_F	32	40	0.9316	0.8868	0.9086
017_F	32	40	0.9024	0.9969	0.9473
018_F	16	40	0.8482	0.7370	0.7887
019_F	16	40	0.8392	0.5980	0.6984
020_F	16	40	0.7711	0.7607	0.7659
021_F	16	40	0.9436	0.9558	0.9497
022_F	16	40	0.9207	0.8929	0.9066
023_F	32	40	0.9201	0.7278	0.8128
024_F	32	40	0.9649	1.0000	0.9822
025_F	16	40	0.8081	0.9048	0.8537
026_F	16	40	0.9671	0.9461	0.9565
027_F	16	40	0.9295	0.9271	0.9283
028_F	16	40	0.9767	0.9444	0.9603
029_F	16	40	0.9911	0.9734	0.9822
030_F	16	40	0.9970	1.0000	0.9985
031_F	16	40	0.8873	0.8849	0.8861
032_F	16	40	0.9016	0.9394	0.9201
033_F	16	40	0.8856	0.9728	0.9272
034_F	32	40	0.9709	0.8668	0.9159
035_F	32	40	0.9609	0.9902	0.9754
036_F	any	any	0.000	0.0000	0.0000
037_F	16	40	0.0043	0.2066	0.0084
038_F	16	40	0.9114	1.0000	0.9536
039_F	16	40	0.7508	0.6961	0.7224
040_F	32	40	1.0000	0.9823	0.9911

a Forged image (005_F) b Expected result (005_B) c Result for zero mean noise with variance of 0.00001

Fig. 6.9 Detection of images with noise



a Forged image (005_F) b Expected result (005_B)

c Result without any pre-processing d Result after filtering

Fig. 6.10 Detection of images with Gaussian zero mean noise (variance=0.0001) after filtering

a Forged image (005_F) b Expected result (005_B) c Actual result

Fig. 6.11 Detection of blurred image

6.5 Future Work

The current method is developed with the aim to cope with detection of plain CMF. However, beside simple plain CMF, there are many different variations of forgery where some additional transformation of the copied region is applied. Transformation can include some linear change of size (scaling) or orientation (rotation) of the copied region. Furthermore, transformation can be done as a nonlinear function where the transformation factor is not the same in all directions. Finally, more than one transformation can be applied to the copied region.

The problem of detection of transformed copied area requires the definition of more complicated neighbourhoods. Detection of rotated regions can be solved by using a circular neighbourhood for every block of image. The learning process would remain the same as in the case when 1D CA is used. On the other hand, scaling of the copied region would require using more than one neighbourhood for each block. Those neighbourhoods should have different size to allow detection of changes in sizes of the copied region.

Furthermore, transformation of the copied region introduces small changes in pixels values due to intensity interpolation effects. Those changes do not impact the global texture of the copied region, but they affect the local thresholding process when neighbourhood binary values are defined. Coping with this problem is possible by application of more advanced thresholding methods for representation of images in a binary way.

6.6 Conclusion

Digital image forgery has become a huge problem due to the simplicity of processing of digital images. However, techniques for detection of forged images were introduced in parallel with the development of forgery attacks. Detection methods have some kinds of advantages and disadvantages, but there is still no unique method for detection of all kinds of forgeries. One very common forgery type is copy-move forgery (CMF) where part of the image is selected, copied and moved to a new

location in the same image. Even if this kind of forgery is easy to carry out, successful detection is still a problem of great interest.

Thanks to its property that very simple rules can result in very complex behaviour, cellular automata (CA) are commonly used for various image processing tasks. The basic idea for using a CA for CMF detection is based on an assumption that similar areas on the image should produce similar rules. The main approach is one variation of block-based methods, where the image is divided into overlapping blocks and each block's properties are analysed. Thereby, CA are applied on every overlapping block of forged image with the aim to generate a set of rules. This process can be present as a selection of a subset of rules that describe the texture of block from all possible rules.

Application of a CA on a greyscale image leads to a large number of possible rules and an even larger number of possible subsets of those rules. Reduction of number of rules can be accomplished by a proper binary representation of image, resulting in only two possible values of cells states (as opposed to 256 in case when a greyscale image is used). Thresholding of a greyscale image by global threshold leads to binary image where much information about texture is lost, and usage of binary planes is highly noise sensitive. In order to solve these issues, a new representation of the image based on local binary pattern (LBP) is introduced. LBP defines binary values of neighbourhood pixels based on a difference between central and neighbourhood pixels. Consequently, LBP preserves local information but also keeps enough global images' information.

Detection of copy-move forgery is accomplished using simple 1D CA where the neighbourhood for every pixel is defined as a group of pixels from the row above the pixel under consideration. In our experiments, a neighbourhood size of 7 pixels showed best results according to computational time and detection accuracy. Thresholding to binary values is based on the mean value of all pixels in the neighbourhood and the pixel under consideration. The method is very reliable in different testing conditions, but there are some cases when detection is difficult, such as in the presence of homogeneous areas with small difference between pixels values. In those situations many blocks are false detected as forged. The problem is caused by the presence of similar values of pixels, leading to the same binary representation and the same set of rules for different blocks.

The presented method shows robustness to blurring of the image with averaging filters, but adding noise strongly affects the detection results. However, coping with noise is possible by a simple pre-processing using an averaging filter to smooth the image prior to applying the CA.

References

1. Amerini, I., Ballan, L., Caldelli, R., Bimbo, A.D., Serra, G.: A SIFT-based forensic method for copy-move attack detection and transformation recovery. IEEE Transactions on Information Forensics and Security 6(3), 1099–1110 (2011)
2. Bashar, M., Noda, K., Ohnishi, N., Mori, K.: Exploring duplicated regions in natural images. IEEE Transactions on Image Processing (2010) (accepted for publication)

3. Bayram, S., Sencar, H., Memon, N.: An efficient and robust method for detecting copy-move forgery. In: IEEE International Conference on Acoustics, Speech, and Signal Processing, pp. 1053–1056 (2009)
4. Bravo-Solorio, S., Nandi, A.K.: Exposing duplicated regions affected by reflection, rotation and scaling. In: International Conference on Acoustics, Speech and Signal Processing, pp. 1880–1883 (2011)
5. Christlein, V., Riess, C., Jordan, J., Riess, C., Angelopoulou, E.: An evaluation of popular copy-move forgery detection approaches. IEEE Information Forensics and Security 7(6), 1841–1854 (2012)
6. Farid, H.: Image forgery detection: A survey. IEEE Signal Processing Magazine 26(2), 16–35 (2009)
7. Fridrich, J.: Digital image forensics. IEEE Signal Processing Magazine 26(2), 26–37 (2009)
8. Chen, M., Fridrich, J., Lukáš, J., Goljan, M.: Imaging sensor noise as digital X-ray for revealing forgeries. In: Furon, T., Cayre, F., Doërr, G., Bas, P. (eds.) IH 2007. LNCS, vol. 4567, pp. 342–358. Springer, Heidelberg (2008)
9. Fridrich, J., Soukal, D., Lukás, J.: Detection of copy move forgery in digital images. In: Proc. Digital Forensic Research Workshop (2003)
10. Johnson, M.K., Farid, H.: Exposing digital forgeries by detecting inconsistencies in lighting. In: Proc. ACM Multimedia and Security Workshop, pp. 1–10 (2005)
11. Kang, X., Wei, S.: Identifying tampered regions using singular value decomposition in digital image forensics. In: International Conference on Computer Science and Software Engineering, New York, vol. 3, pp. 926–930 (2008)
12. Li, G., Wu, Q., Tu, D., Sun, S.: A sorted neighborhood approach for detecting duplicated regions in image forgeries based on dwt and svd. In: IEEE International Conference on Multimedia and Expo, pp. 1750–1753 (2007)
13. Lin, C.Y., Wu, M., Bloom, J., Cox, I., Miller, M., Lui, Y.: Rotation, scale, and translation resilient watermarking for images. IEEE Transactions on Image Processing 10(5), 767–782 (2001)
14. Lin, H., Wang, C., Kao, Y.: Fast copy-move forgery detection. WSEAS Transactions on Signal Processing 5(5), 188–197 (2009)
15. Luo, W., Huang, J., Qiu, G.: Robust detection of region-duplication forgery in digital images. IEEE Information Forensics and Security 4, 746–749 (2006)
16. Ojala, T., Pietikainen, M., Maeenpaa, T.: Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. IEEE Transactions on Pattern Analysis and Machine Intelligence 24(7), 971–987 (2002)
17. Popescu, A., Farid, H.: Exposing digital forgeries by detecting duplicated image regions. Tech. rep. tr2004-515, Dartmouth College (2004)
18. Rosin, P.L.: Training cellular automata for image processing. IEEE Transaction on Image Processing 15(7), 2076–2087 (2007)
19. Ryu, S.-J., Lee, M.-J., Lee, H.-K.: Detection of copy-rotate-move forgery using zernike moments. In: Böhme, R., Fong, P.W.L., Safavi-Naini, R. (eds.) IH 2010. LNCS, vol. 6387, pp. 51–65. Springer, Heidelberg (2010)
20. Shivakumar, B.L., Baboo, S.: Detecting copy-move forgery in digital images: A survey and analysis of current methods. Global Journal of Computer Science and Technology 10(7), 61–65 (2010)
21. Shivakumar, B.L., Baboo, S.: Detection of region duplication forgery in digital images using surf. International Journal of Computer Science Issues 8(4), 199–205 (2011)
22. Sun, X., Rosin, P.L., Martin, R.R.: Fast rule identification and neighborhood selection for cellular automata. IEEE Transactions on Systems, Man, and Cybernetics - Part B 41(3), 749–760 (2011)

23. Tralic, D., Rosin, P.L., Sun, X., Grgic, S.: Detection of duplicated image regions using cellular automata. In: International Conference on Systems, Signals and Image Processing (2014) (accepted for publishing)
24. Tralic, D., Zupancic, I., Grgic, S., Grgic, M.: Comofod-new database for copy-move forgery detection. In: Proc. 55th International Symposium ELMAR 2013, pp. 49–54 (2013)
25. Wang, J., Liu, G., Li, H., Dai, Y., Wang, Z.: Detection of image region duplication forgery using model with circle blocks, pp. 25–29 (2009)
26. Wang, J., Liu, G., Zhang, Z., Dai, Y., Wang, Z.: Fast and robust forensics for image region-duplication forgery. Acta Automatica Sinica 35(12), 1488–1495 (2009)

Chapter 7
Active Image Forgery Detection Using Cellular Automata

Ahmad Pahlavan Tafti and Hamid Hassannia

Abstract. The adequate potential of digital images and the ease in their storage and distribution is such that they are more and more exploited to supply information in this digital epoch. As a consequence, they indicate a public source of evidence in our everyday life. Beside their benefits, the accessibility of them could bring a major detriment as they can be modified easily by a media processing application.

Detection of tampering with digital images is still an open work in the image processing domain. Over the past years there has been a swift expansion in the designing and developing of image forgery detection algorithms plus related software applications. All these algorithms are divided into two groups: active and passive. In the active approaches, we create and embed invaluable data as a cipher key into the original image to protect it against the forgery, while in the passive methods we only investigate some features of the image such as statistical anomalies, correlations and compressions to detect forgery.

This chapter presents an in-depth exploration of issues related to active digital image forgery detection algorithms which are derived from cellular automata. The aim of this chapter is to give a brief but comprehensive overview of the usage of cellular automata to develop active image forgery detection techniques. We conclude with experimental results in this topic and discuss future works in image forgery detection using cellular automata.

7.1 Introduction

We are living in an age where security of digital information such as digital images and videos are becoming more important than ever [5]. The expressive potential of

Ahmad Pahlavan Tafti
University of Wisconsin Milwaukee, USA
e-mail: pahlava2@uwm.edu

Hamid Hassannia
Department of Advanced Computing, Fan Pardaz Higher Education Institute, Iran
e-mail: hamid.h.h@ieee.org

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_7, © Springer International Publishing Switzerland 2014

visual media and the ease in their storage, and transmission are more and more exploited to convey information. Together with undoubted benefits, the accessibility of digital images brings a major drawback. With development of low cost, powerful image editing tools, the craft of tampering visual content is no more restricted to experts, so they can easily change image content and also it's meaning without leaving any traceable effect.

Image forgery detection methods in computer vision are quite able to authenticate the entire content of a digital image and protect them against tampering. A reliable images forgery detection system will be useful in many areas such as surveillance systems, medical imaging, criminal investigation, journalism, visa and immigration documents, insurance processing and forensic investigation.

The forgery detection techniques that are developed for digital images are mainly classified into two major classes, active and passive [2, 5] While in the active methods we would like to insert data or signature at the time of digitizing, the passive methods operate in the absence of any data or signature [2, 5]. In the active methods, we embed data into the original image to protect it against the forgery, where in the passive methods we don't have the original image and we should investigate some features such as statistical anomalies, correlations, compressions and measurements of objects in the existence image to detect forgery [4, 5].

Active approaches can be divided into two categories by the embedding in the position of spatial domain or frequency domain data [4]. Spatial domain techniques have already developed and are easier to implement but are limited in robustness [3]. Data embedding in the spatial domain consists of insertion and detection stages. The insertion algorithms are used to embed the data into the digital image and detection algorithms extract those data.

On validation and authentication aspects, the data which is embedded in a spatial domain should be unpredictable, invisible and also sensitive to any modification [3, 5].

In 2013 Anoop et al. [1] presented a full image encryption algorithm base on transform domain and stream cipher. In 2009 Krikor et al. [9] used DCT and stream cipher for digital image encryption. In 2003 Pommer et al. [13] provided an image encryption approach using selective encryption of wavelet packet. In 2003 Droogenbroeck et al. [19] developed Triple DES and IDEA based approach for the purpose of digital image encryption.

Using cellular automata as a discrete model is another way to generate such intricate information. This information would be embedded in a particular domain of an image for the purpose of image encryption and digital image forgery detection. In 2013 Xiaoyang et al. [20] used elementary cellular automata state rings to encrypt and decrypt QR code binary image. In 2012 Jin [8] developed an image encryption approach using the behavior of a number of Elementary Cellular Automata (ECA) with periodic boundary conditions. In 2011 Malakooti et al. [16] proposed a method to use the one dimensional cellular automata including statistical information of a digital image as an operational and practical way for image forgery detection. See also chapter 5.6 in this book which describes a passive method for copy-move forgery detection using cellular automata.

The goal of this chapter is to introduce such a framework, to propose cellular automata for implementing image forgery detection system and to give experimental results. In this chapter we proposed two active methods to detect digital image forgery in a reliable manner. The aim of this work is to develop a framework to active image forgery detection using cellular automata. The proposed methods take a digital image as input and compute some statistical information from its Lower Upper (LU) and Singular Value Decomposition (SVD). We use singular value decomposition and also lower upper decomposition plus one dimensional cellular automata to generate a cipher key. This key has the image features and completely related to digital image that every small change in the content of digital image will change the key value without any exception.

The rest of the chapter is arranged as follows. In Sections 7.2 and 7.3 we develop two different scenarios to image forgery detection based on a cellular automata. Section 7.4 introduces our sample dataset and describes the experimental results. Section 7.5 discusses limitations of the proposed models. Conclusion and areas for future development is considered in Section 7.6.

7.2 Scenario 1: Using Cellular Automata and LU Decomposition

LU decomposition is a kind of matrix decomposition which composes a matrix by the product of a lower and an upper triangular matrix [6]. Let A be a square matrix. An LU decomposition is a matrix decomposition of the form $A = LU$, where L and U are lower and top triangular matrices of the same dimensions. This means that L has just zeros overhead the diagonal and U has only zeros underneath the diagonal [6]. For a 3×3 matrix, this becomes:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

Section 7.2.1 describes the usage of LU decomposition for the proposed active forgery detection algorithm. Let's a little describing one dimensional cellular automata we want to use in our proposed model. In this book, there are some chapters with details descriptions on cellular automata, so we just give a really brief overview which is necessary for the proposed model. Figure 7.1 shows a simple two state and one dimensional cellular automata with a line of cells. The state of X at the time $t+1$ will be determined by the states of the cells within its neighborhood at the time t [10, 17, 18].

We can define and set our own local rule for each cellular automata. You can see just two example of how we may define a local rule and how does it work. Let us to consider two following rules to estimate the value of cell X at time $t+1$ based on the value of cell X at the time t:

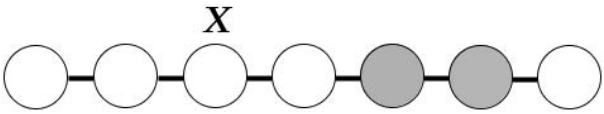


Fig. 7.1 One dimensional cellular automata with three neighborhoods for cell *X*

Rule 1: Cell[*X*] (*t*+1) = Cell[*X*-1] (*t*) (OR) Cell[*X*+1] (*t*)
Rule 2: Cell[*X*] (*t*+1) = Cell[*X*-1] (*t*) (AND) Cell[*X*+1] (*t*)

As long as we consider Rule 1, and the input sequence equals to 01100, then output sequence will be 11110. Table 7.1 shows the output of this cellular automata using an OR local rule.

By using Rule 2, while the input sequence equals to 01110, then output sequence will be 00100 (Table 7.2).

In this scenario, we propose a digital image forgery detection algorithm based on the cellular automata and LU decomposition. Experiments for this scenario will be described in detail in Section 7.4.

7.2.1 Proposed Model

The main idea of this proposed algorithm is to protect a digital image against forgery by creating and embedding an unpredictable cipher key into the spatial domain of an image. We embed the bit sequence of the cipher key into the LSB (Least Significant Bit) of the particular pixels in the original image because it takes less time on embedding. Section 7.4 shows some experimental validations obtained from embedding into different places. Using LSB decreases time consumption and may cause to reduce sensitivity to some attacks.

Our proposed algorithm performs on a grayscale images and generates a .png file including lossless PNG image with the grayscale model. We generate PNG image because generating any other formats like JPEG image would result into losing secret key embedded into pixel's LSB. The input type is not important in the proposed method and it may perform the same process on the different types of digital images such as RGB or CMYK, and different formats like .bmp, .gif, .pgm and etc. For

Table 7.1 An example of cellular automata

Cell Number	**0**	**1**	**2**	**3**	**4**
Input Sequence (time t)	0	1	1	0	0
Cellular Automata Rule	$Cell[X]^{t+1} = Cell[X-1]^{t}$ (OR) $Cell[X+1]^{t}$				
Output Sequence (time $t+1$)	1	1	1	1	0

Table 7.2 An example of cellular automata

Cell Number	**0**	**1**	**2**	**3**	**4**
Input Sequence (time t)	0	1	1	1	0
Cellular Automata Rule	$Cell[X]^{t+1} = Cell[X\text{-}1]^t$ (AND) $Cell[X+1]^t$				
Output Sequence (time $t+1$)	0	0	1	0	0

drawing a block diagram of our method (Figure 7.2), we just assume that the input type would be a .jpeg image.

Cellular automata have been implemented to create the required cipher key bit sequence. The XOR local rule used to generate the result in this chapter. We generate a cipher key by using specific cellular automata with an XOR local rule on six cells (see Table 7.3). In 2013 Sharma et al. [14] proposed a text security approach based on a XOR rule within 2D cellular automata and get well formed results. In 2011 Prasad Panda et al. [12] proposed a cellular automata encryption and decryption algorithm for block cipher based on XOR rules. Our experiments also show that a XOR logical operation often suffices to obtain a very sensitive cipher key. We have achieved a better rate for 'True alert' indicates true forgery detection, by using XOR rule rather than some other logical rules and also arithmetic rules (Table 7.6). Based on the experimental validations, our proposed XOR rule could improve PSNR (Table 7.7). Furthermore, it can be done extremely fast on contemporary CPUs that mostly provide a specific instruction to do a XOR operation [7].

We only use three number of statistical information of the LU decomposition matrices of the original image to generate the cipher key. This information consists of arithmetic mean, median, and the statistics range (Table 7.3). If anybody wants to modify a digital image, then the statistical information of these particular matrices will be changed, so the output of the proposed cellular automata will be damaged.

In this proposed method we consider single iteration cellular automata, implementing zero padding to obtain a valid value for boundary cells (Cells 0 and 5 in Table 7.3). We add zeros to both front and rear of our cellular automata so that the

Table 7.3 Proposed six cells for the cellular automata with XOR local rule: $Cell[X]^{t+1} = Cell[X-1]^t$ (XOR) $Cell[X+1]^t$

Cell Number	**Input Value (time t)**
0	Mean of the values in the L Matrix from LU decomposition of the original image
1	Mean of the values in the U Matrix from LU decomposition of the original image
2	Median of the values in the L Matrix from LU decomposition of the original image
3	Median of the values in the U Matrix from LU decomposition of the original image
4	Range of the values in the L Matrix from LU decomposition of the original image
5	Range of the values in the U Matrix from LU decomposition of the original image

rule could be applied to boundary cells. Since we are facing with the real input values (i.e. mean and range) for our proposed cellular automata, and we intend to apply XOR logical operation as a local rule, so we have to convert them to binary values. We foremost convert from real to integer values to avoid existing alteration complexities and limitations, applying recursive integer to binary alteration algorithm.

Finally, we embed the output of the proposed cellular automata into the LSB (Least Significant Bit) of the first eight pixels in the original image. LSB substitution is the process of modifying the least significant bit of the pixels of the input image. By doing this process, the value of a pixel is changed slightly, so the changes are not reflected physically in the output image. The proposed algorithm has been applied on grayscale images with pixel values between 0 and 255 in which the specific statistical information could be in the same range. Therefore, a byte should be enough to store output value as a cipher key. The experiments indicate that the LSB usage of the first eight pixels in this step is almost proper to obtain a well result on both time consuming and PSNR.

Here we briefly describe the statistical operations which are used in the proposed cellular automata.

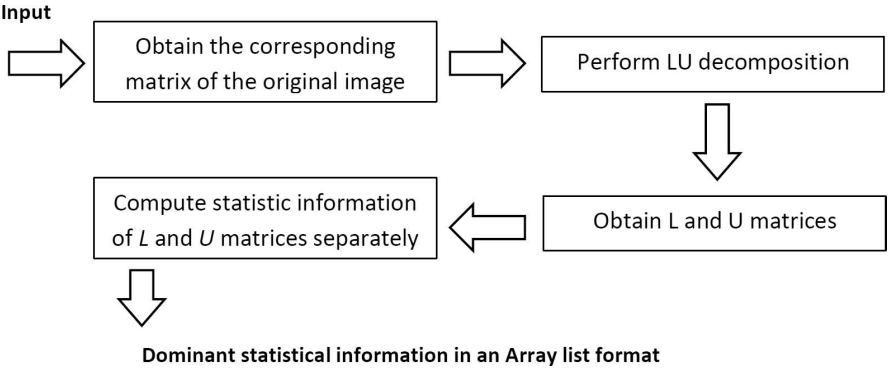


Fig. 7.2 Block diagram of the proposed method

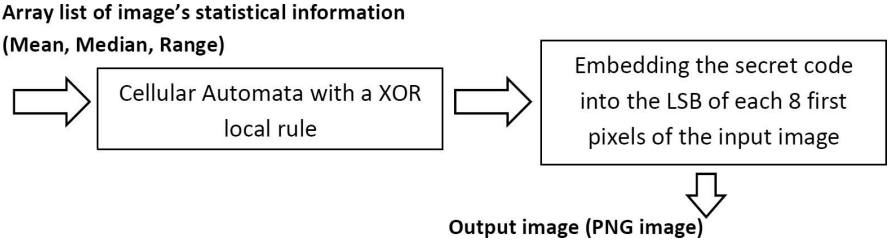


Fig. 7.3 Block diagram of the proposed cellular automata for cipher key creation

Figure 7.2 shows the block diagram of the proposed method and Figure 7.3 illustrates the diagram of the proposed cellular automata to create a cipher key, based on statistical information of the L and U matrices.

The embedding algorithm in a spatial domain of the original image will be described in detail as follows:

Data Embedding Algorithm

Input: grayscale image to apply data embedding to it for forgery detection.

Output: .PNG grayscale image file.

Step1: Open the original image and get the corresponding matrix form of that. We know that a matrix is the certain underlying object of each digital image.

Step2: Perform the LU decomposition and obtain L and U matrices.

Step3: Calculate the statistical information for each L and U matrices separately, converting them to binary, and create the array list as the input values of the cellular automata.

Step 4: Zero padding to apply the rule to the boundaries.

Step5: Perform the cellular automata rule according to the Table 7.2. This rule performs on the array list to create a cipher key. (This step describes in detail in the following section: Forgery Detection Algorithm)

Step6: Convert the cipher keys to the binary representation.

Step7: Select the first eight pixels in the original image and embed the binary sequences of cipher key into the LSB of these eight pixels.

The forgery detection algorithm will be as follows:

Forgery Detection Algorithm

Input: .PNG image that contains the cipher key.

Output: Digital image forgery detection ALARM.

Step1: Open the .PNG input image and make corresponding digital image matrix.

Step2: initial integer variable CipherValue to zero.

Step3: initial integer variable PixelArrayValue to zero.

Step4: Perform the LU decomposition and obtain L and U matrices.

Step5: Calculate the statistical information for each L and U matrices separately and create the array list of these values.

Step6: Perform the cellular automata rule according to the Table 7.3. This rule performs on the array list to create a cipher key.

Step7: Select the first eight pixels of the image and extract the LSB binary value of pixels.

Step8: set CipherValue = value of the cipher key that generated in Step 6.

Step9: set PixelArrayValue = the extraction value in Step 7.

Step10: If PixelArrayValue = = CipherValue then print message ‘False Forgery Alarm’

Else Print message ‘True Forgery Alarm’;

This scenario is applied on grayscale images and it is definitely extendable to apply on color images.

7.3 Scenario 2: Using Cellular Automata and Singular Value Decomposition

Here, we define our second algorithm for digital image forgery detection. This part continues by providing a very brief description of Singular Value Decomposition (SVD), followed by our proposed model. Experiments for the second scenario will be also described in detail in Section 7.4.

SVD [6, 11] is very important in many areas of science. It is a way to very compactly represent what a matrix does to space. SVD can be seen as a generalization of eigenvalues and eigenvectors to a non-square matrix. It is very useful for solving linear algebraic problems like matrix inversion, linear least square estimation and fix-ranked approximation.

7.3.1 Proposed Model

The proposed method here is again based on the active approaches. Two set of dominant attributes that achieve from SVD (eigenvalues and eigenvectors) and one dimensional cellular automata could provide and generate the secret key. Here and in contrast to scenario 1, we prefer to work on RGB digital images to design such a scalable and flexible algorithm. We firstly achieve the eigenvalues and eigenvectors of the Red matrix (Red layer of the RGB image) of the input image and perform the same task for the Green matrix (Green layer of the RGB image). We calculate the eigenvalues and eigenvectors of the image, implementing one dimensional and single iteration cellular automata with a XOR local rule to create the secret key, based on those values. Next, we embed the bit sequence of the secret key into the LSB of the first eight pixels of the Blue layer (Blue Matrix) in the original image. The reason of using XOR logical operation and first eight pixels for embedding purpose was illustrated in previous section. We need to use a real to binary conversion same as the scenario 1.

Table 7.4 shows the main idea of our proposed cellular automata. Only eight number of values of the original image have been used to generate this key. These values consist of sum of eigenvalues, sum of eigenvectors, mean of eigenvalues and mean of eigenvectors.

All of these values are easy to calculate and also exclusive for a particular matrix. Figure 7.4 shows the block diagram of the proposed method and Figure 7.5 illustrates the diagram of the proposed cellular automata to create a secret key, based on these attributes of an image. You see a .png format as an input image in this figure, but our proposed model is suitable and applicable for any format of RGB digital images.

Table 7.4 Proposed eight cells for the cellular automata with XOR local rule: $Cell[X]^{t+1} = Cell[X-1]^t$ (XOR) $Cell[X+1]^t$

Cell Number	Input Value (time t)
0	Sum of all singular values of the Original Image (Red Matrix)
1	Mean of all singular values of the Original Image (Red Matrix)
2	Sum of all right singular vectors of the Original Image (Red Matrix)
3	Mean of all Eigenvectors Numbers of Red Matrix of the Original Image
4	Sum of all Eigenvalues Numbers of Green Matrix of the Original Image
5	Mean of all Eigenvalues Numbers of Green Matrix of the Original Image
6	Sum of all Eigenvectors Numbers of Green Matrix of the Original Image
7	Mean of all Eigenvectors Numbers of Green Matrix of the Original Image

The embedding algorithm into LSB of eight pixels of the Blue layer of the input image will be as follows:

Data Embedding Algorithm

Input: RGB image to apply data embedding to it for forgery protection.

Output: .PNG RGB image file.

Step1: Open the original image and get the corresponding Red, Green and Blue matrices form of that. We know that a matrix is the certain underlying object of each digital image.

Step2: Calculate Eigenvalues and Eigenvectors of the Red Matrix and create the array list as the input values of the cellular automata.

Step3: Calculate Eigenvalues and Eigenvectors of the Green Matrix and create the array list as the input values of the cellular automata.

Step 4: Zero padding to apply the rule to the boundaries.

Step5: Perform the cellular automata rule according to the Table 7.4. This rule performs on the array list to create a Secret key. (This step describes in detail in the following section: Forgery Detection Algorithm)

Step6: Convert the Secret key to the binary representation.

Step7: Select the first eight pixels in Blue Layer (Blue Matrix) and embed the binary sequences of Secret key into the LSB of each pixel.

The forgery detection algorithm will be as follows:

Forgery Detection Algorithm

Input: .PNG image that contains a Secret key.

Output: Digital image forgery detection alarm.

Step1: Open the .PNG input image and make digital image matrix.

Step2: initial integer variable SecretValue to zero.

Step3: initial integer variable EigenVsArrayValue to zero.

Step4: Calculate Eigenvalues and Eigenvectors of the Red Matrix.

Step5: Calculate Eigenvalues and Eigenvectors of the Green Matrix.

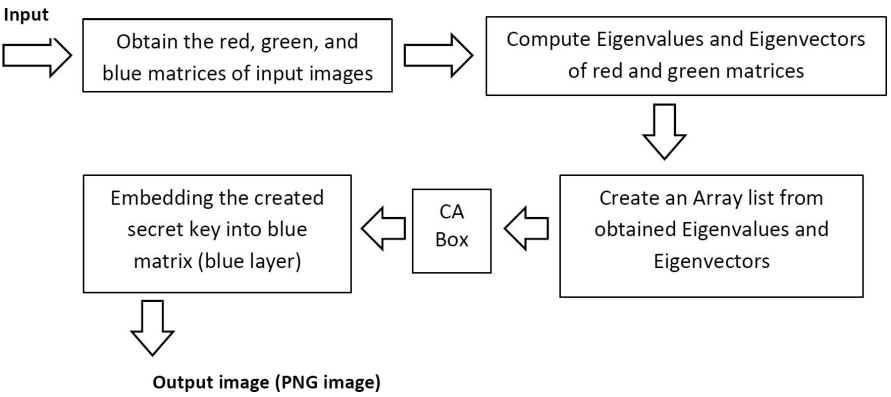


Fig. 7.4 Block diagram of the proposed method

Step6: Perform the cellular automata rule according to the Table 7.2. This rule performs on the array list to create a Secret key.

Step7: Select the first eight pixels of Blue layer and extract the LSB binary value of each pixel.

Step8: set SecretValue = value of the Secret key that generated in Step 4 and 5 and 6.

Step9: set EigenVsArrayValue = the extraction value in Step 7.

Step10: If EigenVsArrayValue = = SecretValue then print message 'False Forgery Alarm'

Else Print message 'True Forgery Alarm';

7.4 Dataset and Experimental Results

Table 7.5 shows our sample datasets and resources we have in our research laboratory in the Department of Advanced Computing, Fan Pardaz Higher Education Institute.

Table 7.5 Our dataset

Image File Format	.png and .jpeg
Image Type	Grayscale and RGB
Image Size	800*800 and 2560*1920
Number of Images	.png = 127 .jpeg = 149
Image Contents	Official digital documents, Medical images, Portrait

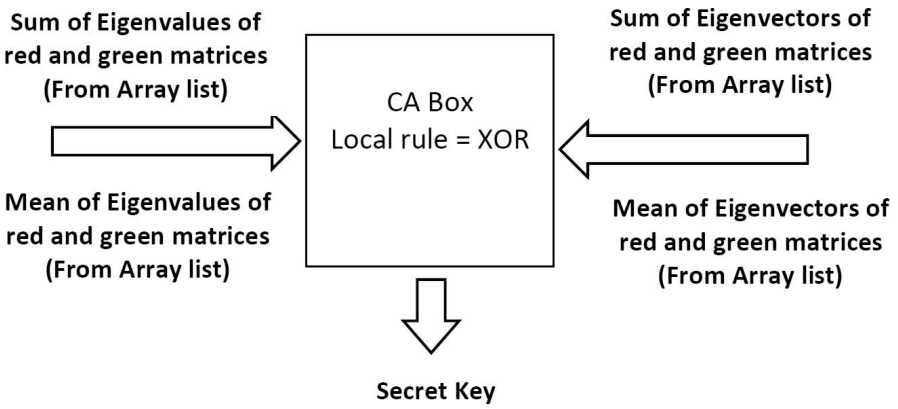


Fig. 7.5 Block diagram of the proposed cellular automata for cipher key creation

To prove the general performance of the proposed forgery detection methods, extensive experiments using real images were carried out. In particular, a configurable system has been built to implement both proposed algorithms. This system was built using JAVA SE with a simple and friendly user interface. All the experiments were carried out on a 3.00 GHz Intel Dual core 4MB cache with 4GB of RAM running 64-bit Windows 7 operating system. Seven experiments will be presented in this section to show the implementation and the results of the proposed methods. These are as follows:

- Performance and visual quality
- Time consumption
- True and False Alert
- PSNR
- Cipher key sensitivity
- Diffusion
- Comparison with non-CA works

In order to evaluate the above aspects of our proposed method, we perform several tests on the dataset (Table 7.5).

7.4.1 Performance and Visual Quality

Figure 7.6 shows the original images and data embedded output .png images which generated via scenario 1 to prove the performance and visual quality of the proposed method. By considering same images, input and output images using scenario 2 are shown in Figure 7.7.

Original Images Output Images

Fig. 7.6 Scenario 1: input and output images

7.4.2 Time Consumption

We select the first eight pixels of the original image to embed the proposed cipher, as we mentioned in Sections 7.2 and 7.3. Figure 7.8 presents the results of different pixels selection with different time consumption in which 50 images for each scenario separately. Figure 7.9 shows the same results with using 100 images. In this experiment we only used 50 and 100 images from the dataset and this number of images is only a random selective set of digital images we had. These figures show that minimum time consumption is obtained by embedding the cipher key into the first eight pixels of the original image.

7.4.3 True and False Alert

In Table 7.6, you can see the percentage of true and false detection of digital image forgery which are performed by the proposed method. We compare these two indices with different local rule for proposed cellular automata in Sections 7.2 and 7.3, plus different position to embed data into the original image. We use exactly same images for both scenarios. In this experiment, three various logical operations (XOR, AND, and OR) and two different arithmetic operations (Addition and Multiply) are implemented as our cellular automata's local rule.

As shown in this table, and by considering previous results (Time consumption), scenario 2 using XOR local rule with data embedding in only first eight pixels of the original images are better in average.

Original Images Output Images

Fig. 7.7 Scenario 2: input and output images

7.4.4 PSNR

Measuring the PSNR (peak signal-to-noise ratio) of our proposed method is the next experiment. It indicates the maximum possible power of a signal and the power of corrupting noise that affects the output. We mentioned that all pixels in both of the input and output images in our proposed method are based on 8 bits. The result of PSNR for our proposed algorithm is shown in Table 7.7. Typical values for the PSNR in lossy image and video compression are between 30 and 50 dB, where higher is better [3], therefore, our proposed algorithm has a good PSNR.

This experiment indicates that scenario 1 produces a very little better PSNR than scenario 2. This kind of experiment in the dataset indicates that using logical operation instead of arithmetic operation was not generating quite different result regarding the PSNR.

7.4.5 Secret Key Sensitivity

An ideal digital image encryption system should be sensitive with respect to the secret key. A little change of a single byte in the secret key should generate a completely different encrypted image and vice versa. Table 7.8 shows the rate of secret key sensitivity. Previous experiments indicate that scenario 2 is better than scenario 1 in average, so in this experiment we just focus on scenario 2. 'Sara' is an image which is shown at the first row of Figure 7.7 and 'Forest' is the second one. Since one of the our goal is to develop sensitive cipher key for image forger detection

Table 7.6 True and false detection of the proposed approaches. Three different logical operations and two arithmetic operations are used to evaluate a local rule of the proposed cellular automata. We use a hundred images for both scenarios.

Approach	True Alert %	False Alert %
Scenario 1: XOR operation + embedding into first eight pixels	94.61	5.39
Scenario 1: XOR operation + embedding into first and middle eight pixels	94.61	5.39
Scenario 1: XOR operation + embedding into first, middle and end eight pixels	94.79	5.21
Scenario 1: AND operation + embedding into first eight pixels	88.67	11.33
Scenario 1: AND operation + embedding into first and middle eight pixels	88.68	11.32
Scenario 1: AND operation + embedding into first, middle and end eight pixels	88.71	11.29
Scenario 1: OR operation + embedding into first eight pixels	88.29	11.71
Scenario 1: OR operation + embedding into first and middle eight pixels	88.29	11.71
Scenario 1: OR operation + embedding into first, middle and end eight pixels	88.32	11.68
Scenario 1: Addition operation + embedding into first eight pixels	81.44	18.56
Scenario 1: Addition operation + embedding into first and middle eight pixels	81.47	18.53
Scenario 1: Addition operation + embedding into first, middle and end eight pixels	81.50	18.50
Scenario 1: Multiply operation + embedding into first eight pixels	91.14	8.86
Scenario 1: Multiply operation + embedding into first and middle eight pixels	91.15	8.85
Scenario 1: Multiply operation + embedding into first, middle and end eight pixels	91.19	8.81
Scenario 2: XOR operation + embedding into first eight pixels	96.23	3.77
Scenario 2: XOR operation + embedding into first and middle eight pixels	96.25	3.75
Scenario 2: XOR operation + embedding into first, middle and end eight pixels	96.29	3.71
Scenario 2: AND operation + embedding into first eight pixels	86.09	13.91
Scenario 2: AND operation + embedding into first and middle eight pixels	86.09	13.91
Scenario 2: AND operation + embedding into first, middle and end eight pixels	86.17	13.83
Scenario 2: OR operation + embedding into first eight pixels	86.01	13.99
Scenario 2: OR operation + embedding into first and middle eight pixels	86.03	13.97
Scenario 2: OR operation + embedding into first, middle and end eight pixels	86.08	13.92
Scenario 2: Addition operation + embedding into first eight pixels	84.73	15.27
Scenario 2: Addition operation + embedding into first and middle eight pixels	84.73	15.27
Scenario 2: Addition operation + embedding into first, middle and end eight pixels	84.77	15.23
Scenario 2: Multiply operation + embedding into first eight pixels	92.32	7.68
Scenario 2: Multiply operation + embedding into first and middle eight pixels	92.32	7.68
Scenario 2: Multiply operation + embedding into first, middle and end eight pixels	92.37	7.63

purposes, we can examine how pixel's value changes results appear in related values of the cipher key. This table shows that even small changes in pixel values of the original image can make a different value for the cipher key.

In this scenario, we propose a digital image forgery detection algorithm based on the cellular automata and LU decomposition. Experiments for this scenario will be described in detail in Section 7.4.

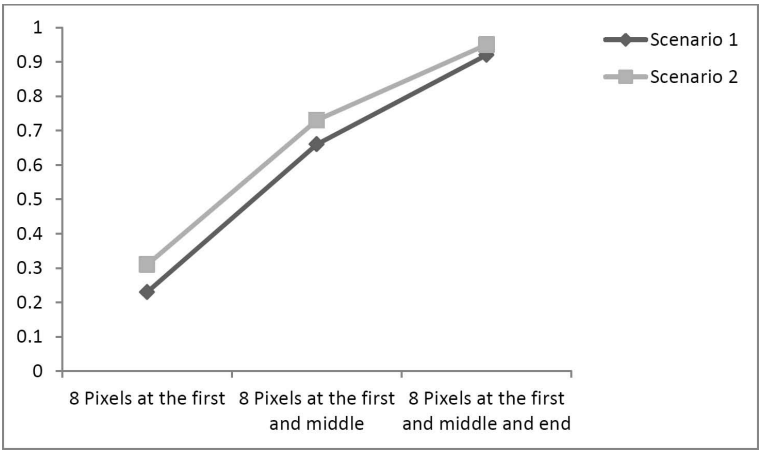


Fig. 7.8 Time consumption for embedding the cipher key into the original image. Here, we used 50 digital images for each scenario. Vertical axis shows the time consumption in seconds.

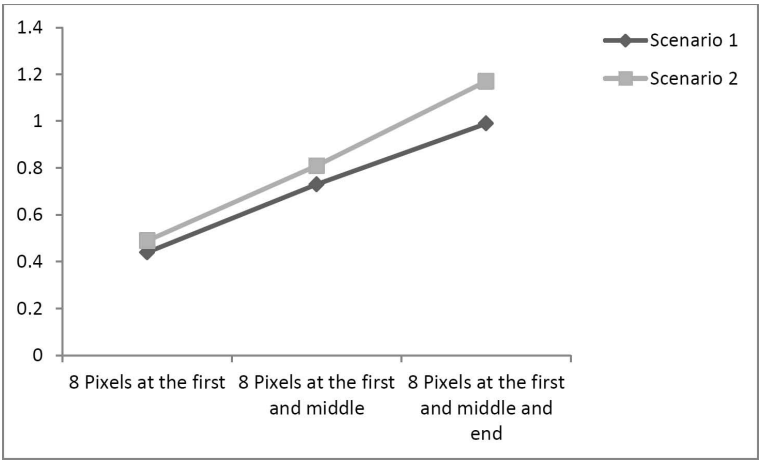


Fig. 7.9 Time consumption for embedding the cipher key into the original image. Here, we used 100 digital images for each scenario. Vertical axis shows the time consumption in seconds.

7.4.6 Diffusion

In this experiment the diffusion of our secret key is considered. Diffusion means that the output bits should depend on the input bits in a very complex way. In a secret key with good diffusion, if one bit of the plaintext is changed, then the secret key should change completely [15]. More generally, one may require that flipping a fixed set of bits should change each output bit with probability one half.

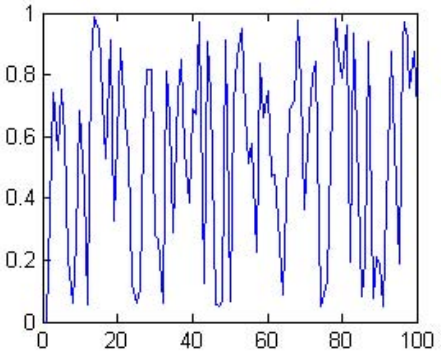


Fig. 7.10 Diffusion Chart for Proposed Secret Key (Average in both scenarios). Row indicates the number of images and column indicates the random number which is between 0 and 1.

Figure 7.10 shows the diffusion chart of our proposed model of generating the secret key. We only examine XOR local rule for this diffusion evaluation. This figure exposes high relative distribution index from 0 to 1, indicating well-founded diffusion for the proposed secret key.

7.4.7 Comparison with Non-CA Works

In Table 7.9, we compared our proposed method with other non-CA algorithms mentioned in literature review (Section 7.1).

There are two level for digital images encryption; high-level and low-level. In the high-level encryption the content of the digital image is completely disordered and the original image is invisible. In low-level encryption, the content of the digital image is understandable and visible. The proposed algorithm in this chapter generates visible images (Figure 7.6 and Figure 7.7) and it is not high-level encryption method in which we will face with really disordered image.

Our proposed CA approach has used the internal information of a digital image instead of the some logo and external information. Using external logo or other data may cause to exceed the size of the output images.

7.5 Limitations

The proposed cellular automata's are only one dimensional and it could be considered as a limitation for our system. The other limitation is using active approach which is needed the original image for forgery detection. Similarly, we do not explicitly model noise as an external effect. The main limitation of the proposed method is sensitivity to post processing.

Table 7.7 PSNR evaluation of the proposed approaches. Three different logical operations and two arithmetic operations are used in this experiment. We used a hundred images for both scenarios, putting the average in this table.

Approach	PSNR
Scenario 1: (XOR operation + embedding into the first and middle eight pixels	39.11
Scenario 1: XOR operation + embedding into first, middle and end eight pixels	39.04
Scenario 1: AND operation + embedding into first eight pixels	39.25
Scenario 1: AND operation + embedding into first and middle eight pixels	39.21
Scenario 1: AND operation + embedding into first, middle and end eight pixels	39.01
Scenario 1: OR operation + embedding into first eight pixels	39.27
Scenario 1: OR operation + embedding into first and middle eight pixels	39.22
Scenario 1: OR operation + embedding into first, middle and end eight pixels	39.16
Scenario 1: Addition operation + embedding into first eight pixels	37.76
Scenario 1: Addition operation + embedding into first and middle eight pixels	37.58
Scenario 1: Addition operation + embedding into first, middle and end eight pixels	37.51
Scenario 1: Multiply operation + embedding into first eight pixels	37.69
Scenario 1: Multiply operation + embedding into first and middle eight pixels	37.57
Scenario 1: Multiply operation + embedding into first, middle and end eight pixels	37.42
Scenario 2: XOR operation + embedding into first eight pixels	38.13
Scenario 2: XOR operation + embedding into first and middle eight pixels	38.07
Scenario 2: XOR operation + embedding into first, middle and end eight pixels	37.92
Scenario 2: AND operation + embedding into first eight pixels	38.19
Scenario 2: AND operation + embedding into first and middle eight pixels	38.11
Scenario 2: AND operation + embedding into first, middle and end eight pixels	38.05
Scenario 2: OR operation + embedding into first eight pixels	38.28
Scenario 2: OR operation + embedding into first and middle eight pixels	38.16
Scenario 2: OR operation + embedding into first, middle and end eight pixels	38.08
Scenario 2: Addition operation + embedding into first eight pixels	36.14
Scenario 2: Addition operation + embedding into first and middle eight pixels	36.10
Scenario 2: Addition operation + embedding into first, middle and end eight pixels	36.01
Scenario 2: Multiply operation + embedding into first eight pixels	36.36
Scenario 2: Multiply operation + embedding into first and middle eight pixels	36.27
Scenario 2: Multiply operation + embedding into the first, middle and end eight pixels)	36.12

Table 7.8 Evaluation of secret key sensitivity and its dependency to the original image's changing

Image	Manipulation	Sum of Eigenvalues (Red Layer)	Mean of Eigenvalues (Red Layer)
Sara	Original Image	51	16
	10 Pixels Changed	43	11
	20 Pixels Changed	38	9
Forest	Original Image	67	21
	10 Pixels Changed	81	15
	20 Pixels Changed	73	14

Table 7.9 Comparison with non-CA algorithms

Developers	**Algorithm**	**Data or items used**	**Encryption level**
Van Droogenbroeck et al. [19]	Triple DES and IDEA	External Logo	high-level
Pommer et al. [13]	Selective Encryption of Wavelet-Packet	External Logo	high-level
Krikor et al. [9]	DCT and Stream Cipher	Pseudo-Random bit sequence. (External Key)	high-level
Anoop et al. [1]	Transform Domains and Stream Ciphers	Stream RC4 key values	high-level
models proposed in this chapter	1D Cellular Automata	Internal properties of the input image	low-level

7.6 Conclusion and Future Work

A cellular automata approach presented here for active image forgery detection. In this chapter we have presented one scenario based on LU decomposition and other scenario based on the SV decomposition with combination of one dimensional cellular automata. The cellular automata rule generates a cipher key which can be used to embed into the image. Both procedures need the original images to notice forgery detection.

Seven different experimental validations have also been done. These experimental results obtained from the methods, specially the diffusion and true and false alert clearly shown the performance and reliability of the models. In future work, we will aim on two dimensional cellular automata forms into the suggested framework.

References

1. Anoop, S., Alakkaran, A.: A full image encryption scheme based on transform domains and stream ciphers. International Journal of Advanced Information Science and Technology 17(17), 5–10 (2013)
2. Birajdar, G.K., Mankar, V.H.: Digital image forgery detection using passive techniques: A survey. Digital Investigation 10(3), 226–245 (2013)
3. Bovik, A.C.: The essential guide to image processing. Academic Press (2009)
4. Cox, I., Miller, M., Bloom, J., Fridrich, J., Kalker, T.: Digital Watermarking and Steganography, 2nd edn. Morgan Kaufmann Publishers Inc., San Francisco (2007)
5. Farid, H.: Image forgery detection. IEEE Signal Processing Magazine 26(2), 16–25 (2009)
6. Golub, G.H., van Van Loan, C.F.: Matrix computations (Johns Hopkins studies in mathematical sciences), 3rd edn. The Johns Hopkins University Press (1996)
7. Intel: Sse4 programming reference. D91561 (2007)
8. Jin, J.: An image encryption based on elementary cellular automata. Optics and Lasers in Engineering (2012)
9. Krikor, L., Shaaban, Z.: Image encryption using DCT and stream cipher. European Journal of Scientific Research 32(1), 47–57 (2009)
10. Lafe, O.: Data compression and encryption using cellular automata transforms. Engineering Applications of Artificial Intelligence 10(6), 581–591 (1997)
11. Mao, K.: Identifying critical variables of principal components for unsupervised feature selection. IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics 35(2), 339–344 (2005)
12. Panda, S.P., Sahu, M., Rout, U.P., Nanda, S.K.: Encryption and decryption algorithm using two dimensional cellular automata rules in cryptography. International Journal of Communication Network & Security 1, 18–23 (2011)
13. Pommer, A., Uhl, A.: Selective encryption of wavelet-packet encoded image data: efficiency and security. Multimedia Systems 9(3), 279–287 (2003)
14. Sharma, P., Lal, N., Diwakar, M.: Text security using 2d cellular automata rules. In: Proceedings of the Conference on Advances in Communication and Control Systems 2013. Atlantis Press (2013)
15. Stallings, W.: Cryptography and Network Security, 3rd edn. Practice Hall (2003)
16. Tafti, A.P., Malakooti, M., Ashourian, M., Janosepah, S.: Digital image forgery detection through data embedding in spatial domain and cellular automata. In: Int. Conf. on Digital Content, Multimedia Technology and its Applications (IDCTA), pp. 11–15 (2011)
17. Toffoli, T., Margolus, N.: Cellular Automata Machines: A new environment for modelling. MIT press (1987)
18. Urias, J.: Cryptography primitives based on a cellular automaton. In: Coding Theory, Cryptography and Related Areas, pp. 244–248 (2000)
19. Van Droogenbroeck, M., Benedett, R.: Techniques for a selective encryption of uncompressed and compressed images. In: Proceedings of the ACIVS Advanced Concepts for Intelligent Vision Systems (2002)
20. Xiaoyang, Y., Yang, S., Yang, Y., Shuchun, Y., Hao, C., Yanxia, G.: An encryption method for QR code image based on ECA. International Journal of Security & Its Applications 7(5) (2013)

Chapter 8
Content-Based Image Retrieval with Cellular Automata

Lynette van Zijl

Abstract. In this contribution, the practical application of cellular automata (CA) to content-based image retrieval is considered. A brief background on the standard content-based image retrieval processes is given, followed by the basic CA models to achieve many of the processes throughout most of the content-based image retrieval pipeline. The chapter concludes with a practical case study.

8.1 Introduction

Given an image, how can one retrieve a similar-looking image from a database of images? The field of content-based image retrieval (CBIR) addresses exactly this problem. This chapter considers the role that cellular automata (CA) can play in implementing the many processes involved in the CBIR pipeline, particularly in the feature analysis of the original image. Although the use of CA is well known in image processing, little work has been done to integrate CA into the whole processing pipeline for CBIR. Exceptions are the work by Van Zijl et al. [2, 32] and Konstantinidis et al. [18].

8.2 Content-Based Image Retrieval: A Background

CBIR is a broad field, with many issues and perspectives at play. For a detailed overview, the reader is referred to some of the well-known books [28] and surveys [8] on the field. In this chapter, the focus is primarily on the use of cellular automata to achieve the implementations of many of the processes involved in CBIR. Prior knowledge of cellular automata is assumed, as for example in [13].

In CBIR, the contents of a source image is used to find the set of nearest matching images in a database of images. For example, given a picture of a bird, one would

Lynette van Zijl
Stellenbosch University, Stellenbosch, South Africa
e-mail: lvzijl@sun.ac.za

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_8, © Springer International Publishing Switzerland 2014

search through a database of images to retrieve all other pictures of similar-looking birds, in order to identify the bird in the original picture. Note that these searches use the image itself and not the *text name* of the picture file to do the search—hence the term *content-based*. Although trivial for humans, this process is algorithmically significantly more difficult than text-based searches. The focus in this chapter will be on similarity searches based on image properties such as colour, shape or texture [18].

In order to find similar-looking images, an image is typically separated into its major regions—this process is known as segmentation. For example, a beach scene might be separated into the sand, the sky, the sun, and a palm tree. Common approaches here are that of k-means clustering and normalised cuts [25]. Then, for each region found in the segmentation process, the qualifying characteristics of the image must be defined. For example, a dog has a body with four legs, and a head with two ears. These characteristics are described mathematically (in terms of shape, size, orientation, and so on) in a so-called feature vector (or set of feature vectors), known as the *signature* or *descriptor* of the image.[1] All the images in the database also have their pre-calculated feature vectors, so that the matching problem becomes one of finding the distance between the feature vector of the input image, and the feature vector of each of the images in the database. The reader must note that it is important that the images in the database are mathematically described to be invariant to rotation, scaling, translation and reflection [34] (affine transformations). Moreover, the image descriptors must be as compact as possible, since typical image databases are huge [5]. The process of establishing a feature vector for an image is known as feature analysis. In this chapter, the focus is on the use of cellular automata for feature analysis.

In feature analysis, it is standard practice to first clean up the given image by performing noise reduction. Then, in order to isolate the distinguishing shapes in the image, edge detection is performed. Lastly, given the edges, one can perform shape recognition. Colours and texture are also analysed, to aid in these procedures. Colours are typically analysed with histogram-based techniques, while texture analysis is often based on wavelet transforms [17]. Shapes may be described by finding the center point of a closed edge, and describing the outline of the shape relative to this center point. Template matching can also be used to find similar shapes. This involves matching a template picture of a certain shape, such as a circle, against the given image. Better results are achieved if two or more components are combined in the search, so that the matching takes place for example on both shape and texture, or both texture and colour.

The actual image matching process involves a similarity comparison between the features of the given image, and the features of the images in the database. This is achieved by comparing the distance between their feature vectors. Typical distances used are the Euclidean, the Hausdorff, and the Earth Mover's distance [33]. The reader should note that, due to the size of the typical image database, it is impractical to do a linear search through the pre-computed feature vectors of the database. The

[1] If the vectors are weighted, and the weights sum to one, the signature may also be considered as a discrete distribution.

feature vectors must therefore be indexed for fast searches, usually in some tree structure. The organisation of such an indexing structure is non-trivial [18, 21].

Note that CBIR is particularly useful in medical imaging. Since this is a constrained domain, many refinements to general CBIR algorithms occur in this field. One example is that of unsupervised segmentation of images [11]. Another specialisation is the use of ant colony optimisation techniques for image processing [12, 24]. Ant colony methods are not explicitly covered in this chapter.

From a CA model point of view, the best rule set to choose for a specific CBIR scenario is of particular interest. In most of the subtasks of CBIR, methods had been proposed to learn the best rule sets for a given scenario. Again, references to this topic is given where relevant, but no detailed discussions are given.

8.3 Cellular Automata in Image Processing and Feature Analysis

A digital image consists of a two-dimensional grid of pixels, and to apply CA to image processing in general and CBIR in particular, this grid of pixels is mapped to a two-dimensional CA, where each cell represents one pixel in the image plane. Furthermore, it is assumed that the individual automata in each cell are identical, and hence one transition function can be defined for the CA as a whole. For black and white images, it is sufficient to consider *binary* CA, where each cell has only the two values 0 and 1 (representing black and white). For grayscale and colour images, cells have more values.

The first uses of CA in image processing came about as parallel implementations to speed up image processing. These were mostly focused on image processing tasks such as noise reduction, edge detection and template matching [2, 4, 9, 27, 32]. This section covers the use of CA in image processing, with the goal of establishing a feature vector for an image.

8.3.1 Noise Reduction

Noise in an image is defined as random variations in the pixels representing an image, and hence the task of noise reduction is to replace each such random pixel with a value as close as possible to the pixel value in the original image. Standard non-CA based techniques for noise reduction include the application of a Gaussian filter, a mean filter (both linear) and a median filter (nonlinear) [26]. Noise reduction techniques depend on the type of noise present in the image, as that influences the algorithm to be applied to remove the noise. The most common types of noise are impulse (or salt and pepper) noise, uniform noise, and Gaussian noise. In impulse noise, the noise pixels have either the minimum or the maximum value over all possible pixel values (for example, in black and white images, a noisy pixel could have the value 0 or the value 255). On the other hand, uniform noise presents as noise pixels which may assume any value over the allowed range.

To apply CA to the task of noise reduction, one considers the neighbourhood of each pixel, and uses an appropriate update rule to replace the value of the current pixel with some average over the pixel values of the neighbourhood. The following issues influence the decision of an appropriate update rule, and hence the eventual practical success of the proposed CA model:

- the type of noise;
- the type of image;
- the noise ratio (how much of the image consists of noisy pixels);
- the neighbourhood (Von Neumann or Moore);
- border effects at the edges of the image; and
- the number of evolutions executed by the CA.

The most basic update rule, as given below, is applicable to impulse noise, and it replaces the current pixel in a Von Neumann neighbourhood with the mean [2] of all the neighbourhood pixels:

$$x_{i,j}(t+1) = \tfrac{1}{5}(x_{i,j-1}(t) + x_{i,j+1}(t) + x_{i,j}(t) + x_{i-1,j}(t) + x_{i+1,j}(t)) \,. \tag{8.1}$$

A better approach for impulse noise is to use the majority update rule [27, 31]. Let all pixel values k fall in the range 0 to n, and take the value $x_{p,r} = k$ in the neighbourhood N which occurs the greatest number of times (the majority value) as the new cell value. If no majority exists, then any random neighbourhood element may replace the current cell [27].

$$x_{i,j}(t+1) = \begin{cases} k, \text{ where } ||(p,r)|k = x_{p,r}(t), (p,r) \in N|| = \max_{m=0}^{n}\{x_{p,r} = m\} \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \text{if } x_{i,j}(t) = 0 \text{ or } x_{i,j}(t) = n \\ \text{random}\{x_{p,r}(t), (p,r) \in N\} \qquad\qquad \text{otherwise} \end{cases} \tag{8.2}$$

Figure 8.1 shows the effect of the majority rule applied to a black and white image and to a colour image with impulse noise [3]. The CA was applied for two evolutions. Note the degradation of the clarity in the colour image after noise was removed.

Improvements and/or variants on the majority update rule include the use of a Moore neighbourhood instead of a Von Neumann neighbourhood, and the use of reflexive boundaries instead of null boundaries [31] to minimise discrepancies on the image borders.

To measure the success of any noise reduction technique, the PSNR (peak signal to noise ratio) is typically used. Consider a clean image C without noise, and introduce random noise to get a noisy image N. Apply the noise reduction technique to N to get a resultant image R. The effectiveness of the noise reduction technique can now be measured by comparing C and R. Assume the images have size $m \times n$. First, find the mean squared error between the individual pixels $C_{i,j}$ of C and $R_{i,j}$ of R:

[2] The median can also be used to remove impulse noise. Note that the median produces less blurring than the mean filter [14].

$$\mathrm{MSE}(C,R) = \frac{\sum_{i=0}^{m-1}\sum_{j=0}^{n-1}(C_{i,j}-R_{i,j})^2}{mn}.$$

Let p represent the maximum possible pixel value (the peak signal) in an image (for example, 255 in a black and white image). Then the PSNR is given by

$$10\log 10\frac{p^2}{MSE(C,R)}.$$

The higher the peak signal to noise ratio, the better the technique. An important issue in reaching the final denoised image, is the number of evolutions that the CA must execute. If too many evolutions are performed, the quality of the image can degrade noticeably. For a general good quality image with only some noise, between one and five evolutions are usually sufficient.

Various authors offer comparisons of the CA-based methods and traditional methods [6, 14, 16, 27, 31] for noise reduction.

Given a denoised image, the next step in image retrieval is to find the feature vector based on shape, colour and/or texture [10].

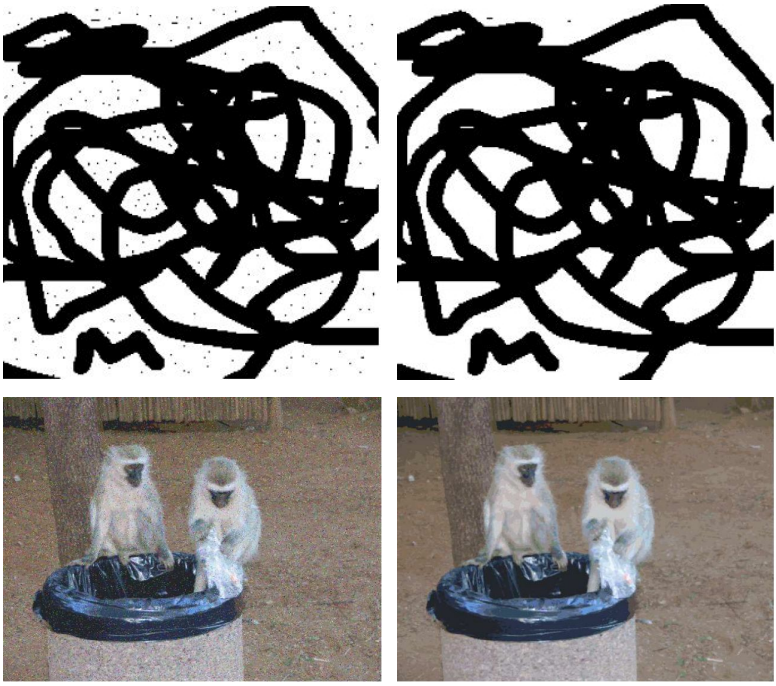


Fig. 8.1 Images with noise before and after the majority update rule CA filter was applied

8.3.2 Edge Detection

An edge in an image is seen as a significant variation in the intensity of pixels in the image at a specific position. Some of the standard edge detectors in image processing include Susan, Sobel and Canny [26]. Since edges are local variations, CA are suited to finding edges, as differences in neighbourhood pixels values are easy to detect. Edge detectors can also thin the edges, to give a sharper edge result.

The method proposed by Popovici et al. [27] gives a standard edge detection CA model. Let $\varphi(a,b)$ give a similarity measure between pixels a and b, such that the value of $\varphi(a,b)$ decreases as the similarity between the pixels increases (this implies that $\varphi(a,a) = 0$). A simple example of such a similarity measure is the Euclidean distance $||a-b||$ in RGB space. In addition, a threshold ε is defined, which can differ according to the application. A Von Neumann neighbourhood is sufficient for edge detection, and one simply needs to test whether the center cell differs more than the specified threshold from each of its neighbours. If so, the center cell is set to zero (the background colour):

$$x_{i,j}(t+1) = \begin{cases} 0, \text{ if} & \varphi(x_{i,j}(t),x_{i,j-1}(t)) < \varepsilon \;\&\; \varphi(x_{i,j}(t),x_{i,j+1}(t)) < \varepsilon \;\& \\ & \varphi(x_{i,j}(t),x_{i-1,j}(t)) < \varepsilon \;\&\; \varphi(x_{i,j}(t),x_{i+1,j}(t)) < \varepsilon \\ x_{i,j}(t), \text{ otherwise.} & \end{cases} \tag{8.3}$$

Note that, as with other non-CA edge detectors, a noisy image may result in false edges, and it is essential to remove as much noise as possible from the image before attempting to detect edges.

Many variants on the basic edge detection CA model exist. For example, Sato and Kanoh [30] propose rule-changing CA, which are trained on the ground truth established by using the Canny edge detector.

Finding the best rule set for edge detection has been investigated by, amongst others, Slatnia and Kazar [16], and Rosin [29]. See chapter 5 for a more detailed overview and comparison of CA edge detectors.

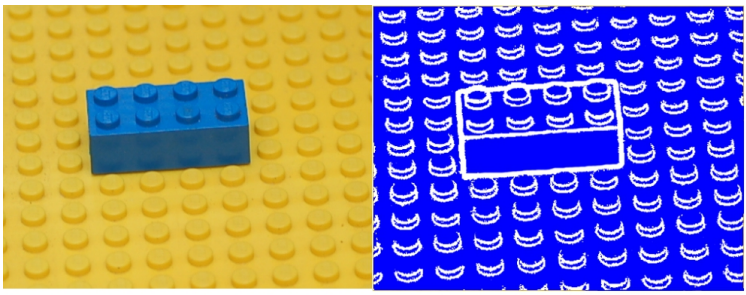


Fig. 8.2 Edges detected with the Popovici CA edge detection model (from [2])

Noise reduction, and to a certain extent edge detection, are used globally on a complete image. However, when shapes, colours and textures need to be compared, a global view may obscure detail in specific regions in an image. Hence, segmentation of images needs to be considered.

8.3.3 Segmentation

Segmentation of an image into smaller homogeneous regions ensures a more local and hence a more specific match to be made between an example and an image in the database. One simple example of segmentation is the separation of an image into a background and foreground [2]. Segmentation based on human interaction is reliable, but impractical for large databases, and hence automatic unsupervised segmentation is desirable.

As a representative example of a CA model of unsupervised segmentation, the unsupervised grow-cut (UCG) CA model of Ghosh et al. [11] is discussed below. Each cell in the CA has a 3-tuple associated with it, containing a label, the pixel intensity, and a so-called cell strength (between 0 and 1). The labels are updated on each evolution, to indicate to which equivalence class a cell belongs. Initially, random pixels are selected to form the first equivalence classes (these pixels get cell strength 1), and the different segments of the image are represented by the different equivalence classes on completion of the process. Suppose that p is the current cell, and consider each of its neighbours q. If p and q are not in the same equivalence class, and the difference in intensity of p and q is too large, then the equivalence class of q is updated to include p and the strength of p is decreased.

The intensity difference between two cells is calculated as follows: let I_p and I_q denote the intensities of p and q respectively, and let ϕ_p indicate the cell strength of cell p. Then

$$1 - \left(\frac{|I_p - I_q|}{\max(I)} \right) \phi_q$$

represents the intensity difference. If cells p and q are not in the same equivalence class, and the intensity difference exceeds a previously defined fixed threshold, then the label of p is updated and added to the equivalence class of q. In addition, the strength of p becomes the intensy difference as calculated above.

Ghosh et al. show that the CA method for unsupervised segmentation compare favourably to non-CA methods as far as results are concerned.

8.3.4 Colour Matching and Histograms

Image analysis based on colour is done by means of histograms. Colours are classified as belonging to a so-called bin, and then the number of pixels in the image that have the same colour as the bin, is counted to form the histogram. Note that the number of bins may become excessively large, as an image can have millions of different colours.

Colour histogram analysis can be effectively combined with CA models for image retrieval [2, 18]. The chosen colour model has an influence on the results of the colour analysis. The well-known standard RGB model simply takes the red, green and blue colour components in a stored image, and any pixel then has a colour constructed as a combination of the red, green and blue values in a cube. In image analysis, colour models such as HSI (hue, saturation and intensity) is preferred, as it is nearer to the human visual colour perception system. Note that conversions between different colour models can be applied, so that the best colour model for a given application can be used. The L*a*b* colour model, for example, has the property that perception of colour differences are uniform. Here, the L* represents luminance, the a* refers to a range describing the relative green-red aspect of the image, and b* refers to the relative blue-yellow aspect of the image. The reader may consult any standard text book on image analysis (such as [26]) for an in-depth discussion of colour models.

A common problem in image analysis is that images that are similar in colour, but with small spatial scene variations, may produce histograms that differ by a large amount. One of the contributing reasons for this phenomenon is that histograms are constructed globally from the whole image. CA approaches counter this effect, as CA implementations stress the local properties in an image without having to resort to the computational and space overhead of a large number of local histograms. For example, Konstantinidis et al. [18] proposed a pre-classification of the image database, based on the use of CA and the L*a*b* colour space. Then, a histogram based on the hue in the HSV (hue, saturation, value) colour model is used for comparison purposes.

Konstantinidis et al. apply two CA with a Moore neighbourhood, one on the a* component and one on the b* component, for each image in the database. The update rule replaces the current pixel value of a cell with the sum of the pixels in the neighbourhood for five evolutions, or until the maximum absolute values in the a* and b* range is reached (that is, $|-128|$ or $|127|$). As the CA evolves through its time steps, the number of pixels that change in each evolution is recorded. These are entered into the bins of the histogram, so that the final histogram contains the number of changing pixels for each time step. Hence, the image database has two histogram values for each image. In the retrieval step, the histogram of the query image (H_Q) is compared to those in the image database (H_C) using the Bhattacharyya distance $-\ln\sum_i \sqrt{H_Q(i) \times H_C(i)}$.

The hundred best matches from the database is then considered as having been preclassified as possible matches. Finally, an HSV histogram colour comparison is done between the query image and the preclassified subset of the database, to find the final best match(es).

Although not directly related to image retrieval, it is interesting to note that cellular automata are also useful for colourisation [19] and colour reduction [23].

8.3.5 Shape Matching

Shape matching requires finding a shape descriptor, and then finding the distance between the query shape descriptor and the shape descriptor of the image in the database. Shape descriptors should be invariant to translation, rotation and scaling. For a basic introduction to shape analysis and matching, the reader may refer to [22].

The first step in shape analysis is to find the outline of the shape using an edge detector (see Sect. 8.3.2 for a CA model for edge detection). A center reference point in the shape has to be determined, and then the shape can be described in terms of how the contour (edge) points fit around the reference point. In [32], a CA model that implements flood filling is used to reach the contour points from the reference point.

Flood filling CA models are often used in shortest path problems [20]. A special cell is indicated as the start cell, and another as the goal cell. When updating the current cell, its relationship to each neighbour cell is used to reflect its new status. In the case of shortest paths, the updated value would reflect a distance from the start cell, based on the distance values of its neighbours. In the shape filling case, the updated value reflects whether this cell is inside the shape, whether it is an edge cell, or whether it is a background cell. Assume the use of a Moore neighbourhood, and a CA where each cell may be marked as belonging to an edge, or belonging to the background, or belonging to the desired shape. Initially, only the center reference point is marked to belong to the shape. Let φ be a specified threshold. During each time step, the update rule counts the number of neighbours that are not edge or background cells. If this value exceeds the threshold, then the current cell belongs to the shape. The sensitivity of the update rule can be changed by giving a predefined neighbourhood size (ns):

$$x_{i,j}(t+1) = \begin{cases} shape, \text{ if} & x_{i,j}(t) \neq shape \;\&\; x_{i,j}(t) \neq background \;\& \\ & (\sum_{m,n} x_{m,n} = shape) > \varphi, \\ & \text{where } i - \frac{ns}{2}, j - \frac{ns}{2} < m,n < i + \frac{ns}{2}, j + \frac{ns}{2}, \\ edge, \text{ otherwise.} & \end{cases}$$

Figure 8.3 shows the effect of the flood filling of a shape. Note that a continuous unbroken edge is necessary from the edge detection phase, as gaps in the edges will cause the flood filling to fail.

Another approach to shape recognition, is based on the geometrical properties [7, 15] of predefined shapes. Here, a standard set of CA rules are defined to find geometrical shapes such as circles, ellipses or triangles. The original image is loaded as the initial configuration of the CA, and the CA executes until it reaches a stable configuration, where an application of the rules do not cause any changes. This stable state is then analysed to deduce the original shape. Note that this technique analyses the single image loaded into the CA as an initial configuration, which means that complex and overlapping shapes cannot be found.

Given the pixels comprising a shape, a mathematical description has to be found that is independent of rotation, translation and scaling. Moreover, the descriptor should preferably have low memory usage and time complexity, while still retaining high accuracy and fast retrieval and comparison rates. For an overview, the reader may consult a standard text such as [26]. For interest's sake, note that Lin et al. used the so-called mountain climbing sequence (MCS) [22], while the Hu set of moments [1] was used in [2].

8.4 Practical Case Study: Recognition of LEGO Bricks

An example of a CBIR system that exploits the use of CA throughout most of the CBIR pipeline, is described in [2, 32]. This seemingly innocuous application requires that a query image of a LEGO brick is matched in a large image database of LEGO bricks, for identification purposes. All the images of the bricks are of a single brick photographed on a background LEGO base. Bricks can be matched by colour and shape, or just shape (see Fig. 8.4). The semantics of a brick is given by its three-dimensional form, as well as the arrangement of the studs on the brick. For example, in Fig. 8.4, bricks number 1, 11 and 12 all have four studs on top, but the shapes of the bricks are all different (slice, circle and square), and the arrangement of the studs on brick 1 is different from that of bricks 11 and 12. Hence, the three bricks are semantically distinct. To contain the number of variations, the initial prototype system catered only for 'standard' bricks without multicoloured stickers or odd shapes without studs (see Fig. 8.4 for examples of nonstandard special bricks).

The feature extraction phase of this CBIR system requires five steps:

- Segmentation: the baseboard plays no role in the semantic definition of the brick, and is removed from the image.
- Edge detection: the edges of the brick will be used to define its form.
- Stud identification: the positions of the studs must be found.

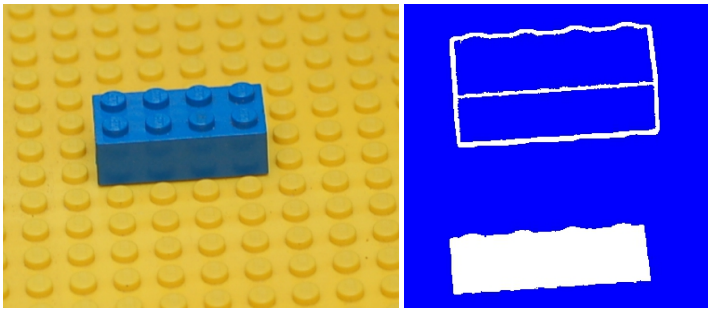


Fig. 8.3 Floodfilling a shape (from [32]). The original image (left), the extracted edges (top right), and the flood-filled top shape of the brick (bottom right).

- Mapping to two dimensions: the semantically defining side of the LEGO brick is isolated, which reduces the three dimensional problem to a two dimensional problem.
- Stud arrangement: the relative alignment of the studs.

It is reasonable to assume that the baseboard will have a colour different to that of the brick itself. Baseboard removal then becomes a simple establishment of colour near the edge of the image, and subtraction of pixels near to that colour. This, however, still leaves the shadowy edges of the studs, because the shadows have a different colour (see Fig. 8.5(a) and (b)).

The background shadows can then be removed using a CA. Define a Von Neumann-type neighbourhood $\vec{n}_x$ for each cell $x_{i,j}$, such that $\vec{n}_x = (\vec{x}_N, \vec{x}_S, \vec{x}_W, \vec{x}_E)$, where

$$\begin{aligned}
\vec{x}_N &= \langle x_{i-\delta,j}, \ldots, x_{i-1,j} \rangle \\
\vec{x}_S &= \langle x_{i+1,j}, \ldots, x_{i+\delta,j} \rangle \\
\vec{x}_W &= \langle x_{i,j-\delta}, \ldots, x_{i,j-1} \rangle \\
\vec{x}_E &= \langle x_{i,j+1}, \ldots, x_{i,j+\delta} \rangle .
\end{aligned}$$

That is, the neighbourhood is taken in the usual four directions up to a distance of δ from the current cell, but instead of using a standard Von Neumann neighbourhood, the neighbourhood forms a rectangle of single cell-width instead of the normal diamond-shape of the Von Neumann neighbourhood. Suppose now that a background pixel in cell $x_{i,j}$ is indicated by $x_{i,j} = 0$. The update rule for the CA can then be defined as

$$x_{i,j}(t+1) = \begin{cases} 0, \text{ if } \exists k_N, k_S, k_W, k_E : x_{i-k_N,j}(t) = 0 \;\&\; \\ \qquad\qquad\qquad\qquad\qquad x_{i+k_S,j}(t) = 0 \;\&\; \\ \qquad\qquad\qquad\qquad\qquad x_{i,j-k_W}(t) = 0 \;\&\; \\ \qquad\qquad\qquad\qquad\qquad x_{i,j+k_E}(t) = 0 \\ 1, \text{ otherwise,} \end{cases}$$

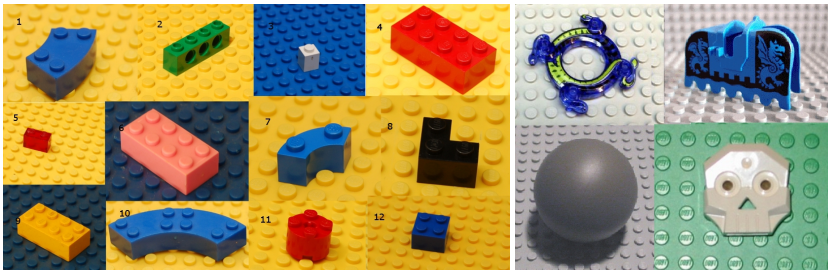


Fig. 8.4 A selection of standard LEGO bricks on the left, and some special bricks on the right

where $1 \leq k_N, k_S, k_W, k_E \leq \delta$. That is, if there is any background pixel found within a distance of δ in all four directions from the current cell, then the cell is taken to be a background cell and will be eliminated. This completes the baseboard elimination phase, and the next step is to detect the edges of the LEGO brick itself.

The edge detection phase was implemented with the CA model of Popovici et al., as described in Sect. 8.3.2. This results in a single LEGO brick, with delineated edges (including the stud edges), as in Fig. 8.5(c). The stud locations on the top of the LEGO brick are found with template matching, where the halfmoon shapes are matched to scaled templates of the same form. The template shapes are simply moved around on the image until a location is found which maximises a match function. In this case, the squared error was used, where f represents the image and T represents the template:

$$\mathrm{SE}(x,y) = \sum_{\alpha=1}^{N} \sum_{\beta=1}^{N} (f(x-\alpha, y-\beta) - T(\alpha,\beta))^2 \ .$$

After the studs had been matched with the templates, their center positions are available for the next step. Given the stud positions, the geometrical arrangement of the studs can now be found, as this forms part of the semantic definition of the brick. This is accomplished by finding a minimal set of straight lines $\mathscr{L} = \{l_1, l_2, \ldots, l_n\}$

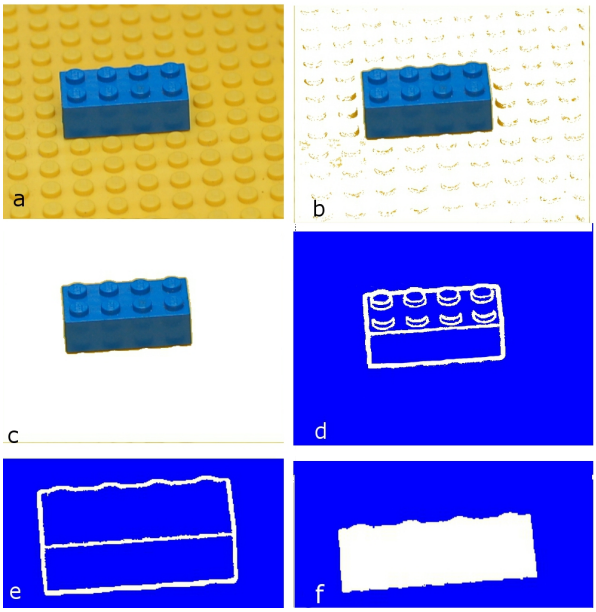


Fig. 8.5 (a) The original brick, (b) after background subtraction, (c) after CA removal of baseboard, (d) edges including studs, (e) edges without studs, (f) top surface after floodfill

where each stud lies on exactly one l_i. For example, in Fig. 8.4, the top left brick is a 2×4 brick, with two parallel rows of four equidistanced studs.

For the final step, the top surface of the brick must be found, in order to define its geometric shape (that is, square, rectangle, or slice). First, the templates found for each stud position is subtracted from the image. This leaves some noise, which is removed with the denoising CA model that was used to do the baseboard elimination. This results in a set of edges delineating the outlines of the brick in a two-dimensional image, but which semantically has a three-dimensional interpretation (see Fig. 8.5(e)). However, the observation that standard LEGO bricks are three-dimensional protrusions from the top surface (containing the studs) to the bottom surface, means that only the two-dimensional top surface is necessary to identify the shape.

In order to find the top surface of the LEGO brick, a CA model is used to floodfill the top surface starting from the known positions of the studs. In this model, each cell may assume four possible states: *background*, *edge*, *top surface* and *not top surface*. All the pixels where there were studs, are identified as top surface, and any cell that is not background, edge or top surface, is identified as *not top surface*.

A Moore neighbourhood with a specified distance *ns* is used. The update rule considers each cell. If it is not a top surface cell, the cell changes into a top surface cell if it is adjacent to any top surface cell in the neighbourhood, and it is neither edge nor background. Moreover, the number of neighbours that are not edge or background, are counted. If this number exceeds a given threshold, then the current cell becomes part of the top surface. Formally, let th be the threshold and ns the neighbourhood size. Let a top surface cell be represented by 0, edges by 1, and background by 2. Then the update rule can be described as:

$$x_{i,j}(t+1) = \begin{cases} 0, \text{ if} & x_{i,j}(t) \neq 0 \;\&\; x_{i,j}(t) \neq 2 \;\& \\ & (\sum_{m,n} x_{m,n} = 0) > th, \\ & \text{where } i - \frac{ns}{2}, j - \frac{ns}{2} < m,n < i + \frac{ns}{2}, j + \frac{ns}{2}, \\ 1, \text{ otherwise.} \end{cases}$$

At this stage, it is now possible to encode the feature vector of a given LEGO brick, based on its number of studs, stud locations, form, and colour. The Hu-set of invariant moments was used to encode the feature vector with the information extracted from the image. Once the feature vector has been calculated for the search image, that vector can be matched against all the pre-calculated feature vectors of images in the database.

An example of the retrieval correctness of the LEGO CBIR system is illustrated in Fig. 8.6. The bricks are shown in best match order (the lower the number, the better the match). Note that an identical brick to the search image was the best match, followed by two other curved bricks, while the rectangular bricks were the worst matches. Experiments showed that the success rate of the system is in the order of 80%.

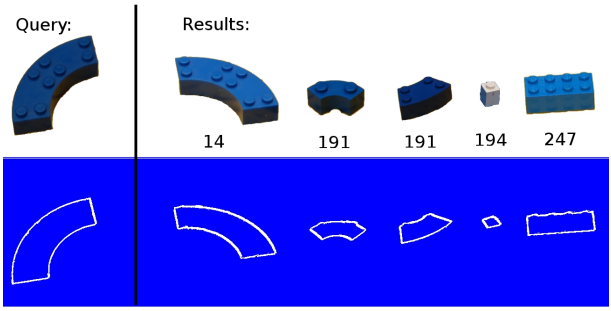


Fig. 8.6 A sample shape retrieval query with match scores presented in thousands

8.5 Conclusion

The advantage of CA in image processing lies in their ability to exploit local regional properties, whilst allowing global image effects. Moreover, CA have bounded space complexity and, in most cases, provide time efficient (real-time) solutions to image processing problems. From a practical point of view, implementation software is easy to implement, maintain and extend. On the negative side, explicit transformation rules may need to be found for different scenarios—however, learning the rules is a possibility.

References

1. Belkasim, S., Shridhar, M., Ahmadi, M.: Pattern recognition with moment invariants: a comparative study and new results. Pattern Recognition 24(12), 1117–1138 (1991)
2. Botha, L., Van Zijl, L., Hoffmann, M.: Realtime LEGO brick image retrieval with cellular automata. Journal of Universal Computer Science 15(14), 2765–2785 (2009)
3. Chamberlain, M.: Cellular automata for image processing. Tech. rep., Stellenbosch University, South Africa (2008)
4. Chang, C., Zhang, Y., Gdong, Y.: Cellular automata for edge detection of images. In: Proceedings of the 2004 International Conference on Machine Learning and Cybernetics, vol. 6, pp. 3830–3834. IEEE (2004)
5. Chatzichristofis, S., Zagoris, K., Boutalis, Y., Papamarkos, N.: Accurate image retrieval based on compact composite descriptors and relevance feedback information. International Journal of Pattern Recognition and Artificial Intelligence 24(2), 207–244 (2010)
6. Dalhoum, A., Al-Dhamari, I., Ortega, A., Alfonseca, M.: Enhanced cellular automata for image noise removal. In: Proceedings of the Asian Simulation Technology Conference, pp. 69–73 (2011)
7. D'Alotto, L.A., Sinha, D.: Cellular automata and real-time geometrical shape recognition. In: Photonics West 1998 Electronic Imaging, pp. 60–69. International Society for Optics and Photonics (1998)
8. Datta, R., Joshi, D., Li, J., Wang, J.: Image retrieval: ideas, influences, and trends of the new age. ACM Computing Surveys 40(2), 300–360 (2008)
9. de Saint Pierre, T., Milgram, M.: New and efficient cellular algorithms for image processing. CVGIP: Image Understanding 55(3), 261–274 (1992)

10. Gagaudakis, G., Rosin, P.: Shape measures for image retrieval. Pattern Recognition Letters 24(15), 2711–2721 (2003)
11. Ghosh, P., Antani, S., Long, L., Thoma, G.: Unsupervised grow-cut: Cellular automata-based medical image segmentation. In: Proceedings of the First IEEE International Conference on Healthcare Informatics, Imaging and Systems Biology (HISB), pp. 40–47. IEEE (2011)
12. Han, Y., Shi, P.: An improved ant colony algorithm for fuzzy clustering in image segmentation. Neurocomputing 70(4), 665–671 (2007)
13. Ilachinski, A.: Cellular automata: a discrete universe. World Scientific Publishing Company (2001)
14. Jana, B., Pal, P., Bhaumik, J.: New image noise reduction schemes based on cellular automata. International Journal of Soft Computing and Engineering 2(2), 98–103 (2012)
15. Karafyllidis, I., Ioannidis, A., Thanailakis, A.: Geometrical shape recognition using a cellular automaton architecture and its VLSI implementation. Real-Time Imaging 3(4), 243–254 (1997)
16. Kazar, O., Slatnia, S.: Evolutionary cellular automata for image segmentation and noise filtering using genetic algorithms. Journal of Applied Computer Science and Mathematics 5(10), 33–40 (2011)
17. Kokare, M., Biswas, P., Chatterji, B.: Texture image retrieval using new rotated complex wavelet filters. IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics 35(6), 1168–1178 (2005)
18. Konstantinidis, K., Sirakoulis, G., Andreadis, I.: Content-based image retrieval using cellular automata. In: Proceedings of the 5th International Conference on Technology and Automation (ICTA 2005), Thessaloniki, Greece, pp. 371–375 (2005)
19. Konushin, V., Vezhnevets, V.: Interactive image colorization and recoloring based on coupled map lattices. In: Proceedings of Graphicon 2006, Novosibirsk Akademgorodok, Russia, pp. 231–234 (2006)
20. Köster, G., Hartmann, D., Klein, W.: Microscopic pedestrian simulations: From passenger exchange times to regional evacuation. In: Operations Research Proceedings 2010, pp. 571–576. Springer (2011)
21. Lew, M., Sebe, N., Djeraba, C., Jain, R.: Content-based multimedia information retrieval: State of the art and challenges. ACM Transactions on Multimedia Computing, Communications, and Applications (TOMCCAP) 2(1), 1–19 (2006)
22. Lin, H.J., Kao, Y.T., Yen, S.H., Wang, C.J.: A study of shape-based image retrieval. In: Proceedings of the 24th International Conference on Distributed Computing Systems Workshops, pp. 118–123. IEEE (2004)
23. Michailidis, G., Andreadis, I.: Color processing – a new approach based on discrete dynamic systems. In: Proceedings of the CREATE 2010 International Conference, Norway (2010), `http://www.create.uwe.ac.uk/norway_paperlist/michailidis.pdf`
24. Nezamabadi-pour, H., Saryazdi, S., Rashedi, E.: Edge detection using ant algorithms. Soft Computing 10(7), 623–628 (2006)
25. Pal, N., Pal, S.: A review on image segmentation techniques. Pattern Recognition 26(9), 1277–1294 (1993)
26. Petrou, M., Petrou, C.: Image processing: the fundamentals. Wiley (2010)
27. Popovici, A., Popovici, D.: Cellular automata in image processing. In: Proceedings of the Fifteenth International Symposium on Mathematical Theory of Networks and Systems (2002)
28. Rao, C., Kumar, S.: Content based image retrieval fundamentals and algorithms: basics, concepts and novel algorithms. LAP Lambert Academic Publishing (2012)

29. Rosin, P.: Training cellular automata for image processing. IEEE Transactions on Image Processing 15(7), 2076–2087 (2006)
30. Sato, S., Kanoh, H.: Evolutionary design of edge detector using rule-changing cellular automata. In: Proceedings of the Second World Congress on Nature and Biologically Inspired Computing (NaBIC), pp. 60–65. IEEE (2010)
31. Selvapeter, P.J., Hordijk, W.: Cellular automata for image noise filtering. In: Proceedings of the World Congress on Nature and Biologically Inspired Computing (NaBIC), pp. 193–197 (2009)
32. Van Zijl, L., Botha, L.: Feature extraction for image pattern matching with cellular automata. In: Holub, J., Zdárek, J. (eds.) Proceedings of the Prague Stringology Conference, Prague, Czech Republic, pp. 3–14 (2009)
33. Veltkamp, R.C., Hagedoorn, M.: Shape similarity measures, properties and constructions. In: Laurini, R. (ed.) VISUAL 2000. LNCS, vol. 1929, pp. 467–476. Springer, Heidelberg (2000)
34. Zheng, Z., Leung, C.: Graph indexes of 2D-thinned images for rapid content-based image retrieval. Journal of Visual Communication and Image Representation 8(2), 121–134 (1997)

Chapter 9
The Application of Cellular Automaton in Medical Semiautomatic Segmentation

Yonghui Gao and Jie Yang

Abstract. Efficient image segmentation is the basic premise of the medical diagnosis and analysis. However, this task exits several major challenges due to the special anatomy and topological changes. Since fully automated techniques can not guarantee the efficiency and precision in general clinical applications, interactive scheme is often alternatively performed with a special or multiple algorithms. To evaluate the performance of an interactive segmentation, three indicators should be considered: interactive efficiency, implementation complexity and accuracy. An efficient solution should minimize the interaction and the implementation complexity under the premise of ensuring higher segmentation accuracy.

This chapter describes an interactive segmentation framework based on Cellular Automaton (CA). User interaction is performed in the selected image planes to indicate the position and main features of the object and background. The models are extracted efficiently by a Cellular Automaton evolution on homogeneous cells – pixels (voxels), regions and blocks with the corresponding attacking and merging mechanism. CA based strategy does not consider the marker or non-marker state of cells, which grants it with the flexibility in parallel implementation and the ability to deal with multi-class problem. It is easy to implement because only some simple operations are needed. It is accurate because a more general linear or nonlinear model can be learned.

Yonghui Gao
University of Shanghai for Science and Technology,
School of Medical Instrument and Food Engineering,
Shanghai, China
e-mail: gaoyonghui1978@163.com

Jie Yang
Shanghai Jiaotong University,
Institute of Image Processing and Pattern Recognition, China
e-mail: jieyang@sjtu.edu.cn

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_9, © Springer International Publishing Switzerland 2014

9.1 Introduction

Reliable image segmentation is an important yet challenging task in medical image analysis [16]. However, an entirely autonomous method can vary significantly in accuracy. Even the ground truth segmentation may differ between physicians. Therefore, interactive segmentation methods have increased in popularity, such as the intelligent scissors [23] [22] and the classical region growing method [13]. An improved region growing method has been proposed for lung nodule analysis [7], which combines fuzzy connectivity, distance and intensity information as the growing mechanism and peripheral contrast as the halting criterion.

In addition, watershed algorithm [18], fast level set [28] and graph cut method [1–3] for the segmentation of volumetric data set, can provide fast and accurate excellent results in medical applications. Figure 9.1 demonstrates a flowchart of interactive segmentation in a surgery system.

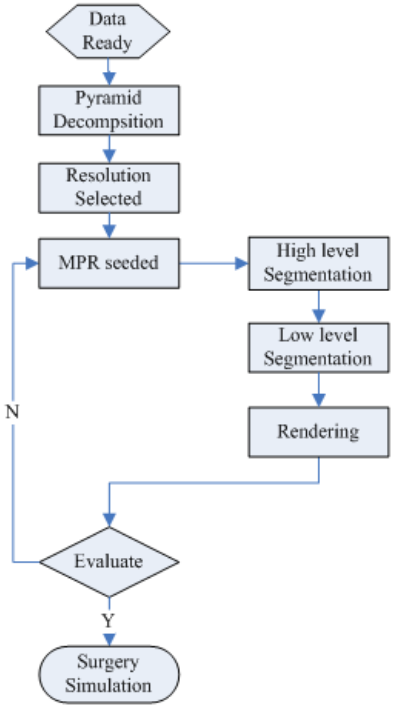


Fig. 9.1 Flowchart of interactive Segmentation

Additionally, methods that allow the use of multiple label classifications include Growcut [27] with intensity based transition CA rules for image segmentation, and Seeded ND Cellular Automata [15] are popular due to the robustness. The Seeded

ND-CA method uses CA to compute the Ford-Bellman shortest path for segmentation on GPU (Graphic Processing Unit). Although the Growcut method and Seeded ND-CA method seem dissimilar, it states that their results are identical and proves the equivalence between them.

One extension of CA is the GrowCut segmentation framework over video sequences [26]. This was done by introducing mean-shift filtering as a pre-processing step, as well as providing improvements and addressing problematic areas in the original formulation, providing a good increase in accuracy. Results showed that the proposed scheme achieves better results than the original formulation, and good segmentation results were achieved using a considerably less complicated framework. However, there are many areas available for future research. Specifically, GrowCut can be transferred to a parallel framework, such as OpenMP, to significantly decrease the run time. In addition, the mean-shift preprocessing step can done on a temporal basis, or other colour clustering methods could be examined. Also, different cost functions and colour spaces may provide better accuracy. The contribution of these factors will inevitably increase accuracy, while achieving real-time results.

Another extension is interactive matting algorithm for natural images [19]. By employing a cellular automaton to iteratively optimize an energy function, the matte is gradually extracted. One attractive feature of the proposed approach is that it does not require a well-defined trimap, but only a few user-specified foreground and background strokes. Whatąś more, additional user input can be added to refine the matte not only after the matting process is over but also in the middle of the matting process. Experimental results show that the proposed approach is capable of producing high quality mattes for many complex natural images. Due to the intrinsic parallelism of cellular automata, the proposed approach has the potential of being accelerated by using parallel computing technique. The parallel implementation will be researched in the future. And the extension of the proposed approach to video matting is also the future work.

In medical applications, no single method is suitable for all kinds of segmentation, and further, there is no general algorithm to segment special tissues from all kinds of medical images. In segmentation, the region of our concern is labeled as foreground and the rest is labeled as background. Mostly, manual segmentation is time-consuming and needs a method that segments an image automatically or with few interactions of the user to provide accurate results.

This chapter is drawn to the CA method for several reasons. First, the CA method supports n-dimensions and m-label categories where the number of labels does not increase computational time or complexity. This ability makes it suitable for simultaneously segmenting multiple tissues. Second, for an effective interactive segmentation, speed and usability are crucial. Since CA is an iterative method where each cell independently follows a set of rules, this naturally lends itself to an efficient parallel implementation. Further, the iterative nature of CA method enables the user to visualize the results during the segmentation process.

The underlying concept behind using CA for segmentation is that a data set is composed of pixels, regions or voxels that can be treated as cells. The segmentation is based on the properties of the data such as intensity and other derived properties. The segmentation also depends on the properties of the neighborhood of the cells, i.e. the classification of a cell as foreground or background also depends on the neighboring cells.

9.2 Cellular Automaton Segmentation Rule

Cellular Automata (CA) were originally introduced by Ulam and Von Neumann with the purpose of obtaining models of biological self-reproduction. Later on, a simple replicator based on parity or modulo-two rules was proposed by Amoroso and Fredkin. Finally actual work in this field started only after Stephen Wolfram developed the CA theory. CA is currently used in a large number of fields and branches derived from physics, chemistry, biology, economics, and information systems. CA is also used in digital image processing such as image denoising, edge detection, restoration, enhancement, segmentation, compression, feature extraction and pattern recognition.

CA is a discrete dynamical system, which consists of a series of rules and no physical equation or function need to be strictly defined. Space, time, and the states of the system are discrete. Each point in a regular spatial lattice, called a cell, can have any one of a finite number of states. A local transition function is defined to update the state of a cell. The local transition function takes the present state of the cell and states of the neighborhood cells as input and outputs the next state of the cell. Another specialty of CA is that the updation phase is applied to the whole lattice of cells simultaneously. Thus the state of the entire lattice advances in discrete time steps.

CA has three fundamental properties [8]: parallelism, locality and homogeneity. These properties mean that cells is updated independently of each other and the CA system is parallel when its constituents evolve simultaneously and independently. The new state of a cell only depends on its and the neighborhood state, the evolution rule is universal in the whole space. Moreover, it is possible to build any type of automata by playing on structural and functional rules.

A cellular automaton can be described as a triplet $A = (S, N, \delta)$, where S is a non-empty state set, each cells can be in one of a finite number of k possible states. N is the neighborhood system, the state of a cell is determined by the previous states of a surrounding neighborhood of cells. δ is the evolution rule (transition function), the state of a cell is updated synchronously in discrete time steps according to this local, identical interaction rule. N is commonly selected as von Neumann or Moore neighborhood of current cell p. A CA evolution rule can be described as:

Algorithm 1: CA Evolution Rule

$$
\begin{aligned}
&while(!Converged)\\
&\{\\
&\quad \forall p \in P\\
&\quad\quad l_p^{t+1} = l_p^t;\\
&\quad\quad \theta_p^{t+1} = \theta_p^t;\\
&\quad\quad Converged = true;\\
&\quad For\ \ \forall q \in N(p)\\
&\quad\quad \textbf{Attack Rule}(p,q)\\
&\quad Endfor\\
&\}
\end{aligned}
\tag{9.1}
$$

where P is the set of cells inside user input, which can be changed in the next step. t is the time of iteration. l_p^t is the current cell label (state set), θ_p^t is the property function (always is strength) of the current cell. The related attack rule can be described as:

Algorithm 2: CA Attack Rule

$$
\begin{aligned}
&If\ g(|q-p|) \cdot \theta_q^t > \theta_p^t\\
&\quad l_p^{t+1} = l_q^t; \theta_p^{t+1} = g(|q-p|) \cdot \theta_q^t;\\
&\quad Converged = false;\\
&\quad Break;\\
&Endif
\end{aligned}
\tag{9.2}
$$

g is a monotonous decreasing function to guarant the iteration to converge. g and θ_p are bounded to $[0,1]$.

$$g(x) = 1 - \frac{|x|}{\max(C)} \tag{9.3}$$

where C is the image gray level and $|x|$ is the difference of p and q. The cell status will be updated once neighbor cells attacking successfully occurs.

Thus, CA provides a general framework derived from the evolution with simple interactions, which is applied in medical image segmentation can extract complex tissues of interest from two-dimensional or volume data. The pixels, image area and volume data blocks are set as the cells can produce different types of segmentation mechanism. Such as the proposed Vladimir's CA segmentation [27] can be used for N-D cases. But the classic method has the shortcoming of high complexity in computing and interaction.

9.3 Cellular Automaton Interactive Segmentation

In an intuitive user interaction scheme – user specifies certain image pixels (call them seed pixels) that belong to objects, should be segmented from each other. The task is to assign labels to all other image pixels automatically, preferably achieving the segmentation result expecting to get. The task statement and input data is similar to Graph Cuts in [3] and [11], however the segmentation instrument differs.

Interactive segmentation method based on CA is used for solving cell labeling task, which allows (but not requires) human input during labeling process, to provide dynamic interaction and feedback between the user and the algorithm. Important properties of CA method, that would like to outline are:

1. Capable of solving moderately hard segmentation tasks (see examples in this chapter);
2. Works with images of any dimension $N \geqslant 1$;
3. Performs multi-label image segmentation (the computation time is not directly affected by the number of labels);
4. Is extensible, allowing construction of new families of segmentation algorithms with specific properties;
5. Interactivity – as the segmentation is refined with each iteration, user can observe the evolution and refine the segmentation 'on the fly';
6. The algorithm is simple in both understanding and implementation;
7. Using cellular automata allows fast parallel implementation

The proposed method iteratively gives feedback to the user while evolution. This process allows user guidance of the implementation, yet does not require additional efforts where the segmentation is reliably computed.

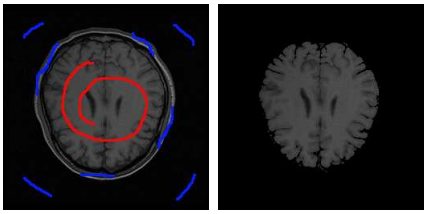
9.3.1 Labels in Image Plane

A digital image is a two-dimensional array of $k \times m$ pixels. An unlabeled image can be considered as a particular configuration state of a CA, where cellular space P is defined by the $k \times m$ array set of the image, and initial states for $\forall p \in P$ are set to:

$$l_p = 0, \theta_p = 0, \overrightarrow{C}_p = RGB_p \tag{9.4}$$

where RGB_p is the three dimensional vector of pixel p's color in RGB space. The final goal of the segmentation is to assign each pixel with one of the K possible labels.

We can treat pixel labeling process as growth and struggle for domination of K types of bacteria. The bacteria start to spread (grow) from the seed pixels and try to occupy all the image. The rules of bacteria growth and competition are obvious – at each discrete time step, each cell tries to 'attack' its neighbors. The attack force is defined by the attacker cell's strength θ_q, and the distance between attacker's and defender's feature vectors $\overrightarrow{C}_q$ and $\overrightarrow{C}_p$. If the attack force is greater than defender's strength – the defending cell is 'conquered' and its label and strength are changed.



a b

Fig. 9.2 Example of labels in image plane. (a) labeled image. (b) segmented image. The red pixels label the object user wish to track. The blue pixels label the background to erase.

The result of these local competitions is that the strongest bacteria occupy the neighboring sites and gradually spread over the image.

The calculation continues until automaton converges to stable configuration, where cell states seize to change.This biological metaphor supplies an intuitive explanation to the pseudo-code above. The examples of image labels and segmentation results are shown in Figure 9.2.

9.3.2 Regional Cellular Automaton Segmentation

To evaluate the performance of an interactive segmentation, three indicators can be considered: Efficiency, complexity and accuracy. A good solution should minimize the interaction and the implementation complexity under the premise of ensuring a higher segmentation accuracy [23] [3] [12].

The prior object edge or initial curve obtained in user interaction [21] [9] [14] is critical to guide the contour extraction. A proper initial information could lead to a good convergence to the true object contour.

As a well preserving edge information segmentation scheme for image, mean shift [5] is often used as pre-segmentation, for example in a maximal similarity based region merging (MSRM) method [25] with better segmentation outputs. MSRM presents a 2D region merging based interactive image segmentation method. The proposed method automatically merges the regions that are initially segmented by mean shift pre-segmentation, users only need to indicate the location or regions of the object and background with rough markers. Two non-marker regions to be merged should have the highest similarity among all adjacent regions. Then the object contours can be reliably extracted by labeling all the non-marker regions as either background or object. The whole process needs not set similarity threshold in advance. According to MSRM, the proposed scheme is divided into two stages: In the first stage, MSRM only merges the marker background regions. For each region $B \in M_B$ and its adjacent region A_i that $A_i \in \overline{S_B}, \overline{S_B} = \{A_i\}_{i=1,2,\ldots,r}$ are merged into one region if

$$\rho(A_i, B) = \max_{j=1,2,\ldots,k} \rho(A_i, S_j^{A_i}), A_i \notin M_B \tag{9.5}$$

and the remained non-marker regions are processed in the second stage. For each non-marker region $P \in N$ and its adjacent region H_i that

$$H_i \in \overline{S_P}, \overline{S_P} = \{H_i\}_{i=1,2,\ldots,p}, \overline{S_{H_i}} = \{S_j^{H_i}\}_{j=1,2,\ldots,p} \tag{9.6}$$

are merged into one region if

$$\rho(P,H_i) = \max_{j=1,2,\ldots,k} \rho(H_i, S_j^{H_i}), H_i \notin M_B, H_i \notin M_O \tag{9.7}$$

where ρ is the similarity measurement. M_B and M_O are the marker regions labeled as background and object respectively. N is the non-marker region set. $\overline{S}_Q = \{S_i^Q\}_{i=1,2,\ldots,q}$ is the formed set of Q's adjacent regions. Obviously, this two-stages process suffers from the complex implementation.

Considering the merit of CA based segmentation that can resolve complex tasks with very simple implementation (evolution rule), a novel Regional Cellular Automaton merging (RCA) rule is presented to facilitate the region merging and overcome the shortcoming of traditional pixel-wise method [10]. In the proposed scheme, an initial mean shift segmentation is required to partition the image into homogeneous regions. After desired object or background regions are labeled with colored markers, the adjacent regions will be iteratively merged and labeled as object or background. Once all the non-marker regions are classified correctly, the object contour can then be readily extracted.

RCA tries to merge similar regions as many as possible until no merging occurs (keeping a stable state). After pre-segmentation, the available regions are set as cells in CA system, and the neighborhood is the adjacent region. This discrete dynamical system consists of homogeneous cells that are synchronously updated in discrete time steps t. The RCA scheme is summarized as: Each segmented homogeneous region (cell)p is assumed to have a eight-cell Moore neighborhood denoted by $N_p = \{q_i\}_{i=1,2,\ldots,8}$. At each time step $t+1$, a cell $p \in N_p$ takes one of k possible states considering its current state σ_p and that of neighbor cells. The cell state is updated according to a transition function Φ based on regional similarity. At start time t^0 the cell states are set according to user imposed labels. Here $k = 2$ and $\sigma_p \in \{0,1\}$. The RCA segmentation is an evolving processing with regions attacking and merging rule.

After user marking, the marker regions cover only a small part of the object and background. To completely extract the object contour, we need to automatically assign each non-marker region with a correct label as either object or background. Therefore, in the automatic region merging process, those regions that belongs to the same class should be identified with highest probabilities and not be merged with regions that belong to different class.

For the convenience of the following development, the set of marker and non-marker regions are denoted as M and $\overline{M}$ respectively. Let q_i be an adjacent region of p and denoted by $N_p = \{q_i\}_{i=1,2,\ldots,k}$, the set of p's adjacent regions. To guide the following region merging process, Bhattacharyya coefficient [6] is used as regional similarity descriptor. The Bhattacharyya coefficient is defined as:

$$\theta(p,q_i) = \sum \sqrt{Hist_p \cdot Hist_{q_i}} \tag{9.8}$$

where $Hist_p$ and $Hist_{q_i}$ are the normalized histograms of p and q_i respectively, and the higher Bhattacharyya coefficient indicates the higher similarity between them.

RCA scheme can be formed as follows:

$$\begin{aligned}
&while(\phi \subset \overline{M}^t)\\
&\{\\
&\quad M^{t+1} = M^t, \overline{M}^{t+1} = \overline{M}^t\\
&\quad \forall p \in M^t, \forall q_i \in N_p\\
&\quad if\theta(p,q_i) = \max_{i=1,2,\ldots,k} \theta(p,N_p)\\
&\quad\quad M^{t+1} = M^t \cup q_i, \overline{M}^{t+1} = \overline{M}^t \setminus q_i\\
&\quad endif\\
&\}
\end{aligned} \tag{9.9}$$

where M^t and $\overline{M}^t$ are the sets of marker regions and non-marker regions (labeled as 0 and 1) at the t time respectively. The iteration stops when the entire $\overline{M}$ shrinks to empty set. The RCA rule is simple and required less user inputs. This fact makes it is conveniently to resolve complex tasks and RCA is essentially a game of life and easily extended to N-D space [15].

Similar to MSRM, RCA is adaptive to image content and it does not need to set the regional similarity threshold in advance. Other color spaces, initial segmentation and distance metrics can also be used in RCA. If the initial segmentation can provide a good basis, RCA may get better results. Different from MSRM, RCA has the ability to deal with multi-class problem according to the number of cell states, which grants it with the flexibility in parallel implementation. Figure 9.3 shows examples to extract single and multiple objects from background. Although the contour of the bird is complex and the skin of polar bears is somewhat similar to the snow background, RCA still successfully separates objects from the background with simple markers. Other color space, initial segmentation and distance metrics can also be used in RCA. If the initial segmentation can provide a good basis, RCA may get better results. Different from MSRM, RCA completes regions merging not considering the marker or non-marker state, which grants it with the greater flexibility in parallel implementation. Moreover, RCA has the ability to deal with multi-class problem according to the number of cell states.

In validation, examples are presented to verify the performance of RCA implemented on the test images woman, church and fish with different color and shape features respectively. As shown in Figure 9.4, although the objects are complex and there is no explicit user input background marker, RCA method can still extract desired objects accurately. In general, RCA algorithm could reliably extract the object contour if the markers cover main features of object or background. However, it may fail when shadow, low-contrast edges and ambiguous areas occur. For example, a parts of the object regions in fish image are very similar to background. Although many markers are used, but the result is not satisfied. Figure 9.5 illustrates the difference between RCA and MSRM segmentation highlighted by red color in the fish

Fig. 9.3 Single and multiple objects segmentation. First column: original images. Second column: initial segmentation and the input markers (the green and blue lines indicate the object and background respectively). Last column: segmentation results by the proposed RCA method.

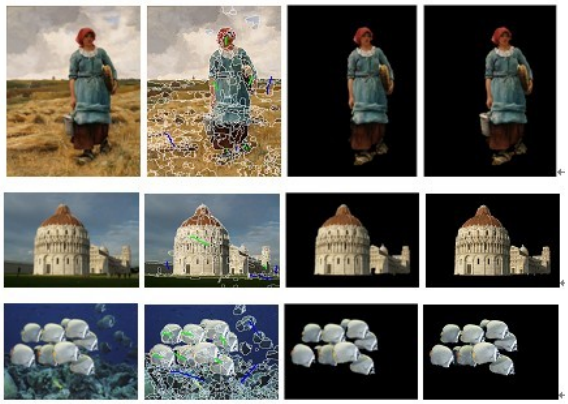


Fig. 9.4 First column: original images. Second column: initial segmentation and the input markers. Third column: segmentation results by the MSRM method. Last column: segmentation results by the proposed RCA method.

image, which implies that more markers may be helpful to provide accuracy results.

To quantitatively evaluate RCA method, the desired objects are manually segmented as ground-truth and three indicators: running time, true positive rate (TPR) and false positive rate (FPR) [25] are computed on the test images. Table 9.1 lists the comparison between two methods with same user markers and programming environment. The numeral results demonstrate that RCA achieves the better segmentation performance. Experiments prove that the time consuming of RCA depends on initial segmentation result, user markers and image content.

Fig. 9.5 Segmentation difference between MSRM and RCA method (highlighted by red regions)

Table 9.1 Comparison of RCA and MSRM

Image	Method	Time (s)	TPR(%)	FPR(%)
Woman	RCA	22.77	97.75	1.26
	MRSM	30.68	94.68	4.49
Church	RCA	25.63	98.07	0.92
	MRSM	37.28	96.32	1.34
Fish	RCA	19.12	96.11	2.87
	MRSM	30.42	95.79	2.93

9.3.3 Volume Cellular Automaton Segmentation

According to traditional 2D interactive method, the straightforward 3D expansion needs laying seeds in each slice, obviously the manual work is hard and not reasonable. In medical system, user interaction is often performed in the selected MPR (Multi-Planar Reformation) image planes to indicate the position and main features of the desired 3D objects. The contours should be extracted by efficient methods from volume data. This operator may satisfy most expert clinician's habits. Interactions in different MPR image planes are demonstrated in Figure 9.6.

9.3.3.1 Brute-Force Algorithm

For the convenience of description, we select a series of MRI ($512\times512\times56$) to demonstrate this strategy. Samples of MRI series are shown in Figure 9.7. The goal is to track the 3D brain soft tissue with complicated shape and process the

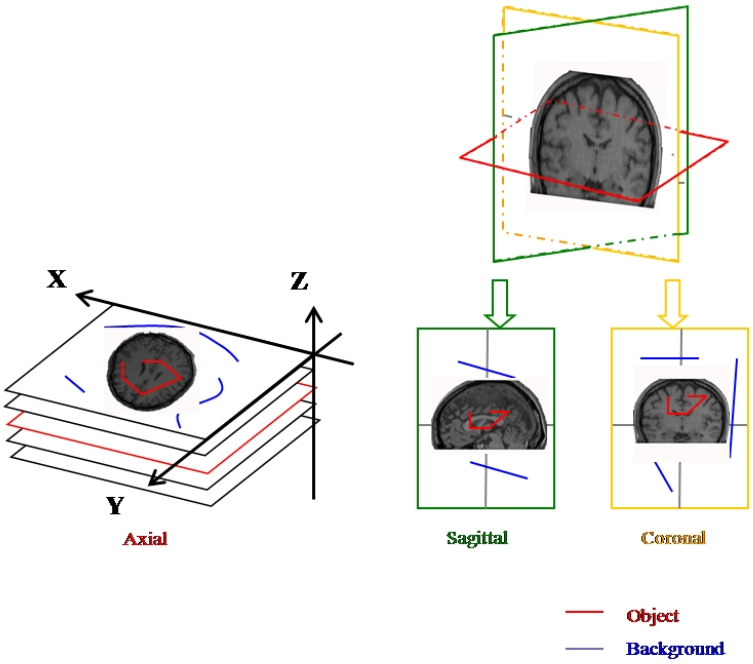


Fig. 9.6 Two-labeling seeds in different MPR image planes

segmentation with few interactions as possible. The software is designed based on object-oriented techniques (VC++ 8.0) to facilitate scheme improvements. The program is implemented on a PC (Intel Dual Core E7200 (2.53GHZ) processor, 2GB RAM, ATI Radeon HD 4850 graphics card with 512MB graphical memory), and the GPU method is performed using OpenGL Shading Language (GLSL).

In MPR image planes, we only select a single axial slice to label desired tissues, sagittal and coronal planes are similar to axial operations, Figure 9.8 demonstrates the labels in axial plane. The total evolution time of implementation on the whole MRI data takes about 812500 millisecond. Average computing time on each slice is 14.5 seconds. The experimental results are listed in Figure 9.9. Another important

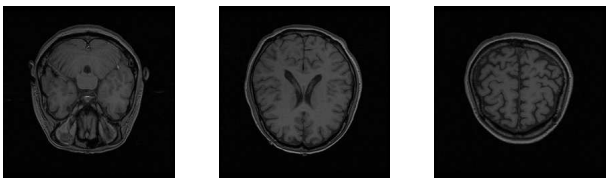


Fig. 9.7 Samples of MRI Series. (a) Location=5. (b) Location =25. (c) Location =50.

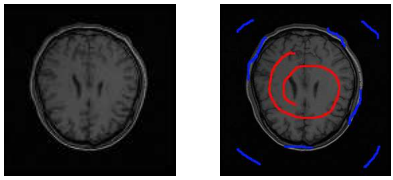


Fig. 9.8 Example of labels in axial plane. (a) Selected slice. (b) labels in image. Red pixels indicate desired objects while blue pixels denote background.

issue is that the local errors can be re-segmented as new object. The additional seeds throughout the computation do not increase the whole working time or only a little time cost. In realistic systems, user can set data boundary to decrease re-computing time largely. Here, eye tissues need to be erased (see Figure 9.10). The validation also indicates that computing time has no explicit relations with slice and seed numbers when the seed number is less than 1/10 of total pixels. The segmented tissues data can be visualized with Marching Cubes [20] [24] and Ray-casting [17] to render (see Figure 9.11).

It is a heavy burden for computing on the whole volumetric data set, so acceleration is crucial for the applications based on CA.

9.3.3.2 Acceleration with Pyramid

In CA evolution, time cost is almost direct ratio to data magnitude, Gaussian and Laplacian pyramid method [4] can be used to produce re-sampled layer data so as to accelerate segmentation. The layer number is specified by user according to interaction precision. In higher level layer, pre-segmentation is performed to get a coarse

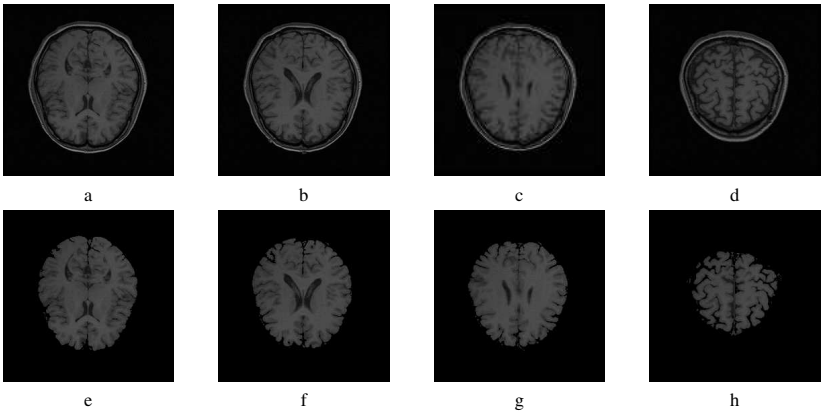


Fig. 9.9 Results of 3D CA segmentation. (a) – (d) Samples of original series. (e) – (h) Segmented tissues of brain (location = 20, 25, 28, 40).

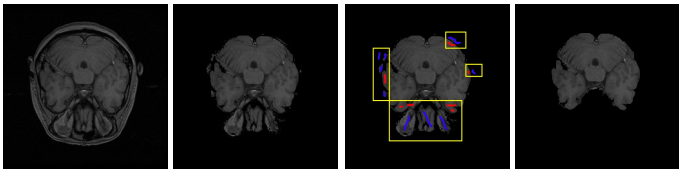


Fig. 9.10 Results of re-segmentation. The yellow rectangles are data boundaries. The computation is limited in these local regions to decrease the computing time.

labeled image as the seed of lower level layer (see Figure 9.1).

Let g_l be the labels of l^{th} Gaussian pyramid, $g_i = EXPAND(g_l, l-i)$ be the result of g_l expanding $l-i$ times. Higher level label image is interpolated as the same size of lower layer with NN (nearest neighbor method). That is $g_{l-1}(i,j) = g_l(\frac{i}{2}, \frac{j}{2}), 0 < l < N$ for levels l^{th} and pixel (i,j), C_{l-1} and R_{l-1} are the width and height of $l-1th$ layer respectively. Thus, $EXPAND$ applied to g_l will yield an array of labeled images. The segmented tissues data can be visualized synchronously with interaction.

If the segmentation supervised by pyramid, acceleration is obvious (see Table 9.2). Here, two additional layers data (1 and 2 level) are produced as the course labeled images. Results are obtained through: 2 level labels image is interpolated to 0 level size directly. The pyramid based method introduces differences inevitably corresponding to different level data, which are unconspicuous in 2D planar and volume rendering results (commonly less than 5.2%). In re-segmentation, user-added new seeds in the local limited rectangle region only cost little time compare to the whole data process, this merit will benefit for the next operations.

Table 9.2 Comparison of CPU and GPU acceleration

Method	Pyramid Initialization	2 Level Seg.	Labels Interpolation	0 Level Seg.	Total time	Speedup Factor
CPU	4288	13146	2501	1429	21364	38.03
GPU	131	134	37	262	564	1440.6

The boundary processing after segmentation mainly focuses on holes in seeded region and ragged boundary, a simple solution is to apply erosion and dilation in the segmented image, but the time cost is a problem. More efficient method is still worth to seek.

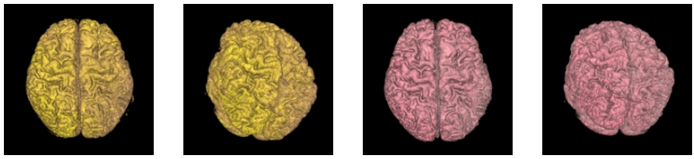


Fig. 9.11 Results of volume rendering

Previously proposed CA methods, e.g. Growcut and Seeded ND-CA, only use local voxel information to guide their evolution, making them prone to errors where boundaries exist. To overcome this problem, [16] present an interactive method that integrates label priors learned from the user interactions in a parallelized CA. The proposed method allows users to easily segment and update the segmentation of any number of different structures in brain tumor segmentation and make the following contributions:

1. Statistical analysis of user input seeds' intensities to create a global model of the intensity distribution for each classification label. The states of the CA are influenced by this model via the newly defined cell evolution rules.
2. A GPU implementation that parallelizes the automata computation, which achieves a 45x speed-up compared to conventional CPU methods and enables the visualization and manipulation of 3D data at interactive rates.

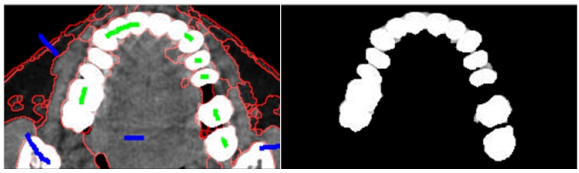
9.4 Block Cellular Automaton Segmentation in Medical Applications

Efficient dental segmentation from volume data provides important assistance for orthodontic surgery and treatment. Quickly and accurately digitized dental model is the basic premise of the surgical simulation and biomechanical evaluation in Orthodontics. To move, arrange and remove individual tooth in a three-dimensional model, each contour need to be accurately extracted. However, this task exits several major challenges due to the special dental anatomy and topological changes. Because optical imaging and dental radiographs cannot provide complete dental structure, CT and CBCT (Cone Beam Computed Tomography) have been intensively applied with guaranteed three-dimensional digital models .

To deal with the dental segmentation problem, a semi-automatic algorithm is proposed to efficiently achieve the desired contours through a simple CA strategy. Firstly, the volume data are partitioned into homogeneous blocks by 3D mean shift classification [5]. Then, user interaction is performed in the selected MPR (Multi-Planar Reformation) image planes to indicate the position and main features of the object and background, i.e. initialize the labeled and unlabeled instances as the input of subsequent processing. The dental contours can be extracted by labeling all the blocks as either object or background (labeled as 1 and 0). The key contribution of the proposed method is treating segmentation as a semi-supervised learning (SSL) mechanism [29], which is adaptive to image content and easy to implement.

BCA merging rule is based on blocks maximal similarity to produce initial labeled and unlabeled instances. This strategy merges local similar blocks as many as possible, which does not need to set similarity threshold in advance. After initial merging, we get a coarse dental model. But there is still numbers of blocks wrongly labeled. The problem is how to adjust this process by a semi-supervised learning based mechanism so that the desired objects can be correctly extracted.

Figure 9.12 demonstrates a view of interaction in 2D MPR image with BCA. Green and blue markers are used to differentiate object and background. The red



a b

Fig. 9.12 User interaction and merged blocks. (a) Initial classification and user markers in 2D MPR image (Green and blue markers indicate the object and background respectively). (b) 2D view of merged blocks by BCA.

lines indicate edges between the homogeneous blocks pre-classified by 3D mean shift. BCA merging method can efficiently separate dental object.

It is worth noting that the initial homogeneous blocks merging is crucial for subsequent learning processing, which can efficiently reduce the dimensions of computing matrix. The proposed segmentation method can be described as a general cluster-then-label algorithm of semi-supervised classification.

Algorithm: Cluster-then-Label.

Input: original voxels data $V_{i\in\Omega}$, an unsupervised clustering algorithm A, labeled blocks $(b_1, y_1), \cdots, (b_l, y_l)$, unlabeled data $b_{l+1}, \cdots, b_{l+u}$, and a semi-supervised learning algorithm SSL .

Output: labels on unlabeled blocks $y_{l+1}, \cdots, y_{l+u}$.

1. Clustered blocks $b_1, \cdots, b_{l+u}$ using A.

2. For each resulting clustered block, let S be the labeled instances. Learn a semi-supervised predictor from S : $f_s = SSL(S)$. Apply f_s to all unlabeled instances.

In step 1, the clustering algorithm A is unsupervised. In step 2, one predictor is we learned using the labeled instances to apply for all the unlabeled instances. This cluster-then-label algorithm does not necessarily involve a probabilistic mixture model.

In validation, we first verify the segmentation algorithm based on blocks maximum similarity (BMSRM) derived from traditional 2D MSRM, and then implement the BCA strategy on the same volume data as a comparison. The semi-supervised learning is used to adjust classification errors, and finally provide the accurate object contour. The program is implemented on a PC (Intel Xeon E5620 2.4 GHz CPU, 24GB RAM, VC++ 9.0 programming environment). The proposed BCA scheme is relying only on simple neighborhood nonlinearities, no optimization or post-processing is required. Figure 9.13 demonstrates a CT sequence ($512\times512\times100$) with resolution of [0.94, 0.94, 2.0] mm. The first row lists the interaction in MPR planes after initial classification; the second row lists the corresponding 2D view of segmentation results of BCA. The green marker represents the dental object while the blue markers represent the background in the selected axial, sagittal and coronal image planes. The marker blocks cover only part but representative features of desired dental objects. The total computing time of the proposed BCA method takes about 2.63 seconds on the whole CT data. The extracted teeth models are listed in Figure 9.14. Another important issue is that the local resulting errors revision needs

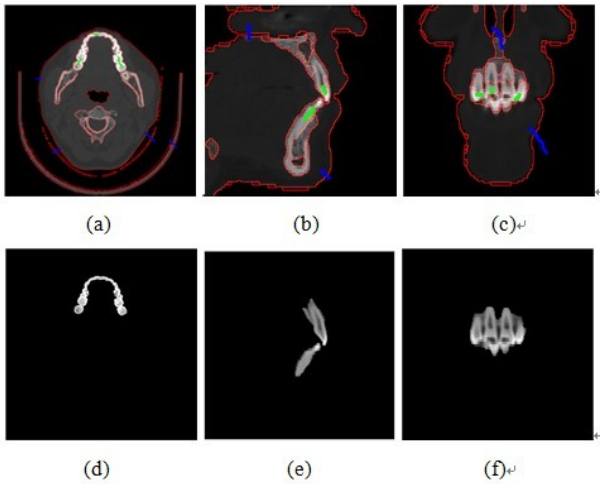


Fig. 9.13 Example of initial classification and interaction in MPR planes. (a)–(c) Selected axial, sagittal and coronal image planes. (d)–(f) Segmented tissues of teeth.

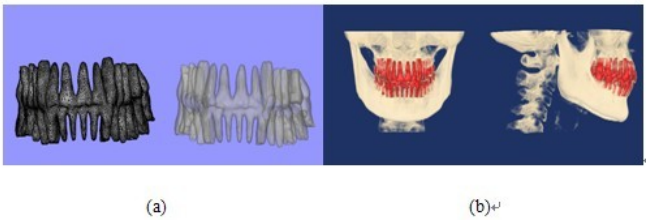


Fig. 9.14 Dental segmentation results. (a) The extracted mesh and volume rendering results. (b) Highlighted rendering with surrounding tissues.

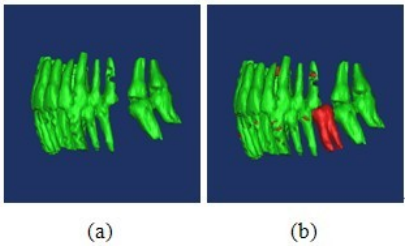


Fig. 9.15 Dental segmentation results. (a) The extracted volume surface model. (b) Highlighted difference between BCA and BMSRM.

only a little more time cost. Using the same input markers, the total evolution time of BMSRM takes about 30.28 seconds. The extracted dental models are listed in Figure 9.15(a). Figure 9.15(b) demonstrates the difference between BMSRM and

Table 9.3 Comparison of BCA and BMSRM

Method	Time (s)	TPR(%)	FPR(%)
BCA	2.63	97.75	1.26
BMSRM	30.28	90.68	8.49

BCA algorithm. BMSRM is easy to fail in some important tissues and wrongly label background blocks as object (highlighted with red color). Here, an individual tooth is lost and many holes occur in surface, that dues to BMSRM only considers local neighborhood similarity while SSL based BCA can achieve a global minimum error. To get the quantitatively comparison, desired objects are manually sculptured as ground truth. Table 9.3 lists TPR and FPR comparison between these two methods. The numeral results imply that BCA achieves the better segmentation performance. BCA is also performed in numbers of CBCT sequences and get the similar results. This validation with low SNR (Signal Noise Ratio) data demonstrates that by grouping the similar pixels into homogeneous blocks, mean shift initial classification improve the robustness of BCA to noises and small pixel variations.

Commonly, the proposed BCA method is not sensitive to initial classification, this fact makes it is more robust and efficient in volume data segmentation tasks. Although BCA has been greatly reducing the requirements of user interaction, but more manual markers are still needed in region with complex structures. Additionally, dental shape and arrangement priors should be considered to increase the precision.

References

1. Boykov, Y., Kolmogorov, V.: An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision. IEEE Transactions on Pattern Analysis and Machine Intelligence 26(9) (2004)
2. Boykov, Y., Veksler, O., Zabih, R.: Fast approximate energy minimization via graph cuts. IEEE Transactions on Pattern Analysis and Machine Intelligence 23(11), 1222–1239 (2001)
3. Boykov, Y.Y., Jolly, M.P.: Interactive graph cuts for optimal boundary and region segmentation of objects in ND images. In: Proceedings of the Eighth IEEE International Conference on Computer Vision (ICCV 2001), vol. 1, pp. 105–112. IEEE (2001)
4. Burt, P.J., Adelson, E.H.: The Laplacian pyramid as a compact image code. IEEE Trans. on Communications COM-31(4), 532–540 (1983)
5. Comaniciu, D., Meer, P.: Mean shift: A robust approach toward feature space analysis. IEEE Transactions on Pattern Analysis and Machine Intelligence 24(5), 603–619 (2002)
6. Comaniciu, D., Ramesh, V., Meer, P.: Kernel-based object tracking. IEEE Transactions on Pattern Analysis and Machine Intelligence 25(5), 564–577 (2003)
7. Dehmeshki, J., Amin, D., Valdivieso, M., Ye, X.: Segmentation of pulmonary nodules in thoracic CT scans: A region growing approach. IEEE Transactions on Medical Imaging 27(4) (April 2008)

8. Ermentrout, G.B., Edelstein-Keshet, L.: Cellular automata approaches to biological modeling. Journal of Theoretical Biology, 97–133 (1993)
9. Felzenszwalb, P.F., Huttenlocher, D.P.: Efficient graph-based image segmentation. International Journal of Computer Vision 59(2), 167–181 (2004)
10. Gao, Y., Yang, J., Xu, X., Shi, F.: Efficient cellular automaton segmentation supervised by pyramid on medical volumetric data and real time implementation with graphics processing unit. Expert Systems with Applications 38(6), 6866–6871 (2011)
11. Grady, L., Funka-Lea, G.: Multi-label image segmentation for medical applications based on graph-theoretic electrical potentials. In: Sonka, M., Kakadiaris, I.A., Kybic, J. (eds.) CVAMIA/MMBIA 2004. LNCS, vol. 3117, pp. 230–245. Springer, Heidelberg (2004)
12. Hadwiger, M., Laura, F., Rezk-Salama, C., Hollt, T., Geier, G., Pabel, T.: Interactive volume exploration for feature detection and quantification in industrial CT data. IEEE Transactions on Visualization and Computer Graphics 14(6), 1507–1514 (2008)
13. Hojjatoleslami, S.A., Kittler, J.: Region growing: A new approach. IEEE Transactions on Image Processing 7(7) (July 1998)
14. Kass, M., Witkin, A., Terzopoulos, D.: Snakes: Active contour models. International Journal of Computer Vision 1(4), 321–331 (1988)
15. Kauffmann, C., Piché, N.: Seeded ND medical image segmentation by cellular automaton on GPU. International Journal of Computer Assisted Radiology and Surgery 5(3), 251–262 (2009)
16. Kim, E., Shen, T., Huang, X.: A parallel cellular automata with label priors for interactive brain tumor segmentation. In: Int. Symposium on Computer-Based Medical Systems (CBMS), pp. 232–237 (2010)
17. Knoll, A., Wald, I., Parker, S., Hansen, C.: Interactive isosurface ray tracing of large octree volumes. In: 2006 IEEE Symposium on Interactive Ray Tracing, pp. 115–124 (2006)
18. Levner, I., Zhang, H.: Classification-driven watershed segmentation. IEEE Transactions on Image Processing 16(5), 1437–1445 (2007)
19. Li, W., Han, G., Zou, K., Gu, G., Zhang, X.: Natural image matting using cellular automata. In: International Conference on Convergence Information Technology, pp. 1746–1751. IEEE (2007)
20. Lorensen, W.E., Cline, H.E.: Marching cubes: A high resolution 3D surface reconstruction algorithm. ACM SIGGRAPH Computer Graphics 21(4), 163–169 (1987)
21. Meyer, F., Beucher, S.: Morphological segmentation. Journal of Visual Communication and Image Representation 1(1), 21–46 (1990)
22. Mortensen, E.N., Barrett, W.A.: Toboggan-based intelligent scissors with a four-parameter edge model. In: IEEE Computer Society Conference on Computer Vision and Pattern Recognition, vol. 2, pp. 452–458. IEEE (1999)
23. Mortensen, E.N., Barrett, W.A.: Interactive segmentation with intelligent scissors. Graphical Models and Image Processing 60(5), 349–384 (1998)
24. Newman, T., Yi, H.: A survey of the marching cubes algorithm. Computers and Graphics 30(5), 854–879 (2006)
25. Ning, J., Zhang, L., Zhang, D., Wu, C.: Interactive image segmentation by maximal similarity based region merging. Pattern Recognition 43(2), 445–456 (2010)
26. Phan, R., Androutsos, D.: Interactive video growcut: A semi-automated video object segmentation framework using cellular automata. In: 2011 24th Canadian Conference on Electrical and Computer Engineering (CCECE), pp. 000,081–000,084. IEEE (2011)

27. Vezhnevets, V., Konouchine, V.: “growcut"-interactive multi-label n-d image segmentation by cellular automata. In: Proc. Graphicon, pp. 150–156 (2005)
28. Vincent, L., Soille, P., Wang, Q.: Segmentation for MRA image: An improved level-set approach. IEEE Transactions on Instrumentation and Measurement 56(4), 1316–1321 (2007)
29. Zheng, Y., Kambhamettu, C.: Learning based digital matting. In: IEEE 12th International Conference on Computer Vision, pp. 889–896 (2009)

Chapter 10
Convex Hulls and Metric Gabriel Graphs

Luidnel Maignan and Frédéric Gruau

Abstract. The convex hulls construction is mostly known from the point of view of 2D Euclidean geometry where it associates to a given set of points called seeds, the smallest convex polygon containing these seeds. For the cellular automata case, different adaptations of the definition and associated constructions have been proposed to fit with the discreteness of the cellular spaces. We review some of these propositions and show the link with the famous majority and voting rules. We then unify all these definitions in a unique framework using metric spaces and provide a general solution to the problem. This will lead us to an understanding of the convex hull construction as a chase for shortest paths. This emphases the importance of Voronoï diagrams and its related proximity graphs: Delaunay and Gabriel graphs. Indeed, the central problem to be solved is that of connecting arbitrary sets of seeds, in a local and finite-state way, while remaining inside the desired convex hull, i.e by shortest paths. This is exactly what will be made possible by a suitable generalization of Gabriel graphs from Euclidean to arbitrary metric spaces and the study of its construction by cellular automata. The general solution therefore consists of two levels: a connecting level using the metric Gabriel graphs and a level completing the convex hull locally as the majority rule does. Both levels can be generalized to compute the convex hull, when the seeds are moving.

10.1 Notational and Naming Convention

In this chapter, the set of all cells of the space is denoted S. The neighborhood of a cell x is denoted $N(x)$ and does not include the cell x itself. A set of cells

Luidnel Maignan
LACL, Université Paris-Est Créteil, France
e-mail: luidnel.maignan@u-pec.fr

Frédéric Gruau
LRI, Université Paris-Sud Orsay, France
e-mail: gruau@lri.fr

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_10, © Springer International Publishing Switzerland 2014

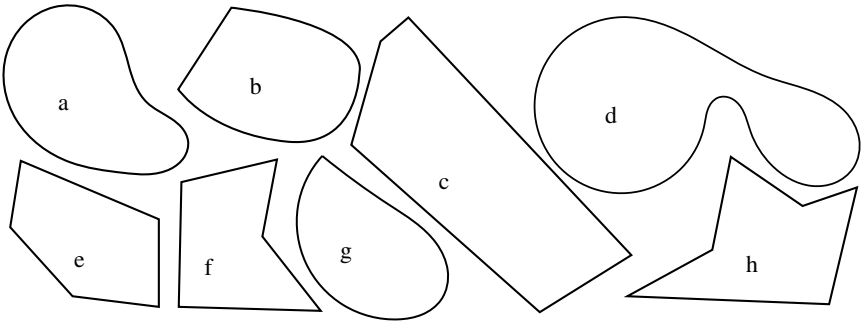


Fig. 10.1 Examples of Euclidean convex and non Euclidean convex shapes

together with a specified neighborhood is called the cellular space. Cells are sometimes called points. The value associated by a configuration c at a cell x is denoted $c(x)$. The configuration obtained at time t by a rule f is denoted f_t, subscripts being used to reduce the impact of repeating the timing information. Hence, $f_t(x)$ is the value associated to a cell x at a time t by a rule f and the transition function of the rule, for example the sum of all the values in the neighborhood, can be denoted as $f_{t+1}(x) = \sum_{y \in N(x)} f_t(y)$. Note that we do not indicate explicitly what is the initial configuration, letting it free to be initialized externally in some sense. Also, rules are often called fields, similarly to electric field and magnetic field. A single cellular automaton can be made of a certain number of fields, in which case they can refers to each other in their transition function, and even to just computed values. For example if there are two rules f and g, $f_{t+1}(x)$ might depend on $g_{t+1}(x)$ as long as there are no circular definition and everything is well-defined. This is why we need to be explicit about the timing information. This is a light notation related to, although a bit more explicit than, the one used in multivariate differential calculus for example.

10.2 Introduction

The convex hull construction is well known in the context of Euclidean geometry. Its definition comes in two steps, one for the word "convex" and one for the added word "hull". A set of points is **convex** if and only if it contains entirely any segment joining any two of its points. As an exercise to link this definition with its geometrical content, the reader should check, for each shape of Fig. 10.1, whether the set of its inside points is convex or not by trying to exhibit a segment not entirely contained in the shape while its extremities are in the shape. Note that some of the convex shapes are polygons, and others are not.

Convex shapes are simple shapes in some interesting sense. In physics, or physics simulations for example, detecting when two arbitrary shapes collide is difficult but this detection is much easier when the shapes are convex. There are also situations

where all the shapes that appear are necessarily convex, such as in a bumble bath, etc. However, there are situations where the shapes are not convex, but we would like to approximate it by a convex shape in order to make life easier. In this case, we have an initial set of points, not necessarily convex, and we look for its **convex hull**, i.e. the smallest convex set containing all the initial points. In the rest of this chapter, these initial points will be called **seeds**. The question is how to compute the convex hull of a set of seeds given as input.

A well known fact is that if there are finitely many seeds, their convex hull is a polygon using these seeds as vertices and many classical algorithms exist to compute this polygon as a list of vertices based on a set of seeds, everything being described using a coordinate system. In contrast, this chapter considers the case where the space is not the usual Euclidean space but a cellular space and the seeds are not represented by coordinates but by a Boolean information holded by each cell and representing whether the cell is a seed or not. So we do not assume any knowledge of the classical algorithms but let us add a few words on why finite sets of seeds have a polygonal convex hull in 2D Euclidean spaces.

The reason is the following: for the set of seeds to be convex, all the segments joining them have to be in the set, so we consider now a set containing the seeds and their pairwise segments. This new set in still not convex in general and we still need to add all the segments joining the newly added points, and so on until no segment is missing. The final result of this iterative adding process is indeed the minimal convex set containing the seeds since only absolutely required segments have been added. But all these additional points are strictly between the firstly added segments, so the border of the convex hull is made of some of the firstly added segment with the seeds as vertices. This is a very basic process, but it worths taking the time to visualize it and to keep it in mind from the start.

In the context of cellular automata, the famous majority and voting rules and some of their variations exhibit behaviors related to Euclidean convex hulls as described in [7]. For instance, let us consider the majority rule where cells have two states, either selected or not selected, and the rule selects a cell if at least 4 or its neighbors are selected. If we denote by P the set of seeds and by the predicate $\text{majo}_t(x)$ the fact that the cell x is selected at time t by the rule majo, this behavior can be written formally as:

$$\text{majo}_{t+1}(x) = x \in P \vee \text{card}\{\, y \in N(x) \mid \text{majo}_t(y) \,\} \geq 4 \tag{10.1}$$

Figure 10.2 shows an evolution of this rule with a dense set of seeds as initial configuration. Here, the Moore neighborhood is used, so each cell has 8 neighbors, and the number 4 is 50% of the neighborhood, hence the name of the rule. So the number would be 2 for a Von Neumann neighborhood, 3 for hexagonal cells which have 6 neighbors, and so on. In all these cases, it is clear that the final result is visually related to Euclidean convex hulls. To make this precise, different approaches have been tried. The goal is not only to understand what sort of convex hull is computed by the majority rule, but also to be able to compute such a convex hull for arbitrary sets of seeds, which is unfortunately not the case for the majority rule.

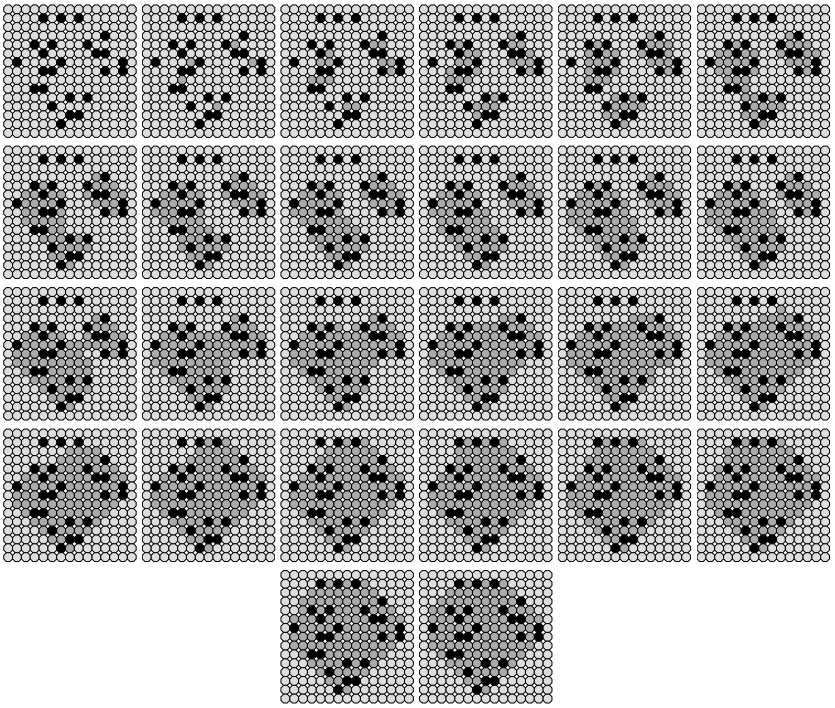
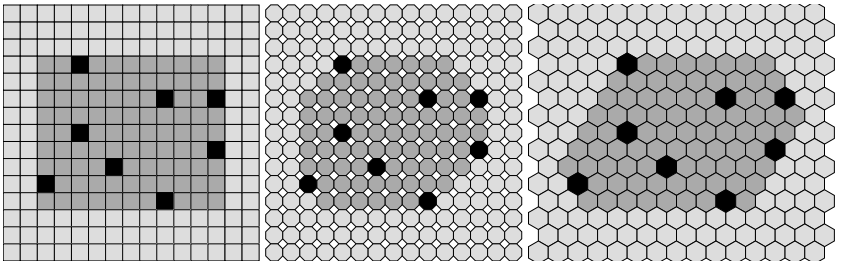


Fig. 10.2 Convergence to one global convex hull

We first describe the early angular point of view to this problem, describing the obtained convex hull in terms of Euclidean convexity with a restriction on the allowed angles in Sect. 10.3. This point of view takes into account the Euclidean positions of the cells. In Sect. 10.4 and Sect. 10.5, we then describe a more general approach describing everything in terms of shortest paths, i.e. in terms of distances, i.e. in terms of metric space. In this case, the focus is on something more intrinsic than the cells position, namely the graph of communication of the cells. We then conclude in Sect. 10.6 by a summary the complete cellular automaton that works for any set of seeds and for any cellular space.

10.3 The Angular Point of View

The first approach is to consider the positions of the cells in Euclidean space. If we take an arbitrary cell and consider the vector leading to each of its 8 neighbors, we can see that they match the directions of the sides of the convex polygon obtained in Fig. 10.2. This idea can be extended to Von Neumann neighborhood with 4 vectors, and to hexagonal cellular spaces with 6 vectors. The trick is therefore to change the definition of convexity and to allow a set to be called convex if and only if it is Euclidean convex polygon and its sides follow one the admitted



a Von Neumann: $\theta = 90$ b Moore: $\theta = 45$ c Hexagonal: $\theta = 60$

Fig. 10.3 θ-convex hulls in cellular spaces

vector directions. The Von Neumann, Moore, and hexagonal cellular spaces thus allow directions whose angles are multiple of 90°, 45°and 60°respectively. This concept is called **θ-convexity**, and examples of θ-convex hull are shown in Fig. 10.3. It might worth noting that, contrary to the Euclidean convex hull, the vertices of the θ-convex hull polygon are not necessarily in the original set of points. Using this definition, let us examine again the behavior the majority rule and see how arbitrary sets of seeds can be handle.

10.3.1 Majority Rule and θ-Convexity

With the definition of θ-convexity, we have an explanation of the results of the evolution depicted in Fig. 10.2. However this behavior is not the general case. When the set of points is not dense enough, we might obtain a disconnected set, corresponding to a collection of partial θ-convex hulls. In fact, it is not really a matter of density, and if we take the initial configuration of Fig. 10.2 and move only one of the six bottom seeds, we obtain the behavior shown in Fig. 10.4. In Fig. 10.2, the moved seed is higher, allowing the partial convex hulls to merge into a bigger convex-hull. So in general, the complete θ-convex hull is not obtained, and it is hard to describe the final result by an other way than saying that this is the fixpoint of the majority rule. But both of these issues will be addressed, firstly with the solution described in the next section, and then further by changing the point of view from a angular to a metrical one.

10.3.2 Complete θ-Convex Hull

In [5, 14], a cellular automaton is proposed that constructs the θ-convex hull of an arbitrary set of seeds, with the little price that the seeds have to be already enclosed in an arbitrary initial connected region. This rule improves this initial region by two stages that eventually transforms it into the θ-convex hull of the seeds. The first stage is to reduce the region to ensure that it is smaller than the wanted θ-convex hull. The second stage is to inflate the region just enough to exactly match

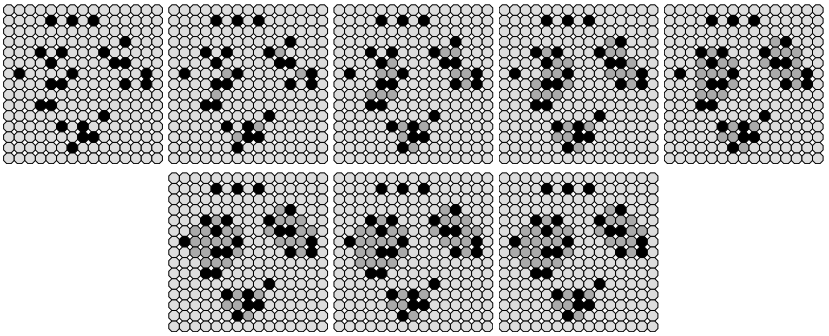


Fig. 10.4 Convergence to many convex hulls

the wanted result. This second stage is simply a modified version of the majority rule described earlier, which is enough because the result of the first stage is a connected region. However, the transition from the first to the second stage requires a mechanism that ensure a coherent global transition to the second stage, as it is not desirable to be in the shrinking and growing stages at the same time. There are of course some little technical difficulties, and the reader is redirected to the original articles for more details. An example of initial configuration is shown in Fig. 10.5, along with the result of the first and second stages on it.

10.4 The Metrical Point of View

The solution described in the previous section is a great improvement on the majority rule since it handles an arbitrary set of seeds. However, one might ask whether it is possible to get rid of the initial region and simply build the region from the seeds. In the case of the Euclidean convex hull, we noted in Sect. 10.2, the definition of the convex hull asks to add all required segments to the initial set of seeds in order

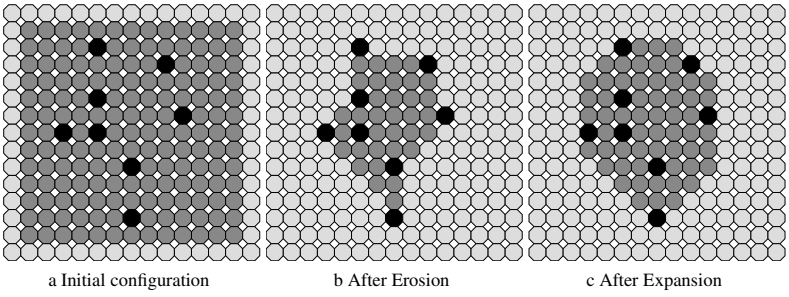


Fig. 10.5 Stages from wrapped seeds to their convex hull: The initial wrapping (a) is shrunk into (b) and is then grown to convex hull (c)

to obtain a convex set. This idea is the main anchor of the rest of the chapter, and it is applied with a change of point of view.

In this new point of view, we forget about any position of the cells in some Euclidean space and simply consider the graph of communication of the cells and its structure. In this context, we can not talk about angles, but we can still talk about distances between pairs of cells. Indeed, Euclidean convexity is a particular case of metric convexity. This latter is defined over metric spaces, and replaces the word *segment* in the definition of Euclidean convexity by the more general concept of *shortest path*. Therefore, a set of points is **metric convex** if and only if it contains entirely any *shortest path* joining any two of its points. As everything else in this chapter will depend on this definition, let us be more precise by giving the formal definition of metric spaces, shortest paths and metric convexity.

A **metric space** is a set of points M together with a distance function d, also called metric function or metric, associating to each pair of points $p_0, p_1 \in M$ a real value $d(p_0, p_1) \in \mathbb{R}$ in a certain appropriate way that allows us to think of $d(p_0, p_1)$ as the distance between the points. The requirement to be appropriate is that the function must be symmetric, i.e. $d(p_0, p_1) = d(p_1, p_0)$, the function should assign the value 0 for all and only for distances between a point and itself, i.e. $d(p_0, p_1) = 0 \Leftrightarrow p_0 = p_1$, and whenever we make a detour via a third point, the total distance should be greater or equal than the direct travel, i.e. $d(p_0, p_1) + d(p_1, p_2) \geq d(p_0, p_2)$. This last requirement is the least trivial one. It is often called the **triangle inequality** and is very important.

Given a path in a metric space, one can consider the length of this path by summing the distance between each pair of successive points of the path. Between two arbitrary points p_0 and p_1, one may ask for the length of the shortest paths among all possible linking paths and expect this value to match $d(p_0, p_1)$. However, this does not follow from the three previous requirements, but it is often convenient to have this property. In this case, the metric of the space is said to be intrinsic, and the space is called a **length metric space**. For example, Euclidean spaces and also graphs with their natural notion of distance are both length metric spaces. In this case, the triangle inequality really says something about the length of the shortest paths, detours and direct travels.

In particular, it is possible to express the fact that a point z is on a shortest path between two points x and y as a **triangle equality**: $d(x,z) + d(z,y) = d(x,y)$, which means that we do not make the path lengthier by making a detour via z. We say that z is **between** x and y, or more explicitly that z is on a shortest path joining x and y. We will use this triangle equality and, equivalently, the notion of betweenness many times, and will denote it as $z \in [x,y]$, which means that $[x,y] = \{ z \in M \mid d(x,z) + d(z,y) = d(x,y) \}$. This is called the **interval** between x and y. In the Euclidean case, this set $[x,y]$ is exactly the points of the segment joining x and y, but in our cellular spaces where there might be many shortest paths between two cells, it corresponds to the union of all these shortest paths.

After all these definitions, let us see what a metric convex hull looks like in our cellular spaces. Firstly, in Fig. 10.6, the interval between two points of various cellular spaces are shown. The figure also shows the "Moore-$\sqrt{2}$" cellular spaces which

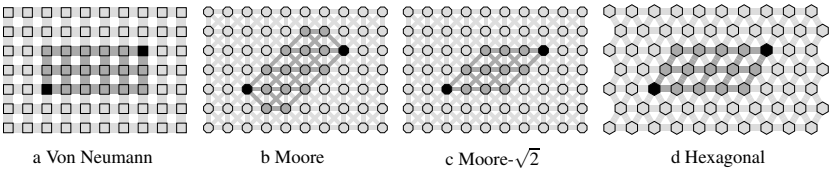


Fig. 10.6 Intervals for different cellular spaces and different metrics

correspond to the communication graph of the Moore cellular space with the length of diagonal assigned the value $\sqrt{2}$ instead of just 1 as graph metric would imply. This difference makes the horizontal and vertical edges shorter, and therefore more preferable than diagonal one. This discards some paths of the Moore cellular space as being shortest. The structure of the intervals dictates the structure of the metric convex hulls. Examples of metric convex hulls are shown in Fig. 10.7. This shows that the definition of metric convex hull is strictly more general than θ-convex hull since the 90, 45, and 60-convex hull corresponds to the metric convexity in Von Neumann, Moore-$\sqrt{2}$ and hexagonal cellular spaces respectively. Note that the Moore-$\sqrt{2}$ can be constructed as the intersection of the Von Neumann and Moore metric convex hull. This allows us to restrict our attention only to Von Neumann, Moore and Hexagonal cellular spaces, i.e. we will always consider just plain graph metric, i.e. the distance between a cell and one of its neighbor is always 1, even for Moore diagonals. This sets precisely what our goal is: to understand the majority rule based on metric convexity and to find a cellular automaton that computes this metric convex hull for arbitrary sets of seeds. Because of the chosen level of generality, the results will also be applicable for a large class of cellular spaces, including 3D and higher dimensional cellular spaces, but those are unfortunately not very good as paper examples.

10.4.1 Majority Rule and Metric Convexity

Coming back to what we already said, computing the convex hull is about adding all the missing shortest paths. In fact, the majority rule can be understood in these terms. To see that, let us consider the neighborhood $N(x) \cup \{x\}$ of the center cell x and the distance function restricted to this neighborhood. If we take a such neighborhood

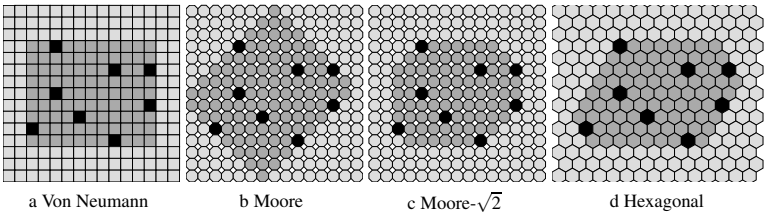


Fig. 10.7 Convex hulls for different cellular spaces and different metrics

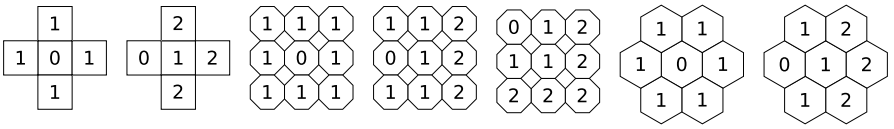


Fig. 10.8 Distances between cells in a neighborhood

and take one cell y in it, we can assign to all cells z its distance $d(y,z)$ to y as depicted in Fig. 10.8. If y is the center x, then all of its neighbors are at distance 1 by definition, as shown in the first, third, and sixth neighborhoods of Fig. 10.8. If y is a neighbor of the center x, then the direct neighbors of y are at distance 1 from y and any other cell z is at distance 2. Indeed, in this last case, the path (y,x,z) is a shortest path of length 2. Now, what is the relation with the majority rule ? The answer is that any time the majority rule selects a cell, it is because half of its neighborhood is already selected as specified in Sect. 10.2. But this implies that there are two selected cells of distance 2 apart from each other. This means that there is a shortest path joining these two cells and passing through the center as we just pointed out. Therefore the center has to be selected for the set of selected cells to have a chance of being metric convex. This explains why the majority rule never produces more than the convex hull, and evolves toward convexity.

Using this understanding, we can replace the majority rule with a more direct one that checks for the existence of two selected cells in the neighborhood such that a shortest path passes through the center:

$$\text{conv}_{t+1}(x) = x \in P \vee \exists y_0, y_1 \in N(x); \text{conv}_t(y_0) \wedge \text{conv}_t(y_1) \wedge x \in [y_0, y_1]; \quad (10.2)$$

Note that there are many ways to write this rule formally, but this one is the more direct one with respect to our intention. Therefore, this formula remains unchanged if we change the neighborhood in size or shape for example, while other ways of writing it might need some adaptation.

With this rule, we obtain for example the evolution depicted in Fig. 10.9. The conv rule has roughly the same global behavior than the majority rule presented in the sense that it generally produces a set of partial convex hulls and that the partial convex hulls can merge to form bigger convex hulls during the evolution. However it has at least two benefits. Firstly, the conv rule is more precise than majo. Indeed, it detects strictly more neighborhoods than the majority rule. As an example, one can note that the initial configuration of Fig. 10.9 is a fixpoint for the majority rule. Secondly, the final result has a clear property that can be expressed without reference to fixpoints: any shortest path of length 2 between any two cells of the selected region belongs to the selected region. The link with the convex hull is far more obvious in this way.

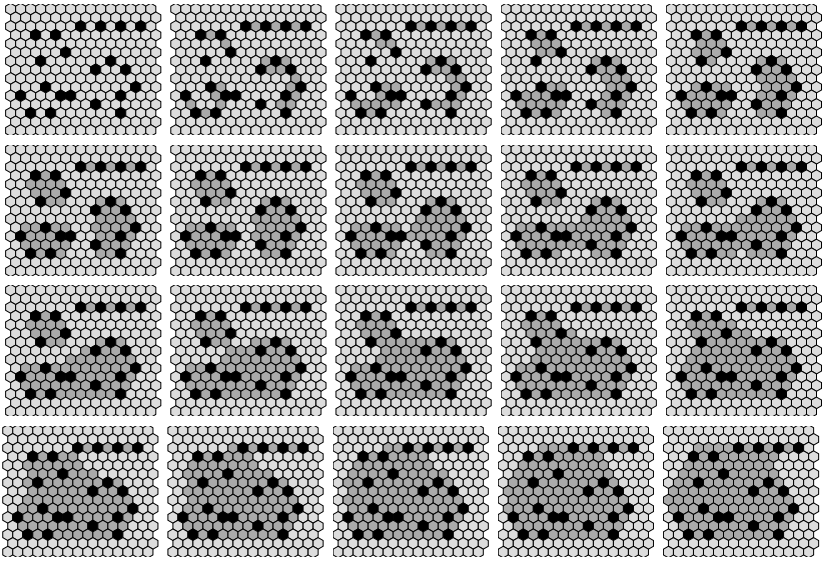


Fig. 10.9 Evolution of the conv rule with neighborhood radius 1

10.4.2 Complete Metric Convex Hulls

Now that we know how to obtain partial metric convex hulls, involving only shortest paths of length at most 2, let us target the detection of shortest paths of arbitrary length. Our first goal is, given two seeds, to select the shortest paths joining them, i.e. all cells of their interval. A first idea might be to compute the distance of all cells to all the seeds and then select the cells where the triangle equality is verified. However we can not handle an arbitrarily large number of integers in each cell in the cellular automata framework. The number of states of the cells has to be finite.

We can reduce this idea and only store in each cell one distance, the distance to the closest seeds, modulo 3. This information is, in fact, enough to detect the middles of the shortest path, and from them, the entire shortest path. A rough intuition for the latter is given by the fact that, along any shortest path connecting two seeds, the distance modulo 3 values stored in the cells is $(0,1,2,0,1,2,0,2,1,0,2,1,0)$ for instance. It is possible to locally distinguish between the middle and the non middle cells, and from these middles, it is possible to join the seeds by following the distance values in decreasing order $(2,1,0)$. This is explained in more details in Sect. 10.4.2.1.

In Sect. 10.4.2.2, we explain that this interval detection is, in fact, enough to solve entirely the problem and we examine why this is the case. In fact, when many seeds are involved, sufficiently many pairs of seeds are connected to allow the conv or majo rules to complete the job. But the investigation of the detection of middles in general cellular spaces of arbitrary dimensions leads to a trip through the notions of

Voronoï diagram, Delaunay and Gabriel graphs in order to make the full connection between the Euclidean and the cellular case, and is therefore postponed to Sect. 10.5.

10.4.2.1 Paths Joining Two Distant Seeds

As just explained, to connect two distant seeds, we need to compute some distance information. The cellular automata framework only allows for a finite number of state for the cells, so we can not store sets of integers, or even single arbitrarily large integers. We are reduced to compute at each cell only one distance value, and to compute it modulo 3: this distance value is the distance of the cell to its closest seeds. We could have called this the distance rule, but we in fact call this the **distance field**.

In the cellular automata case, this distance field corresponds to the computation of the minimal distance modulo k, where k depends on properties of the complete cellular automaton where the distance field is used. It is a very general notion that can be used to solve many problems and can handle the motion of seeds and asynchronicity to some extent for example. Its full description [10] is out of the scope of this book chapter so we present only a reasonable restricted version of it to the case of synchronous static seeds. In this case the value of k is 3.

In this restricted version, the idea is to begin with the value 0 on the seeds, and the value 1 anywhere else as shown in the initial configuration of Fig. 10.10. From this initial configuration, each cell increases its value at each transition until its value becomes greater than one of its neighbor value[1]. This happens exactly when one of the neighbors has a different value since all values only increases then stops at some transition. Formally, the transition function is:

$$\mathrm{dist}_{t+1}(x) = \begin{cases} 0 & \text{if } x \in P \\ \mathrm{dist}_t(x) + 1 \bmod 3 & \text{if } x \notin P \wedge \forall y \in N(x);\ \mathrm{dist}_t(y) = \mathrm{dist}_t(x) \\ \mathrm{dist}_t(x) & \text{if } x \notin P \wedge \exists y \in N(x);\ \mathrm{dist}_t(y) \neq \mathrm{dist}_t(x) \end{cases} \quad (10.3)$$

The meaning of this transition rule can be observed on Fig. 10.10. After the first transition, all values are at 2, except the seeds, which kept their value 0, and the cells around the seeds which kept their value 1 because of the presence of 0 in their neighborhood. After the second transition, all cells values go to 0, except the seeds (because they are seeds), the neighbors of the seeds (because of the presence of the 0 in their neighborhood while their value is 1) and the neighbors of the neighbors of the seeds (because of the presence of 1 in their neighborhood while their value is 2). So circles of cells of same distance value are constructed, and after some transitions, the circles coming from different seeds eventually collide at different cells. The first

[1] Note that because of the modulo three operation, one should only look at the differences between values locally, i.e. in the neighborhood of a given cell. If this cell value is 1, then the order is $0 < 1 < 2$, but if this cell value is 0 or 2, the orders are respectively $2 < 0 < 1$ and $1 < 2 < 0$. So a cell of value v considers, in its local neighborhood, that the value $v - 1 \bmod 3$ is less than its value and $v + 1 \bmod 3$ is greater than its value. This is the way the comparisons have to be understood.

cells where these collisions occur are those of interest: they are exactly at the middle of the shortest paths between the seeds. In Fig. 10.10, this happens after the forth transition. We can see that the distance values in the neighborhood of these middles is special. Each of the middle cells can actually see that the distance pattern in its neighborhood can only appear because of the presence of two seeds at opposite directions.

In Fig. 10.11 theses middle cells are marked in light green. We need to express the general middle detection rule more precisely of course. For a particular grid, it is possible to build this rule by enumerating all local distance patterns corresponding to middles, but expressing this detection rule for general cellular spaces appears to be the main difficulty in the convex hull construction. So let us pretend that we can do it and proceed to the final step. We will come back to the middle detection in the next section.

So if we suppose that we can detect these middle cells, the rest of the evolution is obtained easily and is depicted in Fig. 10.11. Once the middles are detected, we can consider the neighbors of the middles. Let us consider one of them and call it x. It has all required information to know if it belongs to the shortest path or not. Indeed, it knows its distance value $d(s,x) = \text{dist}_t(x)$ to the nearest seed s, the distance value $d(s,y) = \text{dist}_t(y)$ of the marked middle y, and the distance $d(x,y) = 1$. It can thus determine whether it is between the seed and the middle, i.e. whether it is on a shortest path joining s and y, i.e. $x \in [s,y]$. The triangle equality $d(s,y) = d(s,x) + d(x,y)$ to be checked is reduced to $\text{dist}_t(y) = \text{dist}_t(x) + 1$ in this case. So if y is marked as being in the convex hull and this reduced triangle equality is verified, x can deduce that it has to belong to the convex hull too. Formally, we obtain the following transition rule:

$$\text{back}_{t+1}(x) = \text{cent}_{t+1}(x) \vee \exists y \in N(x);\ \text{dist}_t(y) = \text{dist}_t(x) + 1 \bmod 3 \wedge \text{back}_t(y). \tag{10.4}$$

A more simple explanation of the behavior is that to go back to the seeds from the middles, you have to follow the distance value in decreasing order, i.e. the cell x takes the mark from the cell y if $\text{dist}_t(x) = \text{dist}_t(y) - 1$. Both views are simply equivalent, they both describe a trajectory along the shortest possible paths back to the seeds. It is interesting to note that anywhere the notion of shortest path is used, it can somehow be reduced to the triangle equality. Coming back to Fig. 10.11, we can see after 4 transition the detection of the middles of distance value 1, and then, at the next configuration some cells of distance value 0 deduce that they have to be in the convex hull from the presence of a marked cell in their neighborhood and the differences in their distance value, and after that, the same thing happens to some cells of distance value 2 at the next configuration, and so on. It worths checking the distance values on Fig. 10.10 while consulting Fig. 10.11.

10.4.2.2 Pairwise Hull Construction

We now have a way to construct the convex hull of two seeds using one distance field and some detections on it. When considering many seeds, we can obviously not

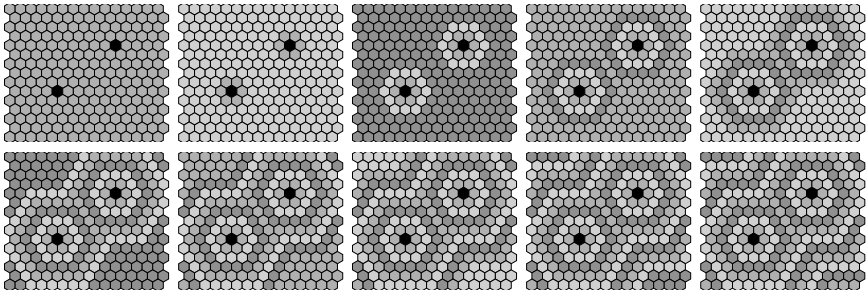


Fig. 10.10 Distance fields from two particles

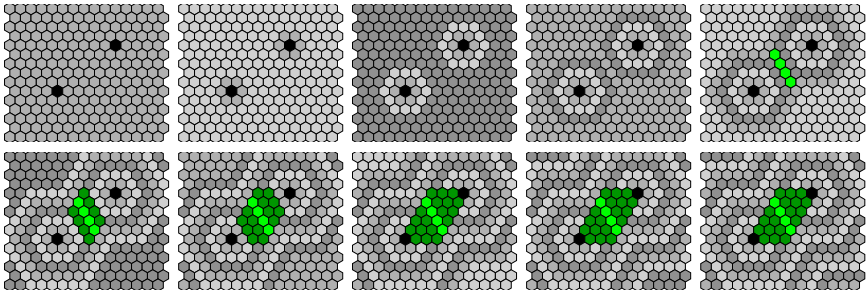


Fig. 10.11 Detected middles and paths back to both particles

build one distance field for each pair of seeds in the space, as we need the number of states of the cells to be finite. However, a single distance field might allow for the detection of middles of more than one pairs of seeds. This can be checked on Fig. 10.12 which shows the evolution of the rules dist and back (using our yet-to-be-defined detection of the middles cent) in the presence of many seeds. In the case of this figure, we can even see that the result is connected. If this is always true that the result of this pairwise construction is connected, then adding conv or majo

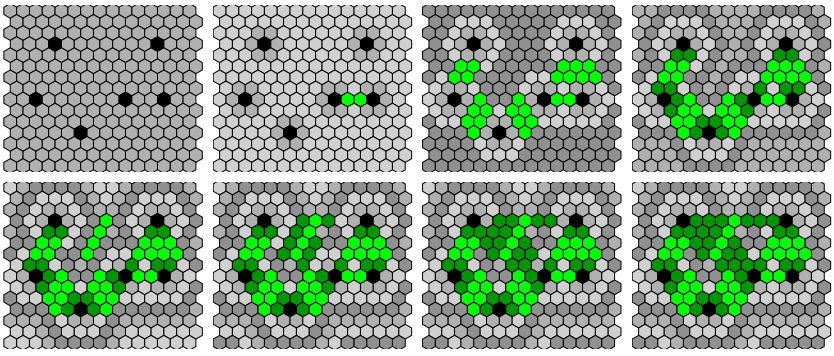


Fig. 10.12 Construction of pairwise convex hull

rule to our previous construction is enough to complete to the complete convex hull. It turns out to be the case from the results of the next section, and we can already say that the final construction of the convex hull is therefore obtain using the dist, cent, back, and conv. Although we used different colors to mark middles, and non middle cells, they can all be marked in the same way is our static case. We therefore use 7 states: i.e. one seed state, and for non-seed cells, there is a binary mark and a modulo 3 value, so 2 * 3 = 6 other states. The binary mark is set to "true" either by the middle detection, the back detection or the conv detection. This is a first possible summary of our metric solution. But understanding why this is indeed a solution in general is the same as understanding how the middles should be detected, which is the subject of the next section. A more detailed summary is given in Sect. 10.6.

10.5 Detectable Middles and Metric Gabriel Graphs

10.5.1 Pairwise Construction in Euclidean Space

In order to understand the detection of the middles, let us look at the pairwise construction described earlier in the context of Euclidean geometry. Figure 10.13 shows very precisely what the result would be in this case, and has to be compared with Fig. 10.12. The distance field would be continuous, and concentric circle would grow similarly as seen in the cellular automata case. When the circles collide, some middles showed in light green are detected, and the corresponding paths from these middles back to the seeds can also be detected. The analogy allows us to use existing tools build from Euclidean spaces to re-expressed the behavior of this construction: we can use now the vocabulary of Voronoï regions and diagrams and two of its associated proximity graphs: Delaunay and Gabriel graphs.

10.5.1.1 Voronoï Diagram and Distance Fields

The Voronoï diagram is a famous construction [1, 3]. It consists of associating to each point of the space its closest seeds. After doing so, each seed has an associated set of points, and this is called the **Voronoï region** of this seed. Some points have many seeds at equal distance. If we highlight these points, the result is called the **Voronoï diagram** and displays the borders of the Voronoï regions, the points where the Voronoï regions are in contact.

Using the distance field implicitly means to have some relation with the Voronoï regions and diagrams since the distance field associates to each cell of the space the distance to the closest seeds. Cells that have more than one closest seeds are those having special patterns of distance value in their neighborhood. This relation is shown in Fig. 10.14a, where the Voronoï regions boundaries, a.k.a. the Voronoï diagram, is displayed in blue. Comparing this with the final configuration of Fig. 10.13 shows the relation with the circles of distance field.

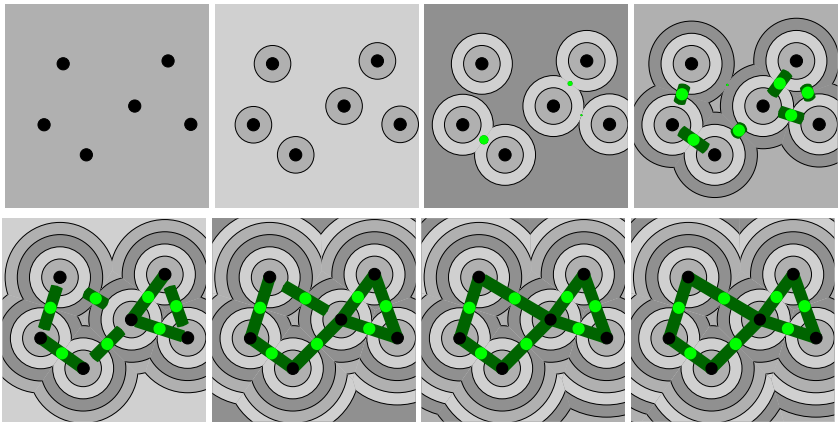


Fig. 10.13 Construction of pairwise convex hull in the Euclidean context

10.5.1.2 Delaunay Graphs as a Tentative

When using Voronoï diagrams, an associated construction is to consider adjacent Voronoï regions. It is formalized in the notion of **Delaunay graph** which has for vertices the seeds, and for edges the pairs for seeds whose Voronoï regions are adjacent. For two regions two be adjacent, there must be at least one point having for closest seeds the seeds of these two regions. This means that the edges of the Delaunay graph can also be described as the pairs of seeds such that there exists a disk containing these two seeds, but having no other seed inside it. This other description has the benefits not to refer explicitly to the Voronoï diagram. Whatever description we use, Delaunay graph edges are strongly related to the pairs of seeds for which we detect the middles in Fig. 10.14b. But if we compare with Fig. 10.13, we can see that there are too many edges.

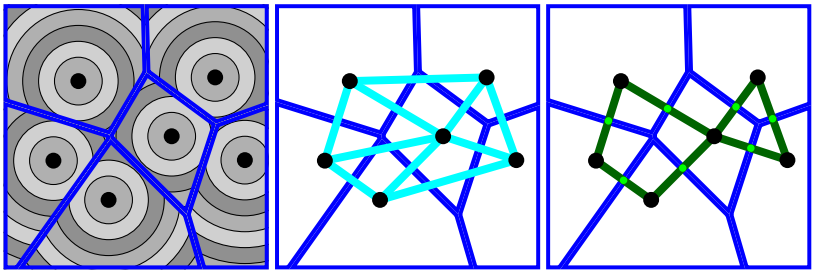


Fig. 10.14 Euclidean distance fields, Voronoï diagram, Delaunay and Gabriel graphs

10.5.1.3 Gabriel Graphs and Middle Detection

In fact, since we detect the edges from their middles and theses middles are detected from special distances patterns on the Voronoï diagram. This means that any edge whose middle is not on the Voronoï diagram is not constructed. Removing those non-detectable edges from Fig. 10.14b reduces it to Fig. 10.14c, and it comes out that this resulting graph is also of great interest in applications relying on graphs in Euclidean spaces: it is the **Gabriel graph** of our seeds. While edges of the Delaunay graphs connect seeds of adjacent Voronoï region, the Gabriel graph retains only edges that do not pass through the Voronoï region of another seed. This can be put in another way which is the most known definition of this construction: there is an edge between two seeds x and y if and only if there is a disc using the segment $[xy]$ as diameter and this disc does not contain any other seed. Given two seeds, such a disc is called the **Gabriel disc** of these seeds and its center is called the **Gabriel center** between these seeds. These Gabriel centers are exactly the middles that are detected in Fig. 10.13, which makes Gabriel graphs very important for our purpose.

Gabriel graphs are proximity graphs introduced in [6] to study sets of geographical points. They are now used in many domains such as wireless [8] and sensors networks for routing and communication management purpose. They also serve as tools to study proximity of points in order to cluster them, in domains like data mining, data and multivariate analysis [2], and machine learning [4, 12]. In computer graphics [9, 13], they can help to convert a set of points into a 3D surfaces and to obtain information about such surfaces. Their wide spread use is due to their numerous properties. Indeed, they are related to Voronoï diagrams, Delaunay graphs as just seen, but also to planar graphs, minimum spanning trees, nearest neighbor graphs, and represent or contain optimal solutions for some classes of energy-minimizing problem [8].

One particularly important property of Gabriel graphs is that *they are always connected*. This is exactly the property what we need to ensure that the construction of the convex hull is always complete, but for the cellular case. So our goal in the next section is to see how this Euclidean construction fits to cellular spaces. In fact, some adaptation will be required. So we will describe a new generalization of Gabriel graphs that we call *metric Gabriel graphs*. The latter carries properties of Gabriel graphs in any metric space, which allows them to be used in most considered cellular spaces in particular. This will provide us with the rule for the detection of middle that we need in general for many cellular spaces.

10.5.2 (Metric) Gabriel Graphs in Cellular Spaces

10.5.2.1 Failure of the Original Definition

In this section, we show that the original Gabriel graph definition does not accommodate the particularities of the cellular space. We highlight the fact that the original definition relies on many uniqueness properties of the Euclidean space, the uniqueness of the segment linking two points for example. Cellular spaces do not

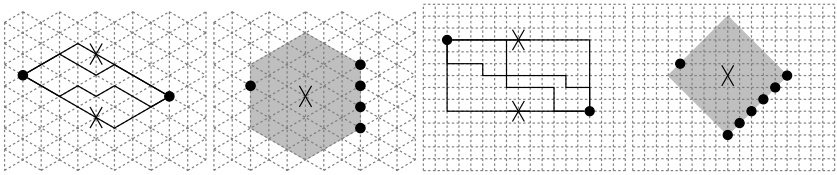


Fig. 10.15 Non-uniquenesses in hexagonal and Von Neumann communication graphs: (a,c) lines indicate possible shortest paths, and crosses indicate many possible centers for the balls, (b,d) the isolated point forms a diameter for the ball with any of the other points

have these uniqueness properties, so a generalization of the original definition is needed.

Indeed, the connectedness of the original definition mainly relies, by definition, on the existence of sufficiently many Gabriel discs and associated centers. In an Euclidean space, for a given arbitrary disc and a given point on the border of this disc, there is a unique diameter segment. Also, for any pair of points, there is a single disc using these points as diameter. Unfortunately, considering cellular spaces, and replacing the notion of disc by the metric notion of balls does not conserve these uniqueness properties as shown in Fig. 10.15. A **ball** is given by a center point and a radius and corresponds to the set of points of the space whose distance to the center point is less or equal to the radius. Looking at (b) and (d), we can see that for a given ball of arbitrary radius r, and one selected cell of its left border, there are many cells on the right forming a diameter for the ball: any of them is at distance $2r$ from the left cell and, as a consequence, has a shortest path joining it with the left cell and passing through the center of the ball. In the same vein, a pair of cells is not a diameter for a unique ball. Indeed, the uniqueness of this ball in Euclidean space is implied by the uniqueness of the shortest path linking two points. When many shortest paths exist between two points, they may have different middles, which leads to different balls, all having the same radius, but each being centered at a different middle as shown in (a) and (c).

Because of these differences between the Euclidean and cellular case, the original Gabriel graph definition in terms of discs does not correspond to a connected graph in the cellular case. Figure 10.16 gives an example of loss of connectedness due to the fact that there might be no ball having only two seeds. In (a), every ball containing two seeds contains an additional third seed, preventing the two points to have a Gabriel edge. In (b), every ball containing two seeds of different lines contains four additional seeds, preventing the lines to be belong to the same connected Gabriel component, and so on.

10.5.2.2 Metric Gabriel Graphs

We therefore need to modify the definition to take into account these particularities in a meaningful way. In [11], an examination of the nature of Gabriel graphs and its transformation for arbitrary metric spaces is given and the following definition

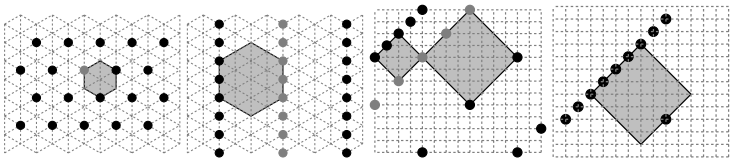


Fig. 10.16 Hexagonal and square grid: the Gabriel graph is not connected. Grey seeds show one of the connected components and gray balls give the reason of the non-connectedness.

of **metric Gabriel graph** is obtained: its vertices are the seeds as before, and two seeds x and y have an edge connecting them if and only if there is a ball such that the seeds contained in this ball can be partitioned in two sets X and Y with $x \in X$, $y \in Y$ and $d(X,Y) = 2r$, r being the radius of the ball[2]. As before, such a ball if called a **metric Gabriel ball** and its center a **metric Gabriel center**.

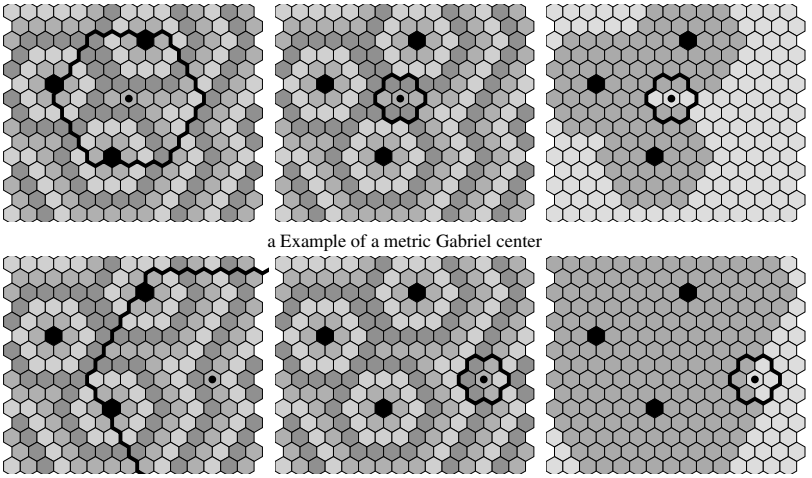
Compared to the original definition of Gabriel graphs, this definition deals with the non-uniqueness of diameters by replacing the requirement of having two seeds x and y such that $d(x,y) = 2r$ (this means precisely that $[xy]$ is a diameter) by the requirement of having two sets of seeds X and Y such that $d(X,Y) = 2r$. The non-uniqueness of balls for a given diameter is managed by requiring only the existence of at least one metric Gabriel ball. This means that whenever diameters and balls are unique, this definition is equivalent with the original one, as it is the case for Euclidean spaces. In fact, metric Gabriel graphs correspond exactly to original Gabriel graphs when the distance function is the Euclidean one. Moreover, *metric Gabriel graphs are always connected* for any set of points in any arbitrary metric space.

So this corresponds exactly to what we want to construct with our detection of middles. More precisely, metric Gabriel centers are precisely the detectable middles. To detect them, we use a very useful relation between metric Gabriel graphs and distance fields: in the same way a ball of radius r is a metric Gabriel ball when the set of seeds that it contains can be separated in two sets of distance $2r$, a distance field neighborhood of radius r corresponds to that of a detectable middle when its set of minimally valued cells can be separated in two sets of distance $2r$. This does not hold in arbitrary metric spaces but is true in cellular spaces.

Figures 10.17a and 10.17b give an example of this correspondence in the case of an hexagonal cellular space with three seeds.

In Fig. 10.17a, the first configuration shows a cell, indicated with a little dot, that is the center of a metric Gabriel ball. Indeed, the two seeds that it contains forms a diameter, which can be checked by exhibiting a shortest path joining them and passing through the center, or simply by checking that they have distance 8, while the ball is of radius 4. When considering only the neighborhood of this cell, in the second configuration, the only available information is the distance values computed

[2] Another way to put it is that there is an edge between two seeds x and y if and only if there is a ball such that x and y are at distance $2r$ and any other seed inside the ball is at distance $2r$ from x or from y.



a Example of a metric Gabriel center

b Example of a cell which is not a metric Gabriel center

Fig. 10.17 Relation between distance fields, metric Gabriel balls, and union of balls

from the seeds, not the actual seeds position. Taking the minimally-valued cells in this neighborhood of radius 1, we can see that they can be partitioned in two sets of distance 2, which means that this partition forms a diameter for the neighborhood. In fact, this neighborhood is a metric Gabriel ball for a *dilation* of the union balls of of radius 3 centered at each of the three seeds, as shown in the third configuration.

In Fig. 10.17b, we can see the converse. The first configuration shows a cell, indicated with a little dot, that is *not* a metric Gabriel center. Indeed, the two seeds do not form a diameter for this ball. When considering only the neighborhood of this cell, in the second configuration, there is no way to partition the three minimally-valued cells of the neighborhood in two sets of distance 2.

To summarize, the distance field values in the neighborhood of a cell allows it to detect whether it is a metric Gabriel center, and using this detection for the detection of middle required in Sect. 10.4.2.1 and Sect. 10.4.2.2, we know that sufficiently many pairs of seeds will be connected to obtained only one connected component and construct the complete convex hull.

10.6 The Complete Cellular Automaton

We are now in position to write the complete cellular automaton constructing the convex hull for arbitrary set of seeds, and for any cellular spaces[3]. It is simply a

[3] For the sake of brevity, some parts are not sufficiently general to be used without modification in all cellular spaces, but we believe that the main "understanding" is in this restriction version. We hope that any reader who needs the extra missing piece of generality will be able to guess it.

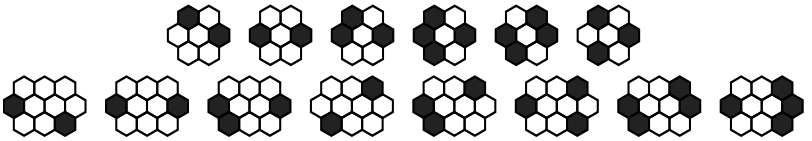


Fig. 10.18 Local configurations of middle cells and middle edges in hexagonal grids

compound of all the building blocks we have introduced throughout the chapter, but we now show explicitly how these building blocks needs to be linked:

$$\mathrm{dist}_{t+1}(x) = \begin{cases} 0 & \text{if } x \in P; \\ \mathrm{dist}_t(x) + 1 \bmod 3 & \text{if } x \notin P \wedge \forall y \in N(x);\ \mathrm{dist}_t(y) = \mathrm{dist}_t(x); \\ \mathrm{dist}_t(x) & \text{if } x \notin P \wedge \exists y \in N(x);\ \mathrm{dist}_t(y) \neq \mathrm{dist}_t(x); \end{cases}$$

$$\mathrm{cent}_{t+1}(x) = \bigvee \begin{cases} \{z \in N(x) \mid \mathrm{dist}_t(z) = \mathrm{dist}_{t+1}(x) - 1\} \text{ diam. } 2\ ; \\ \exists y \in N(x); \{z \in N(xy) \mid \mathrm{dist}_t(z) = \mathrm{dist}_t(xy) - 1.5\} \text{ diam. } 3\ ; \end{cases}$$

$$\mathrm{back}_{t+1}(x) = \mathrm{cent}_{t+1}(x) \vee \exists y \in N(x);\ \mathrm{dist}_t(y) = \mathrm{dist}_t(x) + 1 \bmod 3 \wedge \mathrm{back}_t(y);$$

$$\mathrm{conv}_{t+1}(x) = \mathrm{back}_{t+1}(x) \vee \exists y_0, y_1 \in N(x);\ \mathrm{conv}_t(y_0) \wedge \mathrm{conv}_t(y_1) \wedge x \in [y_0, y_1].$$

Here the metric Gabriel centers detection is written using two cases: one for diameters of even length, and the other for the diameters of odd length. In the first case, it is sufficient to look at the minimally-valued neighbors (those having value $\mathrm{dist}_{t+1}(x) - 1$) and to check if they can be separated in two sets of distance 2, i.e. the diameter of the neighborhood. For the second case, the center is between two cells x and y, and we need to look at the "neighborhood of the edge (x,y)" of radius 1.5. So $N(xy)$ is a shorthand for $N(x) \cup N(y)$, and $\mathrm{dist}_t(xy)$ is a shorthand for $\min(\mathrm{dist}_t(x) + 0.5, \mathrm{dist}_t(y) + 0.5)$ which is the distance of the edge to the closest seeds. If the minimally-valued neighbors (those having value $\mathrm{dist}_t(xy) - 1.5$) can be separated in two sets of distance 3 (the diameter of the neighborhood), then x is the extremity of an edge-like metric Gabriel center. For concreteness, Fig. 10.18 shows the set of minimally-valued neighbors that can be separated in two sets whose distance is the diameter for the case of the hexagonal cellular space. The first line is for cell-centers, and the second line for edge-centers.

The evolution of this cellular automaton for hexagonal cellular space and without the conv field was already shown in Fig. 10.12. The reader can now recheck the detection of the middles in the light of our explanation of the relation with metric Gabriel graphs, balls and centers. When it comes to the number of states, there are two things to say. Firstly, when restricting to the content of this chapter, namely the construction of the convex hull for a statis set of seeds, it is not useful to distinguish between cells having been marked the cent, back and conv rules. These three detections can be reduced to one Boolean field summarizing the five reasons to be marked as belonging to the convex hull: being a seed, being a cell-like metric Gabriel center,

being a edge-like metric Gabriel center, being on the way back from a marked cell and its closest seed, and being between two marked cells in the neighborhood:

$$\mathrm{mark}_{t+1}(x) = \bigvee \begin{cases} x \in P; \\ \{z \in N(x) \mid \mathrm{dist}_t(z) = \mathrm{dist}_{t+1}(x) - 1\} \text{ diam. } 2\ ; \\ \exists y \in N(x); \{z \in N(xy) \mid \mathrm{dist}_t(z) = \mathrm{dist}_t(xy) - 1.5\} \text{ diam. } 3\ ; \\ \exists y \in N(x);\ \mathrm{dist}_t(y) = \mathrm{dist}_t(x) + 1 \bmod 3 \wedge \mathrm{mark}_t(y); \\ \exists y_0, y_1 \in N(x);\ \mathrm{mark}_t(y_0) \wedge \mathrm{mark}_t(y_1) \wedge x \in [y_0, y_1]. \end{cases}$$

This construction therefore requires 7 states: 3 states for the distances modulo 3, multiplied by 2 states for the mark field, plus 1 special state for the seeds that always have dist $= 0$ and mark $= \top$. Its neighborhood radius is 2 because of the edge-like metric Gabriel centers detection. Secondly, if one wants to consider moving seeds for example, it is necessary to keep the distinction between the fields cent, back and conv.

Acknowledgements. The authors would like to thank A. Adamatzky for his important involvement in the development of this work. He was the one who incited them to begin this work and also the one who told them about Gabriel graphs when they were seeking for the appropriate subgraphs of Delaunay graphs which is naturally computed by the pairwise algorithm.

References

1. Adamatzky, A.: Voronoi-like partition of lattice in cellular automata. Mathematical and Computer Modelling 23, 51–66 (1996), doi:doi:10.1016/0895-7177(96)00003-9
2. Aupetit, M., Catz, T.: High-dimensional labeled data analysis with topology representing graphs. Neurocomputing 63, 139–169 (2005)
3. Aurenhammer, F.: Voronoi diagrams - a survey of a fundamental geometric data structure. ACM Comput. Surv. 23(3), 345–405 (1991), http://doi.acm.org/10.1145/116873.116880
4. Chen, H., Wei, W.: Geodesic Gabriel graph based supervised nonlinear manifold learning. In: Huang, D.-S., Li, K., Irwin, G.W. (eds.) ICIC 2006. LNCIS, vol. 345, pp. 882–887. Springer, Heidelberg (2006)
5. Clarridge, A.G., Salomaa, K.: An improved cellular automata based algorithm for the 45-convex hull problem. Journal of Cellular Automata 5(1-2), 107–120 (2010)
6. Gabriel, R.K., Sokal, R.R.: A new statistical approach to geographic variation analysis. Systematic Zoology 18(3), 259–278 (1969)
7. Ilachinski, A.: Cellular Automata: A Discrete Universe. World Scientific Publishing Co., Inc., River Edge (2001)
8. Kanj, I.A., Perković, L., Xia, G.: Local construction of near-optimal power spanners for wireless ad hoc networks. IEEE Transactions on Mobile Computing 8(4), 460–474 (2009), http://doi.ieeecomputersociety.org/10.1109/TMC.2008.132

9. Lee, C., Kim, D.-U., Shin, H., Kim, D.-S.: Efficient computation of elliptic Gabriel graph. In: Gavrilova, M.L., Gervasi, O., Kumar, V., Tan, C.J.K., Taniar, D., Laganá, A., Mun, Y., Choo, H. (eds.) ICCSA 2006. LNCS, vol. 3980, pp. 440–448. Springer, Heidelberg (2006)
10. Maignan, L., Gruau, F.: Integer gradient for cellular automata: Principle and examples. In: Spatial Computing Workshop at IEEE SASO 2008 (2008)
11. Maignan, L., Gruau, F.: Gabriel graphs in arbitrary metric space and their cellular automaton for many grids. ACM Trans. Auton. Adapt. Syst. (2010)
12. Mukherjee, K.: Application of the Gabriel graph to instance based learning algorithms. Master's thesis, SFU CS School (2004)
13. Park, J.C., Shin, H., Choi, B.K.: Elliptic Gabriel graph for finding neighbors in a point set and its application to normal vector estimation. Computer-Aided Design 38(6), 619–626 (2006)
14. Torbey, S., Akl, S.G.: An exact and optimal local solution to the two-dimensional convex hull of arbitrary points problem. J. Cellular Automata 4(2), 137–146 (2009)

Chapter 11
Cellular Automaton Shading for Building Envelopes

Machi Zawidzki

Abstract. This chapter collects the key findings of the pioneering concept of a CA-based shading system for building envelopes, CASS for short. CASS can be considered as a realistic approach to realization of the architect's dream of an adaptive skin of a building which could dynamically change its appearance according to varying external conditions or the inhabitants' will. Since CASS is based on identical modular units it has potential of being relatively robust and inexpensive. Most importantly it takes visual advantage of CA's emergent properties which result in new, intriguing, organic aesthetics. The practical approach focused on tangible and "buildable" results make this material particularly suitable for designers and building engineers — groups that usually are not well addressed in CA research. Additionally, this paper presents an alternative approach to CA which may contribute to the mainstream research in the field.

11.1 Building Envelope and Daylighting

Architectural building envelope (BE) is an example of a real-life engineering design problem, which can be formulated as multi-objective optimization. BE is to serve a number of functions and meet requirements which sometimes are contradictory; the hierarchy of their importance is often ambiguous and strongly user-dependent. BE is an interface between the exterior and interior of a building and usually serves the four main functions:

- protection from external factors, as it improves security and reduces the levels of noise pollution;
- protection from climatic changes (temperature, humidity, glare);
- provision of natural light and visual contact with the environment, or visual isolation from exterior if required;
- aesthetics, as the facade is one of the most articulated visual aspect of a building.

Machi Zawidzki
Massachusetts Institute of Technology, USA
e-mail: zawidzki@mit.edu

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_11, © Springer International Publishing Switzerland 2014

The requirements for the indoor conditions vary among the occupants, however in relatively narrow ranges. For a recent survey of literature on how different factors influence human comfort indoors see [20]. Conversely, the exterior conditions vary substantially both in circadian and annual cycles. Therefore, modern BEs should adapt both to the occupants' requirements and to the variable outdoor conditions. Moreover, in principle, relying exclusively on artificial light indoors is not desirable and in many cases not allowable by building regulations. At humanistic level, according to [50]: *natural light in architecture must be part of a more general philosophy that reflects a more respectful, sensitive attitude in human beings towards the environment in which they live*. The main benefits of daylighting as a design strategy:

- Economical and ecological, as it substantially reduces the energy consumption and greenhouse gases emissions [2] [8];
- Physiological, as it is an effective stimulant to the human visual and circadian systems [10];
- Well-being, as it provides high illuminance and permits excellent color discrimination and color rendering; enables occupants to see both a task and the space well, and to experience some environmental stimulation [9]. Working by daylight is believed to result in less stress and discomfort.
- Social, as those of higher status in organizations are often given spaces closer to/with more windows [9].

Daylight offers a dynamic, changing pattern to stimulate the eye. It also provides a very wide range of illuminance: from 0 to over 25 000 lx, way beyond values commonly required indoors which range from 10 to 1000 lx, that is from the lowest level of color discrimination to the bright appearance [13]. Moreover, its thermal effect and its unique distribution of luminances are desirable in winter and in cold climates and undesirable in summer in hot climates [50]. Moreover, personal preference of illuminance levels and the degree of glare discomfort vary, and desired quantities of additional electric light depend on the type of task and the distance from the window. There are a number of systems for controlling these variances to appropriate levels. For a review see [48]. For a survey of systems that harvest the daylight, usually from the roof, and redistribute it inside the building see [32]. For other related issues in the context of CASS such as daylight design in urban areas, the importance of the outside view, smart windows, etc. see [65]. The complexity of functions performed by this type of a spatial barrier resembles organic skin. A comparison between skin and a building envelope is shown in Tab. 11.1 after [63]. As Tab. 11.1 indicates BE is a multifaceted problem and most importantly, no "conventional" solution that is optimal and universal exits. An alternative idea based on cellular automaton (CA), so called CA-shading system, CASS for short, has been proposed in [66]. In the original CASS the facade is divided into identical units which form a lattice for one-dimensional (1D) CA, and only the edge rows of cells are controlled directly. The pattern on the entire array is the result of local interactions determined

Table 11.1 A comparison between the functions of skin and building envelope

Function	Skin	Building envelope
Protection	S. is an anatomical barrier from pathogens and damage between the internal and external environment in bodily defense [43] [36].	Outer shell to protect the indoor environment.
Sensation	S. contains a variety of nerve endings that react to heat and cold, touch, pressure, vibration, and tissue injury.	Users' sensory contact with the outside (mostly visual, also audial).
Heat regulation	S. contains a blood supply far greater than its requirements which allows precise control of energy loss by radiation, convection and conduction. Dilated blood vessels increase perfusion and heat loss, while constricted vessels greatly reduce cutaneous blood flow and conserve heat.	Temperature control (insulation, solar gain, heat transfer, thermal mass, etc.)
Evaporation control	S. provides a relatively dry and semi-impermeable barrier to fluid loss [36].	Moisture control (e.g. air conditioning)
Storage and synthesis	S. acts as a storage center for lipids and water.	Heat storage (e.g. thermal mass, Trombe wall [49])
Absorption	Oxygen, nitrogen and carbon dioxide can diffuse into the epidermis in small amounts, some animals use s. for their sole respiration organ [12].	Ventilation (indoor air quality, hygiene and public health).
Water resistance	S. acts as a water resistant barrier so that essential nutrients are not washed out of the body.	Water barrier (water condensation).
Pigmentation	Camouflage, mimicry, UV protection, communication, sexual reproduction, warning etc.	Appearance (aesthetics, communication).
Structure	Other animal coverings such the arthropod exoskeleton or the seashell have different developmental origin, structure and chemical composition.	Structural integrity (shell).

by the update rules, the same for each cell. This system is therefore homogeneous like many biological systems [29], and emerging patterns exhibit certain "organic" integrity as shown in Fig. 11.1.

11.2 Why Cellular Automata to Drive Shading of Building Envelopes?

CA combine certain properties which are particularly suitable for application on building facades, as shown in Tab. 11.2. The first property listed there — *modularity* is mostly relevant to the fabrication of CASS described in Subsect. 11.6. The CA modules, although having uniform topology may have various shapes as in the case

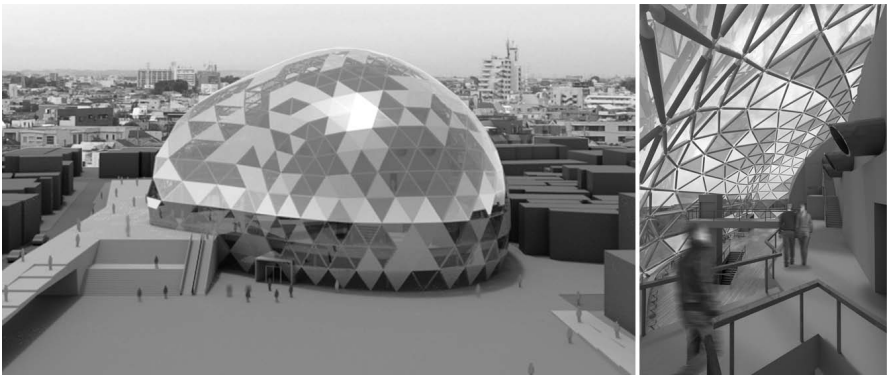


Fig. 11.1 Simulation of CA on a free-form BE comprised of triangular panels of Hita City Auditorium Competition Entry (2003, Oita, Japan, architectural design by Takehiko Nagakura). The 30^{th} time-step of two-color two-dimensional triangular CA rule 9622 is shown (compare with Fig. 11.19).

of CASS on free-form surfaces described in Subsect. 11.4.2. Combination of the last two properties from Tab. 11.2, that is *determinism* with *emergence* is the most challenging. Both derive from the sequence of CA patterns displayed on CASS. These patterns and their transitions must meet the following criteria:

1. Patterns to be visually interesting;
2. Ability of producing patterns of any average density between 0: full transparency, to 1: full opacity;
3. Density transitions to be gradual;
4. Patterns to be evenly distributed.

From these criteria a straightforward constraint can be immediately derived. Namely, only even-number (EN) CA rules should be considered. The shading action starts arbitrarily from full transparency so all cells have value 0. An odd-number rule produces non-zero state from all 0s in the background, so in the first time-step the state of practically entire CASS changes to 1, which precludes the third criterion listed above. In the case of CASS, the visual attractiveness can be attributed to the emergence, defined after [33] as: *a property of a complex system that is not exhibited by*

Table 11.2 Comparison of the key properties of CA and BE

Cellular automaton	**Building envelope**
In principle, all the cells of a particular CA's are identical.	Modularity is highly desirable in building industry, as it allows for mass prefabrication and easy assembly.
Determinism	The control of the state of BE, at least statistical, is crucial.
Capability of strong emergence	Potential of the new, intriguing, organic-like aesthetics

its individual component parts determined from a model of the system. The emergence in CA is manifested by the simultaneous emergence of boundaries between the domains, which take form of boundaries or "particles", that is small regions of cells separating two domains and persisting for a relatively many time steps [24]. Particular type of particle emerges in the so-called "solitons" — moving persistent structures which pass through one another while preserving their identities [52]. The subsequent sections describe various CA which meet the criteria listed above. The review starts from the simple CA, preceded by a brief explanation of terms specific to CASS. In all the instances, the CA update is synchronous.

11.2.1 The Nomenclature

A few practical terms often used in the context of CASS are briefly explained below. For simplicity they were formulated and illustrated with rectangular, two-color (2C) CA.

The Sequence of Initial Conditions (SIC)
In principle, CA pattern is determined by local transition rules (TRs), initial condition (IC) and the boundary conditions (BC). As a result, so is the average pattern density, which is the ratio between the number of black cells (1s) to the number of all cells in the array. An array with all black cells forms a completely opaque facade (density 1), conversely, an array of all white cells (density 0) corresponds to 100% transparency. For an interactive demonstration see [59]. In CASS the pattern remains still until the next alteration in IC. Such changes to IC form a sequence(SIC), which must meet two arbitrary conditions: 1) for $k > 1$, the k^{th} IC has exactly one black cell more than $(k - 1)^{th}$; 2) all the remaining black cells to be preserved. A single IC is a list of {0,1} of length n. SIC is also a n-long list of such lists, forming an $n \times n$ square matrix. Such SIC ensures that the displayed changes of the array will not appear excessively chaotic or disturbing. An alteration of the state of a single cell in IC propagates over the entire array, so the transition from one state of BE to another can be traced, understood and interpreted by the observer. SICs can be conveniently encoded into SIC* using order-based representation(OBR) [23]. A sequence of n ICs is represented by a list of integers of length $n - 1$. Each integer is the "distance to the most recently added 1 among the available positions". Thus SIC* is a "sequence of the leap lengths where consecutive 1s are located". The positioning is done from left to right. Since in the very last step there is only one available position, is it dropped, as illustrated in Fig. 11.2. For detailed explanation of this encoding method see [67]. For the interpretation of the SIC shown in Fig. 11.2 see Fig. 11.3.

Pattern grayness, in other words pattern density, is defined as a ratio between the number of black (opaque) cells to the total number of cells. The grayness applies to

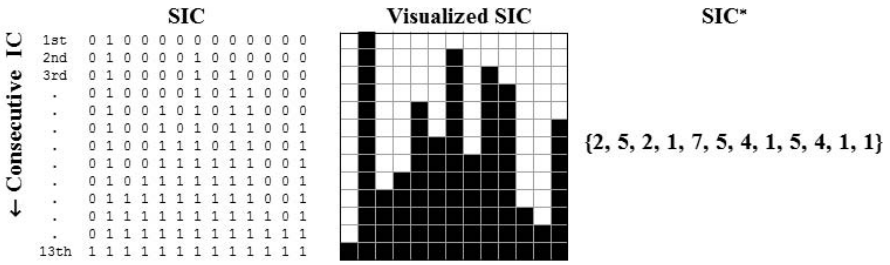


Fig. 11.2 An example of a sequence of 13 initial conditions (SIC) with the corresponding encoded SIC*

any sub-matrix, row of cells or group of cells. A pattern with all white (transparent) cells has grayness 0, with all black (opaque) cells the grayness is 1.

Grayness Function (GF) links the grayness of IC with the grayness of the entire CA array. In other words, GF is a series of graynesses for consecutive ICs. For shading, GF should be monotonic, ideally the grayness of the entire array should be proportional to the grayness of IC. It should also render values from the entire range, that is from 0 to 1. Such GF ensures that the average opacity of the array can be fully controlled. For an interactive demonstration see [41].

Grayness Function Error(GFE) is the difference between the ideal (proportional) GF and the GF of a particular CA at given SIC. For a single CA array $\boldsymbol{A}$ at the given IC it is expressed as:

$$\mathrm{GFE}[\boldsymbol{A}] = \frac{\sum_{w=1}^{W}\sum_{h=1}^{H} \mathrm{a}_{h,w}}{WH} - \frac{\sum_{w=1}^{W} \mathrm{a}_{1,w}}{W} \tag{11.1}$$

Where H and W are the height and width of an array (facade) respectively. GFE for SIC is the result of the summation for all ICs:

$$\mathrm{GFE} = \frac{1}{F}\sum_{i=1}^{F} |\mathrm{GFE}[\boldsymbol{A}_i]| \tag{11.2}$$

Where F is the number of facades' states which equal to the length of SIC, $\boldsymbol{A}_i$ is the i^{th} CA array (for the i^{th} IC of SIC). GFE is usually used for preliminary search for automata suitable for CASS.

Grayness Monotonicity and Pattern Distribution Error(GDE). As discussed in [67], finding the best SIC is a computationally challenging problem. GDE is used for fine-tuning the SIC as it evaluates each CA pattern considering both the grayness function monotonicity and the uniformity of the pattern distribution:

$$\mathrm{GDE}[\boldsymbol{A}, r] = \sum_{k=1}^{\lfloor \frac{W}{r} \rfloor - 1} \frac{\left| \frac{1}{Hr}\sum_{x=kr}^{(k+1)r}\sum_{h=1}^{H} a_{h,w} - \frac{1}{W}\sum_{w=1}^{W} a_{1,w} \right|}{\lfloor \frac{W}{r} \rfloor} \tag{11.3}$$

Where A is a CA array (facade), H and W are the height and width of the array, respectively; r and k are the width and the number of vertical stripes in which the entire array is subdivided, respectively. The summation over all CA patterns produced at each IC from the given SIC:

$$\mathrm{GDE} = \frac{1}{F} \sum_{f=1}^{F} \mathrm{CFA}[\boldsymbol{A}_f] \tag{11.4}$$

Where F is the number of facades' states which equals to the length of SIC, $\boldsymbol{A}_f$ is an f^{th} CA array corresponding to the f^{th} IC in SIC. If each of the r-wide sub-arrays of a given CA pattern have the same average densities, the values of GDE and GFE are equal. For further details see [69]. CA for shading must be both controllable, that is having sufficiently low GDE, and visually appealing. Although the latter criterion is rather arbitrary, in most of the cases, the necessary condition is that the CA pattern is complex, that is of Wolfram class IV behavior [66].

11.3 One-Dimensional Cellular Automata Applied on Surfaces

Although BEs are two-dimensional (2D), the application of one-dimensional (1D) CA for CASS seems the most practical. This is analogous to the common convention for presenting a 1D CA by showing the history of its generation changes, where each row corresponds to a time step in the history. Every row becomes IC for the next row and so forth. This process continues in a cascade and propagates over the entire array. Use of a 2D CA may seem more intuitive because this domain is greater than for one dimensional automata, increasing the chances of finding the best one for CASS. Moreover, the inter-cell wiring seems to be easier to fabricate. Nevertheless, as mentioned in [66], there are major concerns regarding the application of 2D CA. Although their behavior is often intriguing, it is very difficult to control the states of cells. 2D CA continuously updates all the cells until an equilibrium is reached, which almost always leads to an uninteresting, uniform state of the array. All the cells become black or white, often with artifacts (small islands of the opposite state) and usually with locally strobing cells (switching their state at every step forever). In general the exact final state of the 2D CA array is difficult or impossible to predict due to the computational irreducibility. Such 2D array are usually not useful for shading due to strobing. Controlling the state of 2D arrays is difficult, or perhaps impossible. This problem could be solved by freezing the array at a certain step and not allowing it to evolve further, but at present it seems to be a difficult technical problem. With the adopted common convention of displaying 1D CA, this problem does not occur because every row displays the state at a certain step, and once set the state is maintained. The second major problem with 2D CA is the setting of ICs. How is the initial input given to the cells of a 2D array? A possible solution where only cells on the edges of the array are used, as done in the 1D case, has been demonstrated in [63].

11.3.1 Elementary Cellular Automata

There are 256 1D 2C, radius 1 (r1), so called elementary cellular automata (ECA). However, due to symmetries, the number of fundamentally inequivalent ECAs is only 88 [29]. For an interactive demonstration of CASS based on ECAs see [62].

11.3.2 The Original CA for Shading

Since none of ECAs meet both "shading" criteria, for the original CASS 2C1Dr2 CA code {3818817080,2,2} has been introduced in [66]. The first, second and third values in the latter code represent the decimal enumeration of the TR outputs, the number of possible states (colors) and the neighborhood size, respectively. Figure 11.3 shows the best possible, that is the ideal SIC for a 13×13 array at fixed boundary conditions with this CA.

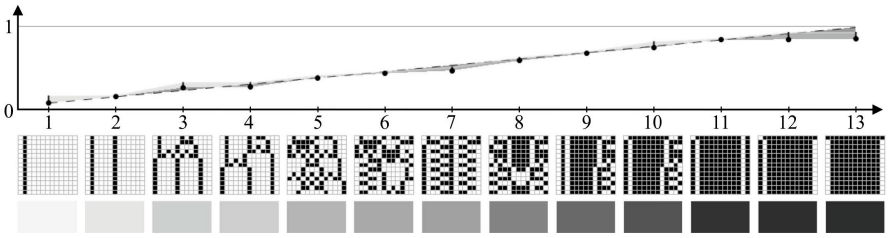


Fig. 11.3 13×13 array {3818817080,2,2} CA at SIC*: {2 5 2 1 7 5 4 1 5 4 1 1} renders the opacity transition with the lowest GDE = 0.58 and GFE = 0.42. From the top: 1): Black dots indicate the value of grayness (the average density) of each CA pattern. Dashed line indicates the referential proportionality. Dark gray filling indicates GFE at particular IC: excessive and insufficient densities are shown over and under the reference line, respectively. Similarly, light gray filling indicates GDE at particular IC. 2): The sequence of patterns. 3): The sequence of gray levels equivalent to the graynesses of the CA patterns shown above.

11.3.3 Half-Distance Automata

So called half-distance rules can be created by shifting the successive rows, so the number of input cells becomes even. If an underlying cell is placed between the two cells above, it is called radius $\frac{1}{2}$ (r-$\frac{1}{2}$) CA. For radii $\frac{3}{2}$ (r-$\frac{3}{2}$) and $\frac{5}{2}$ (r-$\frac{5}{2}$), the underlying cell is placed between the corresponding 4 and 6 cells, respectively. For an interactive demonstration see [40]. There are only 16 r-$\frac{1}{2}$ 1D2C such automata, none of them particularly interesting visually. However, the number of corresponding r-$\frac{3}{2}$ automata is 65,536. Half of them, that is 32,768, being EN which is rather manageable for simple analysis. As Fig. 11.4 indicates many of them have low GFE. As Fig. 11.4 indicates, there are many, namely 720, r-$\frac{3}{2}$ CA with very low GFE, that is below 0.05. The six automata whose GF plots are shown on the right have been selected according to the following criteria:

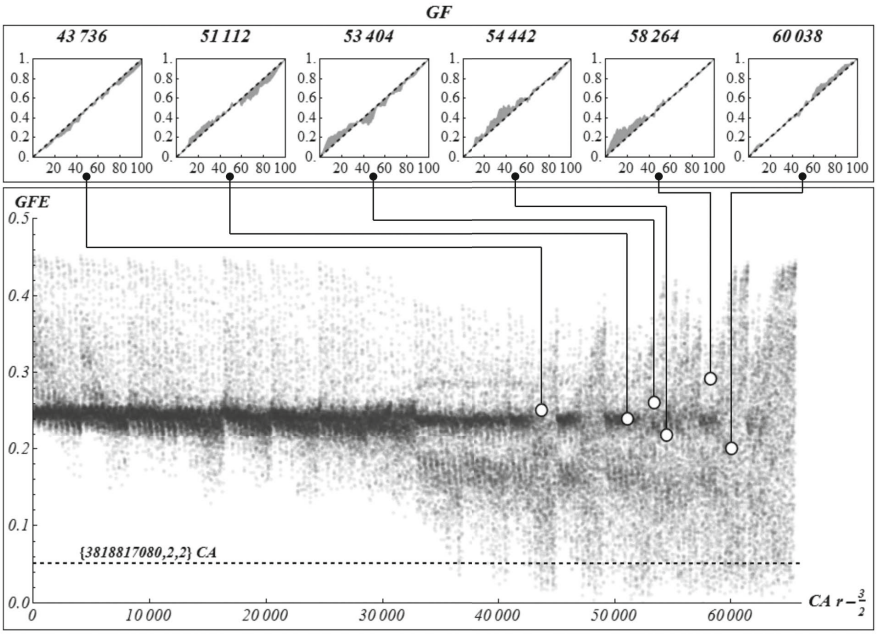


Fig. 11.4 r-$\frac{3}{2}$ CA: On the bottom: the GFE plot for all automata. Some of them with very low GFE (below 0.1). The referential value for {3818817080,2,2} CA is shown with the dotted line. On the top: the GF plots of six selected automata. The referential proportionality and the differences to it are shown with the dotted line and gray filling, respectively.

1. CA Patterns to be vertical, or at least no direction (left or right) should be dominant;
2. Pattens to continue for many steps;
3. Patterns to be distinct, preferably organic; for discussion on "man-made versus organic appearance" see [63].

Evenness of the pattern distribution was not considered. Figure 11.5 shows sample patterns of these automata. Half-distance automata are particularly interesting for CASS, since they can be applied on hexagonal and triangular tessellations, as described in Subsect. 11.3.5.

11.3.4 Higher Order Cellular Automata

A *k*-order cellular automaton is a type of reversible CA where the state of a cell at time t depends not only on its neighborhood at time t - 1, but on its states at $\{t-1,\ldots,t-k\}$. These automata are particularity interesting due to their intriguing properties [3] and relatively simple implementation for CASS. The number of second order 1D2Cr1 CA, or 2-CA is rather unmanageable for a simple search (total of $2^{2^6} = 1.84 \times 10^{19}$, half of them being EN), however, although there are only

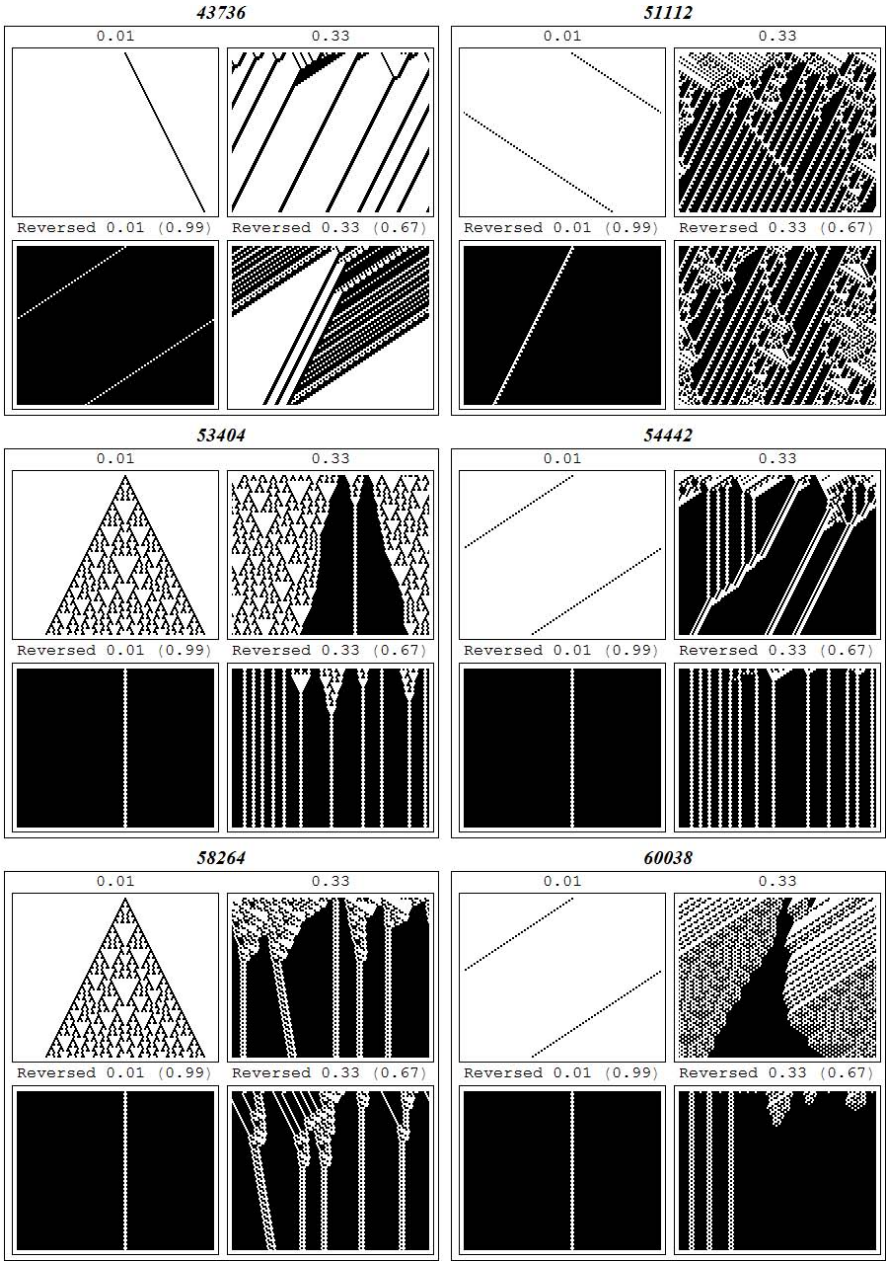


Fig. 11.5 The patterns of r-$\frac{3}{2}$ CA rules: 43736, 51112, 53404, 54442, 58264 and 60038. Patterns at the same ICs are shown for each rule: on the left: a single 1; below: the corresponding reversed IC with a single 0; on the right: $\frac{1}{3}$ rate of 1s and below: the corresponding reversed IC with $\frac{1}{3}$ 0s.

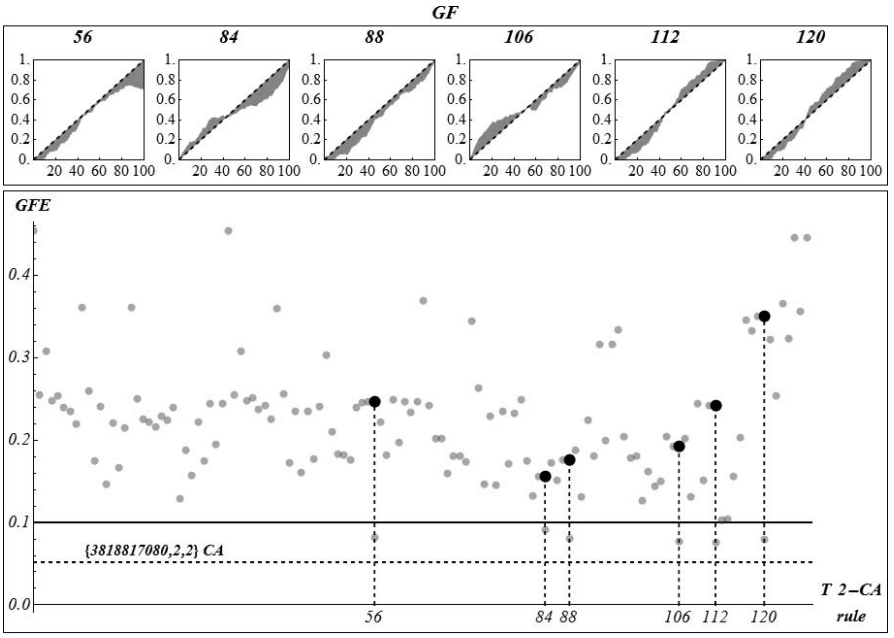


Fig. 11.6 T 2-CA: On the bottom: there are six T 2-CA with relatively low GFE (below 0.1). The referential value for {3818817080,2,2} CA is shown with the dotted line. On the top: the corresponding GF plots. The referential proportionality and the differences to it are shown with the dotted line and gray filling, respectively.

128 totalistic automata of this kind (T 2-CA), of which 64 are EN T 2-CA, some of them have rather good shading properties, as shown in Fig. 11.6. Figure 11.7 shows sample patterns of the selected T 2-CA. Rules 112 and 120 are not shown as they produce overly simple patterns.

11.3.5 Other Regular Tessellations: Hexagonal and Triangular

The square grid is one of three, regular, also called "Platonic" tilings. The remaining two are: triangular and hexagonal. Regular tilings allow edge-to-edge tiling by congruent regular polygons [11]. This property has been widely used in architectural practice since antiquity, and the first systematic mathematical treatment was done in the early XVII century in [30]. The properties of regular tessellations in the context of architectural design and particularly for BE shading are collected in Tab. 11.3. Although the original concept of CASS was based on an opto-mechanical system of square plates made of polarized glass [66], the prototype was based on liquid crystal (LC) technology [68]. The concept of polarized film shading system (PFSS), where shading elements are composed of two polygonal sheets of normally-white polarized films is explored in [65]. One of the parts of PFSS is fixed and the other rotates.

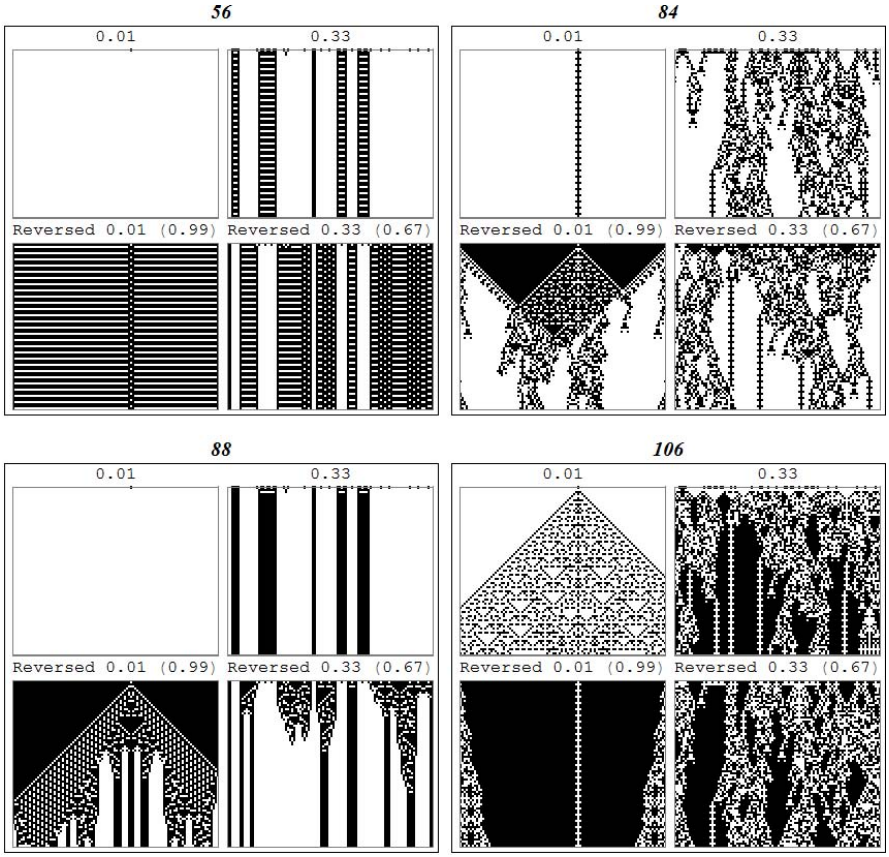


Fig. 11.7 The patterns of T 2-CA rules: 56, 84, 88 and 106. Patterns at the same (coupled) ICs are shown for each rule: a single 1; below: the corresponding reversed IC with a single 0; $\frac{1}{3}$ rate of 1s and the corresponding reversed IC with $\frac{1}{3}$ 0s.

Opacity of the idealized PFSS units is proportional to the angle of rotation: at the parallel position, that is 0°, the opacity is 0 and at the crossed position, that is 90°, it is 1. The minimal and maximal opacities occur at $k \times 180°$ and $k \times 180° + 90°$, respectively; where $k \in \mathbb{N}$. Since the states of a CA are discrete, it is natural to limit the rotation angles of polygonal elements to discrete, so called dihedral rotations (DR). The numbers of DRs for triangle, square, and hexagon are 3, 4 and 6, respectively, as shown in Fig. 11.8. As Fig. 11.8 indicates, PFSS based on triangular, square, and hexagonal tessellations have at maximum 3, 2 and 3 distinct opacities, respectively. However, only the square PFSS renders extreme values from 0 to 1. The triangular and hexagonal PFSS render three shades of gray: 0.16, 0.5 and 0.83. Square PFSS can realize any 2C1D CA, the hexagonal and triangular PFSS can realize both two and three states 1D CA. There are two possible arrangements of cells in the hexagonal tessellation. So called "zigzag" tessellation (hexagon-Z) requires one type of

Table 11.3 The regular tessellations in the context of design and CA shading. *Those two types although have different shapes, are based on identical CA logic.

	Occurrence in nature	**Design**	**Visual attractiveness** **Impact on the pattern**	**CA shading**
□	Extremely rare in macro scale in nature (e.g.: bismuth and galena crystals, cobwebs of *Cyrtophora citricola* [42]).	Rectangular or square meshes are prevalent in built environment.	Not particularly attractive Neutral	The easiest to apply. Each CASS module is identical.
▽	Regular triangular meshes do not occur naturally in macro scale.	Triangles are the only polygons that are rigid in plane [34]. Although this property is particularly useful in architecture and engineering, triangular grid is relatively rarely exposed in the built environment.	Relatively interesting Strong	Some CASS require two types of modules*.
⬡	It is encountered in macro scale in nature more often than other types of regular grids (e.g.: honeycomb, basalt columns). Therefore it carries certain visual organic appeal.	Since it is the only regular tessellation without single points of contact, the patterns appear as the most coherent. Nevertheless, it is relatively rare in human design.	Very attractive Minimal	Some CASS require two types of modules*.

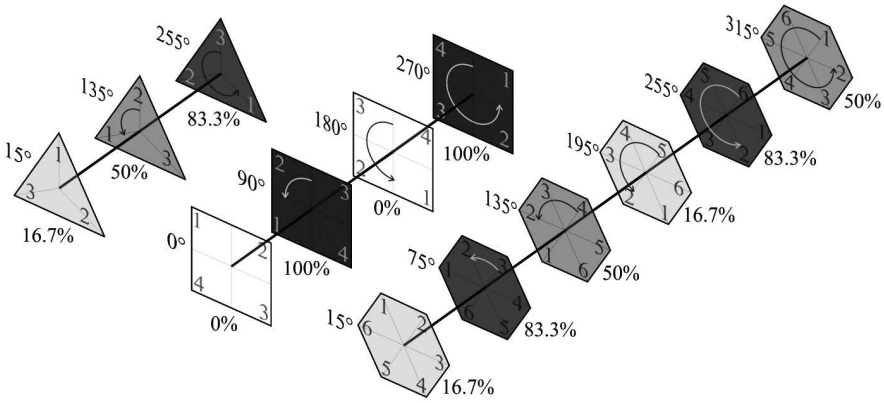


Fig. 11.8 PFSS opacity as a function of dihedral rotation on regular tessellations

module, and the "armchair" tessellation (hexagon-A) requires two modules of different shapes: O and E [6]. For neighborhoods of radii of any positive integer the triangular tessellation also requires two types of modules. Figure 11.9 illustrates the original CASS based on CA rule {3818817080,2,2} implemented by four types of PFSS.

11.3.5.1 PFSS on Hexagonal Grid

PFSS on hexagonal tessellation can execute 2 or 3 state CA. The following examples are based on the former, for 3 state examples see [65]. As mentioned above, depending on orientation, there are two types of hexagonal grids: "zigzag" and "armchair", called hexagon-Z and hexagon-A, respectively.

Hexagon-Z: $\mathbf{PFSS_{HZ}}$
Only one type of module is necessary to emulate 1D CA with $PFSS_{HZ}$. However, as Fig. 11.9 indicates, due to the asymmetry of the module, the pattern normally vertical, becomes skewed. It seems possible to find automata whose behavior will compensate for this, however, implementation of half-distance automata is more straightforward as shown in Fig. 11.10.

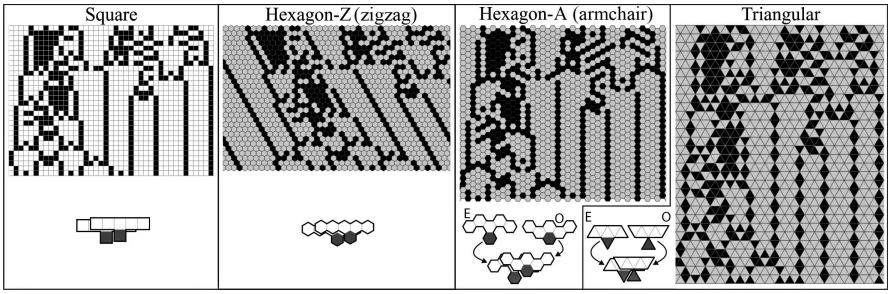


Fig. 11.9 CA rule {3818817080,2,2} realized by PFSS on regular tessellations. Gray levels on arrays represent the opacity values achievable by particular PFSS. For each type two modules for each tessellation are shown schematically. Coupled modules: E and O are shown accordingly.

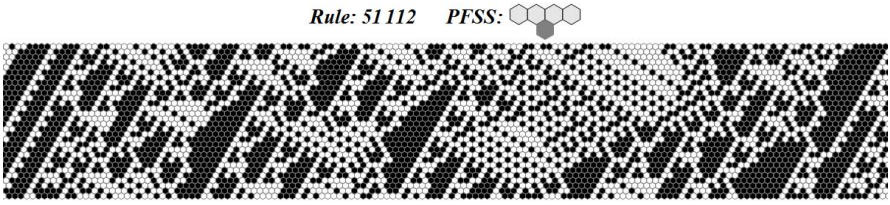


Fig. 11.10 The pattern produced by $PFSS_{HZ}$ driven by 1D2C r-$\frac{3}{2}$ CA rule 51112 from Fig. 11.5.

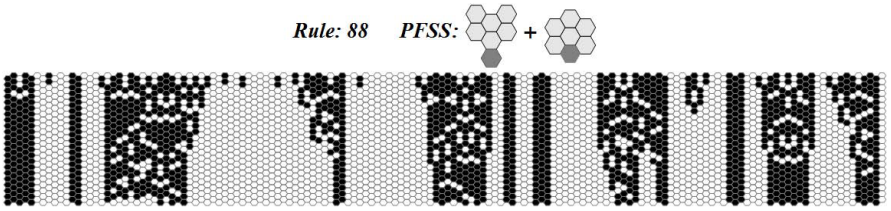


Fig. 11.11 The pattern produced by $PFSS_{HA}$ driven by T2-CA rule 88. IC rate of opacity: 0.5. Two PFSS modules are shown on the top, from the left: E and O.

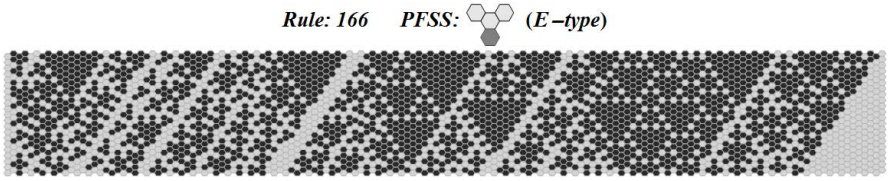


Fig. 11.12 The pattern produced by $PFSS_{HA}$ driven by 2D*2Cr1 $hexagon-A_E$ rule 166. IC rate of opacity: 0.8.

Hexagon-A: $\mathbf{PFSS_{HA}}$

Alternatively, hexagons in a lattice can be arranged so their parallel sides are aligned with the rows of cells. Such arrangement is called hexagon-A, and the corresponding shading system: $PFSS_{HA}$. As mentioned above, in such a case emulating 1D CA requires two types of modules: O and E. Figure 11.11 illustrates the application of totalistic second-order(T 2-CA) rule 88 selected from Fig. 11.7. However, it is also possible to arrange an array of cells using only one type: E or O. Such systems derived from one-dimensional cellular automata, are called two-dimensional cellular automata with specified offsets, and denoted here as 2D*CA. For example there are 256 2D*1Cr1CA analogous to ECA. Although the behavior is also analogous, the patterns produced by the former on $PFSS_{HAE}$ are more visually attractive. For example ECA rule 116 produces very simple patterns at very low GFE. However, the analogous 2D*2Cr1 $hexagon-A_E$ rule 166, although maintains low GFE, produces more intriguing patterns, as shown in Fig. 11.12. For more examples, including $hexagon-A_E$ and 3-state $PFSS_{HA}$ see [65].

11.3.6 PFSS on Triangular Grid: $PFSS_T$

Similarly, in the case of systems based on triangular tessellation ($PFSS_T$), emulating 1D CA with integer-value radii also requires two types of modules, as shown in Fig. 11.9. As noted in Tab. 11.3, the visual impact of the triangular tessellation is very strong. Therefore, it may not be particularly suitable for complex, organic patterns. However, its strong appearance combined with relatively simple CA can produce visually attractive patterns of different nature. As an example, Fig. 11.13 shows $PFSS_T$ based a second-order totalistic automaton (T2-CA) rule 56, the same as in

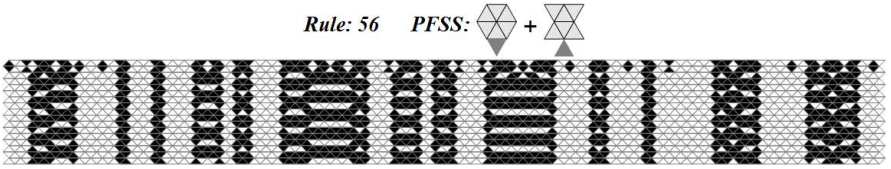


Fig. 11.13 The strong appearance of triangular tessellation can be advantageous. Despite the simplicity of the pattern produced by T2-CA rule 56 it becomes more visually attractive on triangular grid. IC rate of opacity: 0.5. Two PFSS modules are shown on the top, from the left: E and O.

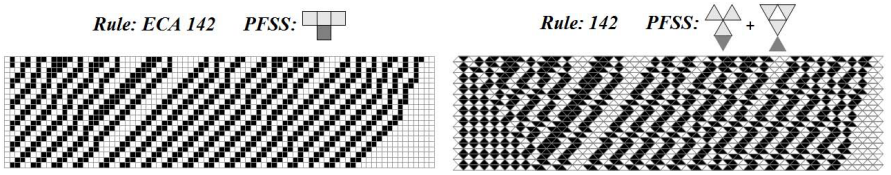


Fig. 11.14 Two patterns showing essentially the same CA at the same IC. On the left: ECA rule 142. On the right: analogous CA realized by combination of two modules (O and E) of $PFSS_T$.

Fig. 11.7. A small modification to the modules shown in Fig. 11.13 allow to emulate $hexagon - A_O$ PFSS, as shown in Fig. 11.14. The logic of both $PFSS_T$ modules shown in Fig. 11.14 is identical to the logic of the corresponding ECA. However, otherwise rather plain patterns gain certain visual attractiveness. The implementation of half-distance CA by $PFSS_T$ also requires two type of modules. However, creating perforated $PFSS_T^*$ by removing every other triangular facet allows to use only one type of module. Moreover, such an arrangement seems particularly interesting for BE, as every void in the tessellation can represent clear glass or solid walls, as shown in figures 11.15 and 11.1. Since the area of $PFSS_T^*$ is perforated in 50%, it should be taken in consideration, that the pattern is not as legible as in other shading systems presented here.

11.4 Two-Dimensional Cellular Automata on Surfaces

As mentioned in the beginning, controlling of the state of a BE surface with 2D CA seems intuitive, but the actual implementation is in fact more challenging than with 1D automata. As previous sections demonstrated, it is rather straightforward to apply any of regular tessellation, that is triangular, square or hexagonal on a planar surface and apply an automaton into it. For the overview of CA, in particular the Game of Life (GOL) in triangular, pentagonal and hexagonal tessellations see [6]; for the corresponding interactive demonstration see [5]. Modest size examples of GOL in triangle, square, and hexagon planar topologies and hierarchical hexagonal grid on the sphere intended for modeling of biogeographical, ecological and

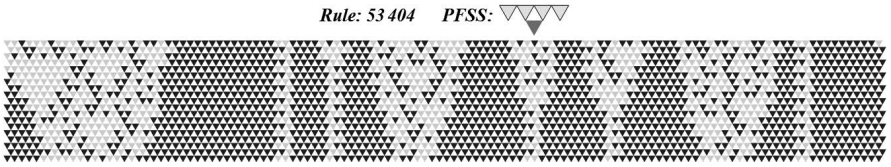


Fig. 11.15 Pattern produced by $PFSS_T^*$ driven by 1D2C r-$\frac{3}{2}$ CA rule 53404 selected from Fig. 11.5

epidemiological processes on the globe have been presented in [31]. Although the sphere is not a particularly practical shape for a building, domes have special position in the history of architecture. More recently, geodesic domes and spheres [47] have gained certain popularity due to their structural and aesthetic properties. A geodesic sphere (GS) is a spherical shell structure or lattice shell based on a network of great circles (geodesics) on the surface of a sphere, which intersect to form rigid triangular elements that also distribute the stress across the structure. Icosahedral geodesic sphere (IGS) is a spherical polyhedron with Euler characteristic $\chi = 2$. All triangular facets of IGS have 6 triangles per vertex, except, according to Euler's polyhedron formula [16], for 12 vertices with 5 triangles, regardless of the recursive subdivisions of the triangles (mesh resolution). The dual polyhedron of IGSs give Goldberg polyhedra [22], which consists of all hexagonal faces, except for also 12 [16], which are pentagons. [57] explores GOL on such a tessellation; for illustrative animation see [56].

The most general case of BE is a free-form surface, which cannot be constructed with regular polygons. Irregular tessellations have been introduced to more realistically model natural phenomena, e.g.: social networks have been simulated on irregular grid structures based on Voronoi-diagrams [18]. A geographic information system (GIS)-based CA on irregularly sized and shaped land parcels, at synchronous and asynchronous development have been used to model land-use change at the land parcel scale in [53]. Incremental matching method for processing of large structures that extend across many neighborhoods intended to enhance the data contained within topographic maps has been proposed in [37]. A cell-based wildfire simulator that uses an irregular grid that produces much faster results comparable in accuracy with traditional fire front propagation schemes has been presented in [28]. Graph-based cellular automata for CA models on irregular lattices have been introduced in [39].

For a free-form BE, however, triangular tessellation is the most suitable. Any 3D surface can be triangulated [45], that is divided into a set of (planar) triangles, with the restriction that each triangle side is entirely shared by two adjacent triangles. Analogous operation is impossible with (planar) quadrilaterals or hexagons. The control of state of such CASS is much more limited than in the previously described 1DCA-based systems. In the examples shown below, there is a single IC cell located arbitrary in the mesh. Alteration of the state of that cell triggers the 2D CA evolution, which propagates over entire mesh until the state of each cell stabilizes. It is imaginable that complex patterns perpetually fluctuating on BE could

be desirable and feasible especially considering the number conserving cellular automata (NCCA) [27] [19]. However, here for simplicity, the CA pattern is expected to become still as soon as the propagation reaches the boundary of BE.

11.4.1 Triangular Cellular Automata

Although the most commonly used lattice for CA regardless of the dimension is an orthogonal grid, several studies have been carried out to examine the properties of triangular cellular automata (TCA), e.g. life-like rules on r1 Moore's neighborhood in a triangular lattice have been studied in [4], the computational universality of an 8-state triangular reversible partitioned CA has been demonstrated in [26]. The effect of simple memory on a particular reversible, structurally dynamic CA in the triangular tessellation has been demonstrated in [1]. TCAs has also been proposed for BE shading in [63] and the applications of totalistic and semi-totalistic [21] TCAs have been further studied in [25]. Figure 11.16 shows an example of the evolution of a semi-totalistic triangular CA (stTCA) on a regular triangular lattice with voids. For an illustrative corresponding demonstration see [61]. The history of evolution of a 2D CA can be conveniently shown as a column of consecutive time steps, where unfolded lists of cells are shown as single rows, as shown in Fig. 11.17. However, unlike the 1D case, the information about the adjacencies is partially lost. From such diagrams some fundamental information for CASS can be derived, namely: whether a certain automaton stabilizes, at how many time steps and with what final opacity rate (FOR). Figure 11.18 shows all stable stTCAs. As Fig. 11.18 indicates, there are eight 2D2Cr1stTCAs suitable for CASS: rules: 50, 114, 118, 178, 242, 246, 250 and 254. After stabilization, the average opacities, called here FOR are: 0.51, 0.66, 0.63, 0,55, 0.77, 1, 0.85 and 1, respectively. For

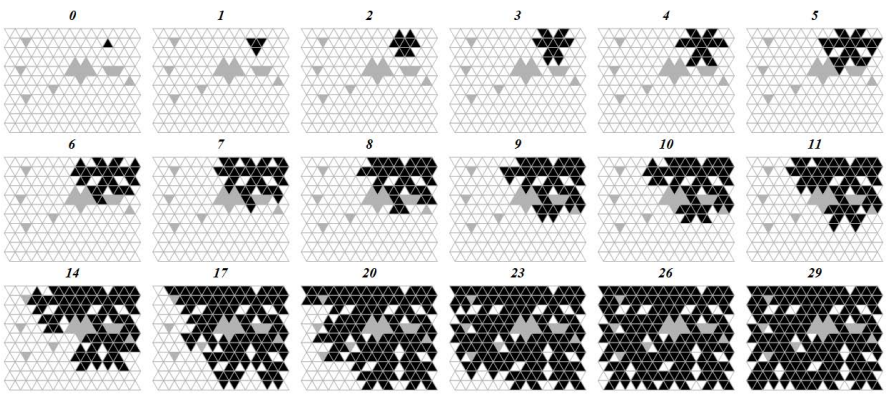


Fig. 11.16 stTCA # 250 at selected time steps starting from a single black cell. The pattern stabilizes at the 29th time step. White, gray and white facets indicate transparent facets, voids in the mesh and opaque facets, respectively.

further investigations including totalistic triangular cellular automata (tTCA) and the stabilization process see [25].

11.4.2 Cellular Automata on Triangulated Free-Form Surfaces

The application of stTCA on a free-form surface has been demonstrated in [63]. However, general rules have potential of allowing more direct control over the CASS mesh than totalistic and semi-totalistic rules. For a corresponding demonstration see [60]. Figure 11.19 shows three arbitrarily selected 2D2C TCAs: 9622, 44862, 65534 and applied on the BE shown in Fig. 11.1. For an interactive demonstration illustrating the application of those TCAs on a free-form surface with voids see [64].

11.5 Application of Evolutionary Algorithms for Optimization of CA Shading

As mentioned in Sect. 11.2.1, the sequence of initial conditions (SIC) is besides the local transition rules (TR) the second most fundamental factor determining the performance of CASS. Finding optimal SIC is much more straightforward than finding an automaton for CASS, nevertheless it is a NP-problem: the number of all possible SICs grows as factorial of the width of a shading array. Since it is impossible to exhaustively explore the entire search space, it is natural to use an meta-heuristics for finding, if not perfect, at least good or competitive solutions. Evolutionary algorithms (EAs) are a well established methodology in this field which has been successfully applied for a number of CA-related problems, e.g.: automated design of CA-based complex systems [54], evolving a non-uniform CA where each cell in the lattice does not use the same rule set [14]; finding Wolfram class IV, that is complex rules [7]; density classification task [38] and the parity problem [58] [35]; discovering and designing cell-state transition functions, where CA are designed to satisfy certain global conditions [15], etc. The application of EA for optimization of SIC for a 100×100 cell CASS has been presented in [67]. In that paper the objective is to minimize grayness monotonicity and pattern distribution error (GDE), which is called there the cost function (CF). The core parts of the

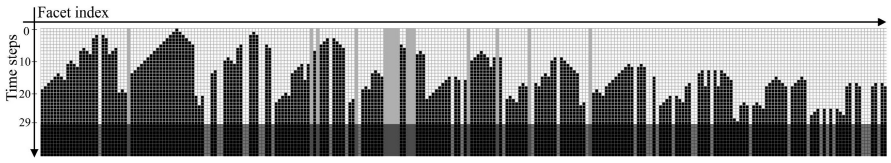


Fig. 11.17 The history of evolution from Fig. 11.16 shown as a single diagram. The shaded part after 29^{th} time step indicates stabilization of the state of the mesh. White, gray and white indicate transparent facets, voids in the mesh and opaque facets, respectively.

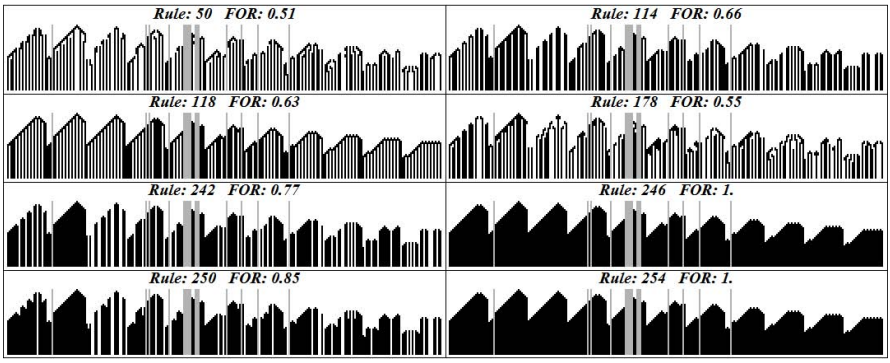


Fig. 11.18 The history diagrams of 40 time steps for stable stTCAs on the same triangular mesh

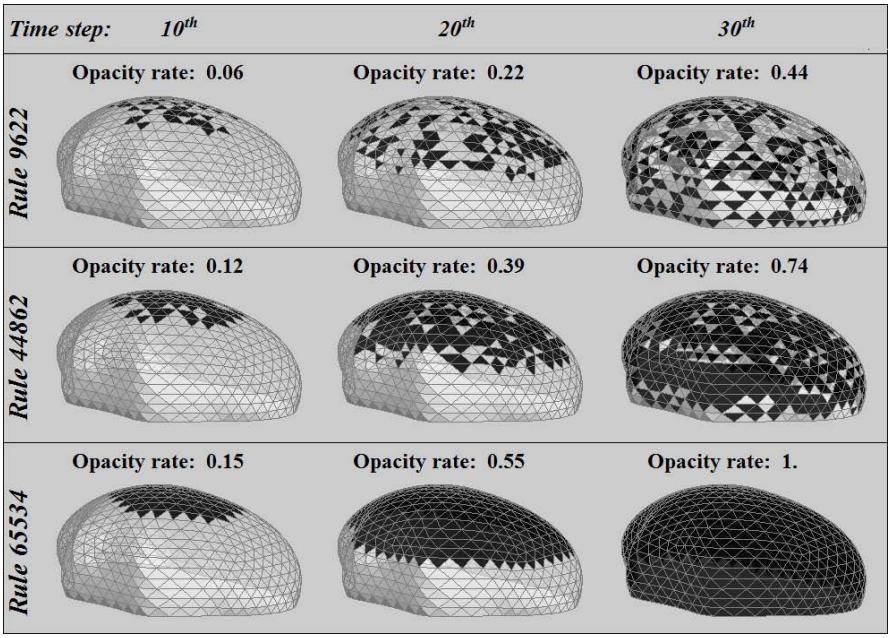


Fig. 11.19 Three general 2C2D TCAs applied on the BE from Fig. 11.1

application of EA for SIC optimization are presented and illustrated below. For further details, as well as the description of the application of evolution strategy [46], that is the simplest EA, where the genetic operations are limited to an intensive mutation and backtracking for the same task see [67]. The genetic operations are performed directly on a SIC* (genotype), that is an OBR-encoded SIC. Figure 11.20 shows examples of mutation and two types of recombination: one-point (OPX) and

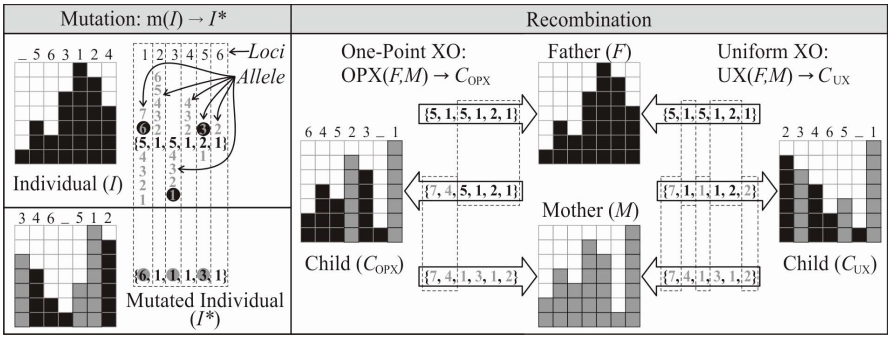


Fig. 11.20 Genetic operations on SIC*: On the left: mutation; on the right: two types of recombination

uniform crossovers (UX). The possible values of the genes, so called *alleles*, in such a genotype are constrained, and depend on the *locus*, that is the position of a certain gene in the genotype. Initially, since there are n available positions in the list, a gene can have n possible values. In the next step, since one position is already occupied, the allele is limited to n - 1, and so forth, up to the last gene in the genotype, which can only have two values: $n-((n-1)-1)=2$, as shown on the left of Fig. 11.20. Mutation is controlled by two parameters: mutation radius m_r the maximal difference of the mutated gene from the original value, and mutation intensity m_i, the maximal number of genes to be mutated, which is expressed as a ratio to the length of a genotype. The values of m_r and m_i in the example shown in Fig. 11.20 are 4 and 0.5, respectively. Figure 11.20 also illustrates two types of recombination: the one-point crossover (OPX) where a single point on both parents' gene strings is selected and the segments of genes are swapped between the two, and the uniform crossover (UX), where each gene is drawn with equal probability from both parents. The major advantage of OBR is that the operation of crossover (XO) is straightforward, and always yields feasible SIC* which represent allowable SIC. The process of adjusting the parameters for various evolutionary techniques for optimization of SIC for a 100×100 cell CASS based on the {3818817080,2,2} CA is described in [67]. Figure 11.21 compares the performance of five methods for an experiment of 10 trials of 200 generations of population of cardinality of 50: evolution strategy (ES), evolutionary algorithm with uniform crossover and mutation rates 0.1 and 0.4 (EA_{UX} 0.1 and EA_{UX} 0.1, respectively), EA with one-point XO and mutation rates 0.1 and 0.4 (EA_{OPX} 0.1 and EA_{OPX} 0.1, respectively). Good results generated by ES indicate that in this problem the operation of mutation is predominant. This influence is also evident in EA_{OPX} where the higher mutation rate produces better solutions. Good results obtained by EA_{UX} can also be attributed to the fact that UX introduces more intensive alterations to a genotype than OPX. However, the best overall result was obtained by EA_{OPX} indicating that the influence of crossover may be an increasing factor in a long run. The selection of the method depends on the designer's strategy. Table 11.4 collects the essential results. In a case of conservative

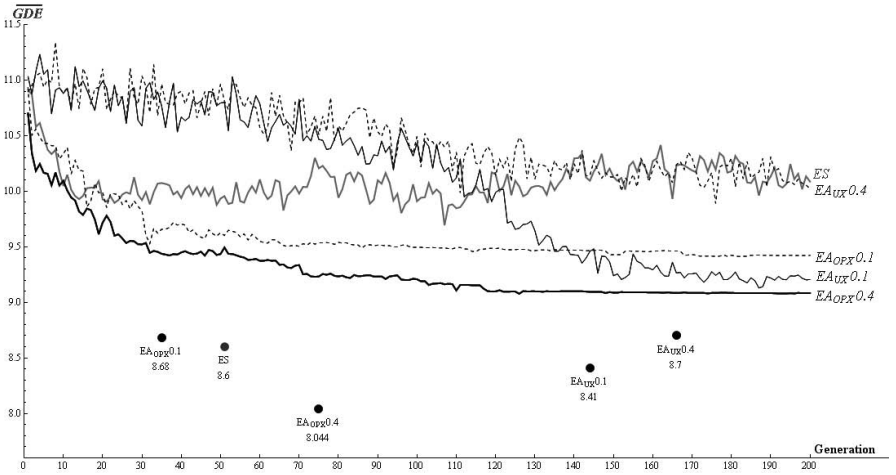


Fig. 11.21 The comparison of the means of the means of GDEs in a population of 50 for each generation in 10 trials for five evolutionary methods. The best solution for each method is indicated by a dot. EA_{OPX} with mutation intensity 0.4 produced consistently the best solutions.

Table 11.4 The results of the experiments for the 100×100 shading array

	ES	**EA**$_{UX}$		**EA**$_{OPX}$	
m_i	1	0.1	0.4	0.1	0.4
GDE$_{min}$	8.6	8.41	8.7	8.68	**8.04**
GDE$_{max}$	**9.39**	9.44	9.41	9.58	9.71
$\overline{\textbf{GDE}}_{min}$	9	**8.9**	9.1	9.1	9
σ_T	0.3	0.3	**0.2**	0.3	0.6

strategy, for example if only one trial is to be performed, ES should be chosen, since the worst result in all ES trials is better than in other methods. EA_{UX} with mutation rate $m_r = 0.4$ is the most predictable, in a sense that the best results in each trial are close and the mean value is competitive. However, EA_{UX} with $m_r = 0.1$ returns the best results on average, while EA_{OPX} with $m_r = 0.4$ gives the best overall result. Higher mutation rate (0.4) results in more scattered results, which, however, have the potential of reaching exceptionally good values. Therefore if a larger number of trials can be afforded, the latter method is recommended.

11.6 Prototypes

Although the research on CA commenced already over half a century ago, the physical devices based on them are still very scarce. The completion of the first hardware

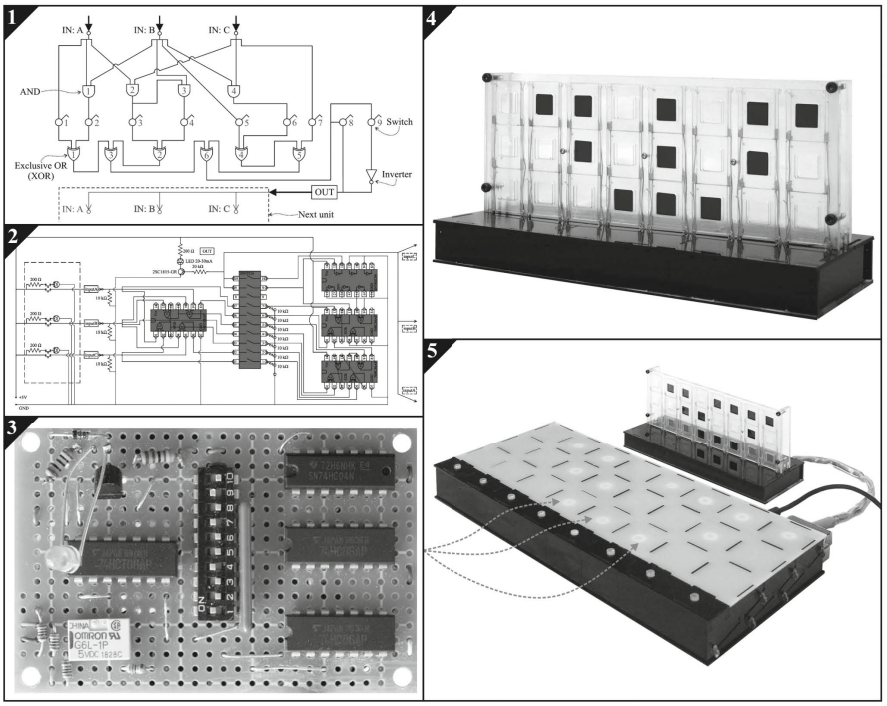


Fig. 11.22 The CASS prototype: 1): Scheme of the logic which emulates 256 ECAs. 2): Electrical diagram of a CA unit. 3): A photograph of a hardware CA unit. 4): SP with some LC elements active (opaque). 5): SP following the pattern produced by CU.

prototype of CASS has been documented in [68]. However, since it is based on ECAs which do not possess good shading properties, it can not be considered as CASS in the strictest sense. Nonetheless, the original motivation was also: to provide a physical educational device driven by CA; as an exercise on electric circuitry; to give students a hands-on experience of CA; to demonstrate the entire process from designing the circuit logic to fabrication of the CA modules; to teach the emergent properties of CA [17]. Therefore the logic was designed to allow emulation of all 256 ECAs by setting manually 8 simple switches at each CA module. Interestingly, this also allows experimentation with non-uniform CA [51], that is automata whose individual cell rules are not the same. Although the concept of CASS is based on interactions among autonomous units, the original prototype, for the reasons mentioned above, is comprised of two components: the CA control unit (CU) and the shading panel (SP) based on liquid crystal (LC) technology. That prototype is fully operational. Since the CA units are based on commonly available integrated circuits (CMOS), they are much larger than intended for the real application, which most likely would combine field programmable gate arrays (FPGA) with printed circuit boards (PCB) [44]. Due to high cost of LC elements, SP is much smaller than

intended. Moreover, CU was built also to demonstrate “hardware automaton” [55], therefore each cell is equipped with a light emitting diode (LED) to display the CA action. This is also the reason why the CA circuits are not integrated with the shading elements, as it would be in the actual shading device. Figure 11.22 shows the prototype documentation.

Acknowledgements. This is part of Singapore University of Technology & Design and Massachusetts Institute of Technology Postdoctoral Program (SUTD-MIT PDP). The research is titled: *Effective computational methods for grid and raster-based modeling of practical problems in architectural and urban design*.

References

1. Alonso-Sanz, R.: A structurally dynamic cellular automaton with memory in the triangular tessellation. Complex-Systems 17(1), 1–15 (2007)
2. Aydinli, S., Seidl, M.: Determination of the economic benefits of daylight in interiors concerned with the fulfillment of visual tasks. In: Adepski, M., McCluney, R. (eds.) Proceedings I: 1986 International Daylighting Conference, Long Beach California USA, pp. 145–151 (1986)
3. Baas, N.A., Torbjorn, H.: Higher Order Cellular Automata. Advances in Complex Systems 8(2-3), 169–192 (2005)
4. Bays, C.: Cellular automata in the triangular tessellation. Complex-Systems 8, 127–150 (1994)
5. Bays, C.: Cellular Automata and the Game of Life in the Hexagonal Grid (2001), `http://www.cse.sc.edu/~bays/h6h6h6/`
6. Bays, C.: Cellular Automata in Triangular, Pentagonal and Hexagonal Tessellations. In: Meyers, R.A. (ed.) Computational Complexity, pp. 434–442. Springer, New York (2012)
7. Bilotta, E., Lafusa, A., Pantano, P.: Searching for complex CA rules with GAs. Complexity 8(3), 56–67 (2003)
8. Bodart, M., De Herde, A.: Global energy savings in offices buildings by the use of daylighting. Energy and Buildings 34(5), 421–429 (2002)
9. Boyce, P.: Why Daylight? In: Proceedings of Daylight 1998, International Conference on Daylighting Technologies for Energy Efficiency in Buildings, Ottawa, Ontario, Canada, pp. 359–365 (1998)
10. Boyce, P., Hunter, C., Howlett, O.: The benefits of daylight through windows: Report. Tech. rep., Rensselaer Polytechnic Institute, Troy, New York (2003)
11. Chavey, D.: Tilings by regular polygons II: A catalog of tilings. Computers & Mathematics with Applications 17(1-3), 147–165 (1989)
12. Connor, S.: The book of skin. Cornell University Press (2003)
13. Cuttle, C.: Lighting by Design. Elsevier, Amsterdam (2003)
14. Das, R., Mitchell, M., Crutchfield, J.: A genetic algorithm discovers particle based computation in cellular automata. In: Davidor, Y., Männer, R., Schwefel, H.-P. (eds.) PPSN 1994. LNCS, vol. 866, pp. 244–353. Springer, Heidelberg (1994)
15. El Yacoubi, S., Jacewicz, P.: A genetic programming approach to structural identification of cellular automata. Journal of Cellular Automata 2, 67–76 (2007)
16. Euler, L.: Elementa doctrinae solidorum. Novi Commentarii academiae scientiarum Petropolitanae 4, 109–140 (1758)
17. Faraco, G., Pantano, P., Servidio, S.: The use of Cellular Automata in the learning of emergence. Computers & Education 47(3), 280–297 (2006)

18. Flache, A., Hegselmann, R.: Do Irregular Grids make a Difference? Relaxing the Spatial Regularity Assumption in Cellular Models of Social Dynamics. Journal of Artificial Societies and Social Simulation 4(4) (2001), http://jasss.soc.surrey.ac.uk/4/4/6.html
19. Formenti, E., Grange, A.: Number conserving cellular automata II: dynamics. Theoretical Computer Science 304(1-3), 269–290 (2003)
20. Frontczak, M., Wargocki, P.: Literature survey on how different factors influence human comfort in indoor environments. Building and Environment 46(4), 922–937 (2011)
21. Garzon, M.: Models of Massive Parallelism: Analysis of Cellular Automata and Neural Networks. European Association for Theoretical Computer Science. Springer, Heidelberg (1995)
22. Goldberg, M.: A Class of Multi-Symmetric Polyhedra. Tohoku Mathematical Journal 43, 104–108 (1937)
23. Grefenstette, J., Gopal, R., Rosimaita, B., Gucht, D.: Genetic Algorithms for the Traveling Salesman Problem. In: Proceedings of the 1st International Conference on Genetic Algorithms and their Applications, pp. 160–168. Psychology Press, Pittsburgh (1985)
24. Hanson, J.E.: Encyclopedia of Complexity and Systems Science. In: Meyers, R.A. (ed.) Cellular Automata, Emergent Phenomena, pp. 768–778. Springer (2009)
25. Zawidzki, M., Nishinari, K.: Controlling the opacity of a building envelope by a triangular two-color two-dimensional cellular automaton. In: Sirakoulis, G.C., Bandini, S. (eds.) ACRI 2012. LNCS, vol. 7495, pp. 194–203. Springer, Heidelberg (2012)
26. Imai, K.: A computation-universal two-dimensional 8 state triangular reversible cellular automaton. Theoretical Computer Science 231(2), 181–191 (2000)
27. Imai, K., Fujita, K., Iwamoto, C., Morita, K.: Embedding a Logically Universal Model and a Self-Reproducing Model into Number-Conserving Cellular Automata. In: Calude, C.S., Dinneen, M.J., Peper, F. (eds.) UMC 2002. LNCS, vol. 2509, pp. 164–175. Springer, Heidelberg (2002)
28. Johnston, P., Kelso, J., Milne, G.J.: Efficient simulation of wildfire spread on an irregular grid. International Journal of Wildland Fire (2008)
29. Kari, J.: Theory of cellular automata: A survey. Theoretical Computer Science 304(1-3), 3–33 (2005)
30. Kepler, J.: Harmonice Mundi. Lincii Austriae (1619)
31. Kiester, R.A., Sahr, K.: Planar and spherical hierarchical, multi-resolution cellular automata. Computers, Environment and Urban Systems 32, 204–213 (2008)
32. Kim, J.T., Kim, G.: Overview and new developments in optical daylighting systems for building a healthy indoor environment. Building and Environment 45(2), 256–269 (2010)
33. Konopka, A.K.: Systems Biology: Principles, Methods and Concepts Taylor and Francis. Taylor & Francis, Boca Raton (2006)
34. Laman, G.: On Graphs and Rigidity of Plane Skeletal Structures. Journal of Engineering Mathematics 4, 331–340 (1970)
35. Lee, K., Xu, H., Chau, H.: Parity problem with a cellular automaton solution. Physical Review E 026702(64), 1–4 (2001)
36. Madison, K.C.: Barrier function of the skin: Ąraison d'etre of the epidermis. Journal of Investigative Dermatology 121(2), 231–241 (2003)
37. O'Donoghue, D.P., Mullally, E.C.: Extending Irregular Cellular Automata with Geometric Proportional Analogies. In: Proceedings of the Geographical Information Science Research UK Conference, NUI Maynooth, Ireland, April 11-13 (2007)
38. de Oliveira, P., Bortot, J., Oliveira, G.: The best currently known class of dynamically equivalent cellular automata rules for density classification. Neurocomputing 70(1-3), 35–43 (2006)

39. O'Sullivan, D.: Exploring Spatial Process Dynamics Using Irregular Cellular Automaton Models. Geographical Analysis 33(1), 1–18 (2001)
40. Pegg, E.: Half-Distance Rules with Low Resolution (2000), http://demonstrations.wolfram.com/HalfDistanceRulesWithLowResolution/ (an interactive demonstration)
41. Pegg, E., Zawidzki, M.: Cellular Shading (2008), http://demonstrations.wolfram.com/CellularShading/ (an interactive demonstration)
42. Peters, H.M.: Functional organization of the spinning apparatus of Cyrtophora citricola with regard to the evolution of the web (Araneae, Araneidae). Zoomorphology 113(3), 153–163 (1993)
43. Proksch, E., Brandner, J.M., Jensen, J.M.: The skin: an indispensable barrier. Experimental Dermatology 17(12), 1063–1072 (2008)
44. Qasim, S.M., Abbasi, S.A., Almashary, B.: An Overview of Advanced FPGA Architectures for Optimized Hardware Realization of Computation Intensive Algorithms. In: Multimedia, Signal Processing and Communication Technologies (IMPACT 2009), pp. 300–303. Institute of Electrical and Electronics Engineers, Aligarh (2009)
45. Rado, T.: Uber den Begriff der Riemannschen Flache (in German). Acta Szeged 2(2), 101–121 (1925)
46. Rechenberg, I.: Evolutionsstrategie: Optimierung Technischer Systeme Nach Prinzipien Der Biologischen Evolution. Ph.D. thesis, Stuttgart (1973) (in German)
47. Rothman, T.: Geodesics, Domes, and Spacetime. In: Science a la Mode. Princeton University Press (1989)
48. Ruck, N., Aschehoug, O., Aydinli, S., Christoffersen, J., Courret, G., Edmonds, I., Jakobiak, R., Kischkoweit-Lopin, M., Klinger, M., Lee, E.S., Michel, L., Scartezzini, J.L., Selkowitz, S.E.: Daylight in buildings: A Source Book on Daylighting Systems and Components. Tech. Rep. LBNL-47493, Lawrence Berkeley National Laboratory, Berkeley, California (2013), http://gaia.lbl.gov/iea21/
49. Saadatian, O., Sopian, K., Lim, C., Asim, N., Sulaiman, M.Y.: Trombe walls: A review of opportunities and challenges in research and development. Renewable and Sustainable Energy Reviews 16(8), 6340–6351 (2012)
50. Serra, R.: Chapter 6 – Daylighting. Renewable and Sustainable Energy Reviews 2(1-2), 115–155 (1998)
51. Sipper, M.: Co-evolving non-uniform cellular automata to perform computations. Physica D 92, 193–208 (1996)
52. Steiglitz, K., Kamal, I., Watson, A.: Embedding computation in one-dimensional automata by phase coding solitons. IEEE Transactions on Computers 37(2), 138–145 (1988)
53. Stevens, D., Dragicevic, S.: A GIS-based irregular cellular automata model of land-use change. Environment and Planning B: Planning and Design 34(4), 708–724 (2007)
54. Terrazas, G., Siepmann, P., Krasnogor, N.: An evolutionary methodology for the automated design of cellular automaton-based complex systems. Journal of Cellular Automata 2(1), 77–102 (2008)
55. Toffoli, T., Margolus, N.: Cellular Automata Machines. MIT Press, Cambridge (1987)
56. Ventrella, J.: Earth Day 2009 – A Spherical Cellular Automaton (2009), http://www.ventrella.com/EarthDay/EarthDay.html
57. Ventrella, J.: Glider Dynamics on the Sphere: Exploring Cellular Automata on Geodesic Grids. Journal of Cellular Automata 6(2-3), 245–256 (2011)
58. Woltz, D., de Oliveira, P.: Very effective evolutionary techniques for searching cellular automata rule spaces. Journal of Cellular Automata 3(4), 289–312 (2008)

59. Zawidzki, M.: Window Opacity Controlled By Cellular Automata (2008), http://demonstrations.wolfram.com/WindowOpacityControlledByCellularAutomata/ (an interactive demonstration)
60. Zawidzki, M.: 2D Triangular Cellular Automata on a Distorted Grid with Holes (2009), http://demonstrations.wolfram.com/2DTriangularCellularAutomataOnADistortedGridWithHoles/ (an interactive demonstration)
61. Zawidzki, M.: 2D2CR1 Cellular Automaton On a Triangular Grid (2009), http://demonstrations.wolfram.com/2D2CR1CellularAutomatonOnATriangularGrid/ (an interactive demonstration)
62. Zawidzki, M.: Delayed CA (2010), http://demonstrations.wolfram.com/DelayedCA/ (an interactive demonstration)
63. Zawidzki, M.: Application of Semitotalistic 2D Cellular Automata on a triangulated 3D surface. International Journal of Design & Nature and Ecodynamics 6(1), 34–51 (2011)
64. Zawidzki, M.: 2D Cellular Automaton on a Triangulated Surface (2013), http://demonstrations.wolfram.com/2DCellularAutomatonOnATriangulatedSurface/ (an interactive demonstration)
65. Zawidzki, M.: Dynamic shading of building envelope based on rotating polarized film system controlled by one-dimensional cellular automata on regular tessellations (triangular, square & hexagonal). Advanced Engineering Informatics (2013) (under review)
66. Zawidzki, M.: Implementing Cellular Automata for Dynamically Shading a Building Facade. Complex-Systems 18(3), 287–305 (2013)
67. Zawidzki, M., Bator, M.: Application of evolutionary algorithm for optimization of the sequence of initial conditions for the cellular automaton-based shading. Journal of Cellular Automata 7(5-6), 363–384 (2013)
68. Zawidzki, M., Fujieda, I.: The prototyping of a shading device controlled by a cellular automaton. Complex-Systems 19(2), 157–175 (2010)
69. Zawidzki, M., Nishinari, K.: Shading for building facade with two-color one-dimensional range-two cellular automata on a square grid. Journal of Cellular Automata 8(3-4), 147–163 (2013)

Chapter 12
Pattern Formation Using Cellular Automata and L-Systems: A Case Study in Producing Islamic Patterns

Seyyed Amir Hadi Minoofam, Mohammad Mahdi Dehshibi,
Azam Bastanfard, and Jamshid Shanbehzadeh

Abstract. The science of pattern formation deals with the visible, (statistically) orderly outcomes of self-organization and the common principles behind similar patterns in nature. Cell pattern formation has an important role in both artificial and natural development. Different methods have been utilized for pattern formation such as geometrical, Cellular Automata (CAs), and L-Systems. In this chapter, we concentrate our aim on introducing the role of CAs and L-Systems in pattern formation and how to extract optimum rules in terms of numbers and functionality for this aim. Because a few works have been reported in the field of script generation, we take generating Ma'qeli script and Holy words patterns, as a case study, in hand. Results of this study show the superiority of the proposed method in comparison with geometrical and fractal approaches in case of the time complexity in word production, simplicity of extracted rules, and possible reusability of the CAs rules in generating other script patterns in other languages. Moreover, the proposed method is shape-resistance, which can be less seen in fractals and geometrical based pattern formation.

Seyyed Amir Hadi Minoofam
Department of Computer Engineering, Islamic Azad University,
Nazar Abad Center, Karaj, Iran
e-mail: minoofam@qiau.ac.ir

Mohammad Mahdi Dehshibi
Department of Computer Engineering,
Islamic Azad University, Science and Research Branch, Tehran, Iran
e-mail: mohammad.dehshibi@piau.ac.ir

Azam Bastanfard
Department of Computer Engineering,
Islamic Azad University, Karaj Branch, Karaj, Iran
e-mail: bastanfard@kiau.ac.ir

Jamshid Shanbehzadeh
Department of Computer Engineering, Kharazmi University, Tehran, Iran
e-mail: jamshid@saba.tmu.ac.ir

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*,
Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_12, © Springer International Publishing Switzerland 2014

12.1 Introduction

According to Wolfram [9], there are no studies that cannot be modeled by cellular automata. Nowadays, the footstep of cellular automata, either synchronous or asynchronous, can be found in different aspects of science [20] from modeling a biological system [22] to produce a virtual social network [23]. Artificial models of cellular development have been proposed over the years with the objective of understanding how complex structures and patterns can emerge from one or a small group of initial undifferentiated cells [6]. This mathematical tool has the capability of easing many proposed solutions and modeling them more intelligently.

On the other hand, an L-system (Lindenmayer system) is a parallel rewriting system and a type of formal grammar. An L-system consists of an alphabet of symbols that can be used to make strings, a collection of production rules that expand each symbol into some larger string of symbols, an initial 'axiom' string from which to begin construction, and a mechanism for translating the generated strings into geometric structures. Lindenmayer [12] used L-systems to describe the behavior of plant cells and to model the growth processes of plant development. L-systems have also been used to model the morphology of a variety of organisms [20] and can be used to generate self-similar fractals such as iterated function systems.

In [11], 'Game of Life' with complex behaviors was characterized by simple synchronous cellular automata's rules. The main contribution towards this work was to design a simple set of rules to study the macroscopic behavior of a population. The Firing Squad [10], Firing Mob [16], and Queen Bee [7] are other games in which synchronization problems are investigated adequately. Piwonska and Seredynski [19] studied the impact of utilizing genetic algorithm on extracting optimum rules for 2D cellular automata. They utilized optimum extracted rules with von Neumann neighborhood in order to reconstruct several patterns. Chavoya et al. [5] consider the problem of growing a solid French flag pattern in a 3D virtual space. They proposed an artificial development model for 3D cell pattern generation based on CAs. Cell replication is controlled by a genome consisting of an artificial regulatory network and a series of structural genes. The genome was evolved by a genetic algorithm in order to generate 3D cell patterns through the selective activation and inhibition of genes. Morphogenetic gradients were used to provide cells with positional information that constrained cellular replication in space.

Bentley et al. [4] describe a novel computer simulation that uses evolution to exhibit some of the functions of cell walls raphid pennate diatom valves. The model of valve morphogenesis used was based on theories that highlight the importance of cytoskeletal elements in valve development. An 'organic' negative imprint is grown in a grid-based system, using both local and global rules to dictate grid cell states. Silica then diffuses out into all remaining grid cells. At each stage of development the generated valves were consistent with observations on real diatom valve growth. Simulated models are extremely useful for investigating, visualizing, and developing theories of morphogenesis. It is the intention of this work to inspire further model-based experiments and to try to bridge the gap between the disparate fields of computer science and biology for the exploration of morphogenesis. This model

of diatom valve morphogenesis is interestingly similar to the negative technique used by artists in batik painting.

In [13], scholars use cellular automata for modeling shell pigmentation of molluscs. They found self-organization into stationary (Turing) structures, travelling waves, chaos, and so-called class IV behavior during their research. Class IV behavior consists of a disordered spatio-temporal distribution of periodic and chaotic patches, which differs from chaos in that it has no well-defined error propagation rate. The calculations of the modes agree well with observations in natural shells. Their results suggest evidence in nature for class IV behavior, a mode that had so far been reported only as the result of simulations. Oya et al. [18] use single-electron circuits to perform dendritic pattern formation with nature-inspired cellular automata. They propose a novel semiconductor device in which electronic-analogue dendritic trees grow on multilayer single-electron circuits. A simple cellular automaton circuit was designed for generating dendritic patterns by utilizing the physical properties of single-electron devices.

Another application of pattern formation, which is a highly demanded topic among computer graphics researchers, is producing cultural-related motifs which are still yet to mature due to their production complexities. Cellular automata show their brilliant capability in this application too; as a result, much research has been devoted to apply this mathematical tool for generating complex patterns [12]. Arata et al. [3] made an effort on applying cellular automata with Margolus neighborhood to model an interactive free-form scheme within a 3D Voxel space for designing virtual clay objects. They assumed each Voxel is allocated a finite state automaton that repeats state transitions according to the conditions of its neighbor Voxels. Typeface and language's script are other attractive patterns, which entices researchers to bring their utility and attractiveness into focus. Xu et al. used a computational approach to digital chinese calligraphy and painting [24]. Ahuja et al. [2] utilized a number of geometric shapes to tessellate the 'Ali' pattern. In [15] a prototype for performing geometrical transformations was introduced with the aim of decoration designs by the Kufic square scripts. Ma'qeli script is a sort of ancient Persian script with amazing features (refer to Fig.12.1); however, a small number of its patterns were produced by Minoofam and Bastanfard [14] utilizing synchronous cellular automata.

This chapter will review the basic foundation of CAs and L-Systems in pattern formation especially those which are related to the cultural heritage. Then, it will be concentrated on generating all forms of the Ma'qeli script with CAs and three holy Islamic words using L-Systems with two clear reasons which one of these is decidedly more glamorous than the other one. The glamorous reason deals with the distinctive features of Persian script, i.e., it is cursive, it depends on the base line, and it is the formal writing typeface of more than 150 million people. All of this has overshadowed the less glamorous side of the reason, which deals with the tessellated nature of Ma'qeli script. This reason seems to have been shown more adaption with cellular automata rather than geometric methods. The focus of this study is not only on generating Ma'qeli script pattern utilizing 2D asynchronous cellular automata and Margolus neighborhood, but also on finding optimum set of rules for this sort

Fig. 12.1 A miniature consists of Ma'qeli pattern of "مبارک باد" which means 'Congratulations'

of script which facilitates reusability possibilities in generating similar patterns. In order to show the simplicity and efficiency of the proposed method, a set of experiments is conducted in which the capability of both synchronous and asynchronous cellular automata with 1D and 2D structures and different neighborhoods are examined. It was observed in the course of experiments that the contribution towards this study has a number of advantages. First, this script, as a graphical primitive, has the capability of being utilized in graphical applications such as computer games, animations, and so forth [8]. Moreover, it will assist industrial applications to show sentences on 7-segments or dot matrix monitors. In short term too, an improvement of this research's understanding can be used as an add-on for CAD software to make it applicable in the domain of cultural heritages, handicrafts and the calligraphies which are woven through the carpets.

The roadmap of this chapter is as follows. Section 2 describes preliminary definition and terminology. The proposed method is explained in Section 3. Experimental results are depicted in Section 4, and finally this chapter is concluded in Section 5.

Fig. 12.2 Six 'Ali' written in Ma'qeli script in a coin such that three are inscribed in shape of hollows and the others are embedded between the hollows

12.2 Preliminary Definitions and Terminologies

The Ma'qeli script is a sort of Persian script which consists of 32 characters. However, it usually appears in 64 different letter's shapes in four different groups of the Initial, Medial, Final, and Isolated forms. The Ma'qeli characters are constructed based on the squares which resemble the markings of a grid paper. There is no curve in this script and both horizontal and vertical lines have odd length in a grid paper. This kind of alphabet is generally utilized to tile shrines with words such as Allah and Muhammad, and has an interesting appearance. In other words, tiling has been done such that only some of these words can be seen in the first impression; however, to look carefully, more words can be recognized in that geometrical object. For instance, an observer could see only three 'Ali' in Fig. 12.2 at first glance, but having a closer look will result in finding six embedded 'Ali' in the golden coin.

This interesting feature motivated us to explore whether this script could be effectively used to generate graphical text patterns. Fig.12.3 shows all Persian alphabet letters in Ma'qeli script.

As shown in Fig. 12.3, the characters with similar shapes are grouped together and the only difference between members of these groups is in existence of additional diacritics including dots and lines, the {س، ش} and {ک، گ} groups are clarifying examples with respect to the former and the latter differences. Production of a word pattern using the Ma'qeli script can be done by the following meta-rules:

- The positions of each letter with regard to the base line.
- Attaching adequate additional diacritics to letters
- The connection or separation of each letter to/from other ones in a word, and space between each words.
- The expansion of letters and words in both vertical and horizontal directions.
- The square cells which fill a grid paper or tessellate a construction.
- Considering a white space before writing a word as the most important principle of calligraphy.

A 2D asynchronous cellular automaton consists of cells arranged in a rectangular array, in which space and time are discrete. Each cell has k finite discrete states with varying values between 0 and k-1. At each discrete time step, the states of all cells are updated asynchronously according to a local rule that depends only upon the state of the cell and its neighbors. Although there is no limitation for defining neighbors in two dimensions cellular automata, models of spatial processes usually use one of the only three regular tessellations in the plane which are squares, hexagons,

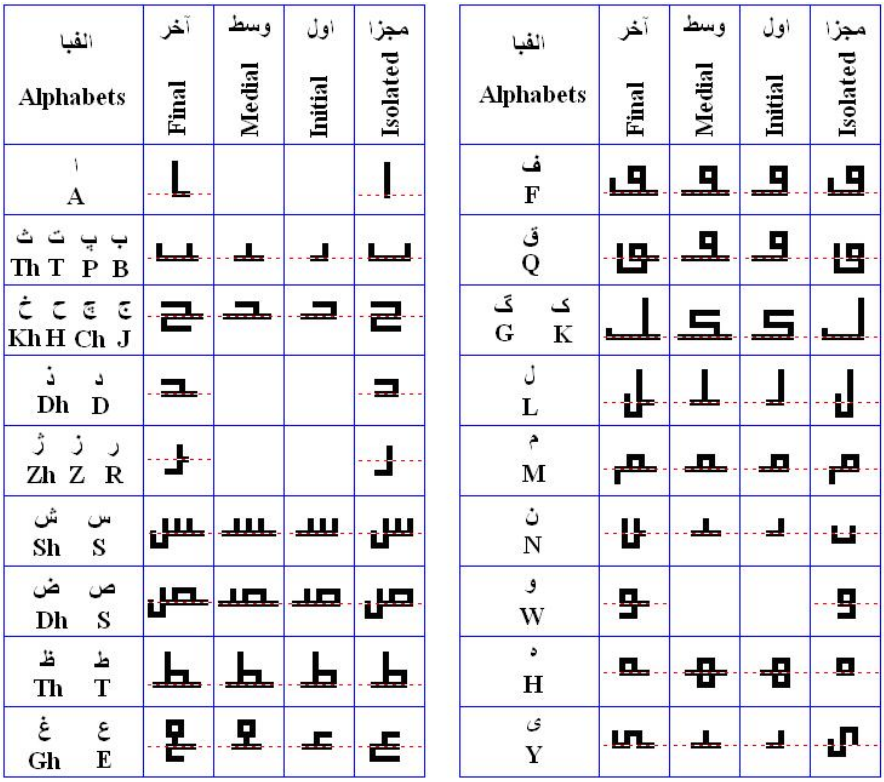


Fig. 12.3 Persian alphabet letters in Ma'qeli script [21]. The dashed lines show the base lines of each letter.

or triangles. The orthogonal neighbors within squares are called the von Neumann neighborhood (Fig. 12.4a), and those plus the diagonal neighbors are called the Moore neighborhood (Fig. 12.4b and Fig. 12.4c). An alternative is the Margolus neighborhood, which is used in Block Cellular Automata [17], and is implemented with two n×n-cell blocked square tessellations. These two square tessellations are offset from each another by one row and one column. Fig. 4d visualizes a specific kind of Margolus neighborhood in which each block consists of 2×2-cell tessellations.

12.3 Proposed Method

As is previously mentioned, letters, words, and sentences expand in two-dimension and this demand inspires us to utilize 2D cellular automata. Since it is important to establish a trade-off between the form of patterns and the number of productive rules, block cellular automata are utilized by dividing the lattice of cells into

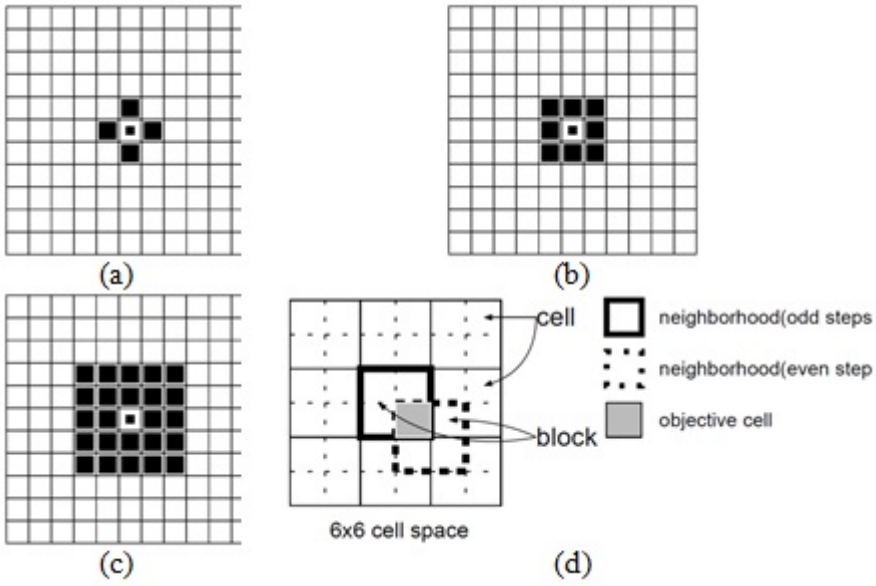


Fig. 12.4 Topologies analyzed in the paper are (a) squares with the von Neumann neighborhood, (b) squares with the Moore neighborhood, (c) squares with the extended Moore neighborhood, and (d) the Margolus neighborhood

non-overlapping blocks. Block cellular automata are useful for generating Ma'qeli scripts, since they are straightforward to choose transition rules that obey writing laws and apply them simultaneously and synchronously to a whole block at a time rather than a single cell. In view of the fact that synchronous cellular automata simultaneously apply all transition rules to all blocks, the procedure of script generation encounters serious problems. In order to achieve a correct script generation, we have to be aware of the fact that each rule must be executed in a correct time step. Not only does this type of supervision make rule extraction process difficult, but also it is impossible in some cases. Therefore, in present study we use asynchronous cellular automata to ease both the pattern formation and the rule extraction process.

12.3.1 Ma'qeli Character Generation Using Margolus Neighborhood

The proposed block scheme consists of Margolus neighborhood itself in which the lattice is divided into 3×3-cell blocked square tessellations, in as much as all Ma'qeli characters can be bounded by such block (Fig. 12.5). An extension to a standard Margolus neighborhood is proposed which does not need to be shifted by one cell on alternate timestamps and can generate preferable patterns in one timestamp. As shown in Fig. 3, 64 characters are just defined without any additional

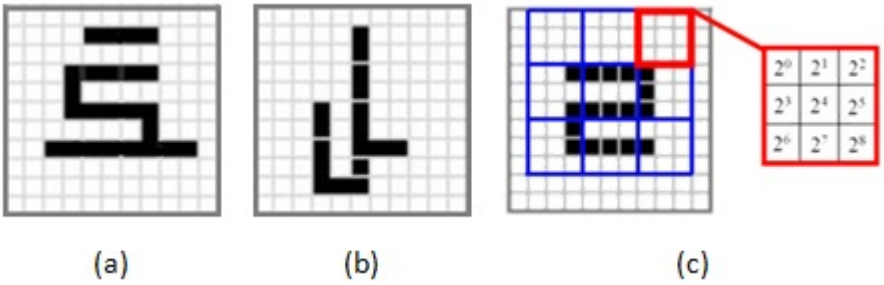


Fig. 12.5 Three samples related to the Ma'qeli alphabets which are (a) Gaf in the form of "ﮔ" (b) Lam in the form of "ﻟ" and (c) Ha in the form of "ﻫ" where its partition and rule's numbers are shown

diacritics. Taking into consideration all of the additional diacritics will result in 114 distinct Persian characters in four different groups. Fig. 12.5 illustrates three out of 114 characters, which are bounded by a 3×3-block.

In order to extract rules of each 3-cells embedded in a 3 × 3-block, binary weight is utilized which is illustrated in Fig. 12.5c. In this example, the letter "ﻫ" fills six 3×3-cells and the binary rules related to this block are 0, 0, 0, 260, 455, 74, 36, 56, and 8, respectively. As is obvious from Fig. 12.3 and Fig. 12.5, all of the 114 characters have their own rules, but extracting separated rules for them seems to be not ideal. As the second contribution towards this study, these rules were decreased down to total of 40 ones covering all 114 Ma'qeli characters. These rules are illustrated in Fig. 6 in which four rules are dedicated to put dot(s) on/under a letter.

12.3.2 Word and Sentence Generation Using Ma'qeli Patterns

In order to generate different words and construct sentences, additional restrictions should be observed. For the reason that Persian script is cursive in the right to left direction, connectivity rules must be obeyed in word generation, and the connection side should be specified. Four letters family {ا، د، ر، و} have no medial and final forms, so they cannot be connected to other letters from left, see Fig. 12.3. It may, however, be noted that other letters' family do not have this limitation and can be connected to other letters from both sides. With regard to the mentioned criteria, if the current block contains a left connected letter, the next block commences with no spaces; otherwise, the next block commences with one cell space. Considering a base line is the most important precondition for generating a well-formed sentence. To achieve this aim, three simple rules are added to each block's rules which are Considering a base line is the most important precondition for generating a well-formed sentence. To achieve this aim, three simple rules are added to each block's rules which are Rule#, Start, and End.Rule# is symbolized by a number and is composed of two parts itself in which the least significant digit shows the group of a

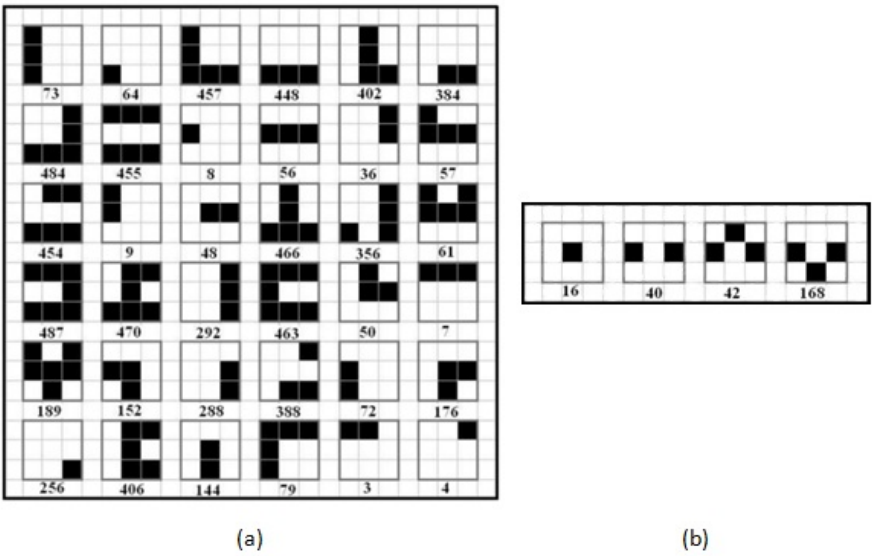


Fig. 12.6 All 114 Ma'qeli script patterns are generated by means of (a) 36 body rules and (b) four dot rules. The rule's number, which is written below each 3×3-cells, presents the sum of binary value of filled cells.

letter (Initial = 1, Medial = 2, Final = 3, and Isolated = 4) and other digit(s) shows the alphabet placement of that letter.

The Start and the End rules are presented in the form of ij, where i (i = 0, 1, …, 8) shows the ith tessellate in the letter block, and j (j = 0, 1, … 8) shows the jth cell in the ith tessellate. In fact, these rules regulate the initial and final cells of a block location such that each letter places on the base line. As an example, Table 1 tabulates 12 rules of a block for producing the letter "␣". The overall procedure of generating a string using Ma'qeli script is as follows below:

12.3.3 Holy Word Formation Using L-Systems

Word formation using L-Systems is a kind of turtle graphics in which there are three attributes including (I) a location, (II) an orientation, and (III) a pen, itself having attributes such as color, width, and up versus down. The turtle moves with commands that are relative to its position, such as "turn left 90 degrees". The pen carried by the turtle can also be controlled, by enabling it, setting its color, or setting its width. From these building blocks one can build more complex shapes like squares, triangles, circles and other composite figures. Combined with control flow, procedures, and recursion, the idea of turtle graphics is also useful in a Lindenmayer system for generating words, as we take it in hand.

L-system grammars are very similar to the semi-Thue grammar. L-systems are now commonly known as parametric L-systems, defined as a tuple G = (V , ω , P),

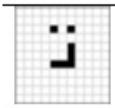
Figure	Rule#	B0	B1	B2	B3	B4	B5	B6	B7	B8	Start	End
	41	-	-	40	-	-	484	-	-	-	58	56
	42	-	40	-	-	448	457	-	-	-	58	46
	43	-	40	-	-	448	457	-	-	-	58	37
	44	-	40	-	-	448	73	-	-	-	56	37

Fig. 12.7 Different rules for producing the pattern of letter " " in Ma'qeli script

where V (the alphabet) is a set of symbols containing elements that can be replaced (variables), ω (start, axiom or initiator) is a string of symbols from V defining the

Generating a string using Ma'qeli script

Input: An array of string, Table of rules

Output: A string in the form of Ma'qeli Script

Output `Ma'qeli_Script` (**Input**)

Begin

Step1: Get the current character and search through the alphabet file to examine whether it is Persian or not

Step2: If the current character does not find in the alphabet file go to **End**

Step3: Scan the left and right neighborhoods of character in the string to find whether it is connective or not

Step4: Based on output of Step 3, select the proper Rule number from rule's table and perform **B0 - B8**, **Start**, and **End** rules

Step5: If the character belongs to the set of space, isolated, final, put a space in output by shifting current block with the amount of 1 cell

Step6: If there is any character go to Step 1.

End

initial state of the system, and P is a set of production rules or productions defining the way variables can be replaced with combinations of constants and other variables. A production consists of two strings, the predecessor and the successor. For any symbol 'A' in V which does not appear on the left hand side of a production in P, the identity production A $\rightarrow$ A is assumed; these symbols are called constants or terminals. The rules of the L-system grammar are applied iteratively starting from the initial state.

The grammars for producing 'Allah', 'Muhammad', and 'Ali' are as follows:

Case Allah

Variables= 'X';

Constants = 'F' 'f' '+' '-';

Start = '-FFFF-f-FFFF+f-F+f+F+F–fF-F-f-fffFF+FF–ffFF-FFFF+FF+FF+FFX'

Rules= 'X' '-ffffFFFF-f-FFFF+f-F+f+F+F–fF-F-f-fffFF+FF–ffFF-FFFF+FF+FF+FFX'

Angle=π/2;

Case Muhammad

Variables= 'X';

Constants = 'F' 'f' '+' '-';

Start ='-FF+FF+FF+FFFFF-FF+FFF+Ff+FFF–fffFF-ffF–f-F-F+f+F+Ff-ffFF-FF-FF-FF–ffFFF-FF+FF+ffff+FF+FFX'

Rules= '-FF+FF+FF+FFFFF-FF+FFF+Ff+FFF–fffFF-ffF–f-F-F+f+F+Ff-ffFF-FF-FF-FF–ffFFF-FF+FF+ffff+FF+FFX'

Angle= π/2;

Case Ali

Variables= 'X';

Constants = 'F' 'f' '+' '-';

Start ='++FF+FF+FF–ffFFF-FFF–fff-FF+FF+FF-FF-FFFFF-FFFFX'

Rules='X' '–fffffff-FF+FF+FF–ffFFF-FFF–fff-FF+FF+FF-FF-FFFFF-FFFFX'

Angle= π/2;

Fig. 12.8 Persian poems, which are written using Ma'qeli script

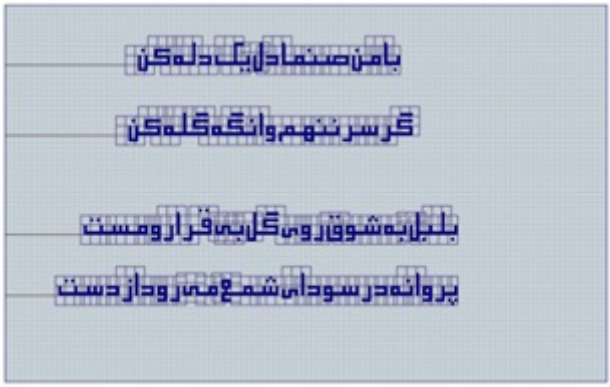


Fig. 12.9 Persian poems, which are written using Ma'qeli script. These outputs are generated in the 'Microsoft Visual Studio.Net 2008' environment.

12.4 Experimental Results

As is stated in the previous section, the proposed method has the capability of producing sentences without any limitation using block cellular automata and Margolus neighborhood such that the base line, as an underlying principle, is carefully observed. Figures 12.8 and 12.9 illustrate two Persian poems, which are written using Ma'qeli script. In order to demonstrate the efficiency of the proposed method, several experiments are conducted, and the capability of generating letters, words, and sentences are examined by means of 1D and 2D synchronous cellular automata with different rules and neighborhoods.

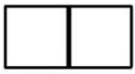


Fig. 12.10 1D synchronous cellular automata in which the rules number 4 is utilized to produce Aleph' environment

12.4.1 Ma'qeli Script Generation Using 1D Synchronous Cellular Automata

1D elementary cellular automata are the simplest form of automata with different applications; however, they have several problems in situation in which they are utilized in generating Ma'qeli script. The most challenging reason is that they should obey predefined rules. In this manner, Aleph, which is written in the form of "ا" is the only letter that can be produced by this type of automata in which the rule's number 4, 12, 36, 44, 68, 76, 100, 108, 132, 140, 164, 172, 196, 204, 228, or 236 needs to be utilized. Figure 12.10 illustrates one cell with rule number 4 for generation Aleph pattern.

Besides the mentioned limitation, this scheme has other shortcomings which are extracting rules for each letter is time consuming and difficult. Also even by utilizing the set of complicated rules, problems concerning the words production are considerable. These problems occur as a result of similarity between letters, and their different appearance form in a word.

12.4.2 Ma'qeli Script Generation Using 2D Synchronous Cellular Automata

The nature of letters and weakness of 1D cellular automaton directed us to examine 2D cellular automata in which three neighborhoods including von Neumann, Moore, and extended Moore are considered for generating Ma'qeli letters and words. Although these neighborhoods have the capability of using in letter and word generation, they suffer from defects, which without loss of generality we describe them by a sample.

The von Neumann neighborhood (see Fig. 12.4 a) can facilitate generating all letters through the use of several rules, although it is not free from defects. As shown in Fig. 12.11, the process of generating the pattern of letter "ـ" faces a major problem regarding the production of the underneath dot, along with the high time complexity and a bad-formed illustration for this letter. This letter can be generated by the following transitional rules:

Moreover, this neighborhood cannot produce words in view of the fact that the word generation process needs to observe the location of each letter based on the

Transitional rules for generation the pattern of letter "ب" using von Neumann neighborhood

1.**for**n=1 to 2 **do**
  a.**if**(N= =1) **then** S=1 **do**
    // where N and S stand for North and South neigh-borhoods, respectively.
1.**for**n=1 to 5 **do**
  a.**if**(E= =1) **then** W=1 **do**
    //where E and W stand for East and West neigh-borhoods, respectively.
1.**for**n=1 to 2 **do**
  a.**if**(S= =1) **then** N=1 **do**

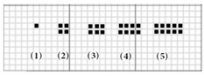


Fig. 12.11 The process of generating the pattern of letter "ب" using von Neumann neighborhoods.

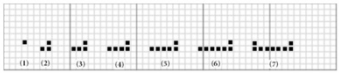


Fig. 12.12 The process of generating the pattern of letter "ب" using Moore neighborhoods.

previous letter, which is impossible in a considerable number of cases using the structure of von Neumann neighborhood. Despite the fact that the Moore neighborhood checks more cells than von Neumann does, it is observed in the course of experiments (Fig. 12.12) that it can produce Kufic form of letter "ب" but it still suffers from same defects and can only reduce the steps of producing letters a bit.

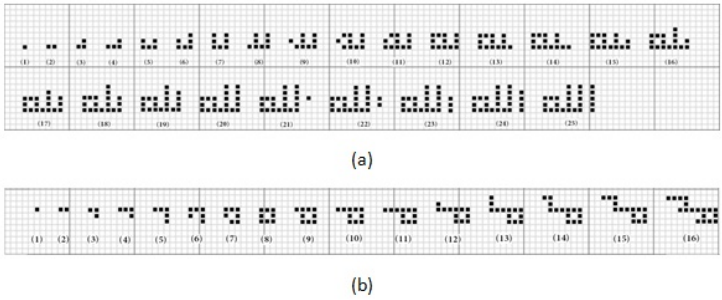
This letter can be produced by the following transitional rules:

In order to resolve the mentioned weakness, the impact of using extended Moore neighborhood is examined, in which the cellular automata can supervise 25 cells simultaneously. In light of the fact that defined rules for this neighborhood have the capability of regulating more cells, it seems to have got rarely eased the process of word generation.

Minoofam and Bastanfard [14] utilized this neighborhood with the aim of simulating the holy word of 'Muhammad' with Ma'qeli script and this neighborhood. They reported this procedure takes 25 steps of rule execution, and the results are promising (see Fig 12.13a. In order to achieve this aim, they combined predefined rules with XOR operator ($\otimes$), which are as follows below: (Consider a central cell,

Transitional rules for generation the pattern of letter "ب" using Moore neighborhood

1.**1.if**(N= =1) **then**
{
S=1;
SW=1; // where SW stands for South-West neigh-borhood
}
2.**for** i=1 to 5 **do**
if(E= =1) **then**
{
W=1;
NW=1; // where NW stands for North-West neighborhood
}
if(S= =1) **then N=1;**



(a)

(b)

Fig. 12.13 The procedure of generating (a) 'Allah' and (b) 'Muhammad' using Ma'qeli script and extended Moore neighborhood

Ci, and apply the following rules to: E, W, N, and S stand for East, West, North, and South neighborhood, respectively.)

Along the lines of that work, the rules for generating holy word of 'Muhammad' is produced by this experiment; however, it is observed that rule which is utilized for generating the letter "ح" deforms the shape of letter "م" which places at the beginning of this word on account of similarity with other rules, see Fig. 12.13b. Rules for producing holy word 'Muhammad' with XOR operator in Ma'qeli script are as follows: (As the rule number 10 and rule number 15 follow the same structure, procedure of generating holy word 'Muhammad' faces problem in step 15.)

With respect to the obtained results, it is reasonable to claim that the proposed method has a considerable number of advantages over the observed competitive methods. For instance, as the proposed method works based on the separated rules, generating patterns of a word can be assigned to a parallel machine. Nevertheless, for other neighbors an adequate algorithm should be design which in the current

Rules for producing holy word 'Allah' with XOR (⊗) operator in Ma'qeli script	
1.1 Ei	1.13. Wi⊗WWi⊗NWWi⊗NNWWi
1.2. Si SWi	1.14. Si SWi SWWi WWi NWWi
1.3. Ei⊗ EEi ⊗NEEi	1.15. Si ⊗SS ⊗SSWWi ⊗SSEEi ⊗SWWi
1.4. Si ⊗SEi⊗SEEi ⊗EEi	1.16. Si⊗SSi⊗SSWi⊗SSWWi⊗SWWi⊗WWi⊗NWWi
1.5. Si⊗SSi⊗SSWi⊗SSWWi⊗SWWi	1.17. Si⊗SSi⊗SSEEi⊗SSWWi
1.6. Si⊗SSi⊗SSEi⊗SSEEi⊗SEEi⊗EEi	1.18. Si⊗SSi⊗WWi⊗NWWi⊗SWWi⊗SSWWi
1.7. Ei⊗EEi⊗NNEi⊗NEi	1.19. Si⊗SSi⊗SSWWi⊗SWWi⊗WWi
1.8. SEi⊗SEEi⊗EEi⊗NEEi	1.20. WWi⊗NWWi⊗NNWWi⊗SWWi⊗SSWW
1.9. SWi⊗SEi⊗SSi⊗SSEi⊗SSEEi⊗Ei	1.21. Ni⊗NWWi⊗WWi⊗SWWi⊗NNWWi
1.10. Ei⊗EEi⊗Ni⊗NEEi⊗NNEi⊗NNEEi	1.22. WWi⊗NWWi⊗NNWWi⊗Ni⊗NNi
1.11. Si⊗SSi⊗SSEi⊗SSEEi⊗SEEi⊗Ei⊗EEi	1.23. Si⊗SSi⊗WWi⊗NWWi⊗SWWi⊗SSWWi
1.12. Wi⊗WWi⊗NWi⊗NNWii	1.24. Si⊗SSi⊗WWi⊗SWWi⊗SSWWi

state, it seems to be impossible. Extendibility and rule extraction are two important criteria for comparing letter generation algorithms. Extendibility means the extracted role can be applied to produce other languages letters or pattern with a bit modification, and this issue can be seen in Margolus neighbor. As is shown in this paper, the extracted rules for Von Neumann, Moore, and extended Moore neighbors for generating letters as well as words are very complicated and in some cases are not applicable for word generation. Therefore, it is reasonable mentioning that the propose algorithm have simple rules and extendable.

Despite the fact that the obtained results are very favorable, and the proposed method tends to have the capability of being utilized in generating similar pattern, we should mention that the proposed method is not free from shortcomings. Initial and final forms of "ڢ" and "ڤ" cannot be produced, albeit this problem can be traced back to the predefined pattern for these letters in Ma'qeli script. If the size of letters is allowed to be modified this problem can be solved adequately; however, this alternation violates the infrastructure of this script.

Rules for producing holy word 'Muhammad' with XOR (⊗) operator in Ma'qeli script	
1.1 Ei	1.9. Ei⊗EEi⊗SEEi⊗SSEEi
1.2. Ni⊗NWi	1.10. Ei⊗EEi
1.3. Ei⊗EEi⊗SEEi	1.11. Si⊗SEi⊗SEEi
1.4. Ni⊗NNi⊗NNWi⊗NNWWi	1.12. Si⊗SSi⊗SSEi⊗SSEEi
1.5. Ni⊗NEi⊗NNEEi⊗EEi⊗SEEi	1.13. Ei⊗SEi⊗SSEi⊗SSEEi
1.6. Ei⊗NEi⊗NNEi⊗NNi⊗NNWi	1.14. Ei⊗EEi⊗SEEi⊗SSEEi
1.7. Ni⊗NNi⊗NNEi⊗NNEEi⊗NEEi⊗EEi⊗Ei	1.15. Ei⊗EEi
1.8. Ei⊗SEi⊗SSEi⊗EEi⊗SSEEi	

12.4.3 Holy Words Formation Using L-Systems

As is stated in the section 3.3, the proposed method has the capability of producing 'Allah', 'Muhammad', and 'Ali' using L-Systems. Figure 12.14 illustrates these words, which are written in Ma'qeli script.

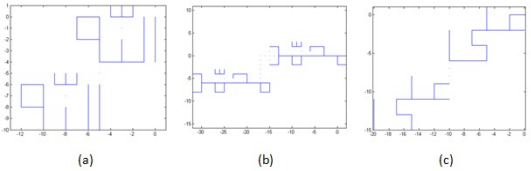


Fig. 12.14 Holy words formation using L-Systems (a) Allah, (b) Muhammad, and (c) Ali

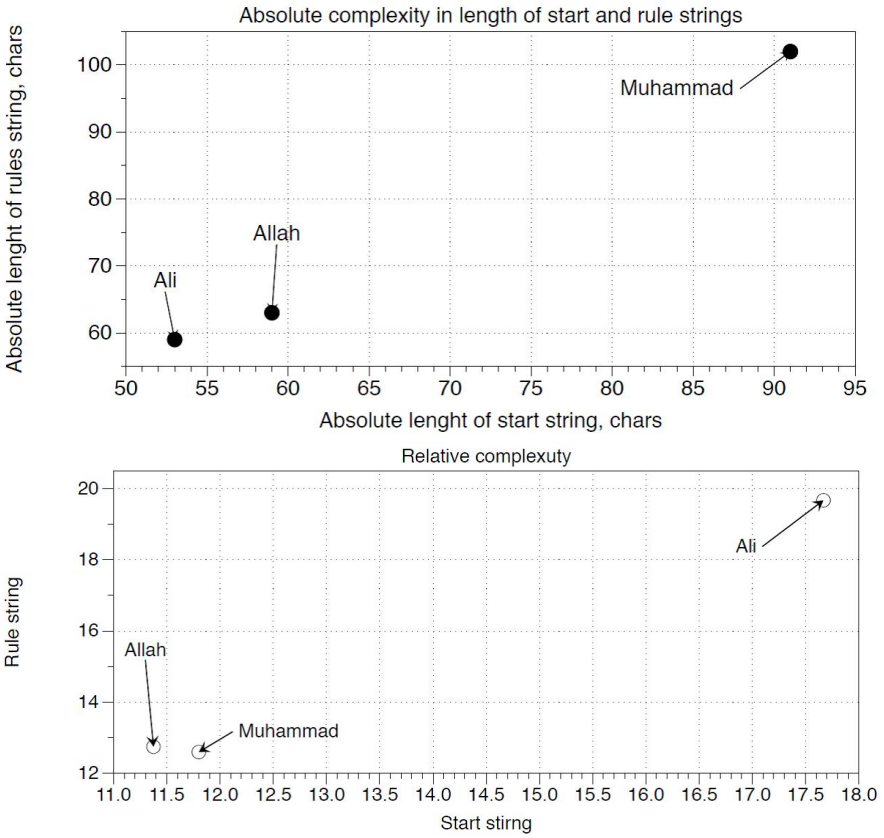


Fig. 12.15 (a) Absolute complexity. (b) Relative Complexity [1].

In order to demonstrate the efficiency of the proposed method, we calculate the absolute and relative complexity. Then we rewrite the rules in compressed format. For calculating absolute complexity, we count the length of the start and rule of each grammar; then, let us relative complexity which is absolute complexity divided by number of chars in Latin spelling of words. Therefore, we have absolute complexity as follows:

- Allah: Start (59), Rule (63)
- Muhammad: Start (91), Rule (102)
- Ali: Start(53), Rule (59)

And, the relative complexities are as follows:

- Allah: Start (11.8), Rule (12.6)
- Muhammad: Start (11.375), Rule (12.75)
- Ali: Start(17.6666666667), Rule (19.6666666667)

Figure 12.15 shows these two kinds of complexities.

12.5 Conclusion

We design cellular automaton and L-Systems based algorithms for generating the ancient Persian script Ma'qeli. We demonstrate how letters of the script can be produced using block cellular automata with Margolus neighborhood. We found a set of optimal (in terms of generation time and richness of letters/words produced) rules to generate complex Persian cursive words. The contribution towards this study has a number of advantages, and has the capability of applying to several domains. First, the script, as a graphical primitive, can be used in computer games and animations. The script can also be used in industrial applications when displaying sentences in 7-segments or dot matrix monitors. We also envisage a possible usage in CAD software in case of cultural heritages, handicrafts and the calligraphies woven through the carpets. In order to demonstrate the efficiency of the proposed method, a set of experiments is conducted on generating holy words 'Allah' and 'Muhammad' with von Neumann, Moore, and Extended Moore neighborhoods so as to compare with Margolus neighborhood. It was observed in the course of experiments that the proposed method is simple for generating words and sentences, as it can produce each letter with just one rule. In conclusion, it is worth mentioning that using L-Systems can eliminate the overhead of constructing initial blocks in the proposed method.

References

1. Adamatzky, A., Bull, L.: Are complex systems hard to evolve? Complexity 14(6), 15–20 (2009)
2. Ahuja, M., Loeb, L.A.: Tessellations in Islamic calligraphy. Leonardo 28, 41–45 (1995)

3. Arata, H., Takai, Y., Takai, N.K., Yamamoto, T.: Free-form shape modeling by 3D cellular automata. In: International Conference on Shape Modeling and Applications, pp. 242–247. IEEE Computer Society (1999)
4. Bentley, K., Cox, E.J., Bentley, P.J.: Nature's batik: a computer evolution model of diatom valve morphogenesis. Journal of Nanoscience and Nanotechnology 5, 25–34 (2005)
5. Chavoya, A., Andalon-Garcia, I.R., Lopez-Martin, C., Meda-Campaña, M.E.: 3D cell pattern generation in artificial development. In: González, J.R., Pelta, D.A., Cruz, C., Terrazas, G., Krasnogor, N. (eds.) NICSO 2010. SCI, vol. 284, pp. 127–139. Springer, Heidelberg (2010)
6. Chavoya, A., Duthen, Y.: A cell pattern generation model based on an extended artificial regulatory network. Biosystems 94, 95–101 (2008)
7. Culik, I.K., Hurd, L., Yu, S.: Formal languages and global cellular automaton behavior. In: Howard, G. (ed.) Cellular Automata, pp. 396–403. MIT Press (1990)
8. Hearn, D., Baker, P.M., Carithers, W.: Computer Graphics with Open GL, 4th edn. Prentice Hal (2010)
9. Ilachinski, A.: Cellular Automata: A Discrete Universe. Emerald Group Publishing Limited (2001)
10. Kari, J.: A counter example to a conjecture concerning synchronizing words in finite automata. In: Bulletin of the European Association for Theoretical Computer Science, p. 146 (2001)
11. Kari, J., Moore, C.: New results on alternating and non-deterministic two-dimensional finite-state automata. In: Ferreira, A., Reichel, H. (eds.) STACS 2001. LNCS, vol. 2010, pp. 396–406. Springer, Heidelberg (2001)
12. Lindenmayer, A., Prusinkiewicz, P., Hanan, J.: Development models of herbaceous plants for computer imagery purposes. SIGGRAPH Computer Graphics 22, 141–150 (1988)
13. Markus, M., Kusch, I.: Cellular automata for modelling shell pigmentation of molluscs. Journal of Biological Systems 3, 999–1011 (1995)
14. Minoofam, S.A.H., et al.: Ad-hoc Ma'qeli Script Generation Using Block Cellular Automata. Journal of Cellular Automata 7(4) (2012)
15. Moustapha, H., Krishnamurti, R.: Arabic calligraphy a computational exploration. In: 3rd International Conference on Mathematics and Design, pp. 294–306 (2001)
16. Nasu, M.: Local maps inducing surjective global maps of one-dimensional tessellation automata. Theory of Computing Systems 11, 327–351 (1977)
17. Nehaniv, C.: Self-reproduction in asynchronous cellular automata. In: Proceedings of the 2002 NASA/DoD Conference on Evolvable Hardware (EH 2002), p. 201 (2002)
18. Oya, T., Motoike, I.N., Asai, T.: Single-electron circuits performing dendritic pattern formation with nature-inspired cellular automata. International Journal of Bifurcation and Chaos 17, 3651–3655 (2007)
19. Piwonska, A., Seredynski, F.: Discovery by genetic algorithm of cellular automata rules for pattern reconstruction task. In: Bandini, S., Manzoni, S., Umeo, H., Vizzari, G. (eds.) ACRI 2010. LNCS, vol. 6350, pp. 198–208. Springer, Heidelberg (2010)
20. Rozenberg, G.: The Mathematical Theory of L Systems, vol. 90. Academic Press (1980)
21. Sakkal, M.: Art of Arabic Calligraphy (1993), `http://www.sakkal.com/ArtArabicCalligraphy.html`

22. Toffoli, T., Margolus, N.: Cellular Automata Machines: A New Environment for Modeling. MIT Press (1987)
23. Wolfram, S.: Cellular Automata and Complexity: Collected Papers. Westview Press (1994)
24. Xu, S., Lau, F., Pan, Y.: A computational approach to digital Chinese painting and calligraphy. Springer (2009)

Chapter 13
Interactive Cellular Automata Systems for Creative Projects

Angus Graeme Forbes

Abstract. The use of cellular automata as a mechanism for generating captivating and unpredictable creative output has been explored by artists and composers since the introduction of the "Game of Life" by the mathematician John Conway and its subsequent popularization via the writings of Martin Gardner. This chapter surveys notable creative projects that incorporate cellular automata systems in order to generate new aesthetic possibilities. In particular, it describes a recent project, *Fluid Automata*, that utilizes a cellular automata system to create fluid simulations and to enable a range of expressivity in different interactive environments.

13.1 Creative Projects Based on Cellular Automata Systems

Cellular automata (hereafter, CA) systems have been used as a mechanism for generating creative output by artists and researchers since the introduction of the "Game of Life" by the mathematician John Conway and its subsequent popularization via the writings of Martin Gardner [33]. These include practitioners in a wide range of fields, such as musical composition [7, 15, 20], urban design [58], cognitive science [66], video game programming [3, 5], and media arts [12, 24, 41].

A rich tradition of using algorithms to generate creative content exists in both music composition and the visual arts. For instance, artists such as Duchamp and writers and artists of the Ouilpo movement placed constraints on their art-making process in order to generate more focused possibilities from a reduced set of choices [17, 43]. The artist Sol LeWitt created algorithmically inspired instructions for the creation of his paintings [39]. Composers from Mozart to John Cage have used randomness as a generator of musical themes [49]. More recently, 20th-century composers, including Xenakis, have developed complex algorithmic compositional systems that guided the creation of their musical scores [67].

Angus Graeme Forbes
University of Arizona, USA
e-mail: angus.forbes@gmail.com
http://angusforbes.com, http://creativecodinglab.com

P. Rosin et al. (eds.), *Cellular Automata in Image Processing and Geometry*, Emergence, Complexity and Computation 10,
DOI: 10.1007/978-3-319-06431-4_13, © Springer International Publishing Switzerland 2014

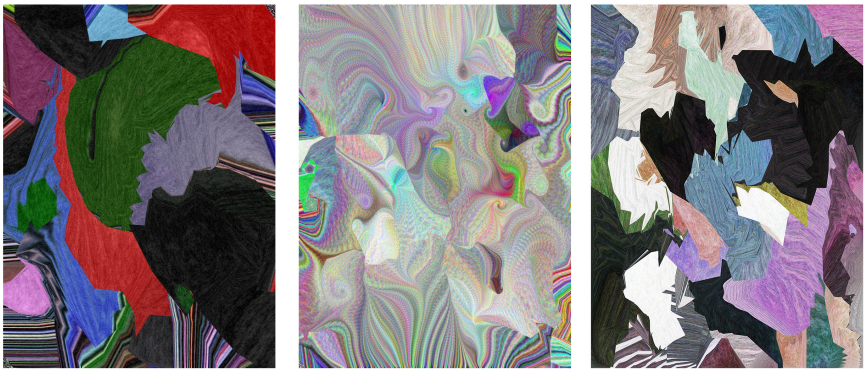


Fig. 13.1 Examples of output from an iPad application utilizing the Fluid Automata system showing the wide range of aesthetic possibilities generated by the system when using different fluid profiles defined by customizing the fluid, visualization, and noise parameters

In a series of essays about the rise of new media art in the 1960s, Edward Shanken explores the ways in which the first media artists developed art installations that were influenced by "cybernetic systems" [52]. These cybernetic works encouraged viewers to think about the relationship between hardware and software, and the metaphorical parallels between the physicality of structure and the emergence of consciousness [53]. Conway's "The Game of Life" explicitly invokes the idea that emergent, like-like structures emerge over time out of simple rules, similar to the way that human consciousness, through a much more complicated series of steps, evolved over of millennia of mutation and selection.

An attractive aspect of CA is its simplicity. Through their discretization, time and space are simplified; everything happens simultaneously, in uniform steps, and within a small uniform grid. Thus the complexity that emerges from the simple rule set governing the system is, in a sense, more tractable. An array of unexpected structures with seeming agency and life-like behaviors (e.g., breeders, gliders, rakes, replicators, etc.) can be explored, analyzed, and modified. For instance, the researcher Dave Greene recently indicated that he successfully built a CA system (in a grid of $14,826,990^2$ cells) that creates a complete copy of itself in $237,228,617$ iterations [34].

Some artists have directly incorporated CA systems into their work. These include Dale Millen's *Cellular Automata Music* [42] and a series of compositions by Peter Beyls that explore the mapping of CA systems to musical structure [8]. Iannis Xenakis used CA in some of his later compositions [54, 67]. Surveys by Dave Burraston and Ernest Edmonds provide a more thorough review of the history of CA systems and composition [13, 14]. Philip Galanter, Margaret Boden, and Ernest Edmonds discuss how CA is more generally situated in the context of "generative art," demonstrating that other processes with emergent behavior are similarly inspirational to artists, including: Belousov-Zhabotinsky reactions [19]; fractal geometry; chaotic attractors; Lindenmayer systems [45]; flocking algorithms; and many other

self-organizing systems based on iterative processes [10, 28]. In particular, Philip Galanter defines a clear distinction between rule-based art and generative art. Rule-based art may not have the "sufficient specificity or autonomy to allow the artist to give up control to the rules, and allow [the rules] to take over and produce the art" [28]. That is, he writes, "the key element in generative art is then the system to which the artist cedes partial or total subsequent control" [27].

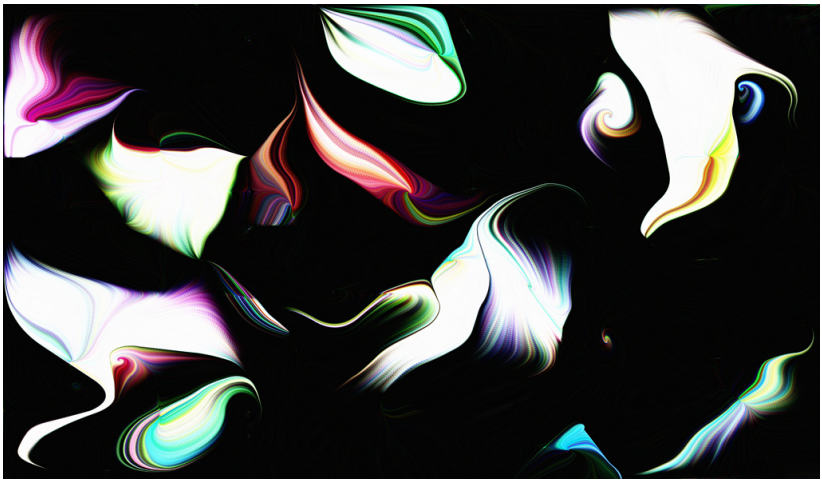


Fig. 13.2 Example output using a custom color palette with high-contrast image processing variables

Various artists and researchers have extended the type and number of elements in CA systems beyond the basic 2D, binary system introduced by Conway. For example, Ritesh Lala's artwork, *Morphon*, uses a 3D CA system to create pleasing visual effects [40]. An implementation developed by Stephan Rafler called "SmoothLife" implements "circularly shaped integrals" in order to situate CA in the continuous domain [46]. Other modifications use non-uniform grids or multiple colors in order to achieve particular effects, such as cyclical behaviors [38]. Paul Brown uses CA systems to develop his "kinetic paintings" [1], and numerous recent contemporary works use or represent generative systems as a primary component. For instance, Casey Reas has created a series of generative works collected in his "Process Compendium Catalog" [47] and described in his "Process Lecture" [48]. George Legrady has produced works that represent generative processes, such as *Kinetic Flow* [41], and one installation, *Blink*, demonstrates "the process of self-organization in a vast matrix of eyes that open and close according to their neighbors' behaviors" [44]. Philip Galanter has also produced generative artworks, including his lightbox drawings and interactive sculptures [29, 30].

Interactive simulations of CA have been available as software since the 1970s, and some artworks using CA methods also incorporate direct interaction. In most cases, the interaction is straightforward, allowing a user with a mouse to change

the state of single cell, or perhaps to add a "gun" or "glider" to the CA system at a particular location. A somewhat more involved way to interact with a CA system is demonstrated in a series of works by the installation artist Bill Vorn. Vorn presents a large CA grid that can be controlled by standing at different locations throughout the installation which are mapped to the cells in the CA system [61]. The artist Charlie Roberts also uses a CA grid as an example of a dynamic interface used to control networked compositions via mobile devices [50, 51].

The next section introduces the Fluid Automata system, based on CA, and subsequently describes technical details about the system, including the simulation engine, the visualization methods, and the single-user and multi-user interaction techniques used in different deployments. It then describes a collaborative art installation, *Annular Genealogy*, involving musical and visual outputs that is based on the Fluid Automata system.

13.2 The *Fluid Automata* Project

The *Fluid Automata* system explores the use of fluid simulation as an aesthetic component for interactive art. While fluid simulation is often used in scientific visualization applications in order to provide a practical understanding of how fluids behave in particular contexts, the use of fluid simulations to create realism and excitement in entertainment contexts is also a widely explored topic. Moreover, fluid systems have been used as a technique in generative media arts contexts, and a number of recent art projects utilize fluid simulation as a component of the work. A method created by Jos Stam in 1999 to create a stable fluid system first made it possible to represent realistic looking fluids at real-time frame rates [55, 56]. Many interactive artworks have made use of this particular technique. For instance, Memo Atken has created a series of demonstrations based upon Stam's method, showcasing them using mobile devices for interaction and making the code available for the OpenFrameworks and Processing multimedia frameworks [4]. Another project that incorporates Stam's method is Graham Wakefield and Haru Ji's *Artificial Nature*. This project uses computer vision techniques to allow participants to interact with a 3D fluid representation through the movement of their bodies [62]. The *Fluid Wall* installation by Naureen Mahmood and Austin Hines similarly enables users to manipulate a wall of fluids using a Kinect interface [6]. Other fluid simulation methods, such as one presented by Martin Guay, et al., are optimized for real-time interaction in video games [36]. Although simulation methods generally focus on producing accurate representations of natural systems, the Fluid Automata system demonstrates that aesthetically-interesting visuals with a wide variation of movement and color can be produced from a simple set of rules that do *not* attempt to exactly reproduce natural systems. In this sense, although influenced by physical simulation, Fluid Automata could more accurately be considered a work of generative art that utilizes a custom CA system [10, 27].

Scientific visualization projects aim to help researchers identify and reason about salient aspects of their data. While an aesthetic sensibility may contribute to the

success of a visualization technique, this is not normally the primary motivation for its creation. Media artists, on the other hand, generally place aesthetic considerations at the forefront of their concerns. Similarly, physical simulations are concerned with accuracy and realism rather than extensibility and interaction, which are central to media arts. However, media arts projects, due to time constraints or limitations in technical knowledge, often incorporate readily-available techniques not originally intended for artistic production, and therefore not as easily adaptable to artistic situations. Fluid Automata provides a flexible system that can be used in many situations, including on less powerful mobile devices, while retaining enough realism to be useful for simulations. Figure 13.1 shows examples of different outputs created with the Fluid Automata system.

13.3 Simulation and Visualization in the *Fluid Automata* Project

The Fluid Automata system is composed of three integrated components: a fluid simulation engine; a flow visualization technique; and an interface that encourages interaction with the visualized fluid simulation [25]. Here I look at the technical details of the fluid simulation and flow visualization components that are shared across multiple deployments of the project.

13.3.1 Fluid Simulation

Fluid Automata introduces a novel fluid simulation technique, inspired by CA systems. While its title was inspired by Tibor Gánti's discussions of "chemotons" [31, 32], the initial kernel of insight for the Fluid Automata system arose from thinking about how to create a simple rule-based system to produce emergent behavior. CA systems demonstrate complex behavior emerging through an iterative system that updates the state of each element (positioned in a uniform grid) based on a set of basic rules. These rules determine the next state of each element by querying each of its neighbors. In Conway's original automata system, each pixel has a binary state, and either "lives" (is set to 1) or "dies" (is set to 0) based upon the number of surrounding pixels that are on or off. Other CA systems, including many that are examined in Stephen Wolfram's "A New Kind of Science" [64], utilize different rule sets and involve multiple states. In particular, a range of CA approaches to fluid simulation have been introduced. For instance, an early paper by Gerard Vichniac describes discretized models for fluid simulation based on CA, but indicates that precise values for specific parameters are necessary in order to avoid "divergences" [60]. Another paper surveys different CA paradigms for fluid simulation, comparing lattice gas, digital fluid, and lattice-Boltzmann strategies [16]. A 1998 paper describes 3D fluids that can be represented with hexagonal and rhombododecahedron grids instead of uniform squares or cubes [11]. Stephen Wolfram also explored general non-linear approximations of fluids via CA [63].

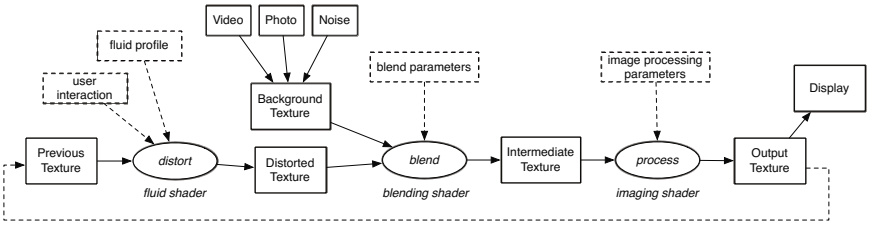


Fig. 13.3 Schematic for the main functionality of the Fluid Automata system. The output texture after one timestep becomes the input for the next timestep.

Unlike Conway's Game of Life and other early CA in which each cell contained only binary information, Fluid Automata's 8-bit CA system contains $2^8 \cdot 2^8$ states per cell: 256 values for the magnitude of a fluid vector and 256 states used to describe its orientation. Retaining the discretization and simplicity central to CA systems makes it possible to create a complex system that is linear and replicable and that effectively simulates the movement of fluid. Unlike the majority of commonly-implemented fluid simulations which utilize the non-linear Navier-Stokes equations, the Fluid Automata algorithm is always stable at any length of timestep. That is, the system is inherently non-realistic since it lacks a mass conservation condition and because the fluid is compressible. Although the system is not physically accurate, it has the advantages of being easy to modify in real-time and relatively straightforward to implement, leading to its incorporation in a range of projects. It distributes a flow of energy throughout the system as follows:

1. The screen is divided into a grid of cells.
2. New energy is added into the grid by user interaction.
3. The energy at each cell is split into three streams, a forward stream and a left and a right stream.
4. In each of the defined directions for each stream defined in step 3, the energy of each cell is moved into the neighboring cell via the following process:
   a. The cell is displaced along the vector describing the energy stream.
   b. For each neighboring cell the displaced cell intersects with, create a "partial" vector by scaling the original vector with the amount of intersection.
   c. This partial is added to the cell it intersects with.
5. When this process has been completed for the entire grid, the partials associated with each cell are combined, creating a new vector that replaces the current vector in the cell.
6. Energy is removed from the system by scaling the energy in each cell by a dampening factor.
7. Steps 2 through 6 are iterated at each timestep until there is no energy left in the system.

More formally, the fluid system acts on a grid G of cells $C_{i,j}$ each containing an energy vector $\vec{E}_{i,j}$, with a particular magnitude m and orientation θ, and where $0 < i < columns$ and $0 < j < rows$. In order to define a "fluid profile" for the system, two ratios are introduced that regulate that behavior of energy as it moves through the cells. The first, the *momentum* ratio, or r_1, defines how much energy moves forward versus moving to the sides. The second, the *directionality* ratio, or r_2, defines how much energy moves to the left versus moving to the right. A parameter, *angularity*, is defined as $\angle\phi \in [0, \pi)$, describing a rotation offset from $\angle\theta$. Additional parameters influencing the fluid profile are the *viscosity* of the system, which acts as a dampening factor defining the rate at which energy is removed from the system, and the *sensitivity* which controls how much energy is added to the system through some form of user interaction. New energy is added into the cells in a particular direction using the multi-touch capabilities of the tablet device. The amount of energy that is added in a particular touch depends upon the *sensitivity* parameter, which can be adjusted during runtime. The magnitude of energy is also determined by how far the current position of the touch is from its previous position. This difference also determines the direction of the added energy. If there is no change in position, then the energy is added in the last known direction. This vector of new energy is added with any existing energy at the currently touched cell to update $\vec{E}_{i,j}$.

At each timestep t during the operation of the fluid system, the energy $\vec{E}$ in $C_{i,j}$ is split into three separate streams: a forward stream, $\vec{F}$, and two "orthogonal" streams, $\vec{L}$ and $\vec{R}$. Using the current fluid profile (the values for the parameters of *momentum, directionality*, and *angularity*), and the current magnitude and orientation for each cell, I define these three streams like so (in Equations 3.1 through 3.3):

$$\vec{F} = \begin{pmatrix} r_1 m \\ \theta \end{pmatrix} \tag{13.1}$$

$$\vec{L} = \begin{pmatrix} (1-r_2)(1-r_1)m \\ \theta + \phi \end{pmatrix} \tag{13.2}$$

$$\vec{R} = \begin{pmatrix} r_2(1-r_1)m \\ \theta - \phi \end{pmatrix} \tag{13.3}$$

Figure 13.5 shows examples of how the current total energy in a cell is split into three streams based on these parameters. Every cell in G thus contains three separate streams of energy, $\vec{F}$, $\vec{L}$, and $\vec{R}$. These streams are used to define the flux of energy moving from each cell into its neighboring cells. The calls to the StreamVector procedure in the code listing in the Appendix also describe how energies are split into different vectors based on r_1 and ϕ.

A "displaced" cell $D(C, \vec{v})$ is a copy of a cell $C \in G$ that moves along a vector $\vec{v}$ positioned at the center of C. This displaced cell D intersects with between 1 and 4 cells (the original cell C itself and up to three neighboring cells). The magnitude of any energy vector is constrained to range between 0 and 1. When displacing a cell, the magnitude is scaled by the length of a cell side. For instance, if the grid is

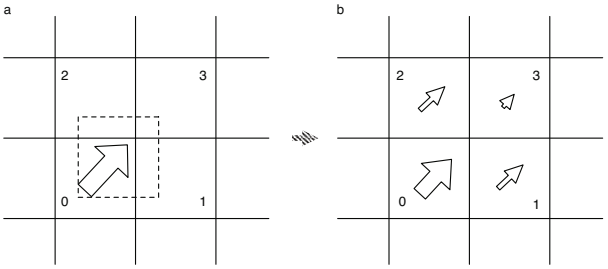


Fig. 13.4 Example showing how a single stream of energy in a single cell is distributed to other cells. In this example, the left grid (a) indicates that the energy in cell 0 is pointing in the direction $\pi/4$ with a magnitude of 0.5. The dotted box shows where the "displaced" cell intersects with its neighbors. The largest intersection of energy stays within the original cell (cell 0); a tiny amount of energy is pushed into cell 3; and a small amount of energy is pushed into cells 1 and 2. The right grid (b) shows the distribution of energy from this single cell moved into its neighbor cells and back into itself.

divided into $10x10$ cells, then each cell side is .1 in length, and so the magnitude in a cell is scaled by 1/10th before being displaced. This ensures that a displaced cell will only ever intersect a cell that is its immediate neighbor (although exceptions to this constraint may have interesting creative possibilities.) The amount of overlap between the displaced cell and the cell it intersects with determines how energy is placed into that cell. An intersection $I(D,C)$ is simply the amount of overlap between the displaced cell and another cell (Equation 4). The value of any intersection can range between 0.0 (no intersection) and 1.0 (full overlap).

$$I(D,C) = \Big(Area(D) \cap Area(C)\Big)/Area(C) \tag{13.4}$$

This value is used to create a "partial" vector $\vec{p}$ via a function $P(N,\vec{v},C)$ for each energy stream. This partial is calculated simply by scaling the stream by the amount it overlaps with the current cell, as defined in Equation 5:

$$P(N,\vec{v},C) = I(D(N,\vec{v}),C) * \vec{v} \tag{13.5}$$

In Equation 5, N refers to a neighbor cell of a cell $C \in G$ and $\vec{v}$ refers to one of the energy vectors ($\vec{F}$, $\vec{L}$, or $\vec{R}$) in that neighbor cell. Figure 13.4 depicts an example of this displacement and the subsequent generation of partials. For each cell $C_{i,j}$ in G, each neighbor is examined to determine how much each of its three streams overlap. The partials created by any intersection between the neighbor streams are summed to generate the new energy vector for the cell (Equations 6 through 9). It is important to note that a cell is considered to be a neighbor of itself. For instance, if a vector of magnitude 0.5 is pushed upwards at 90 degrees, it would intersect with both the current cell and the neighbor cell above it. Since the displaced cell would move a distance of 50% of its height from its current position, it would end up intersecting the current cell and the neighbor cell equally, and thus a copy of the vector, scaled by

50%, would be placed in each of the cells. Figure 13.4 shows a typical example of a single stream of energy in a single cell being distributed to its neighbors. Excluding any dampening factor or any new energy added from user interaction, the system will retain exactly the same amount of energy over each timestep. The PARTIAL and MAKERECT procedures in the code listing provided in the Appendix to this chapter also describe how partials are created through this displacement process.

$$\vec{F}_{i,j,t} = \sum_{p=i-1}^{i+1} \sum_{q=j-1}^{j+1} P(C_{p,q}, \vec{F}_{p,q,t-1}, C_{i,j}) \tag{13.6}$$

$$\vec{L}_{i,j,t} = \sum_{p=i-1}^{i+1} \sum_{q=j-1}^{j+1} P(C_{p,q}, \vec{L}_{p,q,t-1}, C_{i,j}) \tag{13.7}$$

$$\vec{R}_{i,j,t} = \sum_{p=i-1}^{i+1} \sum_{q=j-1}^{j+1} P(C_{p,q}, \vec{R}_{p,q,t-1}, C_{i,j}) \tag{13.8}$$

$$\vec{E}_{i,j,t} = \vec{F}_{i,j,t} + \vec{L}_{i,j,t} + \vec{R}_{i,j,t} \tag{13.9}$$

Other parameters can also be adjusted to create different characteristics for a particular fluid profile. These include: controlling the "jitter", or randomness of the system, and clamping the maximum outflow for the cells within the grid. I also experimented with a toroidal representation of the system where fluid energy wraps around the edges of the screen, instead of bouncing off the edges (that is, $x_{max+1} = x_0$ and $x_{min-1} = x_{max}$). Setting a maximum outflow parameter interestingly creates the sense of ice cracking and melting when a particular threshold is exceeded. Different settings of *viscosity* and *angularity* can create more or less turbulent behaviors. The iterative nature of the system does in fact create a wide variety of fluid-like structures, including the creation of eddies, vortices, and turbulence. Figure 13.1 and Figure 13.6 show examples of fluid systems with different fluid profiles defined by different settings for the *angularity* and *directionality* parameters. The Appendix to this chapter provides a code listing that describes how a cell is updated via querying its neighbors.

Just as simulations for realistic films and video games do not feel constrained by a perfect representation of the physics of a visual effect, so to should media artists not feel constrained by a perfect representation of existing algorithms and equations for a particular kind of effect. By creating a custom fluid system with a wide range of parameter adjustments I was able to extend the aesthetic applicability and variation of the fluid system to capture unusual behaviors not normally depicted with fluid representations. Although the system appears realistic, it in fact sacrifices physical accuracy in order to emphasize interactivity, expressiveness, and experimentation.

13.3.2 Flow Visualization

The main image processing scheme in the Fluid Automata system is based on a feedback loop whereby a high-resolution background image is perpetually blended

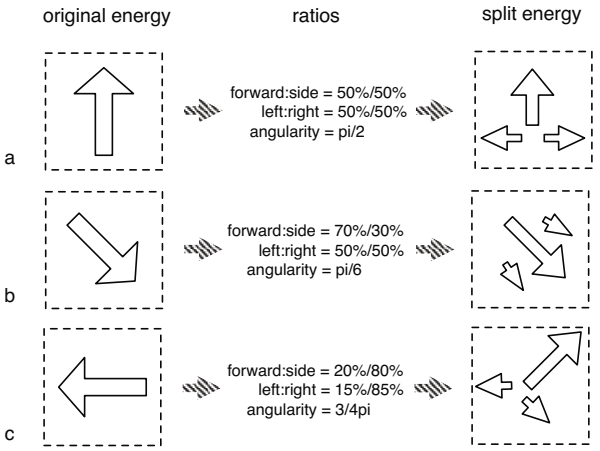


Fig. 13.5 Examples of fluid systems with different characteristics, i.e., fluid profiles, based on different settings. The settings can be changed in real-time. The left side of the chart represents the energy in a single cell; the right side of the chart shows how the energy is split into different streams based upon the parameters defining the ratio between forward and orthogonal momentum; the ratio between left and right momentum; and the angularity of the orthogonal momentum. In the top row (a), energy with a magnitude of 255 and an orientation of pi/2 is split evenly between forward and orthogonal momentum. In the middle row (b), most of the energy in the cell remains moving in the original direction; the orthogonal energy is close to the forward orientation. In the bottom row (c), most of the energy is moving to the side, with a high angularity, and with an uneven distribution between the left and right sides. (Note, the arrows representing energy vectors are not drawn exactly to scale.)

together with a distorted version of itself. The characteristics of the distortion are based directly on the current state of the fluid system, which cause pixels to smear or streak in the direction of the flow vectors. This is similar to Jarke J. van Wijk's "Image Based Flow Visualization," which has been extended for use in a variety of scientific visualization applications, including animated and 3D flows [57, 59].

Since a focus of Fluid Automata is aesthetic exploration, implementations provide the user with a variety of tools to alter aspects of these blending operations, and in addition introduce an image processing layer whereby the user can change a variety of parameters, including: the rate and amount of blending, the type and quality of the background texture, and the brightness, contrast, and saturation of the blended image. The default background texture is a grayscale noise texture at a resolution exactly matching the display, but various other options are possible, including: lower resolution textures; static colored textures; static image textures; dynamic textures that are updated by noise functions; and dynamic textures that are updated by a live video feed. Figure 13.2 shows an image created using a static colored background texture with high contrast and high saturation image processing

Fig. 13.6 Details of high-resolution output demonstrating "unrealistic" fluid simulation using the *Fluid Automata* system. The left image shows the addition of a high amount of vorticity; the right image shows the "spikiness" associated with a high amount of energy.

Fig. 13.7 Photo of iPad application using the Fluid Automata system with a live video feed replacing the background noise texture

parameters. Figure 13.7 shows an image that is created using a live video feed as the background image, rather than an image populated with randomly-colored pixels.

Figure 13.3 provides an overview of the fluid visualization process over the course of a single frame. At t_i, the system: a) distorts the previous image texture (if $i > 0$, otherwise it uses a copy of the background texture) based on user interaction and the current fluid profile; b) blends the background texture in with the distorted texture based on a blending parameter (that can be updated in real-time by a user); and then c) applies image processing filters (based on the image processing parameters described above) to the image to create a final output texture for this frame at t_{i+1}. This output texture is then used as the input texture for the next frame.

13.4 Single-User Interaction Techniques

The Fluid Automata system enables a wide range of interaction techniques. It was originally implemented in a stand-alone version that could be controlled by a single user on an iPhone or iPad [23]. The primary interaction component utilizes the multi-touch capabilities of iOS devices, creating a believable representation of fluids being disrupted by human touch. Users are also able to manipulate a wide number of parameters to control the fluid profile of the simulation as well as a number of image processing components. Much experimentation went into making the user interaction with the fluid system feel responsive and inviting: by tapping the screen the user adds energy to the system; moving a finger (or multiple fingers) across the screen overrides the fluid dynamic system by forcing the vector to move in the indicated direction. Additionally, a user is able to choose saved presets, defined by him or her previously, to quickly jump to states and motions that they find compelling. The iPad application can also be used in performance situations, for example, by choreographers or VJs, to augment a dance performance or musical event, by connecting the iPad to a projector or monitor. Users can also augment the performance by superimposing video playback, a live video stream, or dynamic noise textures onto the fluid system.

Another deployment of Fluid Automata allows the user to view and interact with fluids in a virtual space via the iPad tablet. It utilizes the gyroscope sensor built-in to the iPad in order to transform the tablet's screen into a "magic lens" [9]. The fluid system is placed on the six sides of a cube map (also known as a "skybox" texture), a technique often used in video games to display landscapes or distant horizons. The user is imagined to be inside a virtual fluid system that they can see by spinning in place while rotating the tablet. By touching the screen, a ray is sent to the cube map texture, allowing the user to interact with the surrounding fluid system. Figure 13.8 shows a series of photos of a user interacting with the virtual fluid system.

Fig. 13.8 A user interacting with the virtual Fluid Automata installation using the iPad as a magic lens

13.5 Multi-user Interaction Techniques

In addition to the single-user interaction techniques described above, recent deployments of Fluid Automata enable multiple participants to interact with the system simultaneously. Research investigating interactive flow visualization indicates that engaging interaction techniques can successfully enable collaboration and encourage exploration [37]. In order to investigate these aspects of fluid simulation, a collaborative version of the Fluid Automata system was presented at the 2011 IEEE VisWeek Art Show [22]. In this installation, each user is given an iPad and invited to interact with a single fluid system projected large-scale on a wall. On each iPad, users see the current state of the fluid system's underlying vector representation (represented by rotating and scaling triangles) as well as an indication of the other user's interaction (represented by colored circles) [24].

The Fluid Automata system has also been ported to a multi-touch table, making it easier for multiple people to interact with the system at the same time, and to explore specific gestures that were defined to allow users to alter the intensity of the turbulence of the system. In this installation, touches on the table are mapped to inputs that update the fluid vectors. The table uses a perimeter of LED light sources and light detection sensors to detect touches on the surface of the table. Figure 13.9 shows an image of people interacting with the multi-touch table running an iteration of the Fluid Automata project.

The Fluid Automata system has also been used as an instrument for controlling a multimedia composition, *Study in Brownian (F*) Motion*, in collaboration with the composer Kiyomitsu Odai. The visual output of the composition is projected

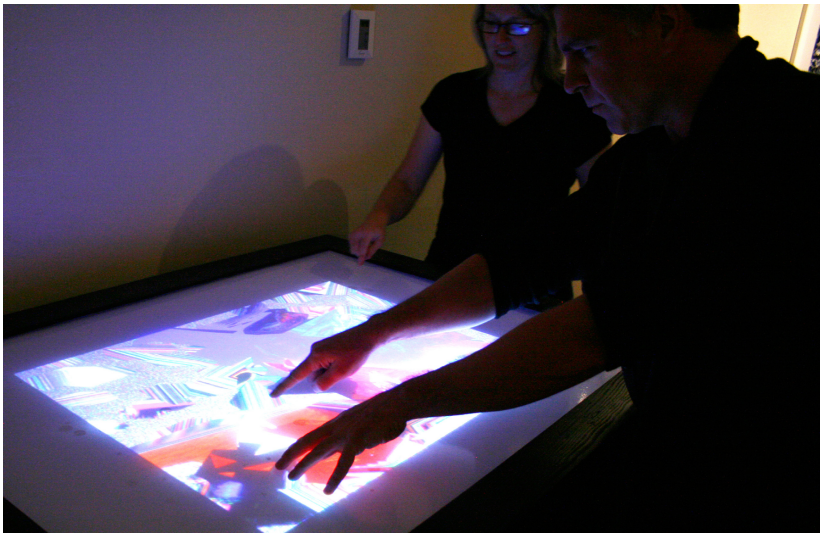


Fig. 13.9 Users gathered around a multi-touch table running a project that uses the Fluid Automata system

onto a large display and the audio output is channeled through multiple speakers. In addition to control via multi-touch, the system responds to Open Sound Control (OSC) [65] messages generated from musical events created on another computer and sent over the network. Additionally, fluid vectors and various fluid parameters can be transmitted wirelessly via OSC to influence the composition.

13.6 The *Annular Genealogy* Project

Annular Genealogy is more ambitious project based on generative systems that involves multi-user interaction. It is an interactive multimedia artwork for two performers using multi-channel speakers, a projector, and a tablet computer. The audio and visual components of the artwork explore feedback processes that the performers interactively shape into appealing, transient structures. The composition engine generates output via a stochastic sequencer that uses Brownian motion as a guiding metaphor. Similarly, the visualization engine, based on the Fluid Automata system, depicts colored fluid energy as a representation of dynamic, ephemeral structures. In addition to exploring these feedback processes independently of each other, each engine also directly influences the other via networked communication: both the visual and audio processes broadcast data via OSC messages which then influence various parameters of the composition and/or visualization. Finally, even the physical interactions are fed into the generative system as contact microphones are used to pick up the tapping and other ambient sounds made during the interaction. The performance aims to bring the layers of feedback into a cohesive compositional experience. These feedback layers are interconnected, and include: the generation of new musical motifs being created from the processing of the output sound; the generation of visual forms from the processing of the output graphics; the vector positions that govern the displacement of the visual forms used as inputs to control music parameters; and the sequencing parameters controlling the generation of the composition used as inputs to control image processing parameters [26].

Other recent multimedia installations have also featured generative compositions that make use of feedback mechanisms between the audio and visual components. For instance, Karen Curley's *Licht und Klang* is an audio-visual installation that generates sounds via optical sensors that use the refractions of light through oil and water as inputs into sound generation software [18]. A work by Joel Ryan and Ray Edgar called *LINA* features both musical and visual output based on mappings from a single CA system [14, 21]. Various electro-acoustic ensembles have explored the use of networked feedback as a tool for improvised performance. Most famously, the new media ensemble, The Hub, creates multimedia performances based on sets of rules that transform signals passed between performers and that are then presented in aural and visual domains [35]. Annular Genealogy similarly creates synaesthetic output based on a synchretic fusion of a mixed audio and visual feedback loop.

By supplying a multi-touch and live coding environment as an interface to and influencer of the generative processes another layer of feedback is added in which the performer responds to and shapes the multimedia output. That is, the performers

are *participants* in a compositional process. The generative software serves to create some structures independently of the performers; it is the role of the performers to guide the generational processes toward more compositionally interesting output and away from output that is overly repetitive, monochromatic, garish, or otherwise less satisfactory. Likewise, the audio and visual engines, via the various feedback processes, continually push against the explicit control of the performers. Overall, the composition is defined by a network of nested feedback loops that link the performer and the algorithm to create an inherent aesthetic tension between the generative and the interactive, the performed and the composed, the random and the intended. Figure 13.10 shows audience members inside an installation of Annular Genealogy in the AlloSphere Research Facility, an immersive CAVE-like environment housed at UC Santa Barbara [2].

While some of the results of interconnecting multiple feedback layers are unpredictable, the performers nonetheless begin to have an intuition as to how their actions will update the overall composition. For example, while there is no direct mapping of how the visualization data will update the compositional structures, after some experience using the iPad interface, it becomes clear that certain gestures during certain kinds of passages generate a particular shaping of the composition. It was also interesting to re-conceive the performers role as "guiders" of aesthetics, rather than as creators. A direction for future versions of the artwork would be to more explicitly highlight the effect that an interaction has as it is transmuted from one medium to the other.

Fig. 13.10 Photograph of viewers wearing 3D active stereo glasses within an installation of *Annular Genealogy* inside the AlloSphere Research Facility at UC Santa Barbara

13.7 Conclusion

This chapter discussed the integration of CA systems into creative projects and presented some of the design decisions that were made to facilitate effective interactivity in standalone and collaborative deployments. Although CA systems are conceptually simple, relatively minor modifications in rule sets can facilitate a wide range of variation. Many creative projects that utilize CA systems tend to either directly represent the CA system or offer a direct mapping of its inputs to an audio or visual output. This chapter instead explored two art projects that use CA as a creative mechanism that generates output which does *not* resemble the CA input. In particular, these two projects enable interaction and improvisation with CA and other generative systems. Fluid Automata extends CA into a more complex system in order to function as an interactive fluid simulation; and Annular Genealogy uses the Fluid Automata system as one input among other iterative feedback processes to generate a dynamic creative experience. By describing these two projects in some detail, we offer an example of how media arts projects may effectively extend CA concepts. The use of generative systems continues to intrigue artists, and new creative projects will incorporate CA system in novel and unexpected ways.

Appendix: Cell-State Transition Algorithm

The following code listing describes the fluid simulation, based on CA, as described in section 3.1. The *cell* parameter refers to a cell positioned in the CA grid (*pos*), with a particular state described by a vector (*vec*) that has a particular magnitude (*mag*) and orientation (*ang*). The *grid* has a pixel width of *width* and pixel height of *height*, and a particular number of columns (*cols*) and rows (*rows*), and contains a double array of cells. The variable *m* refers to the ratio that splits a cell vector into forward and "orthogonal" streams, the momentum parameter, or r_1. The variable ϕ refers to the angularity parameter. Other optional variables, such as the directionality and viscosity parameters, have been excluded for clarity. Note that the NEIGHBORCELLS procedure retrieving a cell's neighbors also includes the cell itself. Finally, the INTERSECTRECTS procedure (referred to but not shown) returns a value between 0.0 and 1.0 indicating the amount that two rectangles of the same size overlap.

Algorithm : UPDATECELLVECTOR($cell, grid, m, \phi$)

comment: Update vector *vec* in a Cell c by querying neighbors in Grid g

procedure NEIGHBORCELLS(c, g)
 $neighborCells \leftarrow$ ARRAY(9)
 for $i \leftarrow -1$ **to** 1
  for $j \leftarrow -1$ **to** 1
  do
   $nc \leftarrow c.col + i$
   $nr \leftarrow c.row + j$
   if $nc < 0,\ nc \leftarrow grid.cols - 1$
   if $nc > grid.cols - 1,\ nc \leftarrow 0$
   if $nr < 0,\ nc \leftarrow grid.rows - 1$
   if $nr > grid.rows - 1,\ nr \leftarrow 0$
   $neighborCells.push(grid[nc][nr])$
 return $(neighborCells)$

procedure MAKERECT(p)
 $w \leftarrow g.width/g.cols$
 $h \leftarrow g.height/g.rows$
 $pLL \leftarrow vec2(p.x - w/2, p.y - h/2)$
 $pUR \leftarrow vec2(p.x + w/2, p.y + h/2)$
 return (RECT(pLL, pUR))

procedure PARTIAL(pos, nc, nv)
 $p \leftarrow$ INTERSECTRECTS(MAKERECT(pos), MAKERECT($nc.pos + nv$))
 return $(nv * p)$

procedure STREAMVECTOR(c, nc, mr, ao)
 return $(vec2(nc.mag * mr, nc.ang + ao))$

main
 $mag \leftarrow ang \leftarrow 0.0$
 $p \leftarrow cell.pos$
 $arr \leftarrow$ NEIGHBORCELLS($cell, grid$)
 for $i \leftarrow 0$ **to** 9
  do
   $nc \leftarrow arr[i]$
   $fv \leftarrow$ STREAMVECTOR($cell, nc, m, 0.0$)
   $lv \leftarrow$ STREAMVECTOR($cell, nc, 1.0 - m/2.0, -\phi$)
   $rv \leftarrow$ STREAMVECTOR($cell, nc, 1.0 - m/2.0, \phi$)
   $v \leftarrow$ PARTIAL(p, nc, fv) + PARTIAL(p, nc, lv) + PARTIAL(p, nc, rv)
   $mag \leftarrow mag + v.mag$
   $ang \leftarrow ang + v.ang$
 $c.vec \leftarrow vec2(mag, ang)$

References

1. Aceti, L., Milovac, T.: Geometries of the sublime. Leonardo Electronic Almanac ("ISEA2011 Istanbul: Uncontainable") 18(5), 329–344 (2011)
2. Amatriain, X., Kuchera-Morin, J., Hollerer, T., Pope, S.T.: The AlloSphere: Immersive multimedia for scientific discovery and artistic exploration. IEEE MultiMedia, 64–75 (2009)
3. Ashley, R.: Spore at the cellular level, `http://www.1up.com/features/spore-cellular-level` (accessed November 2013)
4. Atken, M.: MSA fluid demos (2009), `http://www.memo.tv`
5. Babcock, J.: Cellular automata method for generating random cave-like levels (2005), `http://roguebasin.roguelikedevelopment.org`
6. Baggio, D.L., Emami, S., Escrivá, D.M., Ievgen, K., Mahmood, N., Saragih, J., Shilkrot, R.: Mastering OpenCV with practical computer vision projects. Packt (2012)
7. Beyls, P.: Musical morphologies from self-organizing systems. Journal of New Music Research 19(2-3), 205–218 (1990)
8. Beyls, P.: Cellular automata mapping procedures. In: Proceedings of the International Computer Music Conference, pp. 55–58 (2004)
9. Bier, E.A., Stone, M.C., Pier, K., Buxton, W., DeRose, T.D.: Toolglass and magic lenses: the see-through interface. In: Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques, pp. 73–80. ACM (1993)
10. Boden, M.A., Edmonds, E.: What is generative art? Digital Creativity 20(1-2), 21–46 (2009)
11. Borsani, C., Cattaneo, G., Mattei, V., Jocher, U., Zampini, B.: 2d and 3d lattice gas techniques for fluid-dynamics simulations. In: Bandini, S., Serra, R., Liverani, F. (eds.) Cellular Automata: Research Towards Industry, pp. 67–79. Springer (1998)
12. Brown, P.: Emergent behaviours towards computational aesthetics. Artlink 16(2–3) (1996)
13. Burraston, D.: Generative music and cellular automata: An introduction to the online bibliography. Leonardo 45(2), 165–165 (2012)
14. Burraston, D., Edmonds, E.: Cellular automata in generative electronic music and sonic art: a historical and technical review. Digital Creativity 16(3), 165–185 (2005)
15. Burt, W.A.: Cellular automata as spectra: Beyond sonification into composition. In: Riddell, A., Thorogood, A. (eds.) Proceedings of the Australasian Computer Music Conference, pp. 27–33. Kyoto, Japan (2007)
16. Cappuccio, R., Cattaneo, G., Ciucci, D., Jocher, U.: CA fluid dynamics simulation paradigms lattice gas, digital fluid, lattice boltzmann: A comparison. In: Bandini, S., Worsch, T. (eds.) Theory and Practical Issues on Cellular Automata, pp. 20–28. Springer (2001)
17. Ceric, V.: Algorithmic art: Technology, mathematics and art. In: 30th International Conference on Information Technology Interfaces, ITI 2008, pp. 75–82. IEEE (2008)
18. Curley, K.: Licht und klang (2009), `http://vimeo.com/5333718`
19. Dewdney, A.K.: Computer recreations. Scientific American 259, 104–107 (1988)
20. Dorin, A.: Liquiprism: Generating polyrhythms with cellular automata. In: Proceedings of the International Conference on Auditory Display (ICAD), vol. 5, Kyoto, Japan (2002)
21. Edgar, R., Ryan, J.: Lina. In: Exhibition of the 1986 International Computer Music Conference, ICMA (1986)
22. Forbes, A.G.: Fluid Automata. In: Keefe, D., Campbell, B., Thorson, L. (eds.) IEEE VisWeek 2011 Art Show Catalog (2011)

23. Forbes, A.G.: Fluid automata, http://fluidautomata.com (accessed November, 2013)
24. Forbes, A.G., Höllerer, T., Legrady, G.: Expressive energy: The fluid automata project. In: Proceedings of the International Symposium on Electronic Art (ISEA), Albequerque, New Mexico, pp. 65–70 (2012)
25. Forbes, A.G., Höllerer, T., Legrady, G.: Generative fluid profiles for interactive media arts projects. In: Proceedings of the International Symposium on Computational Aesthetics in Graphics, Visualization, and Imaging (CAe), Anaheim, California, pp. 123–129 (2013)
26. Forbes, A.G., Odai, K.: Iterative synaesthetic composing with multimedia signals. In: Proceedings of the International Computer Music Conference (ICMC), Ljubjiana, Slovenia, pp. 573–578 (2012)
27. Galanter, P.: What is generative art? Complexity theory as a context for art theory. In: Proceedings of the Generative Art Conference, GA (2003)
28. Galanter, P.: Generative art and rules-based art. Vague Terrain 3 (2006)
29. Galanter, P.: Xepa. In: Forbes, A.G., Thorson, L. (eds.) The IEEE VIS 2013 Art Show Catalog, pp. 4–5 (2013)
30. Galanter, P.: Xepa: Intelligent sculptures as experimental platforms for computational aesthetic evaluation. In: Proceedings of the IEEE VIS Arts Program (VISAP), IEEE (2013)
31. Gánti, T.: Biogenesis itself. Journal of Theoretical Biology 187(4), 583–593 (1997)
32. Gánti, T.: The Principles of Life. Oxford University Press (2003)
33. Gardner, M.: Mathematical games: The fantastic combinations of John Conway's new solitaire game "life". Scientific American 223(4), 120–123 (1970)
34. Greene, D.: Re: Geminoid challenges (November 23, 2013), http://www.conwaylife.com/forums/viewtopic.php?f=2&t=1006&p=9917#p9901
35. Gresham-Lancaster, S.: The aesthetics and history of the hub: The effects of changing technology on network computer music. Leonardo Music Journal, 39–44 (1998)
36. Guay, M., Colin, F., Egli, R.: Simple and fast fluids. In: Engel, W. (ed.) GPU Pro 2, pp. 433–444 (2011)
37. Isenberg, T., Hinrichs, U., Carpendale, S.: Studying direct-touch interaction for 2d flow visualization. In: Collaborative Visualization on Interactive Surfaces-CoVIS 2009, p. 17 (2009)
38. Kiwibonga: Another cellular automaton video (April 5, 2011) , www.gamedev.net/blog/844/entry-2249737-another-cellular-automaton-video
39. Lafia, M.: Algorithms and allegories. Digital Creativity 14(2), 125–128 (2003)
40. Lala, R.: Morphon (2011), http://www.riteshlala.net
41. Legrady, G.: George legrady studio: Algorithmic visualizations, http://www.georgelegrady.com (accessed November 2013)
42. Millen, D.: Cellular automata music. In: Proceedings of the 1990 International Computer Music Conference, pp. 314–316 (1990)
43. Motte, W.F.: Oulipo: a primer of potential literature. University of Nebraska Press Lincoln, London (1986)
44. O'Rourke, K.: Walking and Mapping: Artists as Cartographers. MIT Press (2013)
45. Prusinkiewicz, P., Lindenmayer, A., Hanan, J.S., Fracchia, F.D., Fowler, D.R., de Boer, M.J., Mercer, L.: The algorithmic beauty of plants. Springer (1990)
46. Rafler, S.: Generalization of Conway's "game of life" to a continuous domain-smoothlife. arXiv preprint arXiv:1111.1567 (2011)
47. Reas, C.: Compendium 2004-2010. REAS Studio (2010)

48. Reas, C.: A database for Casey Reas, http://reas.com (accessed November 2013)
49. Roads, C.: The computer music tutorial. MIT Press (1996)
50. Roberts, C.: Control: Software for end-user interface programming and interactive performance. In: Proceedings of the International Computer Music Conference (ICMC), pp. 425–428. Huddersfield, UK (2011)
51. Roberts, C., Forbes, A., Höllerer, T.: Enabling multimodal mobile interfaces for interactive musical performance. In: Proceedings of the International Conference on New Interfaces for Musical Expression (NIME), pp. 102–105. Daejeon, Korea (2013)
52. Shanken, E.A.: Art in the information age: Technology and conceptual art. Leonardo 35(4), 433–438 (2002)
53. Shanken, E.A.: Cybernetics and art: Cultural convergence in the 1960s. In: Clarke, B., Henderson, L.D. (eds.) From Energy to Information, pp. 155–177. Stanford University Press, Palo Alto (2002)
54. Solomos, M.: Cellular automata in Xenakis's music. Theory and practice. In: Definitive Proceedings of the International Symposium Iannis Xenakis, Athens (May 2005-2006)
55. Stam, J.: Stable fluids. In: Proceedings of the 26th annual conference on Computer graphics and interactive techniques, pp. 121–128. ACM Press/Addison-Wesley Publishing Co (1999)
56. Stam, J.: Real-time fluid dynamics for games. In: Proceedings of the game developer conference, vol. 18 (2003)
57. Telea, A., van Wijk, J.: 3d ibfv: Hardware-accelerated 3d flow visualization. In: Proceedings of IEEE Visualization (VIS), pp. 233–240. IEEE Computer Society (2003)
58. Torrens, P.M., Benenson, I.: Geographic automata systems. International Journal of Geographical Information Science 19(4), 385 (2005)
59. Van Wijk, J.: Image based flow visualization. ACM Transactions on Graphics (TOG) 21(3), 745–754 (2002)
60. Vichniac, G.: Cellular-automata fluids. In: Tirapegui, E., Villarroel, D. (eds.) Instabilities and Nonequilibrium Structures II, Mathematics and Its Applications, vol. 50, pp. 97–116. Springer (1989)
61. Vorn, B.: Evil / live (1997), http://billvorn.concordia.ca/robography
62. Wakefield, G., Ji, H(H.): Artificial nature: Immersive world making. In: Giacobini, M., Brabazon, A., Cagnoni, S., Di Caro, G.A., Ekárt, A., Esparcia-Alcázar, A.I., Farooq, M., Fink, A., Machado, P. (eds.) EvoWorkshops 2009. LNCS, vol. 5484, pp. 597–602. Springer, Heidelberg (2009)
63. Wolfram, S.: Cellular automaton fluids 1: Basic theory. Journal of Statistical Physics 45(3-4), 471–526 (1986)
64. Wolfram, S.: A New Kind of Science. Wolfram Media, Inc., Champaign (2002)
65. Wright, M.: Open sound control: An enabling technology for musical networking. Organised Sound 10(3), 193–200 (2005)
66. Wuensche, A.: The ghost in the machine: Basins of attraction of random boolean networks. In: Langton, C. (ed.) Artificial Life III, Santa Fe Institute Studies in the Sciences of Complexity, Addison-Wesley (1994)
67. Xenakis, I.: Formalized Music: Thought and Mathematics in Composition. Pendragon Press (1992)

References

1. Abelson, H., Allen, D., Coore, D., Hanson, C., Homsy, G., Thomas, F., Knight, J., Nagpal, R., Rauch, E., Sussman, G.J., Weiss, R.: Amorphous computing. Commun. ACM 43(5), 74–82 (2000)
2. Acerbi, L., Dennunzio, A., Formenti, E.: Conservation of some dynamical properties for operations on cellular automata. Theoretical Computer Science 410(38-40), 3685–3693 (2009)
3. Aceti, L., Milovac, T.: Geometries of the sublime. Leonardo Electronic Almanac (ISEA2011 Istanbul: Uncontainable) 18(5), 329–344 (2011)
4. Adamatzky, A.: Identification of Cellular Automata. Taylor & Francis (1994)
5. Adamatzky, A.: Voronoi-like partition of lattice in cellular automata. Mathematical and Computer Modelling 23(16), 51–66 (1996)
6. Adamatzky, A.: Computing in nonlinear media and automata collectives. IOP Publishing Ltd., Bristol (2001)
7. Adamatzky, A.: Programming reaction-diffusion computers (2005)
8. Adamatzky, A., Bull, L.: Are complex systems hard to evolve? Complexity 14(6), 15–20 (2009)
9. Adamatzky, A., Costello, B.D.L., Asai, T.: Reaction-Diffusion Computers. Elsevier Science Inc., New York (2005)
10. Adleman, L.M.: Computing with DNA. Scientific American 279(2), 54–61 (1998)
11. Ahuja, M., Loeb, L.: Tessellations in Islamic calligraphy. Leonardo, vol. 28, pp. 41–45. The MIT Press (1995)
12. Alonso-Sanz, R.: A structurally dynamic cellular automaton with memory in the triangular tessellation. Complex-Systems 17(1), 1–15 (2007)
13. Altuwaijri, M., Bayoumi, M.: A new thinning algorithm for Arabic characters using self-organizing neural network. In: 1995 IEEE International Symposium on Circuits and Systems, ISCAS 1995, vol. 3, pp. 1824–1827 (1995)
14. Amanatiadis, A., Andreadis, I., Gasteratos, A.: A log-polar interpolation applied to image scaling. In: IEEE International Workshop on Imaging Systems and Techniques, pp. 1–5. IEEE, Cracovia (2007)
15. Amatriain, X., Kuchera-Morin, J., Hollerer, T., Pope, S.T.: The AlloSphere: Immersive multimedia for scientific discovery and artistic exploration. IEEE MultiMedia, 64–75 (2009)
16. Amerini, I., Ballan, L., Caldelli, R., Bimbo, A.D., Serra, G.: A SIFT-based forensic method for copy-move attack detection and transformation recovery. IEEE Transactions on Information Forensics and Security 6(3), 1099–1110 (2011)

17. Ammann, C., Sartori Angus, A.: Fast thinning algorithm for binary images. Image Vision and Computing 3(2), 71–79 (1985)
18. Andreadis, I., Illiades, P., Karafyllidis, Y., Tsalides, P., Thanailakis, A.: Design and VLSI implementation of a new ASIC for colour measurement. IEE Proceedings - Circuits, Devices and Systems 142(3), 153–157 (1995)
19. Andreadis, I., Karafyllidis, I., Tzionas, P., Thanailakis, A., Tsalides, P.: A new hardware system for automated visual inspection based on a cellular automaton architecture. J. Intellig. Robot. Sys. 16, 89–102 (1996)
20. Anoop, S., Alakkaran, A.: A full image encryption scheme based on transform domains and stream ciphers. International Journal of Advanced Information Science and Technology 17(17), 5–10 (2013)
21. Antnio, G.C.: Spatial analysis and GIS: A primer, `citeseer.ist.psu.edu/696989.html`
22. Arata, H., Takai, Y., Takai, N.K., Yamamoto, T.: Free-form shape modeling by 3D cellular automata. In: International Conference on Shape Modeling and Applications, pp. 242–247. IEEE Computer Society (1999)
23. Arcelli, C., di Baja, G.S.: Euclidean skeleton via centre-of-maximal-disc extraction. Image and Vision Computing 11(3), 163–173 (1993)
24. Ashley, R.: Spore at the cellular level, `http://www.1up.com/features/spore-cellular-level` (accessed November 2013)
25. Atken, M.: MSA fluid demos (2009), `http://www.memo.tv`
26. Attali, D., Boissonnat, J.D., Edelsbrunner, H.: Stability and Computation of Medial Axes - a State-of-the-Art Report. In: Mathematical Foundations of Scientific Visualization, Computer Graphics, and Massive Data Exploration, vol. 6, pp. 109–125. Springer, Heidelberg (2009)
27. Aupetit, M., Catz, T.: High-dimensional labeled data analysis with topology representing graphs. Neurocomputing 63, 139–169 (2005)
28. Aurenhammer, F.: Voronoi diagrams - a survey of a fundamental geometric data structure. ACM Comput. Surv. 23(3), 345–405 (1991)
29. Avidan, S., Shamir, A.: Seam carving for content-aware image resizing. ACM Trans. Graph. 26(3), 10 (2007)
30. Aydinli, S., Seidl, M.: Determination of the economic benefits of daylight in interiors concerned with the fulfillment of visual tasks. In: Adepski, M., McCluney, R. (eds.) Proceedings I: 1986 International Daylighting Conference, Long Beach California USA, pp. 145–151 (1986)
31. Baas, N.A., Torbjorn, H.: Higher Order Cellular Automata. Advances in Complex Systems 8(2-3), 169–192 (2005)
32. Babcock, J.: Cellular automata method for generating random cave-like levels (2005), `http://roguebasin.roguelikedevelopment.org`
33. Bachrach, J., Beal, J., Horowitz, J., Qumsiyeh, D.: Empirical characterization of discretization error in gradient-based algorithms. In: SASO 2008: Proceedings of the 2008 Second IEEE International Conference on Self-Adaptive and Self-Organizing Systems, pp. 203–212. IEEE Computer Society, Washington, DC (2008)
34. Başturk, A., Günay, E.: Efficient edge detection in digital images using a cellular neural network optimized by differential evolution algorithm. Expert Syst. Appl. 36(2), 2645–2650 (2009)
35. Baggio, D.L., Emami, S., Escrivá, D.M., Ievgen, K., Mahmood, N., Saragih, J., Shilkrot, R.: Mastering OpenCV with practical computer vision projects (2012)

36. di Baja, G.S., Thiel, E.: Skeletonization algorithm running on path-based distance maps. Image and Vision Computing 14(1), 47–57 (1996)
37. Banatre, J., Metayer, D.L.: Programming by multiset transformation. Commun. ACM 36(1), 98–111 (1993)
38. Bandini, S., Manzoni, S., Umeo, H., Vizzari, G. (eds.): ACRI 2010. LNCS, vol. 6350. Springer, Heidelberg (2010)
39. Baraniuk, R., Cevher, V., Duarte, M., Hedge, C.: Model-based compressive sensing. IEEE Trans. Inform. Theory 56, 1982–2001 (2010)
40. Baron, D., Sarvotham, S., Baraniuk, R.G.: Bayesian compressive sensing via belief propagation. IEEE Trans. Signal Process. 58, 269–280 (2010)
41. Baruch, O.: Line thinning by line following. Pattern Recognition Letters 8(4), 271–276 (1988)
42. Bashar, M., Noda, K., Ohnishi, N., Mori, K.: Exploring duplicated regions in natural images. IEEE Transactions on Image Processing (2010) (accepted for publication)
43. Batouche, M., Meshoul, S., Abbassene, A.: On solving edge detection by emergence. In: Ali, M., Dapoigny, R. (eds.) IEA/AIE 2006. LNCS (LNAI), vol. 4031, pp. 800–808. Springer, Heidelberg (2006)
44. Bayram, S., Sencar, H., Memon, N.: An efficient and robust method for detecting copy-move forgery. In: IEEE International Conference on Acoustics, Speech, and Signal Processing, pp. 1053–1056 (2009)
45. Bays, C.: Cellular automata in the triangular tessellation. Complex-Systems 8, 127–150 (1994)
46. Bays, C.: Cellular Automata and the Game of Life in the Hexagonal Grid (2001), http://www.cse.sc.edu/~bays/h6h6h6/
47. Bays, C.: Cellular Automata in Triangular, Pentagonal and Hexagonal Tessellations. In: Meyers, R.A. (ed.) Computational Complexity, pp. 434–442. Springer, New York (2012)
48. Beal, J., Bachrach, J., Vickery, D., Tobenkin, M.: Fast self-stabilization for gradients. In: Krishnamachari, B., Suri, S., Heinzelman, W., Mitra, U. (eds.) DCOSS 2009. LNCS, vol. 5516, pp. 15–27. Springer, Heidelberg (2009)
49. Belkasim, S., Shridhar, M., Ahmadi, M.: Pattern recognition with moment invariants: a comparative study and new results. Pattern Recognition 24(12), 1117–1138 (1991)
50. Bentley, K., Cox, E.J., Bentley, P.J.: Nature's batik: a computer evolution model of diatom valve morphogenesis. Journal of nanoscience and nanotechnology 5, 25–34 (2005)
51. Betel, H., Flocchini, P.: On the relationship between fuzzy and boolean cellular automata. Theoretical Computer Science 412(8-10), 703–713 (2011)
52. Beyls, P.: Musical morphologies from self-organizing systems. Journal of New Music Research 19(2-3), 205–218 (1990)
53. Beyls, P.: Cellular automata mapping procedures. In: Proceedings of the International Computer Music Conference, pp. 55–58 (2004)
54. Bhanja, S., Sarkar, S.: Thermal switching error versus delay tradeoffs in clocked QCA circuits. IEEE Transactions on Very Large Scale Integration (VLSI) Systems 16(5), 528–541 (2008)
55. Biasotti, S., Attali, D., Boissonnat, J.D., Edelsbrunner, H., Elber, G., Mortara, M., Baja, G.S., Spagnuolo, M., Tanase, M., Veltkamp, R.: Skeletal structures. In: Floriani, L., Spagnuolo, M. (eds.) Shape Analysis and Structuring, Mathematics and Visualization, pp. 145–183. Springer, Heidelberg (2008)
56. Bier, E.A., Stone, M.C., Pier, K., Buxton, W., DeRose, T.D.: Toolglass and magic lenses: the see-through interface. In: Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques, pp. 73–80. ACM (1993)

57. Bilotta, E., Lafusa, A., Pantano, P.: Searching for complex CA rules with GAs. Complexity 8(3), 56–67 (2003)
58. Birajdar, G.K., Mankar, V.H.: Digital image forgery detection using passive techniques: A survey. Digital Investigation 10(3), 226–245 (2013)
59. Blum, H.: An associative machine for dealing with the visual field and some of its biological implications. In: Bernard, E.E., Kare, M.R. (eds.) Biological Prototypes and Synthetic Systems, vol. 1, pp. 244–260. Plenum Press, New York (1962), Proceedings of the 2nd Annual Bionics Symposium, held at Cornell University (1961)
60. Bodart, M., De Herde, A.: Global energy savings in offices buildings by the use of daylighting. Energy and Buildings 34(5), 421–429 (2002)
61. Boden, M.A., Edmonds, E.A.: What is generative art? Digital Creativity 20(1-2), 21–46 (2009)
62. Borsani, C., Cattaneo, G., Mattei, V., Jocher, U., Zampini, B.: 2d and 3d lattice gas techniques for fluid-dynamics simulations. In: Bandini, S., Serra, R., Liverani, F. (eds.) Cellular Automata: Research Towards Industry, pp. 67–79. Springer, London (1998)
63. Botha, L., Van Zijl, L., Hoffmann, M.: Realtime LEGO brick image retrieval with cellular automata. Journal of Universal Computer Science 15(14), 2765–2785 (2009)
64. Bovik, A.C.: The essential guide to image processing. Academic Press (2009)
65. Boyce, P.: Why Daylight? In: Proceedings of Daylight 1998, International Conference on Daylighting Technologies for Energy Efficiency in Buildings, Ottawa, Ontario, Canada, pp. 359–365 (1998)
66. Boyce, P., Hunter, C., Howlett, O.: The benefits of daylight through windows: Report. Tech. rep., Rensselaer Polytechnic Institute, Troy, New York (2003)
67. Boykov, Y., Kolmogorov, V.: An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision. IEEE Transactions on Pattern Analysis and Machine Intelligence 26(9) (2004)
68. Boykov, Y., Veksler, O., Zabih, R.: Fast approximate energy minimization via graph cuts. IEEE Transactions on Pattern Analysis and Machine Intelligence 23(11), 1222–1239 (2001)
69. Boykov, Y.Y., Jolly, M.P.: Interactive graph cuts for optimal boundary and region segmentation of objects in ND images. In: Proceedings of the Eighth IEEE International Conference on Computer Vision (ICCV 2001), vol. 1, pp. 105–112. IEEE (2001)
70. Bravo-Solorio, S., Nandi, A.K.: Exposing duplicated regions affected by reflection, rotation and scaling. In: International Conference on Acoustics, Speech and Signal Processing, pp. 1880–1883 (2011)
71. Brown, P.: Emergent behaviours towards computational aesthetics. Artlink 16(2-3) (1996)
72. Bruckstein, A.M., Holt, R.J., Netravali, A.N.: Holographic representation of images. IEEE Trans. Image Processing 7, 1583–1597 (1998)
73. Bubna, M., Roy, S., Shenoy, N., Mazumdar, S.: A layout-aware physical design method for constructing feasible QCA circuits. In: Proceedings of the ACM Great Lakes Symposium on VLSI, GLSVLSI, pp. 243–248 (2008)
74. Burraston, D.: Generative music and cellular automata: An introduction to the online bibliography. Leonardo 45(2), 165–165 (2012)
75. Burraston, D., Edmonds, E.: Cellular automata in generative electronic music and sonic art: a historical and technical review. Digital Creativity 16(3), 165–185 (2005)
76. Burt, P.J., Adelson, E.H.: The Laplacian pyramid as a compact image code. IEEE Trans. on Communications COM 31(4), 532–540 (1983)
77. Burt, W.A.: Cellular automata as spectra: Beyond sonification into composition. In: Riddell, A., Thorogood, A. (eds.) Proceedings of the Australasian Computer Music Conference, Canberra, Australia, pp. 27–33 (2007)

78. Candes, E., Romberg, J.: Practical signal recovery from random projections. In: Proc. SPIE Conf, Wavelet Applications in Signal and Image Processing XI (2005)
79. Canny, J.: A computational approach to edge detection. IEEE Trans. Pattern Analysis and Machine Intelligence 8, 679–698 (1986)
80. Capellari, L., Milani, S., Cruz-Reyes, C., Calvagno, G.: Resolution scalable image coding with reversible cellular automata. IEEE Trans. on Image Processing 20(5), 1461–1468 (2011)
81. Cappuccio, R., Cattaneo, G., Ciucci, D., Jocher, U.: CA fluid dynamics simulation paradigms lattice gas, digital fluid, lattice boltzmann: A comparison. In: Bandini, S., Worsch, T. (eds.) Theory and Practical Issues on Cellular Automata, pp. 20–28. Springer, London (2001)
82. Cardenas-Barrera, J.L., Plataniotis, K.N., Venetsanopoulos, A.N.: QCA implementation of a multichannel filter for image processing. Mathematical Problems in Engineering 8(1), 87–99 (2002)
83. Caspi, E., Chu, M., Huang, R., Yeh, J., Wawrzynek, J., DeHon, A.: Stream computations organized for reconfigurable execution (SCORE). In: Grünbacher, H., Hartenstein, R.W. (eds.) FPL 2000. LNCS, vol. 1896, pp. 605–614. Springer, Heidelberg (2000)
84. Cattaneo, G., Dennunzio, A., Margara, L.: Solution of some conjectures about topological properties of linear cellular automata. Theoretical Computer Science 325(2), 249–271 (2004), Theoretical Aspects of Cellular Automata
85. Ceric, V.: Algorithmic art: Technology, mathematics and art. In: 30th International Conference on Information Technology Interfaces, ITI 2008, pp. 75–82. IEEE (2008)
86. Cha, Y., Kim, S.: The error-amended sharp edge (EASE) scheme for imaging zooming. IEEE Trans. Image Process. 16, 1496–1505 (2007)
87. Chamberlain, M.: Cellular automata for image processing. Tech. rep., Stellenbosch University, South Africa (2008)
88. Chan, T.M.: Optimal output-sensitive convex hull algorithms in two and three dimensions. Discrete & Computational Geometry 16, 361–368 (1996)
89. Chang, C., Zhang, Y., Gdong, Y.: Cellular automata for edge detection of images. In: Int. Conf. on Machine Learning and Cybernetics, vol. 6, pp. 3830–3834 (2004)
90. Chatzichristofis, S., Zagoris, K., Boutalis, Y., Papamarkos, N.: Accurate image retrieval based on compact composite descriptors and relevance feedback information. International Journal of Pattern Recognition and Artificial Intelligence 24(02), 207–244 (2010)
91. Chatzichristofis, S.A., Mitzias, D.A., Sirakoulis, G.C., Boutalis, Y.S.: A novel cellular automata based technique for visual multimedia content encryption. Optics Communications 283(21), 4250–4260 (2010)
92. Chatzis, V., Pitas, I.: A generalized fuzzy mathematical morphology and its application in robust 2-D and 3-D object representation. IEEE Transactions on Image Processing 9(10), 1798–1810 (2000)
93. Chatzis, V., Pitas, I.: Interpolation of 3-D binary images based on morphological skeletonization. IEEE Transactions on Medical Imaging 19(7), 699–710 (2000)
94. Chauhan, S.: Survey paper on training of cellular automata for image. International Journal of Engineering and Computer Science 2(4), 980–985 (2013)
95. Chavey, D.: Tilings by regular polygons II: A catalog of tilings. Computers & Mathematics with Applications 17(1-3), 147–165 (1989)

96. Chavoya, A., Andalon-Garcia, I.R., Lopez-Martin, C., Meda-Campaña, M.E.: 3D cell pattern generation in artificial development. In: González, J.R., Pelta, D.A., Cruz, C., Terrazas, G., Krasnogor, N. (eds.) NICSO 2010. SCI, vol. 284, pp. 127–139. Springer, Heidelberg (2010)
97. Chavoya, A., Duthen, Y.: A cell pattern generation model based on an extended artificial regulatory network. Biosystems 94, 95–101 (2008)
98. Chen, H., Wei, W.: Geodesic Gabriel graph based supervised nonlinear manifold learning. In: Huang, D.-S., Li, K., Irwin, G.W. (eds.) ICIC 2006. LNCIS, vol. 345, pp. 882–887. Springer, Heidelberg (2006)
99. Chen, J.L., Chang, J.Y., Shieh, K.L.: 2D discrete signal interpolation and its image resampling application using fuzzy rule-based inference. Fuzzy Sets Syst. 114, 225–238 (2000)
100. Chen, M.J., Huang, C.H., Lee, W.L.: A fast edge-oriented algorithm for image interpolation. Image and Vision Computing 23, 791–798 (2005)
101. Chen, R.J., Lai, J.L.: VLSI implementation of the universal 2-D CAT/ICAT systems. In: Proceedings of the 11th IEEE International Conference on Electronics, Circuits and Systems, ICECS, pp. 187–190 (2004)
102. Chen, Y., Yan, Z.: A cellular automatic method for the edge detection of images. In: Huang, D.-S., Wunsch II, D.C., Levine, D.S., Jo, K.-H. (eds.) ICIC 2008. LNCS (LNAI), vol. 5227, pp. 935–942. Springer, Heidelberg (2008)
103. Chen, Y.S.: The use of hidden deletable pixel detection to obtain bias-reduced skeletons in parallel thinning. In: Proceedings of the 13th International Conference on Pattern Recognition, Washington, DC, USA, vol. 2, pp. 91–95 (1996)
104. Chen, Y.S., Hsu, W.H.: Systematic approach for designing 2-subcycle and pseudo 1-subcycle parallel thinning algorithms. Pattern Recognition 22(3), 267–282 (1989)
105. Cho, H., Swartzlander Jr., E.E.: Adder and multiplier design in quantum-dot cellular automata. IEEE Transactions on Computers 58(6), 721–727 (2009)
106. Choi, M., Patitz, Z., Jin, B., Tao, F., Park, N., Choi, M.: Designing layout-timing independent quantum-dot cellular automata (QCA) circuits by global asynchrony. Journal of Systems Architecture 53(9), 551–567 (2007)
107. Chopard, B., Droz, M.: Cellular Automata Modeling of Physical Systems. Cambridge University Press (1998)
108. Christlein, V., Riess, C., Jordan, J., Riess, C., Angelopoulou, E.: An evaluation of popular copy-move forgery detection approaches. IEEE Information Forensics and Security 7(6), 1841–1854 (2012)
109. Chua, L.O.: A nonlinear dynamics perspective of Wolfram's New Kind of Science (Vol.I-IV). World Scientific Series on Nonlinear Science, Series A, vol. 57,68,76. World Scientific Publishing Company (2011)
110. Clarridge, A.G., Salomaa, K.: An improved cellular automata based algorithm for the 45-convex hull problem. Journal of Cellular Automata 5(1-2), 107–120 (2010)
111. Comaniciu, D., Meer, P.: Mean shift: A robust approach toward feature space analysis. IEEE Transactions on Pattern Analysis and Machine Intelligence 24(5), 603–619 (2002)
112. Comaniciu, D., Ramesh, V., Meer, P.: Kernel-based object tracking. IEEE Trans. Pattern Analysis and Machine Intelligence 25(5), 564–577 (2003)
113. Connor, S.: The book of skin. Cornell University Press (2003)
114. Coore, D.: Botanical computing: a developmental approach to generating interconnect topologies on an amorphous computer. Ph.D. thesis, MIT (1999)
115. Cox, I., Miller, M., Bloom, J., Fridrich, J., Kalker, T.: Digital Watermarking and Steganography, 2nd edn. Morgan Kaufmann Publishers Inc., San Francisco (2007)

116. Culik, I.K., Hurd, L., Yu, S.: Formal languages and global cellular automaton behavior. In: Howard, G. (ed.) Cellular Automata, pp. 396–403. MIT Press (1990)
117. Curley, K.: Licht und klang (2009), http://vimeo.com/5333718
118. Cuttle, C.: Lighting by Design. Elsevier, Amsterdam (2003)
119. Dalhoum, A., Al-Dhamari, I., Ortega, A., Alfonseca, M.: Enhanced cellular automata for image noise removal. In: Proceedings of the Asian Simulation Technology Conference, pp. 69–73 (2011)
120. D'Alotto, L.A., Sinha, D.: Cellular automata and real-time geometrical shape recognition. In: Photonics West 1998 Electronic Imaging, pp. 60–69. International Society for Optics and Photonics (1998)
121. Das, D.: A survey on cellular automata and its applications. In: Krishna, P.V., Babu, M.R., Ariwa, E. (eds.) ObCom 2011, Part I. CCIS, vol. 269, pp. 753–762. Springer, Heidelberg (2012)
122. Das, R., Mitchell, M., Crutchfield, J.: A genetic algorithm discovers particle based computation in cellular automata. In: Davidor, Y., Männer, R., Schwefel, H.-P. (eds.) PPSN 1994. LNCS, vol. 866, pp. 244–353. Springer, Heidelberg (1994)
123. Datta, R., Joshi, D., Li, J., Wang, J.: Image retrieval: ideas, influences, and trends of the new age. ACM Computing Surveys 40(2), 300–360 (2008)
124. De Rosa, M., Goldstein, S.C., Lee, P., Campbell, J.D., Pillai, P.: Scalable shape sculpting via hole motion: Motion planning in lattice-constrained module robots. In: Proceedings of the 2006 IEEE International Conference on Robotics and Automation (ICRA 2006), pp. 1462–1468 (2006), www.cs.cmu.edu/~claytronics/papers/derosa-icra06.pdf
125. Dehmeshki, J., Amin, H., Valdivieso, M., Ye, X.: Segmentation of pulmonary nodules in thoracic CT scans: A region growing approach. IEEE Transactions on Medical Imaging 27(4) (2008)
126. Dehon, A., Giavitto, J.L., Gruau, F. (eds.): Computing Media and Languages for Space-Oriented Computation 2006. Dagstuhl international workshop 06361 (2006), http://drops.dagstuhl.de/portals/index.php?semnr=06361
127. Denning, P.J.: The locality principle. Commun. ACM 48, 19–24 (2005)
128. Désérable, D.: Simulation de milieux granulaires par automate cellulaire (in french). Master's thesis, IFSIC (1998)
129. Dewdney, A.K.: Computer recreations. Scientific American 259, 104–107 (1988)
130. Dewdney, A.K.: The cellular automata programs that create wireworld, rugworld and other diversions. Computer Recreations, Scientific American, 146–149 (1990)
131. Dinneen, G.P.: Programming pattern recognition. In: Proceedings of the Western Joint Computer Conference, AFIPS 1955 (Western), pp. 94–100. ACM, New York (1955)
132. Diwakar, M., Patel, P., Gupta, K.: Cellular automata based edge-detection for brain tumor. In: Advances in Computing, Communications and Informatics, pp. 53–59 (2013)
133. Dogaru, I., Dogaru, R.: Algebraic normal form for rapid prototyping of elementary hybrid cellular automata in FPGA. In: Proceedings of the ISEEE Conference, Galati, Romania (2010)
134. Dogaru, I., Dogaru, R., Damian, C.: FPGA implementation of chaotic cellular automaton with binary synchronization property. In: Proceedings of 8th International Conference on Communications (COMM), Bucharest, Romania (2010)
135. Dogaru, R.: CA-VQ: A simple compression scheme using codebooks generated by cellular automata. In: Proceedings NSIP 2007 (International Workshop on Nonlinear Signal and Image Processing), Bucharest, Romania (2007)
136. Dogaru, R.: Systematic design for emergence in cellular nonlinear networks with applications in natural computing and signal processing. SCI, vol. 95. Springer, Heidelberg (2008)

137. Dogaru, R.: A fast method for classification of emergent dynamics in cellular automata based on uncertainty profiles. Journal of Control Engineering and Applied Informatics 11, 18–25 (2009)
138. Dogaru, R.: Hybrid cellular automata as pseudo-random number generators with binary synchronization property. In: Proceedings of the International Symposium on Signals Circuits and Systems, ISSCS 2009, Iasi Romania (2009)
139. Dogaru, R.: HCA101: A chaotic map based on cellular automata with binary synchronization properties. In: Proceedings of the 8th Int. Conference on Communications, COMM 2010, Bucharest, Romania (2010)
140. Dogaru, R.: A low complexity image sensing method using pseudo-random scan and recursive reconstruction with radial basis functions. In: Proceedings of the ISEEE Conference, Galati, Romania (2013)
141. Dogaru, R., Dogaru, I.: Uncertainty profiles for predicting complex nonlinear dynamics in cellular automata: the case of five cells neighborhood. In: Proceedings of the International Symposium on Nonlinear Theory and its Applications, NOLTA 2010, Krakow, Poland, September 5-8 (2010)
142. Dogaru, R., Dogaru, I., Kim, H.: Binary chaos synchronization in elementary cellular automata. Int. J. Bifurcation Chaos 19, 2871–2884 (2009)
143. Dogaru, R., Dogaru, I., Kim, H.: Chaotic scan: A low complexity video transmission system for efficiently sending relevant image features. IEEE Trans. on Circuits and Systems for Video Technology 20, 317–321 (2010)
144. Dogaru, R., Murgan, A.T., Ortmann, S., Glesner, M.: A modified RBF neural network for efficient current-mode VLSI implementation. In: Proceedings Micro-Neuro 1996, February 12-14, pp. 265–270. IEEE Press, Laussane (1996)
145. Dogaru, R., Tetzlaff, R., Glesner, M.: Semi-totalistic CNN genes for compact image compression. In: The IEEE Proceedings of 10th International Workshop on Cellular Neural Networks and Their Applications, Istanbul, Turkey (2006)
146. Dorin, A.: Liquiprism: Generating polyrhythms with cellular automata. In: Proceedings of the International Conference on Auditory Display (ICAD), Kyoto, Japan, vol. 5 (2002)
147. Du, Q., Faber, V., Gunzburger, M.: Centroidal voronoi tessellations: Applications and algorithms. SIAM Rev. 41(4), 637–676 (1999)
148. Dufresne, T.E., Sarwal, A., Dhawan, A.P.: A gray-level thinning method for delineation and representation of arteries. Computerized Medical Imaging and Graphics 18(5), 343–355 (1994)
149. Edgar, R., Ryan, J.: Lina. Exhibition of the 1986 International Computer Music Conference. ICMA (1986)
150. El Yacoubi, S., Jacewicz, P.: A genetic programming approach to structural identification of cellular automata. Journal of Cellular Automata 2, 67–76 (2007)
151. Ens, J., Lawrence, P.: An investigation of methods for determining depth from focus. IEEE Trans. Pattern Analysis and Machine Intelligence 15(2), 97–108 (1993)
152. Ermentrout, G.B., Edelstein-Keshet, L.: Cellular automata approaches to biological modeling. Journal of Theoretical Biology, 97–133 (1993)
153. Euler, L.: Elementa doctrinae solidorum. Novi Commentarii academiae scientiarum Petropolitanae 4, 109–140 (1758)
154. Faraco, G., Pantano, P., Servidio, S.: The use of Cellular Automata in the learning of emergence. Computers & Education 47(3), 280–297 (2006)
155. Farid, H.: Image forgery detection: A survey. IEEE Signal Processing Magazine 26(2), 16–35 (2009)

156. Favre, A., Keller, H.: Parallel syntactic thinning by recoding of binary pictures. Computer Vision, Graphics, and Image Processing 23(1), 99–112 (1983)
157. Feldman, Y., Shapiro, E.: Spatial machines: a more realistic approach to parallel computation. Commun. ACM 35(10), 60–73 (1992)
158. Felzenszwalb, P.F., Huttenlocher, D.P.: Efficient graph-based image segmentation. International Journal of Computer Vision 59(2), 167–181 (2004)
159. Fernandez-Berni, J., Carmona-Galan, R., Carranza-González, L.: FLIP-Q: A QCIF resolution focal-plane array for low-power image processing. IEEE Journal of Solid-State Circuits 46, 669–680 (2011)
160. Feynman, R.E.: Simulating physics with computers. International Journal of Theoretical Physics 21(6), 467–488 (1982)
161. Flache, A., Hegselmann, R.: Do Irregular Grids make a Difference? Relaxing the Spatial Regularity Assumption in Cellular Models of Social Dynamics. Journal of Artificial Societies and Social Simulation 4(4) (2001), http://jasss.soc.surrey.ac.uk/4/4/6.html
162. Fonseca, L.R.C., Korotkov, A.N., Likharev, K.K., Odintsov, A.A.: A numerical study of the dynamics and statistics of single electron systems. Journal of Applied Physics 78(5), 3238–3251 (1995)
163. Forbes, A.G.: Fluid Automata. In: Keefe, D., Campbell, B., Thorson, L. (eds.) IEEE VisWeek 2011 Art Show Catalog (2011)
164. Forbes, A.G.: Fluid automata, http://fluidautomata.com (accessed November 2013)
165. Forbes, A.G., Höllerer, T., Legrady, G.: Expressive energy: The fluid automata project. In: Proceedings of the International Symposium on Electronic Art (ISEA), Albequerque, New Mexico, pp. 65–70 (2012)
166. Forbes, A.G., Höllerer, T., Legrady, G.: Generative fluid profiles for interactive media arts projects. In: Proceedings of the International Symposium on Computational Aesthetics in Graphics, Visualization, and Imaging (CAe), Anaheim, California, pp. 123–129 (2013)
167. Forbes, A.G., Odai, K.: Iterative synaesthetic composing with multimedia signals. In: Proceedings of the International Computer Music Conference (ICMC), Ljubjiana, Slovenia, pp. 573–578 (2012)
168. Formenti, E., Grange, A.: Number conserving cellular automata II: dynamics. Theoretical Computer Science 304(1-3), 269–290 (2003)
169. Fridrich, J.: Digital image forensics. IEEE Signal Processing Magazine 26(2), 26–37 (2009)
170. Chen, M., Fridrich, J., Lukáš, J., Goljan, M.: Imaging sensor noise as digital X-ray for revealing forgeries. In: Furon, T., Cayre, F., Doërr, G., Bas, P. (eds.) IH 2007. LNCS, vol. 4567, pp. 342–358. Springer, Heidelberg (2008)
171. Fridrich, J., Soukal, D., Lukás, J.: Detection of copy move forgery in digital images. In: Proc. Digital Forensic Research Workshop (2003)
172. Frisch, U., Hasslacher, B., Pomeau, Y.: Lattice-gas automata for the navier-stokes equation. Phys. Rev. Lett. 56(14), 1505–1508 (1986)
173. Frontczak, M., Wargocki, P.: Literature survey on how different factors influence human comfort in indoor environments. Building and Environment 46(4), 922–937 (2011)
174. Gabriel, R.K., Sokal, R.R.: A new statistical approach to geographic variation analysis. Systematic Zoology 18(3), 259–278 (1969)
175. Gagaudakis, G., Rosin, P.L.: Shape measures for image retrieval. Pattern Recognition Letters 24(15), 2711–2721 (2003)

176. Galanter, P.: What is generative art? Complexity theory as a context for art theory. In: Proceedings of the Generative Art Conference (GA) (2003)
177. Galanter, P.: Generative art and rules-based art. Vague Terrain 3 (2006)
178. Galanter, P.: Xepa. In: Forbes, A.G., Thorson, L. (eds.) The IEEE VIS 2013 Art Show Catalog, pp. 4–5 (2013)
179. Galanter, P.: Xepa: Intelligent sculptures as experimental platforms for computational aesthetic evaluation. In: Proceedings of the IEEE VIS Arts Program (VISAP). IEEE (2013)
180. Gánti, T.: Biogenesis itself. Journal of Theoretical Biology 187(4), 583–593 (1997)
181. Gánti, T.: The Principles of Life. Oxford University Press (2003)
182. Gao, Y., Yang, J., Xu, X., Shi, F.: Efficient cellular automaton segmentation supervised by pyramid on medical volumetric data and real time implementation with graphics processing unit. Expert Systems with Applications 38(6), 6866–6871 (2011)
183. Gardner, M.: Mathematical games: The fantastic combinations of John Conway's new solitaire game "life". Scientific American 223(4), 120–123 (1970)
184. Garzon, M.: Models of Massive Parallelism: Analysis of Cellular Automata and Neural Networks. In: European Association for Theoretical Computer Science. Springer, Heidelberg (1995)
185. Gasteratos, A., Andreadis, I.: Non-linear image processing in hardware. Pattern Recognition 33(6), 1013–1021 (2000)
186. Georgilas, I., Gale, E., Adamatzky, A., Melhuish, C.: UAV horizon tracking using memristors and cellular automata visual processing (2013)
187. Georgoulas, C., Kotoulas, L., Sirakoulis, G.C., Andreadis, I., Gasteratos, A.: Real-time disparity map computation module. Microprocess. Microsyst. 32(3), 159–170 (2008)
188. Gharehchopogh, F., Ebrahimi, S.: A novel approach for edge detection in images based on cellular learning automata. Int. J. Computer Vision and Image Processing 2(4), 51–61 (2012)
189. Ghosh, P., Antani, S., Long, L., Thoma, G.: Unsupervised grow-cut: Cellular automata-based medical image segmentation. In: Proceedings of the First IEEE International Conference on Healthcare Informatics, Imaging and Systems Biology (HISB), pp. 40–47. IEEE (2011)
190. Giavitto, J.L.: Topological collections, transformations and their application to the modeling and the simulation of dynamical systems. In: Nieuwenhuis, R. (ed.) RTA 2003. LNCS, vol. 2706, pp. 208–233. Springer, Heidelberg (2003)
191. Giavitto, J.L., Michel, O.: MGS: a programming language for the transformations of collections. Tech. Rep. 61-2001, LaMI, Universite d'Evry (2001), `mgs.ibisc.fr/PUBLICATIONS/lami-RR61--mgs.pdf`
192. Gil Montoya, M., Garcia, I.: Implementation of parallel thinning algorithms on multicomputers: analysis of the work load balance. In: Proceedings of the Sixth Euromicro Workshop on Parallel and Distributed Processing, PDP 1998, pp. 257–263 (1998)
193. Gladshtein, M.: Quantum-dot cellular automata serial decimal adder. IEEE Transactions on Nanotechnology 10(6), 1377–1382 (2011)
194. Gojman, B., Rachlin, E., Savage, J.E.: Evaluation of design strategies for stochastically assembled nanoarray memories. J. Emerg. Technol. Comput. Syst. 1(2), 73–108 (2005)
195. Goldberg, M.: A Class of Multi-Symmetric Polyhedra. Tohoku Mathematical Journal 43, 104–108 (1937)
196. Golub, G.H., van Van Loan, C.F.: Matrix computations (Johns Hopkins studies in mathematical sciences), 3rd edn. The Johns Hopkins University Press (1996)
197. González, R.C., Woods, R.E.: Digital image processing. Pearson/Prentice Hall (2008)

198. Gorsevski, P., Onasch, C., Farver, J., Ye, X.: Detecting grain boundaries in deformed rocks using a cellular automata approach. Computers & Geosciences 42, 136–142 (2012)
199. Grady, L., Funka-Lea, G.: Multi-label image segmentation for medical applications based on graph-theoretic electrical potentials. In: Sonka, M., Kakadiaris, I.A., Kybic, J. (eds.) CVAMIA/MMBIA 2004. LNCS, vol. 3117, pp. 230–245. Springer, Heidelberg (2004)
200. Graham, R.L.: An efficient algorithm for determining the convex hull of a finite planar set. Information Processing Letters 1(4), 132–133 (1972)
201. Greene, D.: Re: Geminoid challenges (November 23, 2013), http://www.conwaylife.com/forums/viewtopic.php?f=2&t=1006&p=9917#p9901
202. Grefenstette, J., Gopal, R., Rosimaita, B., Gucht, D.: Genetic Algorithms for the Traveling Salesman Problem. In: Proceedings of the 1st International Conference on Genetic Algorithms and their Applications, pp. 160–168. Psychology Press, Pittsburgh (1985)
203. Gresham-Lancaster, S.: The aesthetics and history of the hub: The effects of changing technology on network computer music. Leonardo Music Journal, 39–44 (1998)
204. Gross, J.L., Yellen, J. (eds.): Handbook of Graph Theory (Discrete Mathematics and Its Applications), 1st edn. CRC (2003)
205. Gruau, F., Eisenbeis, C., Maignan, L.: The foundation of self-developing blob machines for spatial computing. Physica D (2008)
206. Gruau, F., Lhuillier, Y., Reitz, P., Temam, O.: Blob computing. In: Computing Frontiers 2004 ACM SIGMicro (2004)
207. Gruau, F., Moszkowski, G.: The blob division a “hardware-free", time efficient, self-reproduction on 2d cellular automaton. Computing Frontiers 3141, 317–337 (2004)
208. Guay, M., Colin, F., Egli, R.: Simple and fast fluids. In: Engel, W. (ed.) GPU Pro 2, pp. 433–444 (2011)
209. Guo, Z., Hall, R.W.: Parallel thinning with two-subiteration algorithms. Communications of the ACM 32, 359–373 (1989)
210. Guo, Z., Hall, R.W.: Fast fully parallel thinning algorithms. CVGIP: Image Understanding 55, 317–328 (1992)
211. Hadwiger, M., Laura, F., Rezk-Salama, C., Hollt, T., Geier, G., Pabel, T.: Interactive volume exploration for feature detection and quantification in industrial CT data. IEEE Transactions on Visualization and Computer Graphics 14(6), 1507–1514 (2008)
212. Han, Y., Shi, P.: An improved ant colony algorithm for fuzzy clustering in image segmentation. Neurocomputing 70(4), 665–671 (2007)
213. Hanson, J.E.: Encyclopedia of Complexity and Systems Science. In: Meyers, R.A. (ed.) Cellular Automata, Emergent Phenomena, pp. 768–778. Springer (2009)
214. Haralick, R.M., Sternberg, S.R., Zhuang, X.: Image analysis using mathematical morphology. IEEE Transactions on Pattern Analysis and Machine Intelligence PAMI-9(4), 532–550 (1987)
215. Hardy, J., de Pazzis, O., Pomeau, Y.: Molecular dynamics of a classical lattice gas: Transport properties and time correlation functions. Pra. 13, 1949–1961 (1976)
216. Haris, K., Efstratiadis, S.N., Maglaveras, N., Katsaggelos, A.K.: Hybrid image segmentation using watersheds and fast region merging. IEEE Transactions on Image Processing 7(12), 1684–1699 (1998)
217. Hearn, D., Baker, P.M., Carithers, W.: Computer Graphics with Open GL, 4th edn. Prentice Hal (2010)
218. Heath, M., Sarkar, S., Sanocki, T., Bowyer, K.: Robust visual method for assessing the relative performance of edge detection algorithms. IEEE Trans. Pattern Analysis and Machine Intelligence 19(12), 1338–1359 (1997)

219. Heath, M.D., Sarkar, S., Sanocki, T.A., Bowyer, K.W.: Comparison of edge detectors: A methodology and initial study. Computer Vision and Image Understanding 69(1), 38–54 (1998)
220. Zawidzki, M., Nishinari, K.: Controlling the Opacity of a Building Envelope by a Triangular Two-color Two-dimensional Cellular Automaton. In: Sirakoulis, G.C., Bandini, S. (eds.) ACRI 2012. LNCS, vol. 7495, pp. 194–203. Springer, Heidelberg (2012)
221. Hernandez, G., Herrmann, H.: Cellular-automata for elementary image-enhancement. Graphical Models and Image Processing 58(1), 82–89 (1996)
222. Hernandez, O.J., Keohane, T., Steponanko, J.: A combined VLSI architecture for nonlinear image processing filters. In: Conference Proceedings - IEEE SOUTHEASTCON, vol. 2006, pp. 261–266 (2006)
223. Heydorn, S., Weidner, P.: Optimization and performance analysis of thinning algorithms on parallel computers. Parallel Computing 17(1), 17–27 (1991)
224. Hilditch, C.: An application of graph theory in pattern recognition. Machine Intelligence 3, 325–347 (1968)
225. Hinton, G.E., Salakhutdinov, R.: Reducing the dimensionality of data with neural networks. Science 313, 504–507 (2007)
226. Hoekstra, J.: On circuit theories for single-electron tunneling devices. IEEE Transactions on Circuits and Systems I: Regular Papers 54(11 spec. iss.), 2353–2359 (2007)
227. Hojjatoleslami, S.A., Kittler, J.: Region growing: A new approach. IEEE Transactions on Image Processing 7(7) (1998)
228. Holt, C., Stewart, A.: A parallel thinning algorithm with fine grain subtasking. Parallel Computing 10(3), 329–334 (1989)
229. Hongbin, P., Junali, C., Yashe, Z.: Fingerprint thinning algorithm based on mathematical morphology. In: 8th International Conference on Electronic Measurement and Instruments, ICEMI 2007, pp. 2-618–2-621 (2007)
230. Howard, A., Mataric, M., Sukhatme, G.: Mobile sensor network deployment using potential fields: A distributed (2002), `http://citeseer.ist.psu.edu/howard02mobile.html`
231. Huang, J., Momenzadeh, M., Lombardi, F.: Analysis of missing and additional cell defects in sequential quantum-dot cellular automata. Integration, the VLSI Journal 40(4), 503–515 (2007)
232. Huang, Y., Fan, H.: Learning from interpolated images using neural networks for digital forensics. In: IEEE Conference on Computer Vision and Pattern Recognition, pp. 177–182. IEEE, San Francisco (2010)
233. Hwang, J.W., Lee, H.S.: Adaptive image interpolation based on local gradient features. IEEE Signal Process. Lett. 29, 359–362 (2004)
234. Ilachinski, A.: Cellular automata: a discrete universe. World Scientific Publishing Company (2001)
235. Ilachinski, A.: Cellular Automata: A Discrete Universe. World Scientific Publishing Co. Inc., River Edge (2001)
236. Imai, K.: A computation-universal two-dimensional 8 state triangular reversible cellular automaton. Theoretical Computer Science 231(2), 181–191 (2000)
237. Imai, K., Fujita, K., Iwamoto, C., Morita, K.: Embedding a Logically Universal Model and a Self-Reproducing Model into Number-Conserving Cellular Automata. In: Calude, C.S., Dinneen, M.J., Peper, F. (eds.) UMC 2002. LNCS, vol. 2509, pp. 164–175. Springer, Heidelberg (2002)
238. Imre, A., Csaba, G., Ji, L., Orlov, A., Bernstein, G.H., Porod, W.: Majority logic gate for magnetic quantum-dot cellular automata. Science 311(5758), 205–208 (2006)
239. Intel: Sse4 programming reference. D91561- (2007)

240. International Technology Roadmap for Semiconductors: Emerging Research Devices, 2007 online edn., http://www.itrs.net
241. Ioannidis, K., Andreadis, I., Sirakoulis, G.C.: An edge preserving image resizing method based on cellular automata. In: Sirakoulis, G.C., Bandini, S. (eds.) ACRI 2012. LNCS, vol. 7495, pp. 375–384. Springer, Heidelberg (2012)
242. Isenberg, T., Hinrichs, U., Carpendale, S.: Studying direct-touch interaction for 2d flow visualization. In: Collaborative Visualization on Interactive Surfaces-CoVIS 2009, p. 17 (2009)
243. Jain, A.K.: Fundamentals of Digital Image Processing. Prentice-Hall, Upper Saddle River (1978)
244. Jana, B., Pal, P., Bhaumik, J.: New image noise reduction schemes based on cellular automata. International Journal of Soft Computing and Engineering 2(2), 98–103 (2012)
245. Jaromczyk, J.W., Toussaint, G.T.: Relative neighborhood graphs and their relatives. Proc. IEEE, 1502–1517 (1992)
246. Jarvis, R.A.: On the identification of the convex hull of a finite set of points in the plane. Information Processing Letters 2(1), 18–21 (1973)
247. Jiang, H., Moloney, C.: A new direction adaptive scheme for image interpolation. In: International Conference on Image Processing, pp. 369–372. Rochester, New York (2002)
248. Jiao, J., Long, G.J., Grandjean, F., Beatty, A.M., Fehlner, T.P.: Building blocks for the molecular expression of quantum cellular automata. isolation and characterization of a covalently bonded square array of two ferrocenium and two ferrocene complexes. Journal of the American Chemical Society 125(25), 7522–7523 (2003)
249. Jin, J.: An image encryption based on elementary cellular automata. In: Optics and Lasers in Engineering (2012)
250. Johnson, M.K., Farid, H.: Exposing digital forgeries by detecting inconsistencies in lighting. In: Proc. ACM Multimedia and Security Workshop, pp. 1–10 (2005)
251. Johnston, P., Kelso, J., Milne, G.J.: Efficient simulation of wildfire spread on an irregular grid. International Journal of Wildland Fire (2008)
252. Jones, J., Saeed, M.: Image enhancement - an emergent pattern formation approach via decentralised multi-agent systems. Multi-Agent and Grid Systems 3(4), 105–140 (2007)
253. Ju, L., Du, Q., Gunzburger, M.: Probabilistic methods for centroidal voronoi tessellations and their parallel implementations. Parallel Comput. 28(10), 1477–1500 (2002)
254. Kalogeropoulos, G., Sirakoulis, G.C., Karafyllidis, I.: Cellular automata on FPGA for real-time urban traffic signals control. Journal of Supercomputing 65(2), 1–18 (2013)
255. Kang, X., Wei, S.: Identifying tampered regions using singular value decomposition in digital image forensics. International Conference on Computer Science and Software Engineering 3, 926–930 (2008)
256. Kanj, I.A., Perković, L., Xia, G.: Local construction of near-optimal power spanners for wireless ad hoc networks. IEEE Transactions on Mobile Computing 8(4), 460–474 (2009)
257. Karafyllidis, I., Andreadis, I., Tzionas, P., Tsalides, P., Thanailakis, A.: A cellular automaton for the determination of the mean velocity of moving objects and its VLSI implementation. Pattern Recognition 29(4), 689–699 (1996)
258. Karafyllidis, I., Ioannidis, A., Thanailakis, A.: Geometrical shape recognition using a cellular automaton architecture and its VLSI implementation. Real-Time Imaging 3(4), 243–254 (1997)
259. Kari, J.: A counter example to a conjecture concerning synchronizing words in finite automata. Bulletin of the European Association for Theoretical Computer Science, 146 (2001)

260. Kari, J.: Theory of cellular automata: A survey. Theoretical Computer Science 304(1-3), 3–33 (2005)
261. Kari, J., Moore, C.: New results on alternating and non-deterministic two-dimensional finite-state automata. In: Ferreira, A., Reichel, H. (eds.) STACS 2001. LNCS, vol. 2010, pp. 396–406. Springer, Heidelberg (2001)
262. Kass, M., Witkin, A., Terzopoulos, D.: Snakes: Active contour models. International Journal of Computer Vision 1(4), 321–331 (1988)
263. Katis, I., Sirakoulis, G.: Cellular automata on FPGAs for image processing. In: 16th Panhellenic Conference on Informatics, pp. 308–313. IEEE Computer Society, Piraeus (2012)
264. Kauffmann, C., Piché, N.: Seeded ND medical image segmentation by cellular automaton on GPU. International Journal of Computer Assisted Radiology and Surgery 5(3), 251–262 (2010)
265. Kazar, O., Slatnia, S.: Evolutionary cellular automata for image segmentation and noise filtering using genetic algorithms. Journal of Applied Computer Science and Mathematics 5(10), 33–40 (2011)
266. Kepler, J.: Harmonice Mundi. Lincii Austriae (1619)
267. Keys, R.G.: Cubic convolution interpolation for digital image processing. IEEE Trans. Acoust., Speech, Signal Process. 29, 1153–1160 (1981)
268. Kiester, R.A., Sahr, K.: Planar and spherical hierarchical, multi-resolution cellular automata. Computers, Environment and Urban Systems 32, 204–213 (2008)
269. Kim, C.: Segmenting a low-depth-of-field image using morphological filters and region merging. IEEE Transactions on Image Processing 14(10), 1503–1511 (2005)
270. Kim, E., Shen, T., Huang, X.: A parallel cellular automata with label priors for interactive brain tumor segmentation. In: Int. Symposium on Computer-Based Medical Systems (CBMS), pp. 232–237. IEEE (2010)
271. Kim, J.T., Kim, G.: Overview and new developments in optical daylighting systems for building a healthy indoor environment. Building and Environment 45(2), 256–269 (2010)
272. Kim, K., Wu, K., Karri, R.: Towards designing robust QCA architectures in the presence of sneak noise paths. In: Proceedings of the Design, Automation and Test in Europe, DATE 2005, vol. II, pp. 1214–1219 (2005)
273. Kim, K., Wu, K., Karri, R.: Quantum-dot cellular automata design guideline. IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences E89-A(6), 1607–1614 (2006)
274. Kim, K., Wu, K., Karri, R.: The robust QCA adder designs using composable qca building blocks. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 26(1), 176–183 (2007)
275. Kirkpatrick, D.G., Seidel, R.: The ultimate planar convex hull algorithm? SIAM Journal on Computing 15(1), 287–299 (1986)
276. Kirsch, R.A., Cahn, L., Ray, C., Urban, G.H.: Experiments in processing pictorial information with a digital computer. Papers and Discussions Presented at the December 9-13, Eastern Joint Computer Conference: Computers with Deadlines to Meet, IRE-ACM-AIEE 1957 (Eastern), pp. 221–229. ACM, New York (1958)
277. Kiwibonga: Another cellular automaton video (April 5, 2011), `http://www.gamedev.net/blog/844/entry-2249737-another-cellular-automaton-video`
278. Knoll, A., Wald, I., Parker, S., Hansen, C.: Interactive isosurface ray tracing of large octree volumes. In: 2006 IEEE Symposium on Interactive Ray Tracing, pp. 115–124 (2006)

279. Kokare, M., Biswas, P., Chatterji, B.: Texture image retrieval using new rotated complex wavelet filters. IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics 35(6), 1168–1178 (2005)
280. Kolumban, G., Kennedy, M.P., Chua, L.O.: The role of synchronization in digital communication using chaos-part ii: Chaotic modulation and chaotic synchronization. IEEE Trans Circuits and Syst. I 45, 1129–1140 (1998)
281. Konopka, A.K.: Systems Biology: Principles, Methods and Concepts Taylor and Francis. Taylor & Francis, Boca Raton (2006)
282. Konstantinidis, K., Sirakoulis, G., Andreadis, I.: Content-based image retrieval using cellular automata. In: Proceedings of the 5th International Conference on Technology and Automation (ICTA 2005), Thessaloniki, Greece, pp. 371–375 (2005)
283. Konstantinidis, K., Sirakoulis, G.C., Andreadis, I.: Design and implementation of a fuzzy-modified ant colony hardware structure for image retrieval. IEEE Transactions on Systems, Man and Cybernetics Part C: Applications and Reviews 39(5), 520–533 (2009)
284. Konushin, V., Vezhnevets, V.: Interactive image colorization and recoloring based on coupled map lattices. In: Proceedings of Graphicon, Novosibirsk Akademgorodok, Russia, pp. 231–234 (2006)
285. Köster, G., Hartmann, D., Klein, W.: Microscopic pedestrian simulations: From passenger exchange times to regional evacuation. In: Operations Research Proceedings 2010, pp. 571–576. Springer (2011)
286. Krikor, L., Shaaban, Z.: Image encryption using DCT and stream cipher. European Journal of Scientific Research 32(1), 47–57 (2009)
287. Kumar, T., Sahoo, G.: A novel method of edge detection using cellular automata. International Journal of Computer Applications 9(4), 38–44 (2010)
288. Lafe, O.: Data compression and encryption using cellular automata transforms. Engineering Applications of Artificial Intelligence 10(6), 581–591 (1997)
289. Lafe, O.: Cellular Automata Transforms: Theory and Applications in Multimedia Compression. In: Encryption and Modeling. Kluwer Academic Publishers (2000)
290. Lafia, M.: Algorithms and allegories. Digital Creativity 14(2), 125–128 (2003)
291. Lala, R.: Morphon (2011), `http://www.riteshlala.net`
292. Laman, G.: On Graphs and Rigidity of Plane Skeletal Structures. Journal of Engineering Mathematics 4, 331–340 (1970)
293. Lee, C., Kim, D.-U., Shin, H., Kim, D.-S.: Efficient computation of elliptic Gabriel graph. In: Gavrilova, M.L., Gervasi, O., Kumar, V., Tan, C.J.K., Taniar, D., Laganá, A., Mun, Y., Choo, H. (eds.) ICCSA 2006. LNCS, vol. 3980, pp. 440–448. Springer, Heidelberg (2006)
294. Lee, C., Kim, D., Shin, H., Kim, D.S.: Trash removal algorithm for fast construction of the elliptic Gabriel graph using delaunay triangulation. Computer-Aided Design 40(8), 852–862 (2008)
295. Lee, J., Verleysen, M.: Nonlinear dimensionality reduction, 3rd edn. Springer (2007)
296. Lee, K., Xu, H., Chau, H.: Parity problem with a cellular automaton solution. Physical Review E 026702(64), 1–4 (2001)
297. Lee, K.H., Cho, S.B., Choy, Y.C.: Automated vectorization of cartographic maps by a knowledge-based system. Engineering Applications of Artificial Intelligence 13(2), 165–178 (2000)
298. Lee, M., Bruce, L.: Applying cellular automata to hyperspectral edge detection. In: Int. Geoscience and Remote Sensing Symposium, pp. 2202–2205 (2010)
299. Legrady, G.: George legrady studio: Algorithmic visualizations, `http://www.georgelegrady.com` (accessed November 2013)

300. Lengauer, T.: VLSI theory. In: Handbook of Theoretical Computer Science. algorithms and complexity, vol. A, pp. 835–866. MIT Press, Cambridge (1990)
301. Lent, C.S., Isaksen, B.: Clocked molecular quantum-dot cellular automata. IEEE Transactions on Electron Devices 50(9), 1890–1896 (2003)
302. Lent, C.S., Isaksen, B., Lieberman, M.: Molecular quantum-dot cellular automata. Journal of the American Chemical Society 125(4), 1056–1063 (2003)
303. Lent, C.S., Tougaw, P.D.: Lines of interacting quantum-dot cells: A binary wire. Journal of Applied Physics 74(10), 6227–6233 (1993)
304. Lent, C.S., Tougaw, P.D., Porod, W., Bernstein, G.H.: Quantum cellular automata. Nanotechnology 4(1), 49–57 (1993)
305. Levi, F.: On helly's theorem and the axioms of convexity. J. of Indian Math. Soc., 65–76 (1951)
306. Levner, I., Zhang, H.: Classification-driven watershed segmentation. IEEE Transactions on Image Processing 16(5), 1437–1445 (2007)
307. Lew, M., Sebe, N., Djeraba, C., Jain, R.: Content-based multimedia information retrieval: State of the art and challenges. ACM Transactions on Multimedia Computing, Communications, and Applications (TOMCCAP) 2(1), 1–19 (2006)
308. Li, G., Wu, Q., Tu, D., Sun, S.: A sorted neighborhood approach for detecting duplicated regions in image forgeries based on dwt and svd. In: IEEE International Conference on Multimedia and Expo, pp. 1750–1753 (2007)
309. Li, H., Liao, X., Li, C., Huang, H., Li, C.: Edge detection of noisy images based on cellular neural networks. Communications in Nonlinear Science and Numerical Simulation 16(9), 3746–3759 (2011)
310. Li, W., Han, G., Zou, K., Gu, G., Zhang, X.: Natural image matting using cellular automata. In: International Conference on Convergence Information Technology, 2007, pp. 1746–1751. IEEE (2007)
311. Li, X., Orchard, M.T.: New edge-directed interpolation. IEEE Trans. Image Process. 10, 1521–1527 (2001)
312. Lin, C.T., Fan, K.W., Pu, H.C., Lu, S.M., Liang, S.F.: An HVS-directed neural network based image resolution enhancement scheme for image resizing. IEEE Trans. Fuzzy Syst. 15, 605–615 (2007)
313. Lin, C.Y., Wu, M., Bloom, J., Cox, I., Miller, M., Lui, Y.: Rotation, scale, and translation resilient watermarking for images. IEEE Transactions on Image Processing 10(5), 767–782 (2001)
314. Lin, H., Wang, C., Kao, Y.: Fast copy-move forgery detection. WSEAS Transactions on Signal Processing 5(5), 188–197 (2009)
315. Lin, H.J., Kao, Y.T., Yen, S.H., Wang, C.J.: A study of shape-based image retrieval. In: Proceedings of the 24th International Conference on Distributed Computing Systems Workshops, pp. 118–123. IEEE (2004)
316. Lindenmayer, A., Prusinkiewicz, P., Hanan, J.: Development models of herbaceous plants for computer imagery purposes. SIGGRAPH Computer Graphics 22, 141–150 (1988)
317. Liu, L.: 3D thinning on cell complexes for computing curve and surface skeletons. Washington University (2009)
318. Liu, L., Chambers, E.W., Letscher, D., Ju, T.: A simple and robust thinning algorithm on cell complexes. Computer Graphics Forum 29(7), 2253–2260 (2010)
319. Lloyd, S.: Least squares quantization in PCM. IEEE Transactions on Information Theory 28, 129–137 (1982)
320. López-Mancilla, D., Cruz-Hernández, C.: Output synchronization of chaotic systems: Model-matching approach with application to secure communication. Nonlinear Dynamics and Systems Theory 5, 141–156 (2005)

321. Lorensen, W.E., Cline, H.E.: Marching cubes: A high resolution 3D surface reconstruction algorithm. ACM SIGGRAPH Computer Graphics 21(4), 163–169 (1987)
322. Lü, H.E., Wang, P.S.P.: A comment on a fast parallel algorithm for thinning digital patterns. Communications of the ACM 29(3), 239–242 (1986)
323. Luo, W., Huang, J., Qiu, G.: Robust detection of region-duplication forgery in digital images. IEEE Information Forensics and Security 4, 746–749 (2006)
324. van der Maaten, L., Postma, E., van den Herik, J.: Dimensionality reduction: a comparative review. Tilburg University Technical Report TiCC-TR 2009-005 (2009)
325. MacQueen, J.B.: Some methods for classification and analysis of multivariate observations. In: Cam, L.M.L., Neyman, J. (eds.) Proc. of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, vol. 1, pp. 281–297. University of California Press (1967)
326. Madison, K.C.: Barrier function of the skin: Araison d'etre of the epidermis. Journal of Investigative Dermatology 121(2), 231–241 (2003)
327. Maignan, L.: Uniformisation de la répartition spatiale d'un ensemble de points par diagramme de Voronoi: Implémentation sur automate cellulaire. Master's thesis, Université Paris XI, Orsay, France (2007)
328. Maignan, L., Gruau, F.: A 1D cellular automata that moves particles until regular spatial placement (2008)
329. Maignan, L., Gruau, F.: Integer gradient for cellular automata: Principle and examples. In: Spatial Computing Workshop at IEEE SASO 2008 (2008)
330. Maignan, L., Gruau, F.: Convex hulls on cellular automata. In: Bandini, S., Manzoni, S., Umeo, H., Vizzari, G. (eds.) ACRI 2010. LNCS, vol. 6350, pp. 69–78. Springer, Heidelberg (2010)
331. Maignan, L., Gruau, F.: Gabriel graphs in arbitrary metric space and their cellular automaton for many grids. ACM Trans. Auton. Adapt. Syst. (2010)
332. Maniatty, W., Szymanski, B.: Fine-grain discrete Voronoi diagram algorithms in L_1 and L_∞ norms. Mathematical and Computer Modelling 26, 71–78 (1997)
333. Manimaran, M., Snider, G.L., Lent, C.S., Sarveswaran, V., Lieberman, M., Li, Z., Fehlner, T.P.: Scanning tunneling microscopy and spectroscopy investigations of QCA molecules. Ultramicroscopy 97(1-4), 55–63 (2003)
334. Mao, K.: Identifying critical variables of principal components for unsupervised feature selection. IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics 35(2), 339–344 (2005)
335. Maragos, P., Schafer, R.W.: Morphological filters - Part I: Their set - theoretic analysis and relations to linear shift - invariant filters. IEEE Transactions on Acoustics, Speech, and Signal Processing ASSP-35(8), 1153–1169 (1987)
336. Mardiris, V.A.: Design and simulation of quantum cellular automata nanoelectronic circuits. PhD thesis, Democritus University of Thrace (2011)
337. Mardiris, V.A., Karafyllidis, I.G.: Universal cellular automaton cell using quantum cellular automata. Electronics Letters 45(12), 607–609 (2009)
338. Mardiris, V.A., Karafyllidis, I.G.: Design and simulation of modular 2n to 1 quantum-dot cellular automata (QCA) multiplexers. International Journal of Circuit Theory and Applications 38(8), 771–785 (2010)
339. Mardiris, V.A., Karafyllidis, I.G.: Design and simulation of modular quantum-dot cellular automata multiplexers for memory accessing. Journal of Circuits, Systems and Computers 19(2), 349–365 (2010)
340. Markus, M., Kusch, I.: Cellular automata for modelling shell pigmentation of molluscs. Journal of Biological Systems 3, 999–1011 (1995)

341. Martin, D., Fowlkes, C., Malik, J.: Learning to detect natural image boundaries using local brightness, color, and texture cues. IEEE Trans. Pattern Analysis and Machine Intelligence 26(5), 530–549 (2004)
342. Men, H., Zhang, J., Wang, C.: Measurement of inhibition zone based on cellular automata edge detection method. In: Int. Workshop on Education Technology and Computer Science, vol. 2, pp. 357–360 (2009)
343. Meyer, F., Beucher, S.: Morphological segmentation. Journal of Visual Communication and Image Representation 1(1), 21–46 (1990)
344. Michailidis, G., Andreadis, I.: Color processing – a new approach based on discrete dynamic systems. In: Proceedings of the CREATE 2010 International Conference, Norway (2010), http://www.create.uwe.ac.uk/norway_paperlist/michailidis.pdf
345. Michailidis, G., Andreadis, I.: A real-time stereo correspondence algorithm based on 2-D cellular automata. In: Int. Workshop on Advanced Imaging Technology, Kuala Lumbur, Malaysia, pp. 1–6 (2010)
346. Millen, D.: Cellular automata music. In: Proceedings of the 1990 International Computer Music Conference, pp. 314–316 (1990)
347. Minoofam, S.A.H., et al.: Ad-hoc Ma'qeli Script Generation Using Block Cellular Automata. Journal of Cellular Automata 7(4) (2012)
348. Mirzaei, K., Motameni, H., Enayatifar, R.: New method for edge detection and denoising via fuzzy cellular automata. Int. J. Phy. Sci. 6(13), 3175–3180 (2011)
349. Mitrea, C., Ionescu, B., Dogaru, R.: A pseudo-random scan perspective to the motion detection paradigm. In: Proceedings of the ISEEE Conference, Galati, Romania (2013)
350. Mohanty, B.K., Meher, P.K.: New scan method and pipeline architecture for VLSI implementation of separable 2-D FIR filters without transposition. In: IEEE Region 10 Annual International Conference, Proceedings/TENCON (2008)
351. Momenzadeh, M., Huang, J., Lombardi, F.: Defect characterization and tolerance of QCA sequential devices and circuits. In: Proceedings - IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems, pp. 199–207 (2005)
352. Mortensen, E., Barrett, W.: Toboggan-based intelligent scissors with a four-parameter edge model (1999)
353. Mortensen, E.N., Barrett, W.A.: Interactive segmentation with intelligent scissors. Graphical Models and Image Processing 60(5), 349–384 (1998)
354. Motte, W.F.: Oulipo: a primer of potential literature. University of Nebraska Press Lincoln, London (1986)
355. Moustapha, H., Krishnamurti, R.: Arabic calligraphy a computational exploration. In: 3rd International Conference on Mathematics and Design, pp. 294–306 (2001)
356. Mukherjee, K.: Application of the Gabriel graph to instance based learning algorithms. Master's thesis, SFU CS School (2004)
357. Muresan, D., Parks, T.: Adaptively quadratic (aqua) image interpolation. IEEE Trans Image Process 13, 690–698 (2004)
358. Nagpal, R.: Programmable self-assembly: constructing global shape using biologically-inspired local interactions and origami mathematics. Ph.D. thesis. MIT (2001)
359. Nagpal, R.: Programmable self-assembly using biologically-inspired multiagent control. In: AAMAS 2002: Proceedings of the First International Joint Conference on Autonomous Agents and Multiagent Systems, pp. 418–425 (2002)
360. Nalpantidis, L., Amanatiadis, A., Sirakoulis, G.C., Gasteratos, A.: Efficient hierarchical matching algorithm for processing uncalibrated stereo vision images and its hardware architecture. IET Image Processing 5(5), 481–492 (2011)

361. Nasu, M.: Local maps inducing surjective global maps of one-dimensional tessellation automata. Theory of Computing Systems 11, 327–351 (1977)
362. Navi, K., Farazkish, R., Sayedsalehi, S., Rahimi Azghadi, M.: A new quantum-dot cellular automata full-adder. Microelectronics Journal 41(12), 820–826 (2010)
363. Nedzved, A., Ilyich, Y., Ablameyko, S., Kamata, S.: Color thinning with applications to biomedical images. In: Skarbek, W. (ed.) CAIP 2001. LNCS, vol. 2124, pp. 256–263. Springer, Heidelberg (2001)
364. Nehaniv, C.: Self-reproduction in asynchronous cellular automata. In: Proceedings of the 2002 NASA/DoD Conference on Evolvable Hardware (EH 2002), p. 201 (2002)
365. Neto, O.P.V., Pacheco, M.A.C., Hall Barbosa, C.R.: Neural network simulation and evolutionary synthesis of QCA circuits. IEEE Transactions on Computers 56(2), 191–201 (2007)
366. Neumann, J.V.: Theory of Self-Reproducing Automata. University of Illinois Press, Champaign (1966)
367. Newman, D.J.: The hexagon theorem. IEEE Transactions on Information Theory 28(2), 137–138 (1982)
368. Newman, T.S., Hong, Y.: A survey of the marching cubes algorithm. Computers and Graphics 30(5), 854–879 (2006)
369. Nezamabadi-pour, H., Saryazdi, S., Rashedi, E.: Edge detection using ant algorithms. Soft Computing 10(7), 623–628 (2006)
370. Niemier, M.T., Kogge, P.M.: Problems in designing with QCAs: Layout = timing. International Journal of Circuit Theory and Applications 29(1), 49–62 (2001)
371. Niemier, M.T., Kontz, M.J., Kogge, P.M.: Design of and design tools for a novel quantum dot based microprocessor. In: Proceedings - Design Automation Conference, pp. 228–232 (2000)
372. Ning, J., Zhang, L., Zhang, D., Wu, C.: Interactive image segmentation by maximal similarity based region merging. Pattern Recognition 43(2), 445–456 (2010)
373. O'Donoghue, D.P., Mullally, E.C.: Extending Irregular Cellular Automata with Geometric Proportional Analogies. In: Proceedings of the Geographical Information Science Research UK Conference, April 11-13. NUI Maynooth, Ireland (2007)
374. Ojala, T., Pietikainen, M., Maeenpaa, T.: Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. IEEE Transactions on Pattern Analysis and Machine Intelligence 24(7), 971–987 (2002)
375. de Oliveira, P., Bortot, J., Oliveira, G.: The best currently known class of dynamically equivalent cellular automata rules for density classification. Neurocomputing 70(1-3), 35–43 (2006)
376. Orlov, A.O., Amlani, I., Bernstein, G.H., Lent, C.S., Snider, G.L.: Realization of a functional cell for quantum-dot cellular automata. Science 277(5328), 928–930 (1997)
377. O'Rourke, K.: Walking and Mapping: Artists as Cartographers. MIT Press (2013)
378. O'Sullivan, D.: Exploring Spatial Process Dynamics Using Irregular Cellular Automaton Models. Geographical Analysis 33(1), 1–18 (2001)
379. Otsu, N.: A threshold selection method from gray-level histograms. IEEE Trans. SMC 9, 62–66 (1979)
380. Oya, T., Motoike, I.N., Asai, T.: Single-electron circuits performing dendritic pattern formation with nature-inspired cellular automata. International Journal of Bifurcation and Chaos 17, 3651–3655 (2007)
381. Pal, N., Pal, S.: A review on image segmentation techniques. Pattern Recognition 26(9), 1277–1294 (1993)

382. Panagiotopoulos, F.K., Mardiris, V.A., Chatzis, V.: Quantum-dot cellular automata design for median filtering and mathematical morphology operations on binary images. In: Sirakoulis, G.C., Bandini, S. (eds.) ACRI 2012. LNCS, vol. 7495, pp. 554–564. Springer, Heidelberg (2012)
383. Panda, S.P., Sahu, M., Rout, U.P., Nanda, S.K.: Encryption and decryption algorithm using two dimensional cellular automata rules in cryptography. International Journal of Communication Network & Security 1, 18–23 (2011)
384. Park, J.C., Shin, H., Choi, B.K.: Elliptic Gabriel graph for finding neighbors in a point set and its application to normal vector estimation. Computer-Aided Design 38(6), 619–626 (2006)
385. Patwardhan, J.P., Dwyer, C., Lebeck, A.R., Sorin, D.J.: Circuit and system architecture for DNA-guided self-assembly of nanoelectronics. In: Foundations of Nanoscience: Self-Assembled Architectures and Devices (FNANO), pp. 344–358 (2004)
386. Paun, G.: Membrane Computing: An Introduction. Springer-Verlag New York, Inc., Secaucus (2002)
387. Pavlidis, T.: Algorithms for Graphics and Image Processing. Digital system design series. Computer Science Press (1982)
388. Peer, M., Qadir, F., Khan, K.: Investigations of cellular automata game of life rules for noise filtering and edge detection. Int. J. Information Engineering and Electronic Business 4(2), 22–28 (2012)
389. Pegg, E.: Half-Distance Rules with Low Resolution. An interactive demonstration (2000), `http://demonstrations.wolfram.com/HalfDistanceRulesWithLowResolution/`
390. Pegg, E., Zawidzki, M.: Cellular Shading. An interactive demonstration (2008), `http://demonstrations.wolfram.com/CellularShading/`
391. Peña-Cantillana, F., Berciano, A., Díaz-Pernil, D., Gutiérrez-Naranjo, M.A.: Parallel skeletonizing of digital images by using cellular automata. In: Ferri, M., Frosini, P., Landi, C., Cerri, A., Di Fabio, B. (eds.) CTIC 2012. LNCS, vol. 7309, pp. 39–48. Springer, Heidelberg (2012)
392. Perona, P., Malik, J.: Scale-space and edge detection using anisotropic diffusion. IEEE Trans. Pattern Analysis and Machine Intelligence 12(7), 629–639 (1990)
393. Pesaresi, M., Benediktsson, J.A.: A new approach for the morphological segmentation of high-resolution satellite imagery. IEEE Transactions on Geoscience and Remote Sensing 39(2), 309–320 (2001)
394. Peters, H.M.: Functional organization of the spinning apparatus of Cyrtophora citricola with regard to the evolution of the web (Araneae, Araneidae). Zoomorphology 113(3), 153–163 (1993)
395. Petrou, M., Petrou, C.: Image processing: the fundamentals. Wiley (2010)
396. Phan, R., Androutsos, D.: Interactive video growcut: A semi-automated video object segmentation framework using cellular automata. In: 2011 24th Canadian Conference on Electrical and Computer Engineering (CCECE), pp. 81–84. IEEE (2011)
397. Piao, Y., Kim, S., Cho, S.J.: Two-dimensional cellular automata transforms for a novel edge detection. In: IComputability in Europe 2008, Logic and Theory of Algorithms (2008)
398. Pirsch, P., Stolberg, H.J.: VLSI implementations of image and video multimedia processing systems. IEEE Transactions on Circuits and Systems for Video Technology 8(7), 878–891 (1998)
399. Piwonska, A., Seredynski, F.: Discovery by genetic algorithm of cellular automata rules for pattern reconstruction task. In: Bandini, S., Manzoni, S., Umeo, H., Vizzari, G. (eds.) ACRI 2010. LNCS, vol. 6350, pp. 198–208. Springer, Heidelberg (2010)

400. Pluim, J.P.W., Maintz, J.B.A., Viergever, M.A.: Image registration by maximization of combined mutual information and gradient information. IEEE Trans. Med. Imaging 19(8), 809–814 (2000)
401. Poduri, S., Sukhatme, G.: Constrained coverage for mobile sensor networks (2004), http://citeseer.ist.psu.edu/poduri04constrained.html
402. Pommer, A., Uhl, A.: Selective encryption of wavelet-packet encoded image data: efficiency and security. Multimedia Systems 9(3), 279–287 (2003)
403. Popescu, A., Farid, H.: Exposing digital forgeries by detecting duplicated image regions. Tech. rep. tr2004-515, Dartmouth College (2004)
404. Popovici, A., Popovici, D.: Cellular automata in image processing. In: Int. Symp. on the Mathematical Theory of Networks and Systems (2002)
405. Porter, R., Frigo, J., Conti, A., Harvey, N., Kenyon, G., Gokhale, M.: A reconfigurable computing framework for multi-scale cellular image processing. Microprocess. Microsyst. 31(8), 546–563 (2007)
406. Preston, K.: D.J.: Modern Cellular Automata: Theory and Applications. Plenum Press (1984)
407. Priego, B., Bellas, F., Souto, D., López-Peña, F., Duro, R.: Evolving cellular automata for detecting edges in hyperspectral images. In: Int. Conf. on Fuzzy Systems, pp. 1–6 (2012)
408. Progias, P., Sirakoulis, G.C.: An FPGA processor for modelling wildfire spread. Mathematical and Computer Modeling 57(5-6), 1436–1452 (2013)
409. Proksch, E., Brandner, J.M., Jensen, J.M.: The skin: an indispensable barrier. Experimental Dermatology 17(12), 1063–1072 (2008)
410. Prusinkiewicz, P., Lindenmayer, A., Hanan, J.S., Fracchia, F.D., Fowler, D.R., de Boer, M.J., Mercer, L.: The algorithmic beauty of plants. Springer (1990)
411. Pudil, P., Novovicova, J., Kittler, J.: Floating search methods in feature-selection. Pattern Recognition Letters 15(11), 1119–1125 (1994)
412. Qadir, F., Khan, K.: Investigations of cellular automata linear rules for edge detection. Int. J. Computer Network and Information Security 3, 47–53 (2013)
413. Qadir, F., Peer, M., Khan, K.: Efficient edge detection methods for diagnosis of lung cancer based on two-dimensional cellular automata. Advances in Applied Science Research 3(4), 2050–2058 (2012)
414. Qasim, S.M., Abbasi, S.A., Almashary, B.: An Overview of Advanced FPGA Architectures for Optimized Hardware Realization of Computation Intensive Algorithms. In: Multimedia, Signal Processing and Communication Technologies (IMPACT 2009), pp. 300–303. Institute of Electrical and Electronics Engineers, Aligarh (2009)
415. Rabaey, J.M., Gass, W., Brodersen, R., Nishitani, T.: VLSI design and implementation fuels the signal-processing revolution: The design and implementation of signal-processing systems technical committee. IEEE Signal Processing Magazine 15(1), 22–37 (1998)
416. Rado, T.: Uber den Begriff der Riemannschen Flache. Acta Szeged 2(2), 101–121 (1925) (in German)
417. Rafler, S.: Generalization of Conway's "game of life" to a continuous domain-smoothlife. arXiv preprint arXiv:1111.1567 (2011)
418. Rao, C., Kumar, S.: Content based image retrieval fundamentals and algorithms: basics, concepts and novel algorithms. LAP Lambert Academic Publishing (2012)
419. Rauch, E.: Discrete, amorphous physical models. International Journal of Theoretical Physics 42(2), 329–348 (2003)
420. Reas, C.: Process Compendium 2004-2010. REAS Studio (2010)
421. Reas, C.: A database for Casey Reas, http://reas.com (accessed November 2013)

422. Rechenberg, I.: Evolutionsstrategie: Optimierung Technischer Systeme Nach Prinzipien Der Biologischen Evolution. Ph.D. thesis, Stuttgart (1973) (in German)
423. Regev, A., Panina, E.M., Silverman, W., Cardelli, L., Shapiro, E.: Bioambients: an abstraction for biological compartments. Theor. Comput. Sci. 325(1), 141–167 (2004)
424. Roads, C.: The computer music tutorial. MIT Press (1996)
425. Roberts, C.: Control: Software for end-user interface programming and interactive performance. In: Proceedings of the International Computer Music Conference (ICMC), Huddersfield, UK, pp. 425–428 (2011)
426. Roberts, C., Forbes, A., Höllerer, T.: Enabling multimodal mobile interfaces for interactive musical performance. In: Proceedings of the International Conference on New Interfaces for Musical Expression (NIME), Daejeon, Korea, pp. 102–105 (2013)
427. Roberts, L.: Machine Perception of Three-Dimensional Solids. In: Outstanding Dissertations in the Computer Sciences, Garland Publishing, New York (1963)
428. Ronjom, S., Abdelraheem, M., Danielsen, L.E.: TT and ANF representations of boolean functions. Online database of Boolean functions (2013), `http://www.selmer.uib.no/odbf/help/ttanf.pdf` (last accessed September 28, 2013)
429. Rosenfeld, A.: A characterization of parallel thinning algorithms. Information and Control 29(3), 286–291 (1975)
430. Rosin, P.L.: Training cellular automata for image processing. IEEE Trans. on Image Processing 15(7), 2076–2087 (2006)
431. Rosin, P.L.: A simple method for detecting salient regions. Pattern Recognition 42(11), 2363–2371 (2009)
432. Rosin, P.L.: Image processing using 3-state cellular automata. Computer Vision and Image Understanding 114(7), 790–802 (2010)
433. Rosin, P.L., Sun, X.: Cellular automata as a tool for image processing. In: Chen, C.H. (ed.) Emerging Topics in Computer Vision and its Applications, pp. 233–251 (2011)
434. Rothman, T.: Science a la Mode, chap. Geodesics, Domes, and Spacetime. Princeton University Press (1989)
435. Rozenberg, G.: The Mathematical Theory of L Systems, vol. 90. Academic Press (1980)
436. Ruck, N., Aschehoug, O., Aydinli, S., Christoffersen, J., Courret, G., Edmonds, I., Jakobiak, R., Kischkoweit-Lopin, M., Klinger, M., Lee, E.S., Michel, L., Scartezzini, J.L., Selkowitz, S.E.: Daylight in buildings: A Source Book on Daylighting Systems and Components. Tech. Rep. LBNL-47493, Lawrence Berkeley National Laboratory, Berkeley, California (2013), `http://gaia.lbl.gov/iea21/`
437. Rutovitz, D.: Pattern recognition. Journal of the Royal Statistical Society 129, 504–530 (1966)
438. Ryu, S.-J., Lee, M.-J., Lee, H.-K.: Detection of copy-rotate-move forgery using zernike moments. In: Böhme, R., Fong, P.W.L., Safavi-Naini, R. (eds.) IH 2010. LNCS, vol. 6387, pp. 51–65. Springer, Heidelberg (2010)
439. Saadatian, O., Sopian, K., Lim, C., Asim, N., Sulaiman, M.Y.: Trombe walls: A review of opportunities and challenges in research and development. Renewable and Sustainable Energy Reviews 16(8), 6340–6351 (2012)
440. Saeed, K., Tabedzki, M., Rybnik, M., Adamski, M.: K3M: A universal algorithm for image skeletonization and a review of thinning techniques. Applied Mathematics and Computer Science 20(2), 317–335 (2010)
441. Sahota, P., Daemi, M., Elliman, D.: Training genetically evolving cellular automata for image processing. In: Int. Symp. Speech, Image Processing and Neural Networks, pp. 753–756 (1994)

442. de Saint Pierre, T., Milgram, M.: New and efficient cellular algorithms for image processing. CVGIP: Image Understanding 55(3), 261–274 (1992)
443. Sakkal, M.: Art of Arabic Calligraphy (1993), http://www.sakkal.com/ArtArabicCalligraphy.html
444. Saponara, S., Fanucci, L., Terreni, P.: Design of a low-power VLSI macrocell for nonlinear adaptive video noise reduction. Eurasip Journal on Applied Signal Processing 2004(12), 1921–1930 (2004)
445. Sato, S., Kanoh, H.: Evolutionary design of edge detector using rule-changing cellular automata. In: Nature & Biologically Inspired Computing, pp. 60–65 (2010)
446. Selvapeter, J., Hordijk, W.: Genetically evolved cellular automata for image edge detection. In: Proceedings of the International Conference on Signal, Image Processing and Pattern Recognition, SIPP 2013 (2013)
447. Selvapeter, P.J., Hordijk, W.: Cellular automata for image noise filtering. In: Proceedings of the World Congress on Nature and Biologically Inspired Computing (NaBIC), pp. 193–197 (2009)
448. Sen, B., Anand, A.S., Adak, T., Sikdar, B.K.: Thresholding using quantum-dot cellular automata. In: 2011 International Conference on Innovations in Information Technology, IIT 2011, pp. 356–360 (2011)
449. Senthilkumar, S., Piah, A.R.M.: An improved fuzzy cellular neural network (IFCNN) for an edge detection based on parallel RK (5,6) approach. International Journal of Computational Systems Engineering 1(1), 70–78 (2012)
450. Serra, J.: Image Analysis and Mathematical Morphology. Acad. Press (1993)
451. Serra, R.: Chapter 6 – Daylighting. Renewable and Sustainable Energy Reviews 2(1-2), 115–155 (1998)
452. Shamsabadi, A.S., Ghahfarokhi, B.S., Zamanifar, K., Movahedinia, N.: Applying inherent capabilities of quantum-dot cellular automata to design: D flip-flop case study. Journal of Systems Architecture 55(3), 180–187 (2009)
453. Shanken, E.A.: Art in the information age: Technology and conceptual art. Leonardo 35(4), 433–438 (2002)
454. Shanken, E.A.: Cybernetics and art: Cultural convergence in the 1960s. In: Clarke, B., Henderson, L.D. (eds.) From Energy to Information, pp. 155–177. Stanford University Press, Palo Alto (2002)
455. Sharma, P., Lal, N., Diwakar, M.: Text security using 2d cellular automata rules. In: Proceedings of the Conference on Advances in Communication and Control Systems 2013, Atlantis Press (2013)
456. Shi, H., Ward, R.: Canny edge based image expansion. In: IEEE International Symposium on Circuits and Systems, pp. 785–788. IEEE, Scottsdale (2002)
457. Shin, M.C., Goldgof, D.B., Bowyer, K.W.: Comparison of edge detector performance through use in an object recognition task. Computer Vision and Image Understanding 84(1), 160–178 (2001)
458. Shivakumar, B.L., Baboo, S.: Detecting copy-move forgery in digital images: A survey and analysis of current methods. Global Journal of Computer Science and Technology 10(7), 61–65 (2010)
459. Shivakumar, B.L., Baboo, S.: Detection of region duplication forgery in digital images using surf. International Journal of Computer Science Issues 8(4), 199–205 (2011)
460. Siddiqi, K., Pizer, S.: Medial representations: mathematics, algorithms and applications. In: Computational Imaging and Vision. Springer (2008)
461. Singer, I.: Abstract convex analysis. John Wiley & Sons Inc., (1997)
462. Sipper, M.: Co-evolving non-uniform cellular automata to perform computations. Physica D 92, 193–208 (1996)

463. Sirakoulis, G.C., Karafyllidis, I., Thanailakis, A.: A CAD system for the construction and VLSI implementation of cellular automata algorithms using VHDL. Microprocessors and Microsystems 27(8), 381–396 (2003)
464. Sirakoulis, G.C., Karafyllidis, I., Thanailakis, A., Mardiris, V.: A methodology for VLSI implementation of cellular automata algorithms using VHDL. Advances in Engineering Software 32(3), 189–202 (2000)
465. Slatnia, S., Batouche, M., Melkemi, K.: Evolutionary cellular automata based-approach for edge detection. Applications of Fuzzy Sets Theory 4578, 404–411 (2007)
466. Smith, M.A.: Cellular automata methods in mathematical physics. Tech. rep., Cambridge, MA, USA (1994)
467. Smith, S.J., Bourgoin, M.O., Sims, K., Voorhees, H.L.: Handwritten character classification using nearest neighbor in large databases. IEEE Transactions on Pattern Analysis and Machine Intelligence 16(9), 915–919 (1994)
468. Soille, P.: Morphological Image Analysis: Principles and Applications. Springer (2010)
469. Solomos, M.: Cellular automata in Xenakis's music. Theory and practice. In: Definitive Proceedings of the International Symposium Iannis Xenakis, Athens (May 2006)
470. Stallings, W.: Cryptography and Network Security, 3rd edn. Practice Hall (2003)
471. Stam, J.: Stable fluids. In: Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques, pp. 121–128. ACM Press/Addison-Wesley Publishing Co. (1999)
472. Stam, J.: Real-time fluid dynamics for games. In: Proceedings of the Game Developer Conference, vol. 18 (2003)
473. Steiglitz, K., Kamal, I., Watson, A.: Embedding computation in one-dimensional automata by phase coding solitons. IEEE Transactions on Computers 37(2), 138–145 (1988)
474. Stevens, D., Dragicevic, S.: A GIS-based irregular cellular automata model of land-use change. Environment and Planning B: Planning and Design 34(4), 708–724 (2007)
475. Strauss, W.: Digital signal processing. IEEE Signal Processing Magazine 17(2), 52–56 (2000)
476. Sudha, N., Nandi, S., Sridharan, K.: A parallel algorithm to construct voronoi diagram and its VLSI architecture. In: Proceedings of the 1999 IEEE International Conference on Robotics and Automation, 1999, Detroit, MI, vol. 3, pp. 1683–1688 (1999)
477. Sun, X., Rosin, P.L., Martin, R.R.: Fast rule identification and neighborhood selection for cellular automata. IEEE Transactions on Systems, Man, and Cybernetics - Part B 41(3), 749–760 (2011)
478. Suyi, L., Qian, W., Heng, Z.: Edge detection of fabric defect based on fuzzy cellular automata. In: Int. Workshop on Intelligent Systems and Applications, pp. 1–3 (2009)
479. Suzuki, S., Abe, K.: Binary picture thinning by an iterative parallel two-subcycle operation. Pattern Recognition 20(3), 297–307 (1987)
480. Tafti, A.P., Malakooti, M., Ashourian, M., Janosepah, S.: Digital image forgery detection through data embedding in spatial domain and cellular automata. In: Int. Conf. on Digital Content, Multimedia Technology and its Applications (IDCTA), pp. 11–15 (2011)
481. Taskin, B., Hong, B.: Improving line-based QCA memory cell design through dual phase clocking. IEEE Transactions on Very Large Scale Integration (VLSI) Systems 16(12), 1648–1656 (2008)
482. Teja, V.C., Polisetti, S., Kasavajjala, S.: QCA based multiplexing of 16 arithmetic and logical subsystems-a paradigm for nano computing. In: 3rd IEEE International Conference on Nano/Micro Engineered and Molecular Systems, NEMS, pp. 758–763 (2008)

483. Telea, A., van Wijk, J.: 3d IBVF: Hardware-accelerated 3d flow visualization. In: Proceedings of IEEE Visualization (VIS), pp. 233–240. IEEE Computer Society (2003)
484. Terrazas, G., Siepmann, P., Krasnogor, N.: An evolutionary methodology for the automated design of cellular automaton-based complex systems. Journal of Cellular Automata 2(1), 77–102 (2008)
485. Toffoli, T.: Cellular automata as an alternative to (rather than an approximation of) differential equations in modeling physics. Physica D: Nonlinear Phenomena 10(1-2), 117–127 (1984)
486. Toffoli, T.: Programmable matter methods. Future Gener. Comput. Syst. 16(2-3), 187–201 (1999)
487. Toffoli, T., Margolus, N.: Cellular Automata Machines: A New Environment for Modeling. MIT Press (1987)
488. Torbey, S., Akl, S.G.: An exact and optimal local solution to the two-dimensional convex hull of arbitrary points problem. J. Cellular Automata 4(2), 137–146 (2009)
489. Torrens, P.M., Benenson, I.: Geographic automata systems. International Journal of Geographical Information Science 19, 4–385 (2005)
490. Tougaw, P.D.: A device architecture for computing with quantum dots. Proceedings of the IEEE 85(4), 541–557 (1997)
491. Tougaw, P.D., Lent, C.S.: Logical devices implemented using quantum cellular automata. Journal of Applied Physics 75(3), 1818–1825 (1994)
492. Tralic, D., Rosin, P.L., Sun, X., Grgic, S.: Detection of duplicated image regions using cellular automata. In: International Conference on Systems, Signals and Image Processing (accepted for publishing 2014)
493. Tralic, D., Zupancic, I., Grgic, S., Grgic, M.: Comofod-new database for copy-move forgery detection. In: Proc. 55th International Symposium, ELMAR 2013, pp. 49–54 (2013)
494. Tseng, P.C., Chang, Y.C., Huang, Y.W., Fang, H.C., Huang, C.T., Chen, L.G.: Advances in hardware architectures for image and video coding - a survey. Proceedings of the IEEE 93(1), 184–197 (2005)
495. Tyrrell, A.M., Sánchez, E., Floreano, D., Tempesti, G., Mange, D., Moreno, O., Rosenberg, A.L., Villa, A.E.P.: POEtic tissue: An integrated architecture for bio-inspired hardware. In: Tyrrell, A.M., Haddow, P.C., Torresen, J. (eds.) ICES 2003. LNCS, vol. 2606, pp. 129–140. Springer, Heidelberg (2003)
496. Umeo, H., Kubo, K.: A seven-state time-optimum square synchronizer. In: Bandini, et al. (eds.) [38], pp. 219–230
497. Urias, J.: Cryptography primitives based on a cellular automaton. In: Coding Theory, Cryptography and Related Areas, pp. 244–248 (2000)
498. Valvassori, M.: Biologically inspired self-organized blob structures on amorphous computers. In: SASO, pp. 359–362. IEEE Computer Society (2007), `http://dblp.uni-trier.de/db/conf/saso/saso2007.html#Valvassor%i07`
499. Van Droogenbroeck, M., Benedett, R.: Techniques for a selective encryption of uncompressed and compressed images. In: Proceedings of the ACIVS Advanced Concepts for Intelligent Vision Systems (2002)
500. Van Wijk, J.: Image based flow visualization. ACM Transactions on Graphics (TOG) 21(3), 745–754 (2002)
501. Van Zijl, L., Botha, L.: Feature extraction for image pattern matching with cellular automata. In: Holub, J., Zdárek, J. (eds.) Proceedings of the Prague Stringology Conference, Prague, Czech Republic, pp. 3–14 (2009)

502. Vankamamidi, V., Ottavi, M., Lombardi, F.: A line-based parallel memory for QCA implementation. IEEE Transactions on Nanotechnology 4(6), 690–698 (2005)
503. Vankamamidi, V., Ottavi, M., Lombardi, F.: A serial memory by quantum-dot cellular automata (QCA). IEEE Transactions on Computers 57(5), 606–618 (2008)
504. Vankamamidi, V., Ottavi, M., Lombardi, F.: Two-dimensional schemes for clocking/timing of QCA circuits. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 27(1), 34–44 (2008)
505. Veltkamp, R.C., Hagedoorn, M.: Shape similarity measures, properties and constructions. In: Laurini, R. (ed.) VISUAL 2000. LNCS, vol. 1929, pp. 467–476. Springer, Heidelberg (2000)
506. Ventrella, J.: Earth Day 2009 – A Spherical Cellular Automaton (2009), http://www.ventrella.com/EarthDay/EarthDay.html
507. Ventrella, J.: Glider Dynamics on the Sphere: Exploring Cellular Automata on Geodesic Grids. Journal of Cellular Automata 6(2-3), 245–256 (2011)
508. Vezhnevets, V., Konouchine, V.: “growcut"-interactive multi-label n-d image segmentation by cellular automata. In: Proc. Graphicon, pp. 150–156 (2005)
509. Vichniac, G.: Cellular-automata fluids. In: Tirapegui, E., Villarroel, D. (eds.) Instabilities and Nonequilibrium Structures II. Mathematics and Its Applications, vol. 50, pp. 97–116. Springer, Netherlands (1989)
510. Vincent, L., Soille, P., Wang, Q.: Segmentation for MRA image: An improved level-set approach. IEEE Transactions on Instrumentation and Measurement 56(4), 1316–1321 (2007)
511. Vorn, B.: Evil / live (1997), http://billvorn.concordia.ca/robography
512. Wakefield, G., Ji, H(H.): Artificial nature: Immersive world making. In: Giacobini, M., Brabazon, A., Cagnoni, S., Di Caro, G.A., Ekárt, A., Esparcia-Alcázar, A.I., Farooq, M., Fink, A., Machado, P. (eds.) EvoWorkshops 2009. LNCS, vol. 5484, pp. 597–602. Springer, Heidelberg (2009)
513. Walus, K., Dysart, T.J., Jullien, G.A., Budiman, R.A.: QCADesigner: A rapid design and simulation tool for quantum-dot cellular automata. IEEE Transactions on Nanotechnology 3(1 spec. iss.), 26–31 (2004)
514. Wang, J., Liu, G., Li, H., Dai, Y., Wang, Z.: Detection of image region duplication forgery using model with circle blocks, pp. 25–29 (2009)
515. Wang, J., Liu, G., Zhang, Z., Dai, Y., Wang, Z.: Fast and robust forensics for image region-duplication forgery. Acta Automatica Sinica 35(12), 1488–1495 (2009)
516. Wang, J.M., Sukhwani, B., Padmanabhan, U., Ma, D., Sinha, K.: Simulation and design of nanocircuits with resonant tunneling devices. IEEE Transactions on Circuits and Systems I: Regular Papers 54(6), 1293–1304 (2007)
517. Willebeek-LeMair, M.H., Reeves, A.P.: Strategies for dynamic load balancing on highly parallel computers. IEEE Trans. Parallel Distrib. Syst. 4(9), 979–993 (1993)
518. Winfree, E.: Self-healing tile sets, architecture for dna-guided self-assembly of nanoelectronics. In: Nanotechnology: Science and Computation (2006), http://www.dna.caltech.edu/DNAresearch_publication
519. Wolfram, S.: Cellular automaton fluids 1: Basic theory. Journal of Statistical Physics 45(3-4), 471–526 (1986)
520. Wolfram, S.: Theory and Applications of Cellular Automata: Including Selected Papers, 1983-1986. Advanced Series on Complex Systems. World Scientific Publishing Company, Incorporated (1986)
521. Wolfram, S.: Random sequence generators. US Patent 4691291A (1987)
522. Wolfram, S.: Cellular Automata and Complexity: Collected Papers. Westview Press (1994)

523. Wolfram, S.: A New Kind of Science. Wolfram Media, Inc., Champaign (2002)
524. Woltz, D., de Oliveira, P.: Very effective evolutionary techniques for searching cellular automata rule spaces. Journal of Cellular Automata 3(4), 289–312 (2008)
525. Wongthanavasu, S.: Cellular automata for medical image processing. In: Salcido, A. (ed.) Cellular Automata – Innovative Modelling for Science and Engineering, pp. 395–410 (2011)
526. Wongthanavasu, S., Lursinsap, C.: A 3-D CA-based edge operator for 3-D images. In: Int. Conf. Image Processing, pp. 235–238 (2004)
527. Wongthanavasu, S., Sadananda, R.: A CA-based edge operator and its performance evaluation. J. Visual Communication and Image Representation 14(2), 83–96 (2003)
528. Wright, M.: Open sound control: An enabling technology for musical networking. Organised Sound 10(3), 193–200 (2005)
529. Wuensche, A.: The ghost in the machine: Basins of attraction of random boolean networks. In: Langton, C. (ed.) Artificial Life III, Santa Fe Institute Studies in the Sciences of Complexity. Addison-Wesley (1994)
530. Xenakis, I.: Formalized Music: Thought and Mathematics in Composition. Pendragon Press (1992)
531. Xiaoyang, Y., Yang, S., Yang, Y., Shuchun, Y., Hao, C., Yanxia, G.: An encryption method for QR code image based on ECA. International Journal of Security & Its Applications 7(5) (2013)
532. Xiong, R., Ding, W., Ma, S., Gao, W.: Improved autoregressive image model estimation for directional image interpolation. In: 28th Picture Coding Symposium, Nagoya, Japan, pp. 442–445 (2010)
533. Xu, S., Lau, F., Pan, Y.: A computational approach to digital Chinese painting and calligraphy. Springer (2009)
534. Yamins, D.: A theory of local-to-global algorithms for one-dimensional spatial multiagent systems. Ph.D. thesis, Harvard University Cambridge, Massachusetts (2007)
535. Yang, C., Ye, H., Wang, G.: Cellular automata modeling in edge recognition. In: 7th Int. Symp. on Artificial Life and Robotics, pp. 128–132 (2002)
536. Ye, Q.Z., Danielsson, P.E.: Inspection of printed circuit boards by connectivity preserving shrinking. IEEE Transactions on Pattern Analysis and Machine Intelligence 10(5), 737–742 (1988)
537. Zardalidis, G., Karafyllidis, I.G.: SECS: A new single-electron-circuit simulator. IEEE Transactions on Circuits and Systems I: Regular Papers 55(9), 2774–2784 (2008)
538. Zawidzki, M.: Window Opacity Controlled By Cellular Automata. An interactive demonstration (2008), `http://demonstrations.wolfram.com/WindowOpacityControlledByCellularAutomata/`
539. Zawidzki, M.: 2D Triangular Cellular Automata on a Distorted Grid with Holes. An interactive demonstration (2009), `http://demonstrations.wolfram.com/2DTriangularCellularAutomataOnADistortedGridWithHoles/`
540. Zawidzki, M.: 2D2CR1 Cellular Automaton On a Triangular Grid. An interactive demonstration (2009), `http://demonstrations.wolfram.com/2D2CR1CellularAutomatonOnATriangularGrid/`
541. Zawidzki, M.: Delayed CA. An interactive demonstration (2010), `http://demonstrations.wolfram.com/DelayedCA/`
542. Zawidzki, M.: Application of Semitotalistic 2D Cellular Automata on a triangulated 3D surface. International Journal of Design & Nature and Ecodynamics 6(1), 34–51 (2011)

543. Zawidzki, M.: 2D Cellular Automaton on a Triangulated Surface. An interactive demonstration (2013), http://demonstrations.wolfram.com/2DCellularAutomatonOnATriangulatedSurface/
544. Zawidzki, M.: Dynamic shading of building envelope based on rotating polarized film system controlled by one-dimensional cellular automata on regular tessellations (triangular, square & hexagonal). In: Advanced Engineering Informatics (2013) (under review)
545. Zawidzki, M.: Implementing Cellular Automata for Dynamically Shading a Building Facade. Complex-Systems 18(3), 287–305 (2013)
546. Zawidzki, M., Bator, M.: Application of evolutionary algorithm for optimization of the sequence of initial conditions for the cellular automaton-based shading. Journal of Cellular Automata 7(5-6), 363–384 (2013)
547. Zawidzki, M., Fujieda, I.: The prototyping of a shading device controlled by a cellular automaton. Complex-Systems 19(2), 157–175 (2010)
548. Zawidzki, M., Nishinari, K.: Shading for building facade with two-color one-dimensional range-two cellular automata on a square grid. Journal of Cellular Automata 8(3-4), 147–163 (2013)
549. Zhang, K., Zhang, W., Yuan, J.: Edge detection of images based on cloud model cellular automata. In: Chinese Control Conference, pp. 249–253 (2008)
550. Zhang, T.Y., Suen, C.Y.: A fast parallel algorithm for thinning digital patterns. Communications of the ACM 27(3), 236–239 (1984)
551. Zhang, Y.Y., Wang, P.S.P.: A parallel thinning algorithm with two-subiteration that generates one-pixel-wide skeletons. In: Proceedings of the 13th International Conference on Pattern Recognition, 1996, vol. 4, pp. 457–461 (1996)
552. Zheng, Y., Kambhamettu, C.: Learning based digital matting. In: IEEE 12th International Conference on Computer Vision 2009, pp. 889–896 (2009)
553. Zheng, Z., Leung, C.: Graph indexes of 2D-thinned images for rapid content-based image retrieval. Journal of Visual Communication and Image Representation 8(2), 121–134 (1997)
554. Zipf, P., Hinkelmann, H., Shao, H., Dogaru, R., Glesner, M.: An area-efficient FPGA realisation of a codebook-based image compression method. Proceedings of FPT 2008, 349–352 (2008)

Index

acceleration 175
aesthetic exploration 261
algebraic normal form 5, 20
algorithmic art 253
algorithmic music 253
angularity 261
average pattern density 209

binary representations 112
  binary planes 113
  local binary pattern 114
  thresholding 113
binary synchronization 1, 3, 9, 10, 12
binary wire 70
block cellular automata 238, 239
boundary detection 86
building envelope (BE) 205, 207

CA rule 3, 4, 9, 11, 18, 166
CA-VQ 4, 15, 16, 19, 20
Canny edge detector 26, 29, 85, 152
cartography 49
cell complex 50
cellular automata
  copy-move 106, 112, 117
  rule identification 116
cellular automaton 25
  boundary conditions 33
  diffusion model 91
  half-distance 212, 220
  hardware 28, 227
  higher-order (HO) 213, 219
  image processing 27
  Margolus neighborhood 235, 238, 239, 244
  Moore neighborhood 30, 238, 246
  non-uniform 227
  reversible 213
  rule changing 90
  rule extraction 239
  rules 25
  stereo vision 27
  transformation 91
  triangular (TCA) 222
  von Neumann neighborhood 29, 238, 245, 246
  with specified offsets 219
cellular automaton-based shading system (CASS) 205, 206
cellular learning automaton 91
cellular neural network 91
central processing unit (CPU) 49, 56
chaotic counter 1, 6–8, 10, 11
chaotic scan 2, 4, 7, 15, 19
chemoton 257
codebook 4, 15, 18, 19
colour matching 153
CoMoFoD 107
compute unified device architecture (CUDA)™ 49
content-based image retrieval 147
convexity
  θ-convexity 187
  convex hull 185
  convex set 184
  metric convexity 189
coronary arteries 49

crossover (XO) 225
cybernetics 253

data embedding algorithm 133, 135
daylighting 206
Delaunay graph 197
dental segmentation 177
diffusion 140
digital image forensics 105, 128
dihedral rotation (DR) 216
dilation 68
distance field 193

edge
  detection 85, 152
  linking 92
  magnitude 85, 91
  thresholding 85, 86, 92, 93
emergence 208
emergent behavior 2
encryption 1, 2, 15, 18, 20
erosion 68
evolution strategy (ES) 224
evolutionary algorithm (EA) 223

false positive rate (FPR) 172, 180
feature vector 148
feedback 261, 266
flood filling 155
flow visualization 261
fluid automata 256
fluid profile 258, 259
fluid simulation 256, 257
forgery detection 106
  active 128
  algorithm 133
  blurred image 119
  copy-move 105, 107, 109
  noisy image 119
  passive 128
  plain copy-move 116, 119
  pre-preocessing 119
FPGA 2, 4, 5, 20, 21, 227
fuzzy cellular automata 91
fuzzy cellular neural network 91

Gabriel graph
  Euclidean 198
  metric 200
gene expression programming 90
generative art 254
genetic algorithm 87
genetic programming 90
geodesic sphere 221
  icosahedral geodesic sphere 221
graphics processors unit (GPU) 49, 56
grayness
  function (GF) 210
  function error (GFE) 210
  monotonicity and pattern distribution error (GDE) 210
grid
  hexagonal 218
  irregular 221
  regular 215
  triangular 219
  with voids 222
Guo and Hall
  algorithm 51
  conditions 52

histogram 153
human fingerprints 49
hybrid cellular automata 1, 10–12, 21

image authentication 105, 128
  active methods 105
  block-based methods 108, 109
  feature vectors 111
  keypoint-based methods 107
  passive methods 105
image compression 4, 15
image forgery 105
  combination 107
  copy-move 105, 107
  distortion 107
  plain copy-move 107
  post-processing 107, 119
  rotation 107
  scaling 107
image interpolation
  bicubic 26
  bilinear 26
  fuzzy 26
  linear 26
  near edge orientation 29
  nearest neighbor 26
  neural network 26
  nonlinear Partial Differential Equation 26

spline 26
zero-ground 29
image processing 149, 261, 264
image resizing 25
2-D lattice 32
adaptive algorithms 26
binary image 31
Cameraman 35
Canny edge detector 27
filtering 30
hysteresis thresholding 30
intensity gradient 30
maximum suppression 30
color image 31
edge cell 32
edge detection 31
edge-directed techniques 25
high resolution 34
Lena 35
low resolution 34
NEDI 35
non-adaptive algorithms 26
non-edge cell 32
processing time 25
PSNR 25
Teddy 35
undefined cell 32
weights 34
interaction 264, 265
interactive cellular automata 255
interactive segmentation 164, 165, 168
block CA segmentation (BCA) 177
regional CA segmentation (RCA) 169
interval 189
Islamic words 235

L-system 234, 241, 243
labels 168
LU Decomposition 129

Ma'qeli script 235, 237, 244–248
magic lens 264
mathematical morphology 67
media arts 253
medial axis transformation 48, 49
metric space 189
motifs 235
multi-planar reformation (MPR) 173
multi-touch 265
multimedia 266
mutation 225

Navier-Stokes equations 257
noise reduction 149

optical character recognition (OCR) 49
order-based representation 209

particle 209
pattern formation 235, 239
peak signal to noise ratio (PSNR) 138, 150
performance 264
evaluation 86
Persian script 235, 240
polarized film shading system (PFSS) 215
printed circuit boards 49
pyramid 175

quantum-dot cellular automata (QCA) 69
cell 69
circuit design 73
circuit operational stability 74
circuit simulation 75
implementation 75
inverter 71
logic gates 71
majority gate 70
technology 66

regional merging 169–171
rhombododecahedron 257
rotational symmetric pattern 87

scan 5
scientific visualization 256
segmentation 148, 153
dental 177
interactive 164
unsupervised 153
semi-supervised learning 177
sequence of initial conditions (SIC) 209
sequential floating forward search 92
shape matching 155
single electron technology 69
Singular Value Decomposition (SVD) 134
skeletonization 47
skin 206, 207
soliton 209
state transition rule 87

stream vector 259
strobe effect 211
structuring element 67
surface
  free-form 223
  triangulated 221
synaesthetic 266

tessellations 238
texture advection 261
thinning 48
threshold decomposition 91
thresholding 89, 91
training data 96
triangle inequality 189
true and false alert 138
true positive rate (TPR) 172, 180
turbulence 261
turtle graphics 241

vector quantization 4, 17
video texture 262
virtual environment 264
viscosity 261
Voronoï diagram 196